

ARC12 ▪ Computer Architecture and Laboratory

RISC-V on the PRG32 platform

University of Naples “Parthenope” • A.Y. 2026/27

Computer Architecture

Program First, Formalise Later

From the first RISC-V instruction to a published game:
information, logic, memory and the processor, taught on the PRG32 platform
through the course-long project Verm1

Raffaele Montella

Department of Science and Technology
University of Naples “Parthenope”

Verm1 reference implementation contributed by Giuseppe Coviello

Textbook of the lectures of *Computer Architecture and Computer Architecture Laboratory* (ARC12), Bachelor’s Degree in Computer Science, first year, first semester.

The assembly listings of this book are read from files that are assembled and tested automatically.

Platform: https://github.com/riscv-prg32/PRG32
Desktop host and debugger: https://github.com/riscv-prg32/PRG32-QT

Computer Architecture: Program First, Formalise Later.
Textbook of the lectures of ARC12, Computer Architecture and Computer Architecture Laboratory, University of Naples "Parthenope", academic year 2026/27.

The book follows, chapter by chapter, the twenty-five lectures of the course: Chapter N is Lecture N. Each lecture has two slide decks, one for the concepts and one for the code. The decks, the program of each lecture, the reference programs of the Verm1 project and the tests that assemble and run every listing are kept together with the sources of this book; the listings printed here are read from the same files that the tests execute.

The book is built around PRG32, an open educational runtime for RISC-V assembly and C games that targets the ESP32-C6 microcontroller, the Espressif QEMU emulator and the PRG32-QT desktop host. The platform, its source code and its documentation are distributed under the MIT License by their authors: Raffaele Montella (academic supervisor and project lead), Ivan Cafiero and Simone Boscaglia (student contributors), University of Naples "Parthenope". Readers who use PRG32 in coursework, reports, theses or teaching material are asked to cite it through the `CITATION.cff` metadata of its repository (Montella et al., 2026a). The companion volume, *Playing with RISC-V* (Montella et al., 2026b), is the laboratory manual of the platform.

The reference implementation of Verm1, the course-long project of the book (the four programs of Appendix D), was contributed by Giuseppe Coviello.

The text and the original figures of this book are released under the Creative Commons Attribution–ShareAlike 4.0 International licence (CC BY-SA 4.0). The programs are released under the MIT License. Historical photographs are reproduced from the lecture slides for teaching purposes and remain the property of their respective owners.

RISC-V is a registered trademark of RISC-V International. ESP32, ESP32-C3, ESP32-C6 and ESP-IDF are products and trademarks of Espressif Systems. GitHub is a trademark of GitHub, Inc. All other trademarks are the property of their respective owners and are used here for identification only.

Edition compiled October 13, 2026. Typeset with LaTeX in Latin Modern.

Contents

Preface **xi**

How to Use This Book **xiii**

I Start **1**

0 Introduction: Program First, Formalise Later **3**
0.1 The course at a glance . . . 3
0.2 A new approach . . . 4
0.3 The machine: PRG32 and RISC-V . . . 5
0.4 The project: Verm1 . . . 8
0.5 The course in blocks . . . 9
0.6 Syllabus and learning outcomes . . . 9
0.7 Organisation and assessment . . . 11
0.8 Material . . . 12
0.9 First contact: who are you? . . . 13
0.10 Before the next chapter . . . 13
0.11 The program of this chapter . . . 14

1 PRG32 and PRG32-QT: Setting Up Your Workshop **17**
1.1 Four levels: install what you need . . . 17
1.2 The platform, in detail . . . 18
1.3 Step 1: install PRG32-QT . . . 20
1.4 Step 2: install the toolchain . . . 21
1.5 Step 3: your first build . . . 24
1.6 Optional: QEMU and the kit . . . 26
1.7 When something does not work . . . 27
1.8 The program of this chapter . . . 28
1.9 Check your understanding . . . 30
1.10 Exercises . . . 30
1.11 Glossary . . . 31
1.12 Summary and reading . . . 31

2 First Contact: RISC-V Assembly on PRG32 **35**
2.1 The machine . . . 35
2.2 Your first cartridge . . . 36
2.3 Reading what you wrote . . . 42

2.4 What comes next . . . 44
2.5 The program of this chapter . . . 44
2.6 Check your understanding . . . 46
2.7 Exercises . . . 47
2.8 Glossary . . . 47
2.9 Summary and reading . . . 47

II Information **49**

3 History of Computing **51**
3.1 What is informatics? . . . 51
3.2 Before electricity . . . 52
3.3 The twentieth century . . . 54
3.4 From home computers to RISC-V . . . 56
3.5 Informatics as an applied science . . . 58
3.6 The program of this chapter . . . 59
3.7 Check your understanding . . . 61
3.8 Exercises . . . 62
3.9 Glossary . . . 62
3.10 Conclusions and reading . . . 62

4 Representing Digital Information **63**
4.1 Information . . . 63
4.2 Text . . . 66
4.3 Data structures . . . 68
4.4 Data and algorithms . . . 69
4.5 The program of this chapter . . . 72
4.6 Check your understanding . . . 74
4.7 Exercises . . . 74
4.8 Glossary . . . 75
4.9 Conclusions and reading . . . 75

5 Number Representation: Integers **77**
5.1 Warm-up on the machine . . . 77
5.2 Bases and conversions . . . 78
5.3 Natural numbers and overflow . . . 80
5.4 Signed integers . . . 81
5.5 Back to the machine . . . 85
5.6 The program of this chapter . . . 87
5.7 Check your understanding . . . 90
5.8 Exercises . . . 91
5.9 Glossary . . . 91
5.10 Conclusions and reading . . . 91

6 Number Representation: Real Numbers **93**
6.1 Warm-up on the machine . . . 93
6.2 More encodings for signed integers . . . 94
6.3 Fixed point . . . 95
6.4 Floating point . . . 97

6.5 Back to the machine . . . 100
6.6 The program of this chapter . . . 100
6.7 Check your understanding . . . 102
6.8 Exercises . . . 103
6.9 Glossary . . . 103
6.10 Conclusions and reading . . . 103

7 Verm1, Part 1: A Head That Crosses the World 105
7.1 The project . . . 105
7.2 Step 0: the working loop . . . 107
7.3 Step 1: three functions that return . . . 107
7.4 Step 2: a grid and a head . . . 108
7.5 Step 3: movement . . . 110
7.6 Step 4: look inside with PRG32-QT . . . 112
7.7 Check your understanding . . . 113
7.8 Make it yours . . . 114
7.9 Where we are, and what comes next . . . 114

III Logic 117

8 Boolean Algebra: Operators and Properties 119
8.1 Warm-up on the machine . . . 119
8.2 The computer as a logic machine . . . 120
8.3 Boolean algebra . . . 122
8.4 Back to the machine . . . 124
8.5 The program of this chapter . . . 126
8.6 Check your understanding . . . 129
8.7 Exercises . . . 129
8.8 Glossary . . . 130
8.9 Conclusions and reading . . . 130

9 Boolean Algebra: Functions, Canonical Forms and Minimisation 131
9.1 Warm-up on the machine . . . 131
9.2 Boolean functions . . . 132
9.3 Canonical forms . . . 134
9.4 Karnaugh maps . . . 136
9.5 Back to the machine . . . 137
9.6 The program of this chapter . . . 138
9.7 Check your understanding . . . 141
9.8 Exercises . . . 142
9.9 Glossary . . . 142
9.10 Conclusions and reading . . . 142

10 Electronics: What a Bit Is Made Of 145
10.1 Warm-up on the machine . . . 145
10.2 Electricity in four quantities . . . 146
10.3 Semiconductors . . . 148
10.4 From volts to bits . . . 149
10.5 Back to the machine . . . 151

10.6 The program of this chapter . . . 153
10.7 Check your understanding . . . 155
10.8 Exercises . . . 156
10.9 Glossary . . . 156
10.10 Conclusions and reading . . . 157

11 Combinational Machines **159**
11.1 Warm-up on the machine . . . 159
11.2 Combinational machines . . . 160
11.3 Codes . . . 161
11.4 Decoders and encoders . . . 163
11.5 Multiplexers and demultiplexers . . . 165
11.6 Comparing and checking . . . 166
11.7 The arithmetic logic unit . . . 168
11.8 The program of this chapter . . . 169
11.9 Check your understanding . . . 171
11.10 Exercises . . . 171
11.11 Glossary . . . 172
11.12 Conclusions and reading . . . 172

12 Verm1, Part 2: Steering, Food and a Score **173**
12.1 Start from a Part 1 that runs . . . 173
12.2 Step 1: names for bits . . . 174
12.3 Step 2: steering . . . 175
12.4 Step 3: food, a score, a sound . . . 178
12.5 Step 4: watch a decision . . . 182
12.6 Check your understanding . . . 183
12.7 Make it yours . . . 184
12.8 Where we are, and what comes next . . . 184

IV State and Memory **185**

13 Sequential Machines **187**
13.1 Warm-up on the machine . . . 187
13.2 Time and state . . . 188
13.3 Finite-state machines . . . 189
13.4 Latches . . . 192
13.5 Flip-flops . . . 194
13.6 Back to the machine . . . 195
13.7 The program of this chapter . . . 198
13.8 Check your understanding . . . 200
13.9 Exercises . . . 200
13.10 Glossary . . . 201
13.11 Conclusions and reading . . . 201

14 Elementary Machines: Registers and Buses **203**
14.1 Warm-up on the machine . . . 203
14.2 Word gates and registers . . . 204
14.3 Buses . . . 204

14.4 Three-state buffers . . . 205
14.5 Shift registers . . . 207
14.6 Back to the machine . . . 208
14.7 The program of this chapter . . . 210
14.8 Check your understanding . . . 212
14.9 Exercises . . . 213
14.10Glossary . . . 213
14.11Conclusions and reading . . . 213

15 Memory **215**
15.1 Warm-up on the machine . . . 215
15.2 What is a memory? . . . 216
15.3 A little history: rings of ferrite . . . 216
15.4 Memory today . . . 217
15.5 The main memory . . . 218
15.6 Processor and memory . . . 220
15.7 Memory and input/output . . . 221
15.8 The program of this chapter . . . 222
15.9 Check your understanding . . . 225
15.10Exercises . . . 225
15.11Glossary . . . 226
15.12Conclusions and reading . . . 226

16 Permanent Memory **227**
16.1 Warm-up on the machine . . . 227
16.2 The components of a computer . . . 228
16.3 Non-volatile memories . . . 229
16.4 The cache . . . 230
16.5 Secondary memory . . . 232
16.6 Optical disks . . . 233
16.7 Storage on several levels . . . 234
16.8 The program of this chapter . . . 235
16.9 Check your understanding . . . 238
16.10Exercises . . . 238
16.11Glossary . . . 239
16.12Conclusions and reading . . . 239

17 Verm1, Part 3: A Body, Rules and States **241**
17.1 The design, before the code . . . 242
17.2 Step 1: the memory of the game . . . 243
17.3 Step 2: three states . . . 244
17.4 Step 3: a worm in memory . . . 248
17.5 Step 4: rules . . . 251
17.6 Step 5: look at memory move . . . 256
17.7 Check your understanding . . . 257
17.8 Make it yours . . . 257
17.9 Two functions that did not change . . . 258
17.10Where we are, and what comes next . . . 258

V The Machine **259**

18 The Processor **261**
18.1 Warm-up on the machine . . . 261
18.2 The processor . . . 262
18.3 Registers . . . 264
18.4 The stack . . . 266
18.5 Processor models . . . 267
18.6 The ALU and the whole picture . . . 268
18.7 The program of this chapter . . . 269
18.8 Check your understanding . . . 271
18.9 Exercises . . . 272
18.10 Glossary . . . 272
18.11 Conclusions and reading . . . 272

19 Instructions and Interrupts **275**
19.1 Warm-up on the machine . . . 275
19.2 Instructions . . . 276
19.3 The processor cycle . . . 278
19.4 Interrupts . . . 280
19.5 The RISC-V solution . . . 282
19.6 The program of this chapter . . . 284
19.7 Check your understanding . . . 287
19.8 Exercises . . . 288
19.9 Glossary . . . 288
19.10 Conclusions and reading . . . 289

20 Buses and the Video Card **291**
20.1 Warm-up on the machine . . . 291
20.2 Peripherals and controllers . . . 292
20.3 Buses . . . 292
20.4 The video card . . . 293
20.5 GPU and GPGPU . . . 296
20.6 The program of this chapter . . . 297
20.7 Check your understanding . . . 298
20.8 Exercises . . . 299
20.9 Glossary . . . 300
20.10 Conclusions and reading . . . 300

21 Input and Output **301**
21.1 Warm-up on the machine . . . 301
21.2 Displays . . . 302
21.3 Keyboard and mouse . . . 304
21.4 Printers . . . 305
21.5 Back to the machine . . . 305
21.6 The program of this chapter . . . 306
21.7 Check your understanding . . . 309
21.8 Exercises . . . 309
21.9 Glossary . . . 310
21.10 Conclusions and reading . . . 310

22 Verm1, Part 4: The Finished Game **311**
22.1 Step 1: the score in digits . . . 312
22.2 Step 2: levels . . . 314
22.3 Step 3: `step` tells what it changed . . . 316
22.4 Step 4: draw only what changed . . . 319
22.5 Step 5: measure . . . 324
22.6 Check your understanding . . . 325
22.7 Make it yours . . . 326
22.8 The functions that did not change . . . 326
22.9 Where we are, and what comes next . . . 329

VI Publication and Outlook **331**

23 Verm1 on the Cartridge Store **333**
23.1 Why publish? . . . 333
23.2 Step 1: one source, two cartridges . . . 334
23.3 Step 2: an identity . . . 335
23.4 Step 3: the cartridge carries its papers . . . 337
23.5 Step 4: the bundle . . . 339
23.6 Step 5: publication . . . 339
23.7 Check your understanding . . . 340
23.8 Your release . . . 341
23.9 What you built . . . 341

24 Beyond One Core: Superscalar RISC-V, HPC, GPUs and FPGAs **343**
24.1 Warm-up on the machine . . . 343
24.2 Superscalar processors . . . 344
24.3 Vectors: one instruction, many data . . . 347
24.4 High-performance computing . . . 348
24.5 GPU accelerators . . . 350
24.6 FPGA accelerators . . . 351
24.7 Back to the machine . . . 353
24.8 The program of this chapter . . . 354
24.9 Check your understanding . . . 356
24.10Exercises . . . 357
24.11Glossary . . . 357
24.12Conclusions and reading . . . 357

VII Appendices **359**

A RISC-V Reference **361**
A.1 The name of an instruction set . . . 361
A.2 Registers . . . 361
A.3 Instruction formats . . . 361
A.4 RV32I: the base integer instruction set . . . 362
A.5 The M extension: multiplication and division . . . 365
A.6 The A extension: atomic instructions . . . 365
A.7 The C extension: compressed instructions . . . 366

A.8 Pseudo-instructions . . . 367
A.9 The assembler: directives, labels and expressions . . . 368
A.10 The calling convention . . . 369
A.11 Privilege, control and status registers, traps . . . 370
A.12 Encoding and decoding by hand . . . 371
A.13 Idioms . . . 371

B PRG32 Reference **375**
B.1 The platform at a glance . . . 375
B.2 The contract . . . 376
B.3 Screen, colours and input . . . 377
B.4 The programming interface . . . 377
B.5 The cartridge file . . . 381
B.6 The command-line tools . . . 382
B.7 The hosts . . . 384
B.8 Testing a program on the three hosts . . . 385
B.9 The Cartridge Store . . . 387
B.10 When something does not work . . . 388

C Git and GitHub Reference **389**
C.1 Why version control . . . 389
C.2 Concepts . . . 389
C.3 Installing and configuring . . . 390
C.4 Getting a repository . . . 391
C.5 The everyday cycle . . . 391
C.6 Looking at the history . . . 392
C.7 Undoing . . . 392
C.8 Branches . . . 393
C.9 Working with GitHub . . . 393
C.10 Versions, tags and releases . . . 396
C.11 This book and Overleaf . . . 396
C.12 Citing software . . . 397
C.13 Command summary . . . 397

D Verm1: The Complete Sources **399**
D.1 Part 1: a head that crosses the world . . . 399
D.2 Part 2: steering, food and a score . . . 401
D.3 Part 3: a body, rules and states . . . 405
D.4 Part 4: the finished game . . . 413
D.5 The files of the Store bundle . . . 423

E Answers to the Questions **427**

F English–Italian Glossary **433**

Preface

Most courses on computer architecture are built from the bottom up. They begin with the bit, climb through number systems, Boolean algebra and logic networks, assemble a processor out of gates, and only in the last weeks let the student write a program for the machine that the previous months have described. The order is logically impeccable, and for many first-year students it is also the reason why the subject feels remote: for ten weeks every answer arrives before its question.

This book is the written form of a course that turns the order round without giving up any of the rigour. *Program first, formalise later*: in the first week the reader installs a small console, types a dozen lines of RISC-V assembly and sees a rectangle move across a screen. From then on every formal topic is introduced by something that the reader's own program has already done. Two's complement arrives after a position has walked off the left edge of the screen and the debugger has shown `0xFFFFFFFF`. Boolean algebra arrives after a mask has been used to ask whether a button is pressed. The flip-flop arrives after a counter has survived from one frame to the next, and the instruction cycle after the reader has pressed **Step** a few hundred times and watched the program counter advance by four.

Nothing is removed from the classical syllabus. The reader will find positional notation and the IEEE 754 formats, axioms and theorems of Boolean algebra with their proofs, canonical forms and Karnaugh maps, the transistor as a switch, adders, multiplexers and decoders, latches and flip-flops, finite state machines, registers and buses, memory hierarchies and permanent storage, the datapath and the control unit, interrupts, the video card and the peripheral devices. What changes is the direction of the argument: each chapter opens on the machine, leaves it for as long as the theory requires, and comes back to it with a section called *Back to the machine*, where the same rules are read again in the instructions of a real processor.

The machine

The processor is a RISC-V core: the RV32IMAC instruction set, as implemented by the Espressif ESP32-C6. RISC-V is an open standard, small enough to be learnt completely in one semester and real enough to be found in billions of devices; it replaces here the Motorola 68000 of the previous editions of the course, which survives only in a few historical comparisons.

The environment is PRG32, an open educational platform developed at the University of Naples "Parthenope". A PRG32 program is a *cartridge*: a small file of native machine code that supplies three functions, one called once and two called at every frame. The same cartridge runs on a breadboard kit, on the QEMU emulator and on PRG32-QT, a desktop

application whose built-in debugger shows the thirty-two registers and the memory while the program runs one instruction at a time. No board is needed to follow the book: a laptop is enough.

The project

A platform that is used only for exercises is a gymnasium. PRG32 is also a workshop, and the book has a project that runs through it: Verm1, read *vermone*, "the big worm", a tribute to Snake written from an empty file in RISC-V assembly. After every block of four chapters a tutorial chapter adds a part of the game, using only what those four chapters have taught: first a head that crosses a world without edges, then steering, food and a score, then a body that grows and the rules that end a game, then digits, levels and sound. A last tutorial packages the finished game for the PRG32 Cartridge Store. Every tutorial prints every line of the program, tells which lines are new and which have changed, and stops at checkpoints where the program must run and behave in a stated way.

What "verified" means in this book

A listing that does not assemble teaches the wrong lesson. The assembly code in this book is not typed into the text: it is read from files that are assembled, run on the upstream QEMU and on the headless runner of PRG32-QT, and compared with expected results by automatic tests; the four parts of Verm1 are played by a scripted player that also takes the screenshots printed in the tutorial chapters. Where a fragment could not be run on the author's machine (the CUDA and Verilog fragments of the last chapter, the installation steps for operating systems other than the one used to write the book) the text says so.

Acknowledgements

Mariacarla Staffa teaches the laboratory half of the course, where the exercises of this book are done with pencil, paper and keyboard. Ivan Cafiero and Simone Boscaglia contributed to PRG32 as students and showed that the platform can be understood, and extended, by the people for whom it is meant. Giuseppe Coviello contributed the reference implementation of Verm1: the four programs that the tutorial chapters build line by line, and that the automatic tests play, are his. The students of the previous editions of the course wrote, with their questions, most of the paragraphs that begin with "Watch out".

Raffaele Montella
Naples, October 2026

How to Use This Book

One chapter, one lecture

The book has twenty-five chapters, numbered from 0 to 24 as the lectures of the course are, and as almost everything else in this book is: bits, registers, rows and columns all start from zero. Chapter N follows Lecture N, which has two slide decks. The *lecture deck*, of at most twenty slides, holds the concepts; the *code deck* holds everything that is typed and run: each listing with its explanation, the complete program of the lecture, what it shows, and the way to test it on PRG32-QT, on QEMU and on the board. The chapter is the long form of both: the order of the sections is the order of the slides, the listings are the same files, the questions at the end are those shown in class, and what twenty slides have no room for is here. A reader who has attended the lecture finds the sentences that a slide cannot hold; a reader who has missed it finds the lecture.

Most chapters end with a section, *The program of this chapter*, that prints in full the program from which the listings of the chapter are taken, shows the frame that it produces and gives the commands to run it. The programs are in the folder `code` of the repository of the course, one per lecture, and every one of them is built and run by the automatic tests; Section B.8 is the step-by-step guide for the three hosts.

Part	Chapters	Then, the project
Start	0 Introduction, 1 Setting up, 2 First contact	
Information	3 History, 4 Digital information, 5 Integers, 6 Reals	7 Verm1, part 1
Logic	8–9 Boolean algebra, 10 Electronics, 11 Combinational machines	12 Verm1, part 2
State and memory	13 Sequential machines, 14 Registers and buses, 15 Memory, 16 Permanent memory	17 Verm1, part 3
The machine	18 Processor, 19 Instructions and interrupts, 20 Video card, 21 Input and output	22 Verm1, part 4
Publication, outlook	24 Beyond one core	23 Verm1 on the Cartridge Store

The shape of a topical chapter

Every chapter that presents a topic has the same five movements.

1. **Warm-up on the machine.** A few lines of RISC-V on PRG32 that produce a fact

needing an explanation. Run them before reading on.

2. **The formal treatment.** Definitions, theorems, methods and worked examples.
3. **Back to the machine.** The same rules, as RISC-V implements them.
4. **Check your understanding.** Multiple-choice questions in the style of the midterm test. Answers, with a line of explanation each, are in Appendix E: read them after answering, not instead.
5. **Exercises, glossary, reading.** Exercises are done on paper first and on PRG32 afterwards; the glossary gives the Italian term for every English one, since the course is taught in a bilingual setting.

The shape of a tutorial chapter

Chapters 7, 12, 17, 22 and 23 build Verm1. They are self-contained and meant to be followed at the keyboard. Each step shows a few lines, says whether they are **new**, **changed** or **unchanged**, and ends with a *Run it* checkpoint stating what the screen must show. If a program refuses to work at the beginning of a tutorial, start from the reference file of the previous part (Appendix D): nobody should be left behind by a bug.

Boxes

Concept

A **Concept** box states an idea that the rest of the chapter depends on.

Definition 0.1 (how definitions look)

Formal definitions and theorems are numbered within the chapter.

Try It Yourself

A **Try It Yourself** box asks for something to be done at the keyboard, and says what must be observed.

Watch Out

A **Watch Out** box marks a mistake that is made every year.

Back to the Machine

A **Back to the Machine** box ties a formal result to an instruction, a register or a screen of the debugger.

Historical Note

A **Historical Note** box tells where an idea comes from.

Notation

Numbers without a mark are decimal. Hexadecimal numbers are written as the assembler wants them, `0x07E0`; binary numbers as `0b1010` in code and with a subscript, 1010_2, in formulas. Bits are numbered from the right starting at zero, so that bit i has weight 2^i. Registers (`a0`, `sp`), instructions (`addi`) and file names are set in a typewriter face. In Boolean expressions the complement of x is $\overline{x}$, the sum is $+$ and the product is written by juxtaposition. Multiples of bytes follow the binary prefixes where precision matters: 1 KiB $= 2^{10}$ bytes, 1 MiB $= 2^{20}$ bytes.

The appendices

Appendix A is a complete reference of the RISC-V instruction set used in the book: registers, formats, every instruction of RV32I and of the M, A, C and Zicsr extensions, pseudo-instructions, assembler directives, the calling convention and the machine-mode control registers. Appendix B is the reference of PRG32: the contract, the programming interface, the cartridge format, the command-line tools, the hosts and the Cartridge Store. Appendix C is a reference of Git and GitHub, the tools with which the platform, this book and the reader's project are kept and shared. Appendix D prints the four reference programs of Verm1 in full; Appendix E the answers to all the questions; Appendix F the complete English–Italian glossary.

Part I

Start

CHAPTER 0

Introduction: Program First, Formalise Later

"The Matrix is a system, Neo. That system is our enemy."
— *The Matrix*, Lana and Lilly Wachowski, 1999

In this chapter

- What the course is, who teaches it and how its ninety-six hours are used.
- The method: a program that runs comes first, its theory comes after.
- The machine: PRG32, RISC-V and the debugger PRG32-QT.
- The project: Verm1, a game written from an empty file and published.
- The rules: syllabus, learning outcomes, assessment, material.

A computer is the most complicated object that most people use every day, and the one whose inner working they are least encouraged to look at. Its makers have spent seventy years hiding the machine behind layers that are each easier to use than the one below: logic gates behind instructions, instructions behind languages, languages behind libraries, libraries behind applications, and now applications behind conversational assistants. The layers are what make computing useful. They are also a system in the sense of the film quoted above: one can live inside it for a lifetime without knowing that there is anything underneath.

Computer architecture is the discipline that looks underneath. It studies how a machine that can only switch currents on and off comes to represent numbers, text, pictures and sound; how a handful of logical operations, repeated thousands of millions of times a second, become arithmetic, decisions and memory; and how the whole is organised so that a sequence of very simple orders, a *program*, can make it do anything that can be computed at all. This chapter explains how the course approaches that study, with which machine and through which project, and then states the practical rules.

0.1 The course at a glance

Computer Architecture and Computer Architecture Laboratory, code ARC12, is a first-year, first-semester course of the Bachelor's Degree in Computer Science of the University of Naples "Parthenope". It is worth 12 ECTS credits, belongs to the scientific area INFO-01/A (Informatics) and consists of 96 hours, divided equally between lectures and laboratory

(Table 1).

Table 1. The course at a glance.

Name	Computer Architecture and Computer Architecture Laboratory (ARC12)
Position	First year, first semester, Bachelor's Degree in Computer Science
Credits	12 ECTS, 96 hours; scientific area INFO-01/A (Informatics)
Lectures	Raffaele Montella, 6 ECTS, 48 hours
Laboratory	Mariacarla Staffa, 6 ECTS, 48 hours
Office hours	Room 420, Tuesday 14:00–16:00, by appointment; also on Microsoft Teams (team code `c7rf33g`)
Extra credits	Accenture SuperSkills! gives 2 ECTS as further knowledge (https://richiediaccesso.uniparthenope.it)
Prerequisites	None

There are no formal prerequisites. No previous experience of programming, of electronics or of mathematics beyond the school curriculum is assumed: the only real prerequisite is a passion for computing and for the technologies that make it possible.

The two halves of the course are two views of the same matter. In the **lectures** the ideas are presented: information, numbers, Boolean algebra, logic networks, memory, the processor, input and output. Every topic opens with a program that can be run and is then formalised with definitions, methods and proofs; every lecture closes with multiple-choice questions in the style of the midterm test; and every four lectures a tutorial adds the next part of Verm1, the game of the course. In the **laboratory** the same ideas are practised: number representation and conversion techniques; analysis, synthesis and minimisation of logic circuits; RISC-V assembly programming on the PRG32 platform; and, finally, the student's own project, a variant of Verm1 or a game of the student's invention. This book is the text of the lectures; its exercises are the natural material of the laboratory.

0.2 A new approach

Program first. Formalise later.

From the first days the student writes RISC-V assembly that runs on a real processor. Each formal topic then answers a question that the student's own program has already raised.

The traditional order of the subject is deductive: representation, then logic, then circuits, then the processor, then, at last, a program. It is the order in which the machine is *built*, and it has the elegance of every construction from first principles. It is not the order in which the machine is *understood*. A student who meets two's complement in the third week of a deductive course has never seen a negative number inside a computer, and has no reason to care how it is written. A student who, the week before, has moved a rectangle to the left of the screen until its coordinate, shown by the debugger, has turned from `0x00000000` into `0xFFFFFFFF` has a question, and two's complement is its answer.

The course is therefore organised as a spiral, repeated for every topic (Figure 1).

First one *plays*: a few instructions are typed, a cartridge is built and run. Then one *observes*:

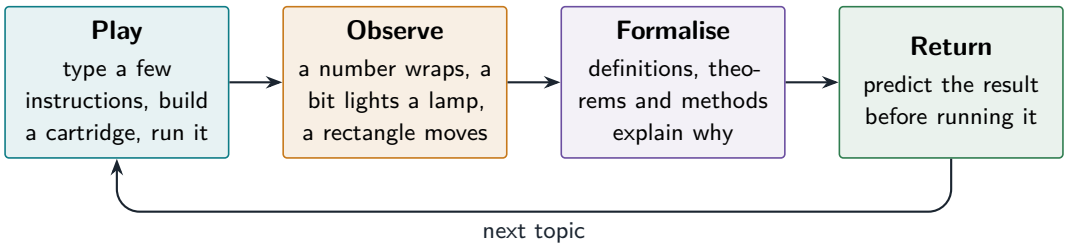


Figure 1. The learning spiral, repeated for every topic of the course.

something happens that was not expected, or was expected and can now be seen. Then one *formalises*: definitions and theorems explain exactly why. Finally one *returns* to the machine, and the measure of understanding is the ability to predict a result before running the program. A mark is an opinion; a paddle that leaves the screen is a fact. The course uses facts, and then explains them.

0.2.1 What the course aims at

The aims are those of every serious course on the subject; only the path is different. The course

- presents the fundamental aspects of the architecture of modern computers;
- uses, in the laboratory, the basic concepts needed to program them efficiently in low-level languages: machine language and assembly;
- treats rigorously the tools from which a processor is built, namely Boolean algebra with the synthesis of combinational and sequential logic networks, and binary representation with binary arithmetic;
- describes in detail the functional and physical structure of the CPU, of the memory and of the input/output subsystem;
- develops assembly code that makes visible how a processor really works and how the basic units of a computer interact.

0.3 The machine: PRG32 and RISC-V

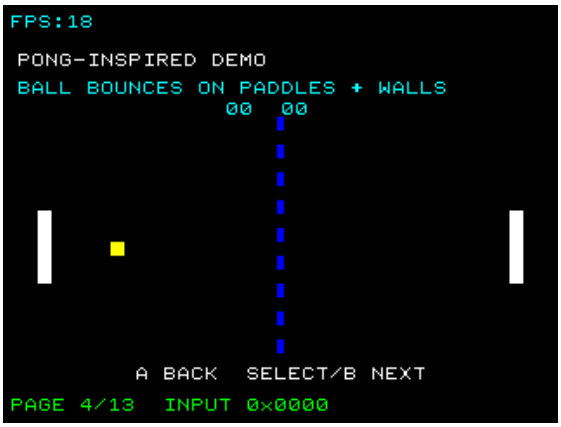
0.3.1 A retro console programmed in assembly

PRG32 is an open-source educational runtime for teaching RISC-V assembly and C, developed at the University of Naples "Parthenope" (Montella et al., 2026a). To the eye it is a small games console of the kind that was sold in the early 1980s (Figure 2); to the programmer it is a deliberately thin contract.

- The student's code runs *natively*, as 32-bit RISC-V machine code of the RV32IMAC instruction set. It is not interpreted by a teaching simulator: the instructions written are the instructions executed.
- The contract consists of three functions that the student writes: `init` runs once; `update` and `draw` run once per frame.
- Games travel as small *cartridges*, files with the extension `.prg32`.
- The machine offers a viewport of 320 by 200 pixels, a joystick with a few buttons, and sound: small enough to be held in one's head, rich enough to exercise the whole



(a) The menu of the resident firmware.



(b) Pong: native RISC-V machine code.

Figure 2. PRG32, the machine of the course.

course.

0.3.2 The philosophy behind PRG32

Six principles, taken from the textbook of the platform (Montella et al., 2026b), shape both PRG32 and the way this book uses it.

Small enough to understand.
One processor, a screen of 320 by 200 pixels, a joystick, a small speaker. Nothing is hidden: the whole machine can be held in one's head.

Real, not simulated.
The instructions written are those that the chip executes. What is learnt here is true on every RISC-V processor, from a sensor to a supercomputer.

Limits make you creative.
A few kilobytes and three functions. The games of the 1980s were born from limits like these: a constraint is an invitation.

One thing at a time.
Start with functions that only return. Add one thing, run, look. `update` changes the state, `draw` shows it. Sound comes last.

Look inside.
When something is surprising, stop the program and read registers and memory. The debugger is part of the method, not an emergency tool.

Make, then share.
A cartridge is a file that can be given to a friend. The Cartridge Store is the shelf where finished work meets its players.

The platform has, accordingly, two uses in the course. It is a *gymnasium*: every chapter opens with a short program that runs and closes by going back to the machine; an exercise consists of a prediction and of a check; and the machine never flatters, because a wrong mask lights the wrong lamp. And it is a *workshop*: a blank cartridge and a screen, on which rules, colours, sounds and levels are design decisions that belong to the author. The project of the course starts guided and ends free, and what is finished can be published. First-year students rarely expect a course on architecture to ask them for an idea; this one does.

0.3.3 PRG32-QT: the debugger of the course

PRG32-QT is PRG32 on a laptop: an application that runs the same cartridges with no board and no compiler (Montella, 2026). In debug mode it opens the machine (Figure 3): the program can be paused, resumed and advanced one instruction at a time; breakpoints can be placed on any instruction of `init`, `update` or `draw`; the thirty-two registers and 128 bytes of memory at any address are displayed; the speed can be set between 0.01 and 4 times the real one; and counters report frames, instructions executed and frames that arrived late.

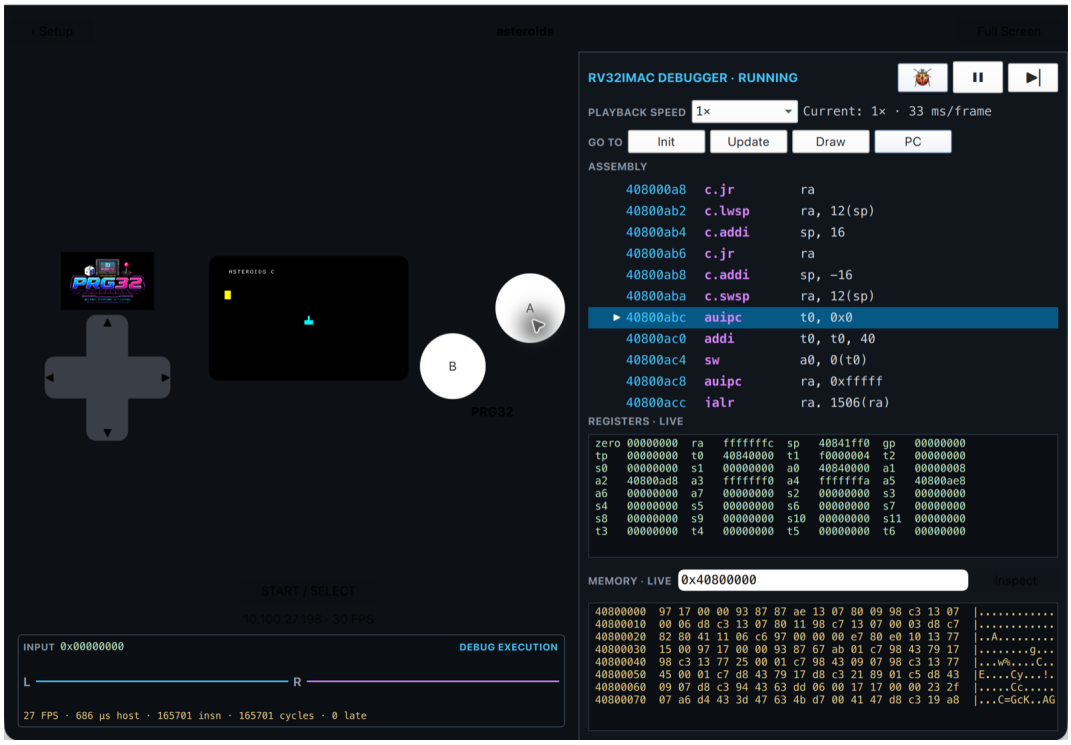


Figure 3. PRG32-QT in debug mode: the game on the left, the machine on the right.

The debugger is used in every chapter: to check a prediction, to find a mistake, to measure. It also supplies the working definition of understanding adopted in this book.

When is a program understood?

A program is understood when one can say what each register will hold *before* pressing **Step**.

0.3.4 Why RISC-V?

The choice of the processor is not neutral, and four reasons led to RISC-V.

It is open.
RISC-V is an open standard instruction set: anyone may design, build and sell a RISC-V processor without paying a licence for the architecture (RISC-V International, 2024c).

It is small.
The base integer instruction set has about forty instructions and thirty-two registers.

All of it can be learnt in one semester, and Appendix A lists all of it.

It is real.
`addi`, `lw`, `sw` and `beqz` are not a teaching dialect. Industry ships RISC-V cores by the billion, and they execute exactly these instructions.

It is European.
Europe treats RISC-V as a lever for technological sovereignty: the European Chips Act and projects such as DARE (Digital Autonomy with RISC-V in Europe, a EuroHPC Joint Undertaking project) build processors on it (EuroHPC Joint Undertaking, 2025).

0.3.5 The map: every topic is met twice

Table 2 is the course on one page. Each formal topic of the left column is met first on the machine, in the form given by the right column, and only then in its general form. It is worth coming back to this table at the beginning of each part of the book.

Table 2. Every formal topic is met first on PRG32.

Formal topic	First met on PRG32 as...
Positional notation, hexadecimal	the colour `0x07E0` and the value shown by the debugger
Two's complement, overflow	a position that moves left past zero and shows `0xFFFFFFFF`
Fixed and floating point	a ball that moves 0.375 pixels per frame on a chip with no floating-point unit
Boolean algebra, bit masks	`andi t3, t0, 1` to test whether **LEFT** is pressed
Electronics, logic gates	the button that joins a pin of the chip to ground
Combinational networks, adders	what `add` actually does to thirty-two bits
Sequential networks, registers	the counter that survives from one frame to the next
CPU, datapath, instruction cycle	single-stepping one instruction in PRG32-QT
Memory and input/output	`lw` and `sw`, the stack, the joystick and the display

0.4 The project: Verm1

Verm1, read *vermone*, "the big worm", is a tribute to Snake: the game of the arcade machine *Blockade* (1976) and of the Nokia telephones (1997). A worm moves on a grid, eats, grows, and must not bite itself. The student writes every line of it in RISC-V assembly, with no engine and no library beyond the calls of PRG32. The game is small enough for a first-year student and rich enough to need every topic of the course; at the end it is ready for the Cartridge Store with its author's name on it. The reference implementation, against which every reader can compare a program of their own, was contributed by Giuseppe Coviello (Appendix D).

The project is the thread of the book. It is built in four parts and a publication (Table 3, Figure 4), and each part uses exactly what the four chapters before it have taught. Each part ends with a game that runs: the reader never goes home with a broken program.

A tutorial chapter obeys four rules. It is *self-contained*: it shows the whole program, and every line is either new, changed, or reprinted unchanged. It proceeds *step by step*: a few lines at a time, and after each step the program is built, run and compared with a stated result. It uses *the debugger*: each tutorial has a session in PRG32-QT, with a breakpoint, a register to watch and a region of memory to read. And its result belongs to the reader: each

Table 3. Verm1: four parts and a publication.

Chapter	After the part on	Verm1 gains	With
7 Part 1	information and numbers	a grid, a head that moves, a world without edges	colours, shifts, fixed point
12 Part 2	logic and circuits	steering, food, a score	masks, XOR, tables
17 Part 3	state and memory	a body that grows, collisions, three screens	a state machine, arrays
22 Part 4	the machine	digits, levels, sound, fast drawing	calls, division, the frame buffer
23 The Store	the whole course	a name, an icon, credits, a bundle	the PRG32 tools



(a) Part 1: a head **(b)** Part 2: food, score **(c)** Part 3: a body **(d)** Part 4: finished

Figure 4. Verm1 grows with the reader: the same file, four sittings. The screenshots are taken from the reference programs by the automatic tests of the course material.

tutorial ends with changes to be made independently, because the game that is published should not be identical to the one printed here.

0.5 The course in blocks

The order of the topics was chosen for learning. First comes what a bit means; then how bits are combined; then how they are remembered; then the machine that does all three. The processor is presented before the devices that it drives. After each block the project uses it. The last lecture is an outlook, held before a visit to the computing systems of the Department (Table 4).

0.6 Syllabus and learning outcomes

0.6.1 Syllabus

Computer organisation.
Hardware and software; application and system software; the von Neumann model; operating principles of a computer.

Boolean algebra and logic.
Boolean functions, truth tables; combinational networks and their minimisation; adders; finite-state machines; sequential networks; flip-flops, registers.

Data representation.
Encoding; redundant codes, error detection and correction; decoders, multiplexers; positional systems; naturals, integers, reals; arithmetic.

Structure of a computer.
The CPU: control unit, ALU, registers; the instruction cycle; the datapath; main memory; the I/O subsystem; microprogrammed architectures.

Table 4. The blocks of the course. Numbers are those of the lectures and of the chapters of this book.

Block	Lectures	Then
Start	0 Introduction; 1 PRG32 and PRG32-QT: setting up; 2 First contact with RISC-V	
Information	3 History; 4 Digital information; 5 Integers; 6 Reals	7 Verm1, part 1
Logic	8 and 9 Boolean algebra; 10 Electronics; 11 Combinational machines	12 Verm1, part 2
State and memory	13 Sequential machines; 14 Registers and buses; 15 Memory; 16 Permanent memory	17 Verm1, part 3
The machine	18 Processor; 19 Instructions and interrupts; 20 Video card; 21 Input and output	22 Verm1, part 4
Publication		23 Verm1 on the Cartridge Store
Outlook	24 Beyond one core: superscalar RISC-V, HPC, GPUs and FPGAs	the visit to the computing systems

RISC-V assembly.
Instruction set; registers, addressing; arithmetic and logic; branches; subroutines, calling convention, stack; memory and I/O.

Project.
Verm1: a game in RISC-V assembly on PRG32, built in four guided parts, then varied, documented and packaged by the student or by a small team.

The detailed syllabus is published on the university's information system and on the e-learning portal, https://elearning.uniparthenope.it.

0.6.2 Learning outcomes

The outcomes are stated, as usual in European higher education, along the five Dublin descriptors.

Knowledge and understanding.
A sound knowledge of the binary system, of digital logic and of the operating principles of computers. No previous computing background is assumed.

Applying knowledge.
The ability to apply theory to exercises and to RISC-V assembly programs, and to recognise both the capabilities and the limits of computers on real problems.

Making judgements.
The ability to write assembly programs that represent and process real problems in binary, choosing suitable data representations and control structures.

Communication skills.
The ability to take an active part in teams that design, implement and document assembly applications, using correct technical terminology.

Learning skills.
The ability to connect representation, logic networks and processors into one picture, and to keep up to date independently from reliable technical sources.

0.7 Organisation and assessment

This section records the rules of the academic year 2026/27. Dates that had not been fixed when the book was compiled are announced in class and on the e-learning portal, which prevail over this text in case of difference.

0.7.1 Timetable

From September to December 2026 the course meets four times a week in Aula 1: lectures on Monday (14:00–16:00) and Wednesday (09:00–11:00), laboratory on Tuesday and Friday (09:00–11:00). An internal calendar with the actual distribution of lectures and laboratory sessions is shared with the students.

0.7.2 One midterm test

There is one midterm test, taken in person and on paper in about 45 minutes: **ten multiple-choice questions plus two exercises**. It is held on a Monday, 14:00–16:00, in Aula 1, on a date to be announced; students may be split into groups and the test run in shifts. Candidates bring an identity document and a pen, and must know their institutional e-mail address and student number.

The test assesses both sides, students and teachers: it is needed to know where the course is going. It closes the first three blocks (Table 5); the fourth, the machine, is examined in the oral. RISC-V instructions appear in the examples of the test: the first-contact material of Chapter 2 is assumed, and the code of Verm1, parts 1 to 3, is a natural source of questions. Every chapter of this book ends with questions of the same kind, and the best preparation is to answer them week by week.

Table 5. What the midterm test covers.

Block	Lectures	Topics
Information	3 to 6	history of computing; digital information; number representation: integers and reals
Logic	8 to 11	Boolean algebra; electronics and logic gates; combinational machines
State and memory	13 to 16	sequential machines: latches, flip-flops, state machines; registers and buses; memory and permanent storage
Not in the midterm	18 to 24	the processor, instructions and interrupts, the video card, input and output, the outlook: part of the oral exam

No tech. No GPT.

The midterm test and the exam are *#nogpt*. No technological or human help of any kind is allowed: no devices, no ChatGPT or other large language models, no generative AI. Cheating cancels the test at once; more serious behaviour has more serious consequences.

0.7.3 Assessment

The exam has two parts, and both must be passed.

- **Theory (50%)**: an oral examination with impromptu questions on the whole syllabus; some answers need a diagram or a calculation on the board. The oral can also lower the mark obtained in the midterm test.
- **Practical (50%)**: the laboratory exam with Prof. Staffa.

The final mark is the mean of the theory mark and of the practical mark. Midterm and exam results are kept within the current academic year.

To finalise the exam a positive mark is needed in both theory and practice, and finalisation happens only during a theory exam session. The laboratory exam is usually held one week before the theory session, and both sessions are booked on https://uniparthenope.esse3.cineca.it. Two questions are asked every year. *Can the theory exam be taken before the practical one has been passed?* Yes: the result is kept for one academic year, but the exam is finalised only after the practical part has been passed. *Can the theory session be booked without the practical test?* Yes, the case is the same.

0.7.4 Midterm week and the closing event

During the midterm week the lectures are replaced by seminars on emerging computer architectures. At the end of the course, a student or a team that presents its game at the closing ceremony, a variant of Verm1 or a game of its own, can earn bonus points for the exam. The closing morning has two halves: the last lecture, beyond one core, from superscalar RISC-V to supercomputers (Chapter 24), and a visit to the supercomputing systems of the Department of Science and Technology. Finished games can be published on the PRG32 Cartridge Store, as Chapter 23 shows.

0.8 Material

Slides.
Each lecture has two slide decks: the lecture deck, of at most twenty slides, for the concepts, and the code deck, with every listing, the complete program of the lecture and the guide for testing it on PRG32-QT, on QEMU and on the board. They are always available in a shared Google Drive folder and are periodically published on https://elearning.uniparthenope.it. They are generated from source: corrections and new exercises are welcome as contributions.

Programs.
The program of each lecture is in the folder `code` of the repository of the course, https://github.com/riscv-prg32/computer-architecture-with-prg32, with the decks, this book and the tests that run every listing.

Platform.
PRG32, free and open source, at https://github.com/riscv-prg32/PRG32, with its own textbook, *Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform* (Montella et al., 2026b).

Textbooks.
Patterson and Hennessy (2020), *Computer Organization and Design, RISC-V Edition*, and Tanenbaum and Austin (2013), *Structured Computer Organization*, are the two references of the course. The syllabus also lists Stallings (2021) and Bucci (2017).

Standards.
The official RISC-V documentation at https://riscv.org (RISC-V International, 2024c).

The teaching material lives in a folder shared with the students, who can contribute to it; to obtain access a student sends the address of an account linked to the service to the lecturer. All other communication with the lecturer goes through the institutional address of the student.

Cineforum

Thirty films and series about computing accompany the course, as a reminder that the subject has a culture as well as a technique: *The Social Network* (2010), *Steve Jobs* (2015), *Pirates of Silicon Valley* (1999), *The Imitation Game* (2014), *Hackers* (1995), *Ex Machina* (2014), *Her* (2013), *Tron* (1982), *Tron: Legacy* (2010), *The Matrix* (1999), *WarGames* (1983), *Sneakers* (1992), *Jobs* (2013), *Antitrust* (2001), *Blackhat* (2015), *The Fifth Estate* (2013), *The Circle* (2017), *Silicon Valley* (2014–2019), *Source Code* (2011), *Chappie* (2015), *Mr. Robot* (2015–2019), *Ghost in the Shell* (1995), *Minority Report* (2002), *Transcendence* (2014), *Ready Player One* (2018), *A.I. Artificial Intelligence* (2001), *Revolt* (2017), *The Thirteenth Floor* (1999), *Eagle Eye* (2008) and *Terminator 2: Judgment Day* (1991). *Ready Player One* is also the cultural reference of the PRG32 textbook: fluency with the foundational layers is a literacy that keeps its value.

0.9 First contact: who are you?

The first lecture ends with questions put to the room, and a reader working alone may put them to themselves. Their purpose is not to select but to find where to begin.

- **Hardware.** Have you ever assembled a computer? Have you played with an Arduino or an ESP32?
- **Software.** How many programming languages do you know? Have you ever read a line of assembly?
- **Culture.** What are retrocomputing and retrogaming? Have you ever visited a computer museum?
- **Concepts.** Can you tell the difference between ROM, RAM and storage? What is cloud computing? And the Internet of Things?
- **RISC-V.** Have you heard of RISC-V? Which processor is inside your telephone, your laptop, your router?

In class the questions are followed by a short anonymous questionnaire, the *Sorting Hat*, on what attracts each student most in the future of computing and of science; its results are discussed and used to tune examples and projects.

0.10 Before the next chapter

Chapter 1 installs everything, step by step, on Windows, macOS and Linux. It goes faster if the long downloads are already done.

Exercise 0.1 (preparation)

1. Have a laptop on which you are allowed to install programs.

2. Download the PRG32-QT installer for your system from the *Releases* page of https://github.com/riscv-prg32/PRG32-QT.
3. On a good connection, start the long download: ESP-IDF, following the `README` of https://github.com/riscv-prg32/PRG32 for your system.
4. If something fails, do not fight it alone: keep the error message.

Expected result. You begin Chapter 1 with the downloads done. In two hours you have PRG32-QT running and a cartridge built on your own laptop.

0.11 The program of this chapter

The listings of this chapter are parts of one program, `code/l00.S` in the repository of the course, printed in full in Listing 1. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 5 is the frame that the test wrote.

On the screen.
The word ARC12 in white on a blue screen, and under it HELLO, RISC-V.

What to press.
Nothing: the program runs by itself.



Figure 5. `l00.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 00: Course Introduction
# l00.S: the first program of the course, to read before you can write it
#
# On the screen: the word ARC12 in white on a blue screen, and under it HELLO,
    RISC-V.
# Build: python3 -m prg32 cartridge build l00.S --portable \
#            --entry-prefix l00 --name l00 --out l00.prg32
.option norelax

.equ BLUE,  0x001f           # a colour is a 16-bit number (Lecture 04)
.equ WHITE, 0xffff

.section .text
.global l00_init
.global l00_update
.global l00_draw

l00_init:                    # called once, when the cartridge starts
```

```
    ret

l00_update:                 # called once per frame: nothing changes here
    ret

l00_draw:                   # called once per frame: paint the screen
    addi sp, sp, -16        # a function that calls others saves ra
    sw   ra, 12(sp)
    li   a0, BLUE
    call prg32_gfx_clear    # fill the screen with one colour
    li   a0, 140            # x, in pixels
    li   a1, 88             # y
    la   a2, line1          # the ADDRESS of the text
    li   a3, WHITE          # foreground
    li   a4, BLUE           # background
    call prg32_gfx_text8
    li   a0, 104
    li   a1, 104
    la   a2, line2
    li   a3, WHITE
    li   a4, BLUE
    call prg32_gfx_text8
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata            # constants: two strings, each ending with a zero
    byte
line1: .asciz "ARC12"
line2: .asciz "HELLO, RISC-V"
```

Listing 1. The complete program of the chapter, `code/l00.S`.

To run it on PRG32-QT the commands are those of Listing 2; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l00.S --portable \
    --entry-prefix l00 --name l00 --out l00.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l00.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 2. Building `l00.S` and running it on PRG32-QT.

CHAPTER 1

PRG32 and PRG32-QT: Setting Up Your Workshop

"Give me six hours to chop down a tree and I will spend the first four sharpening the axe."
— attributed to Abraham Lincoln

In this chapter

By the end of this chapter you will have

- a clear picture of the pieces of PRG32: runtime, cartridge, tools, hosts, Store;
- PRG32-QT installed on your laptop, to play any cartridge and look inside it;
- the RISC-V toolchain installed and checked, to build cartridges of your own;
- one cartridge built by you, running in PRG32-QT and stopped in the debugger;
- a list of what to do when something does not work.

A craftsman's first job in a new workshop is to put the tools where the hand expects them. This chapter is that job. It contains no theory, and it is the only chapter of the book whose content will age: version numbers are those of October 2026, and the release pages of the projects should be checked before following the steps. Everything else in the book depends on the last line of the summary above: the loop of building a cartridge, sending it to a host and stopping it in the debugger is repeated in every chapter that follows.

The commands of Section 1.5 were run for this book on macOS against PRG32-QT. The installations for Windows and Linux follow the official instructions of the projects and were not run for this text.

1.1 Four levels: install what you need

Not everything has to be installed, and not everything at once. Table 1.1 orders the pieces by need. The course, and the book, can be followed with levels 1 and 2 only: *no board is required.*

Level 1 takes five minutes. Level 2 is a large download and takes from half an hour to an hour: it is best done at home, on a good connection. Levels 3 and 4 are for the curious, and for the laboratory.

Table 1.1. Four levels of installation.

Level	You install	You can	Needed
1	PRG32-QT	play cartridges, download from the Store, use the debugger	by everybody, at once
2	ESP-IDF and the PRG32 repository	build your own cartridges from assembly or C	by everybody, before Chapter 2
3	the QEMU emulator	run the real firmware on your laptop	optional
4	the kit: an ESP32-C6 board	run on real hardware	optional

1.2 The platform, in detail

1.2.1 A runtime, a file format, a set of tools

The name PRG32 covers four things.

- A **resident firmware**, also called the *runtime*: the program that owns the screen, the buttons and the sound.
- A **cartridge format**: a game as one small file, `.prg32`.
- A **tool** on the laptop: the command `python3 -m prg32`, which builds, uploads and packages.
- A **Cartridge Store**, where finished games are shared.

The code of a cartridge runs as real 32-bit RISC-V machine code, RV32IMAC, and the contract between a game and the runtime is three functions: `init`, `update` and `draw`. The runtime calls `init` once and then `update` and `draw` at every frame; Chapter 2 starts from this contract.

1.2.2 From a source file to the screen

Figure 1.1 follows a program from the editor to the screen. The source, in assembly or in C, is translated into machine code by the RISC-V toolchain; the compiler is `riscv32-esp-elf-gcc`, installed by ESP-IDF, the development kit of Espressif. The result is packed into a cartridge, made of code, data and a header. The cartridge is sent over the network to a *host*, where the runtime loads it and begins to call its three functions.

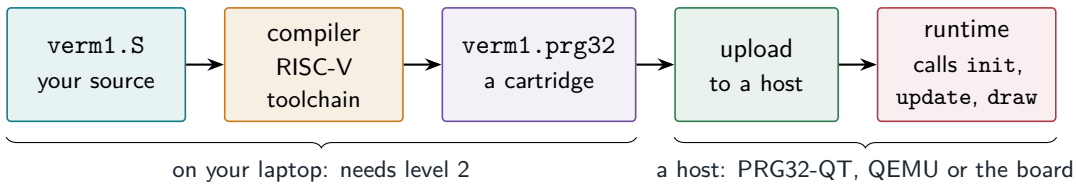


Figure 1.1. From a source file to the screen.

A cartridge is loaded at address `0x40800000` and reaches the runtime only through a table of functions, the *application binary interface* (ABI). This is the reason why the same file runs on three different hosts: it never touches the hardware directly, and each host supplies the same table.

1.2.3 The kit: a console on a breadboard

The hardware of PRG32 (Figure 1.2) is built around an Espressif ESP32-C6: one RISC-V core at 160 MHz, with Wi-Fi. A 2.8 inch colour display of 320 by 240 pixels (an ILI9341 controller on an SPI bus) shows the games, which draw in a viewport of 320 by 200. One or two small audio amplifiers (MAX98357A, on an I2S bus), a joystick and buttons are wired straight to the pins of the chip. A game reaches the board over Wi-Fi: the board creates a network called `PRG32`, and the cartridge is uploaded to it. The bill of materials and the wiring are in the folder `docs/hardware` of the repository.

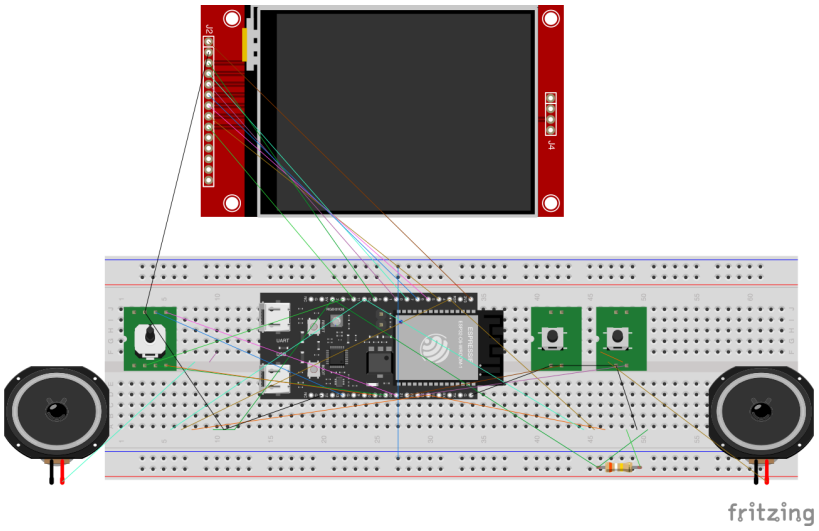


Figure 1.2. The PRG32 kit on a breadboard.

1.2.4 Three ways to run the same cartridge

The cartridge file is the same on every host; what changes is how its instructions are executed and what can be observed (Table 1.2).

Table 1.2. Three hosts for the same cartridge.

	ESP32-C6 board	**Espressif QEMU**	**PRG32-QT**
How code runs	natively on the chip	natively on an emulated ESP32-C3	interpreted RV32IMAC guest
Needs a toolchain	yes (ESP-IDF)	yes (ESP-IDF)	no, to play and debug
Cartridge arrives by	Wi-Fi upload, Store	staging into the flash image	file import, Store, HTTP upload
Instruction stepping	with JTAG and GDB	with GDB	built-in debugger
Runs on	the hardware kit	Windows, Linux, macOS	desktop, Raspberry Pi, iOS, Android, TV

The rule of thumb is: use the board when you want the truth about *time*, and PRG32-QT when you want the truth about a *register*. The emulator models an ESP32-C3 rather than a C6 because Espressif maintains a mature emulator with graphics for that chip; it executes the same RISC-V instructions, and the cartridge does not notice.

1.2.5 PRG32-QT: the console on the laptop

PRG32-QT is an application for Windows, macOS, Linux, Raspberry Pi, telephones, tablets and televisions. It contains an interpreter of RV32IMAC, which executes a cartridge instruction by instruction and offers it the same functions as the firmware. It has three speeds, chosen in *Settings*: *Accurate*, timed like the ESP32-C6, which is the default; *Optimal*, at 30 frames per second; and *Unlimited*, as fast as the laptop goes. The accurate profile charges each instruction and each call a cost in cycles of a 160 MHz clock: it is not a cycle-exact model of the chip, but it is close enough to see when a frame is late. PRG32-QT also contains a browser of the Store and the debugger described in Chapter 0.

The controls of the console map onto the keyboard as in Table 1.3. A USB or Bluetooth game controller is recognised by PRG32-QT when it is plugged in.

Table 1.3. The controls, on a keyboard.

On the kit	In PRG32-QT	In QEMU
Joystick	arrow keys, or **W A S D**	the same, typed in the terminal
Button A	**Z** or **J**	**Z** or **J**
Button B	**X** or **K**	**X** or **K**
Start / Select	**Enter** or **Space**	**Enter** or **Space**
Leave the game	**Escape**: back to Setup	close QEMU

Watch Out

QEMU reads the keyboard from the terminal that launched it, not from the window with the picture. It is the classic surprise of the first session.

1.3 Step 1: install PRG32-QT

Five minutes; no compiler, no account. From the page https://github.com/riscv-prg32/PRG32-QT, under *Releases*, take the latest version of the file for your system (Table 1.4). A file `SHA256SUMS-platforms.txt` on the same page lists the checksums; builds for Android and for televisions are there too.

Table 1.4. Installers of PRG32-QT (version 0.4.1, October 2026; the names follow this pattern with the current number).

System	File	Kind
Windows 10 or 11	`PRG32-QT-0.4.1-Windows-X64-Setup.exe`	installer
macOS, Apple Silicon	`PRG32-QT-0.4.1-macOS-ARM64.dmg`	disk image
macOS, Intel	`PRG32-QT-0.4.1-macOS-Intel-X64.dmg`	disk image
Linux, 64-bit PC	`PRG32-QT-0.4.1-Linux-X64.deb`	Debian package
Raspberry Pi OS, 64-bit	`PRG32-QT-0.4.1-Raspberry-Pi-OS-ARM64.deb`	Debian package

Windows. Download the file ending in `Windows-X64-Setup.exe` and run it. If Windows shows a blue SmartScreen warning, choose *More info* and then *Run anyway*. Accept the proposed folder: the installer brings the Qt libraries and the compiler runtime, and nothing

else has to be installed. Start *PRG32* from the Start menu; if the firewall asks, allow it on private networks. The question appears because PRG32-QT listens on port 8080 for cartridges sent by the tools.

macOS. From the Apple menu choose *About This Mac*: "Chip Apple M..." means ARM64, "Processor Intel" means X64. Open the `.dmg` and drag *PRG32* onto the Applications folder. At the first start, if macOS says that the developer cannot be verified (the application is distributed outside the App Store), right-click *PRG32* and choose *Open*, or go to *System Settings, Privacy and Security, Open Anyway*. Accept incoming connections and local-network access when asked: they serve the device interface on port 8080 and its announcement on the local network.

Linux. The package needs Qt 6.5 or newer, which Debian 13 and Ubuntu 25.04 provide; on older systems (Ubuntu 24.04 has Qt 6.4) the program is built from source (Listing 1.1). On Raspberry Pi OS use the ARM64 package, or the script `scripts/build-raspbian.sh`.

```
# Debian 13, Ubuntu 25.04 and later, or another system with Qt 6.5 or newer
cd ~/Downloads
sudo apt install ./PRG32-QT-0.4.1-Linux-X64.deb
PRG32 &

# On older systems (Ubuntu 24.04 has Qt 6.4: too old): build from source
sudo apt install git cmake ninja-build g++ qt6-base-dev qt6-declarative-dev \
    qt6-multimedia-dev qt6-websockets-dev
git clone https://github.com/riscv-prg32/PRG32-QT
cd PRG32-QT
./scripts/build-linux.sh
```

Listing 1.1. PRG32-QT on Linux.

Run it: your first cartridge, from the Store

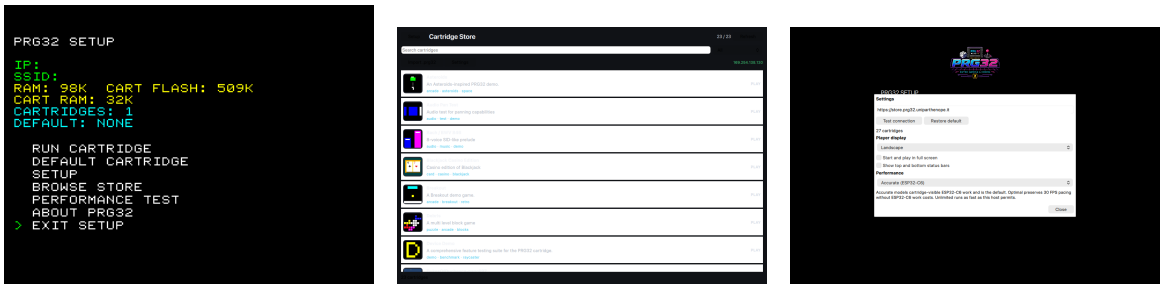
1. Start PRG32-QT: the *Setup* screen appears, the same that the firmware shows on the kit.
2. Choose *Browse Store*: the catalogue of the Cartridge Store is downloaded.
3. Pick a game, download it, and choose *Run Cartridge*.
4. Play it with the arrow keys, **Z** and **X**. **Escape** brings you back to Setup.
5. Open *Settings*: look at the three performance profiles and at the layout choices.

Expected result. A game from the Store runs on your laptop (Figure 1.3). This is level 1: from now on you can run any cartridge, and look inside it. If the list of the Store stays empty, the laptop is not reaching the Internet or a proxy is in the way: *Settings* shows the address of the Store.

1.4 Step 2: install the toolchain

1.4.1 One idea behind every installation

The compiler for the ESP32 family comes with ESP-IDF, the development kit of Espressif (Espressif Systems, 2026). PRG32 needs ESP-IDF 5.4 or newer, installed for two chips:



(a) The Setup menu **(b)** Browse Store **(c)** Settings

Figure 1.3. What you should see: the same three screens on every system.

`esp32c6`, the kit, and `esp32c3`, the emulator. After that, one command does everything: `python3 -m prg32`.

ESP-IDF is not active by itself

Every new terminal must switch ESP-IDF on before the tools work. Almost every installation problem is ESP-IDF not being active *in the terminal you are typing in*. The symptom is always the same message:

`idf.py: command not found`

(or `riscv32-esp-elf-gcc` not found). On Windows, activating means opening the *ESP-IDF PowerShell*; on macOS and Linux, running `export.sh`.

1.4.2 Windows

Install Git for Windows and Python 3, then run the ESP-IDF Tools Installer and work in the PowerShell window that it creates (Listing 1.2). Use the ESP-IDF PowerShell, never a plain terminal; on Windows the command is `python`, not `python3`; `pip` installs into the environment of ESP-IDF, which is intended. Avoid folders with spaces or accented letters in their path. Those who prefer an editor may use Visual Studio Code with the Espressif ESP-IDF extension, which installs the same toolchain.

```
# 1. Install Git for Windows (git-scm.com) and Python 3 (python.org)
# 2. Run the ESP-IDF Tools Installer (dl.espressif.com/dl/esp-idf):
#      - choose an ESP-IDF release 5.4 or newer
#      - select the targets esp32c3 and esp32c6
#      - optional: tick the QEMU RISC-V emulator
# 3. Open the shortcut "ESP-IDF PowerShell" that the installer created
# 4. In that window:

cd $HOME\Documents
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
pip install pyaudio zeroconf
python -m prg32 doctor
```

Listing 1.2. The toolchain on Windows.

1.4.3 macOS

Homebrew comes first, because it provides `git`, `cmake` and `python` (Listing 1.3); `git` may first ask to install the Xcode command line tools. The recursive clone and `install.sh` fetch a few gigabytes: be patient. The dot before `export.sh` matters, because it makes the script change *this* terminal. The procedure works on Apple Silicon and on Intel.

```
# 1. Homebrew (brew.sh), then the prerequisites
brew install git cmake ninja dfu-util ccache libusb python curl zip portaudio

# 2. ESP-IDF 5.4 for the two chips (a large download)
cd $HOME
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
pip install pyaudio zeroconf

# 3. PRG32
cd $HOME
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
python3 -m prg32 doctor
```

Listing 1.3. The toolchain on macOS.

1.4.4 Linux (Debian, Ubuntu)

The prerequisites are one long `apt` line, to be copied whole (Listing 1.4); steps 2 and 3 are the same as on macOS. Other distributions have the same packages under other names. For the kit only, the user must belong to the group `dialout`, which gives access to the serial port of the board: `sudo usermod -aG dialout $USER`, then log out and in again.

```
# 1. Prerequisites
sudo apt update
sudo apt install -y git wget flex bison gperf python3 python3-venv python3-pip
    \
    python3-zeroconf cmake ninja-build ccache libffi-dev libssl-dev dfu-util \
    libusb-1.0-0 curl zip portaudio19-dev

# 2. ESP-IDF 5.4 for the two chips (a large download)
cd $HOME
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
pip install pyaudio zeroconf

# 3. PRG32
cd $HOME
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
python3 -m prg32 doctor
```

Listing 1.4. The toolchain on Linux.

1.4.5 Every new terminal: switch ESP-IDF on

`export.sh` acts on the terminal in which it runs, and only on that one. A new window is a new terminal, and must be activated again. Two aliases save typing (Listing 1.5); they are a convenience, not a requirement. The `prg32` commands are run from the PRG32 folder.

```
# macOS and Linux: in every new terminal, before anything else
. $HOME/esp-idf/export.sh
cd $HOME/PRG32

# save typing: add these two lines to ~/.zshrc or ~/.bashrc
alias get_idf=". $HOME/esp-idf/export.sh"
alias prg="python3 -m prg32"

# then, in a new terminal
get_idf
prg doctor

# Windows: open "ESP-IDF PowerShell" from the Start menu, then
cd $HOME\Documents\PRG32
```

Listing 1.5. Activating ESP-IDF in a new terminal.

See the error once, on purpose

Close the terminal, open a new one and run `python3 -m prg32 doctor` *without* activating ESP-IDF. Read the message. It is better to meet it here than in the middle of a tutorial.

1.4.6 The doctor

The command `doctor` checks the installation (Listing 1.6). The line about `idf.py` says that ESP-IDF is active; the one about `riscv32-esp-elf-gcc` that the RISC-V compiler is reachable; those about partitions that the PRG32 folder is complete. The four cartridge slots, `cart0` to `cart3`, are regions of the flash memory of the board, of 131 072 bytes each. Any line marked `ERROR` says what to do. If the doctor passes, level 2 is done.

```
$ python3 -m prg32 doctor
[OK] idf.py found
[OK] riscv32-esp-elf-gcc found
[OK] partitions file found: .../PRG32/profiles/partitions_prg32.csv
[OK] partition cart0 parsed (offset=0x210000, size=131072)
...

# and when ESP-IDF is not active in this terminal:
[ERROR] idf.py missing (you must source ESP-IDF export.sh)
```

Listing 1.6. The doctor (messages of October 2026).

1.5 Step 3: your first build

1.5.1 Build a cartridge from the examples

The repository contains complete games. Building one of them proves that the compiler works before a single line has been typed (Listing 1.7).

```
# in the PRG32 folder, ESP-IDF active
python3 -m prg32 cartridge build examples/games/pong/c/game.c \
    --portable --entry-prefix pong_c --name pong --out pong.prg32

[OK] built pong.prg32 name=pong load=0x40800000
[INFO] code=3004 mem=3028 audio=0

# what did we build?
python3 -m prg32 cartridge summary pong.prg32
```

Listing 1.7. Building Pong, in C, from the examples.

The command `cartridge build` takes a source and produces a `.prg32` file. The option `--entry-prefix` gives the name that precedes `_init`, `_update` and `_draw` in the source; `--portable` asks for a cartridge that runs on every host, and not on one firmware only. A whole Pong is three thousand bytes. In Chapter 2 the source will be the reader's own, in assembly, and the command will be the same. On Windows write `python` and put the command on one line.

1.5.2 Send it to PRG32-QT

PRG32-QT answers on port 8080 exactly as a board does (Listing 1.8). The command `runtime` prints the state of the host and is a quick test of the connection; `upload-and-run` stores the cartridge in a slot and starts it. The command says `esp32c6` because, for the tool, PRG32-QT *is* a board. If `localhost` does not answer, read the address shown on the Setup screen of PRG32-QT and use it. Without the tools, the same result is obtained in PRG32-QT by importing the file `pong.prg32` and choosing *Run Cartridge*; with `--url http://192.168.4.1` the same command sends the game to a real board.

```
# PRG32-QT must be running on this computer

# 1. is it listening?
python3 -m prg32 runtime --url http://localhost:8080

# 2. upload the cartridge to slot cart0 and start it
python3 -m prg32 esp32c6 upload-and-run pong.prg32 \
    --url http://localhost:8080 --slot cart0

{"loaded":false,"ok":true,"slot":"cart0","stored":true}
{"ok":true,"slot":"cart0"}
[OK] Finished upload and run on ESP32C6
```

Listing 1.8. Sending the cartridge to PRG32-QT, running on the same computer.

1.5.3 Look inside: the debugger

Open the machine

1. In PRG32-QT open *Settings* and enable *Run cartridges in RISC-V debug mode*.
2. Run the cartridge again: the game is on the left, the machine on the right (Figure 1.4).
3. Press **Pause** (the button with two bars): the game freezes, and one instruction is highlighted in the listing.
4. Press **Step** (the triangle with a bar) a few times: the highlight moves, and some registers change value.
5. Press **Update**, then **Draw**: the listing jumps to those two functions of the cartridge.
6. Press **Resume** (the triangle): the game goes on. Lower the speed to 0.1x and watch it in slow motion.

The three transport buttons of the debugger are drawn as on a video player; this book calls them **Pause**, **Resume** and **Step**. This is only a first look: the next chapter explains what the registers are. The point here is that the debugger opens and responds. Debug mode stays on until it is switched off.

1.6 Optional: QEMU and the kit

1.6.1 Level 3: the QEMU emulator

QEMU emulates a whole chip and runs the real firmware (Listing 1.9). The command `qemu build` compiles the firmware and takes several minutes the first time, because it downloads components; `qemu upload` stages a cartridge into the flash image, and QEMU must be closed while it runs. If the upload fails, run QEMU once so that the flash image exists, close it, and upload again. Since October 2026 the firmware starts on the *Setup* menu by default, on QEMU and on the board: choose *Run Cartridge*, then the slot `cart0`. This procedure comes from the documentation of PRG32 (`docs/usage/qemu.md`) and was not run for this book.

```
# libraries for the window of the emulator
brew install libgcrypt glib pixman sdl2 libslirp                    # macOS
sudo apt install -y libgcrypt20 libglib2.0-0 libpixman-1-0 \
    libsdl2-2.0-0 libslirp0                                          # Linux

# the emulator itself (Windows: tick it in the ESP-IDF installer)
python $IDF_PATH/tools/idf_tools.py install qemu-riscv32
. $IDF_PATH/export.sh

# build the firmware for QEMU, stage a cartridge, run
python3 -m prg32 qemu build
python3 -m prg32 qemu upload pong.prg32
python3 -m prg32 qemu run
```

Listing 1.9. Level 3: the QEMU emulator.

1.6.2 Level 4: the kit

The firmware is written to the board once, through a USB cable; after that, games arrive over Wi-Fi and no cable is needed (Listing 1.10). The upload command is the same as for PRG32-QT, with another address. If the serial port is not found it is passed explicitly: `COM`*x* on Windows, `/dev/ttyACM0` on Linux, `/dev/cu.usbmodem...` on macOS. A black screen after flashing usually means that the firmware for QEMU was flashed instead of the one for the board. This procedure, too, comes from the documentation and was not run for this book.

```
# connect the board with a USB cable; ESP-IDF active, in the PRG32 folder
python3 -m prg32 esp32c6 build-and-flash

# the board starts and creates a Wi-Fi network
#     name: PRG32     password: prg32game     address: 192.168.4.1
# join it with your laptop, then
python3 -m prg32 esp32c6 upload-and-run pong.prg32 \
    --url http://192.168.4.1 --slot cart0

# or let the board join the Wi-Fi of the classroom
python3 -m prg32 wifi set --ssid "NAME" --password "PASSWORD" --mode sta
```

Listing 1.10. Level 4: the kit.

1.7 When something does not work

Table 1.5 lists the symptoms met most often; its first row covers most cases. The file `docs/usage/troubleshooting.md` of the repository has the complete list.

Table 1.5. Troubleshooting.

You see	It means	Do this
`idf.py: command not found`	ESP-IDF is not active in this terminal	run `export.sh`, or open the ESP-IDF PowerShell
`riscv32-esp-elf-gcc` missing	the compilers were not installed, or not activated	`./install.sh esp32c3,esp32c6`, then `export.sh`
`No module named prg32`	you are not in the PRG32 folder	`cd` to the folder that you cloned
upload: connection refused	PRG32-QT is not running, or a firewall blocks port 8080	start it; allow it; use the address on its Setup screen
macOS will not open PRG32	the application is from outside the App Store	right-click, Open; or Privacy and Security, Open Anyway
Linux: PRG32 does not start	the Qt of the system is older than 6.5	a newer distribution, or build from source
QEMU ignores the keys	the keyboard goes to the terminal	click the terminal that launched QEMU

How to ask for help

Paste the command and the *whole* message as text. Never send a photograph of the screen.

1.7.1 Ready for the next chapter?

Five checks, and the workshop is ready.

1. PRG32-QT starts and runs a game from the Store (step 1).
2. In a new terminal, after activating ESP-IDF, `doctor` shows only `OK` (step 2).
3. `cartridge build` produces `pong.prg32` (step 3).
4. `upload-and-run` starts it in PRG32-QT (step 3).
5. In debug mode, **Pause** and **Step** work (step 3).

1.8 The program of this chapter

The listings of this chapter are parts of one program, `code/l01.S` in the repository of the course, printed in full in Listing 1.11. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 1.5 is the frame that the test wrote.

On the screen.
Four vertical bars, red, green, blue and white, and the words PRG32 OK.

What to press.
Nothing: the program runs by itself.

```
# ARC12 Computer Architecture - Lecture 01: PRG32 and PRG32-QT: Setting Up Your Workshop
# l01.S: a test pattern that proves the installation works
#
# On the screen: four vertical bars, red, green, blue and white, and the words PRG32
# OK.
# Build: python3 -m prg32 cartridge build l01.S --portable \
#            --entry-prefix l01 --name l01 --out l01.prg32
.option norelax

.equ BLACK, 0x0000
.equ WHITE, 0xffff

.section .text
.global l01_init
.global l01_update
.global l01_draw

l01_init:
    ret

l01_update:
    ret
```

```
23
24 l01_draw:
25     addi sp, sp, -16
26     sw   ra, 12(sp)
27     sw   s0, 8(sp)
28     li   a0, BLACK
29     call prg32_gfx_clear
30     li   s0, 0              # bar number, 0 to 3
31 1:  la   t0, bars           # the table of the four colours
32     slli t1, s0, 2          # 4 bytes per word
33     add  t0, t0, t1
34     lw   a4, 0(t0)          # a4 = colour of bar s0
35     slli a0, s0, 6          # 64 * bar
36     addi a0, a0, 40         # x = 40 + 64 * bar
37     li   a1, 40             # y
38     li   a2, 48             # width
39     li   a3, 100            # height
40     call prg32_gfx_rect
41     addi s0, s0, 1
42     li   t0, 4
43     blt  s0, t0, 1b
44     li   a0, 124
45     li   a1, 160
46     la   a2, ok
47     li   a3, WHITE
48     li   a4, BLACK
49     call prg32_gfx_text8
50     lw   s0, 8(sp)
51     lw   ra, 12(sp)
52     addi sp, sp, 16
53     ret
54
55 .section .rodata
56 bars: .word 0xf800, 0x07e0, 0x001f, 0xffff     # red, green, blue, white
57 ok:   .asciz "PRG32 OK"
```

Listing 1.11. The complete program of the chapter, `code/l01.S`.

To run it on PRG32-QT the commands are those of Listing 1.12; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l01.S --portable \
    --entry-prefix l01 --name l01 --out l01.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l01.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 1.12. Building `l01.S` and running it on PRG32-QT.

1.9 Check your understanding

Q1.1 In a new terminal, `python3 -m prg32 doctor` says: `idf.py missing`. What is the most likely cause?

A. ESP-IDF must be installed again
B. ESP-IDF is not active in this terminal
C. PRG32-QT is not running
D. the board is not connected

Q1.2 Which host lets you run and debug a cartridge without installing a compiler?

A. the ESP32-C6 board
B. QEMU
C. PRG32-QT
D. none of them

Q1.3 Why does the same `pong.prg32` run on the board, on QEMU and on PRG32-QT?

A. it is recompiled at each start
B. it contains three programs
C. it calls the runtime through the same table of functions on every host
D. PRG32-QT translates it to C

1.10 Exercises

Exercise 1.1 (the workshop survives a restart)

1. Finish the five checks above on your own laptop.
2. Download three cartridges from the Store, and find in each the first instruction of `update` with the debugger.
3. Build one of the other examples in `examples/games`, and run it.
4. Close everything, restart the computer, and get back to a running cartridge without this book.
5. Optional: install QEMU and run the same cartridge there.

Expected result. Your workshop survives a restart, and you can open it without help. The fourth task is the real test: it shows whether the activation of ESP-IDF has been understood.

1.11 Glossary

English	Italiano
toolchain	catena di strumenti di sviluppo
cartridge	cartuccia
emulator	emulatore
to build	compilare (costruire)
host	macchina ospite
firmware	firmware
runtime	ambiente di esecuzione
interpreter	interprete
to flash	scrivere nella memoria flash
breakpoint	punto di interruzione

1.12 Summary and reading

PRG32 is a runtime, a cartridge format, a tool and a Store; a cartridge runs unchanged on the board, on QEMU and on PRG32-QT. PRG32-QT is enough to play and to debug; ESP-IDF and the PRG32 repository are needed to build. ESP-IDF must be activated in every new terminal: remember this and most problems disappear. You have built a cartridge, sent it to PRG32-QT and stopped it in the debugger: this is the loop of the whole book.

Reading. The `README` and the folder `docs/usage` of the PRG32 repository (Montella et al., 2026a); the `README` and the documentation of PRG32-QT (Montella, 2026); the chapters on setup and on PRG32-QT of *Playing with RISC-V* (Montella et al., 2026b). Appendix B of this book is a reference of the platform and of its commands.

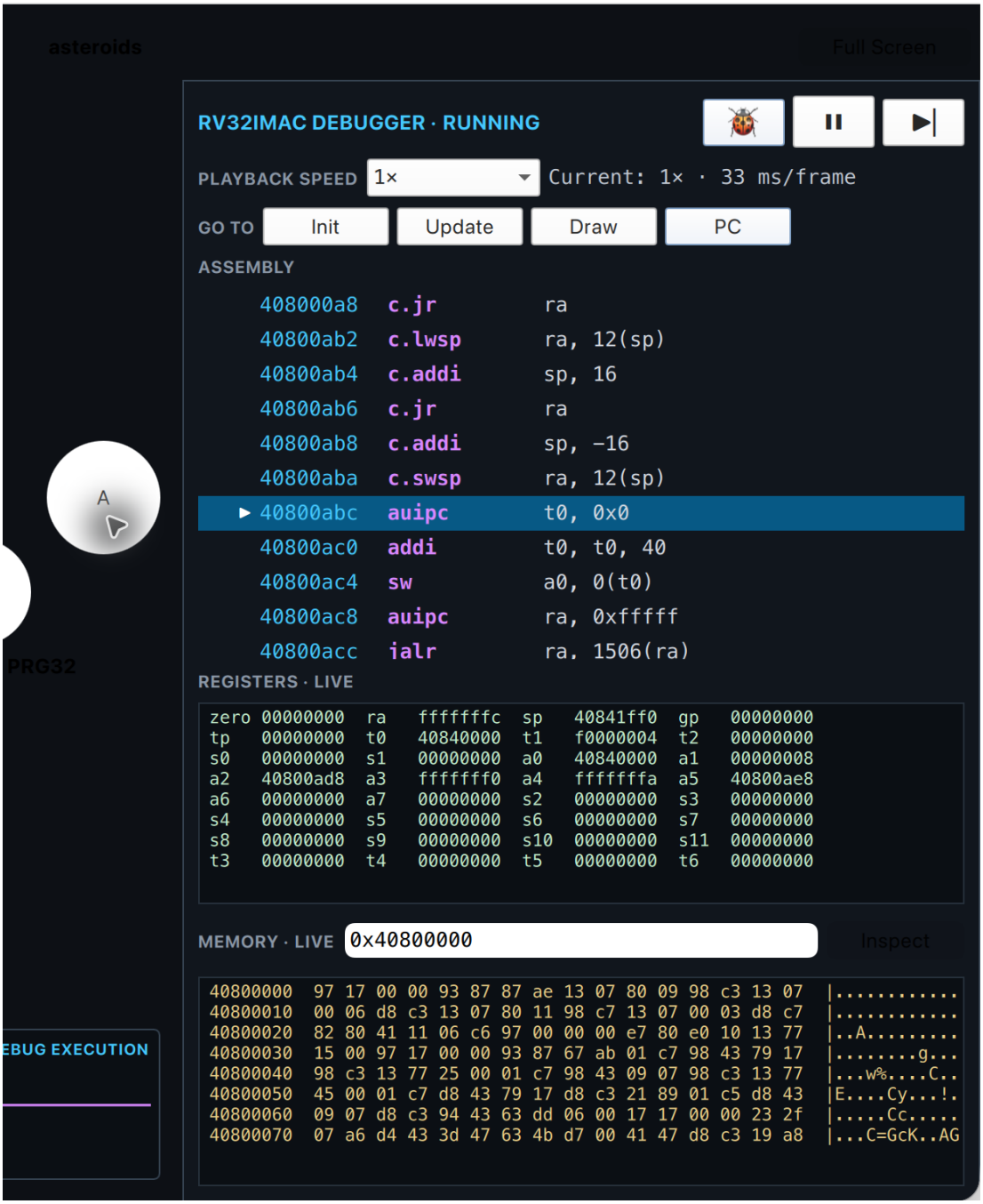


Figure 1.4. The machine, open: assembly with the current instruction, the 32 registers, memory, and the counters of the frame. If you see this with your own cartridge, you are ready for the rest of the book.

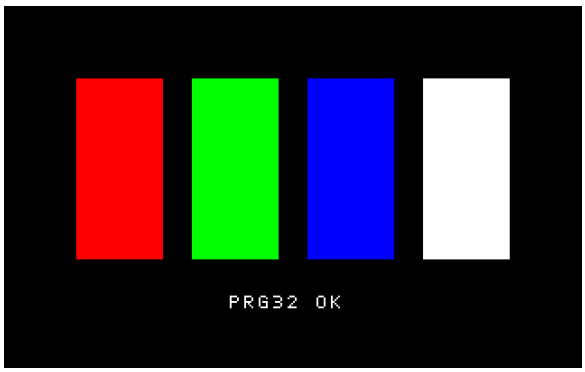


Figure 1.5. `101.S` on PRG32-QT.

CHAPTER 2

First Contact: RISC-V Assembly on PRG32

"Talk is cheap. Show me the code."
— Linus Torvalds, Linux kernel mailing list, 2000

In this chapter

By the end of this chapter you will be able to

- build a cartridge from an assembly file and run it on PRG32;
- say what a register is, and name the ones used every day;
- call a function of the framework, passing arguments in registers;
- keep a value in memory from one frame to the next: load, modify, store;
- read one instruction as the 32-bit number that it really is;
- list the questions that this program raises, which the rest of the book answers.

This chapter is the "play" turn of the spiral for the whole first half of the book: every later chapter points back to something typed here. The reader is *not* expected to understand everything in it. This chapter is for seeing; the following ones are for explaining. Four small programs are written, each complete and each running; they are the verified steps of the first assembly laboratory of the PRG32 textbook (Montella et al., 2026b).

2.1 The machine

2.1.1 Registers, and three kinds of instruction

A RISC-V processor, seen by the programmer, is a very small thing.

Thirty-two registers.
Each *register* holds one 32-bit value. The processor can compute only on values that are in registers.

Compute.
An arithmetic or logic operation between registers: `add`, `sub`, `and`, `or`, `xor`, and their *immediate* forms, such as `addi`, in which one operand is a small constant written in the instruction.

Move.
A transfer between a register and memory: `lw` loads a word, `sw` stores a word. Nothing else touches memory.

Jump.
A change in which instruction runs next: branches such as `beqz`, calls and returns.

That is very nearly the whole machine. The richness of computing comes not from a large instruction set but from combining a small one in long sequences.

2.1.2 The registers actually used

The hardware calls the registers `x0` to `x31` and, with one exception, treats them alike. Programmers call them by other names, which record the *role* that each plays (Table 2.1). The roles are fixed not by the hardware but by an agreement, the *calling convention*, part of the application binary interface (ABI). Following it is what allows a function written in assembly and the C code of the framework to call each other.

Table 2.1. The registers used every day. The complete table is in Appendix A.

Register	Name	Role	Preserved across a call?
`a0`–`a7`	arguments	values passed into a call; `a0` also carries the return value	no
`t0`–`t6`	temporaries	scratch values	no
`s0`–`s11`	saved	values that a call promises to preserve	yes
`ra`	return address	where `ret` jumps back to	no: save it yourself
`sp`	stack pointer	top of the stack, kept 16-byte aligned	yes
`zero`	zero	always reads as 0; writes are ignored	–

2.1.3 The contract: three functions

A PRG32 game is three functions that the programmer writes and the firmware calls (Figure 2.1). `init` is called once, when the cartridge starts, and sets up the state of the game. `update` is called once per frame: it reads the input and changes the state. `draw` is called once per frame: it turns the state into a picture. The firmware finds the three functions by name: a prefix chosen by the programmer, followed by `_init`, `_update` and `_draw`. That is the whole contract between a game and the machine.

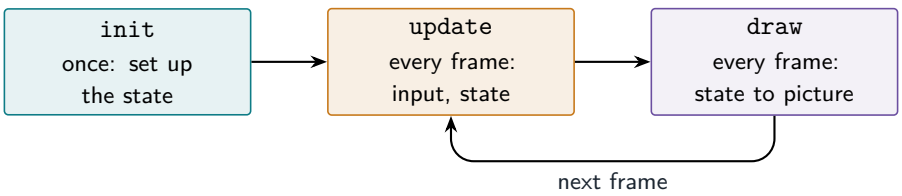


Figure 2.1. The contract between a cartridge and the runtime.

2.2 Your first cartridge

2.2.1 Step 1: the smallest program that runs

Listing 2.1 is a complete cartridge. Create a folder, type the program into a file called `step1_skeleton.S`, and resist the temptation to copy it: the fingers learn too.

```
# Assembly Lab 1: the smallest program that runs
.option norelax

.section .text
.global first_init
.global first_update
.global first_draw

first_init:
    ret

first_update:
    ret

first_draw:
    ret
```

Listing 2.1. Step 1: three functions that do nothing, correctly.

Lines that begin with a dot are *directives*: orders to the assembler, not instructions for the processor. `.option norelax` keeps the layout of the cartridge predictable, and every PRG32 example starts with it. `.section .text` says that what follows is executable code. `.global` makes a *label* visible outside the file, so that the firmware can find the functions by name. A label, a name followed by a colon, stands for the address of what follows it. The only instruction of the program, written three times, is `ret`: return to the caller.

The cartridge is built and run with the commands met in Chapter 1 (Listing 2.2); the cartridge file is byte for byte the same for QEMU, for the board and for PRG32-QT, where the address is `http://localhost:8080`.

```
# 1. build a portable cartridge from one assembly file
python3 -m prg32 cartridge build step1_skeleton.S --portable \
    --entry-prefix first --name step1 --out build/step1.prg32

# 2a. run it in QEMU on your laptop
python3 -m prg32 qemu upload build/step1.prg32
python3 -m prg32 qemu run

# 2b. or send it to a board, or to PRG32-QT, over the network
python3 -m prg32 esp32c6 upload-and-run build/step1.prg32 \
    --url http://192.168.4.1 --slot cart0
```

Listing 2.2. Building and running a cartridge.

Run it

The expected result is a black screen. The achievement is real all the same: the framework has found the three functions, has called them frame after frame, and has survived.

2.2.2 Step 2: say hello

To put something on the screen the program calls two functions of the framework: `prg32_gfx_clear`, which fills the screen with a colour, and `prg32_gfx_text8`, which writes a string with a font of 8 by 8 pixels (Listing 2.3, Figure 2.2). At the top of the file two `.equ` directives give names to two constants: `.equ BLACK, 0x0000` and `.equ WHITE, 0xffff`.

```
first_draw:
    addi sp, sp, -16      # prologue: a 16-byte frame
    sw   ra, 12(sp)       # save the return address

    li   a0, BLACK
    call prg32_gfx_clear

    li   a0, 32           # x
    li   a1, 40           # y
    la   a2, title        # address of the string
    li   a3, WHITE        # foreground
    li   a4, BLACK        # background
    call prg32_gfx_text8

    lw   ra, 12(sp)       # epilogue: restore ra
    addi sp, sp, 16       # release the frame
    ret

.section .rodata
title:
    .asciz "HELLO PRG32"
```

Listing 2.3. Step 2: the new `first_draw`.

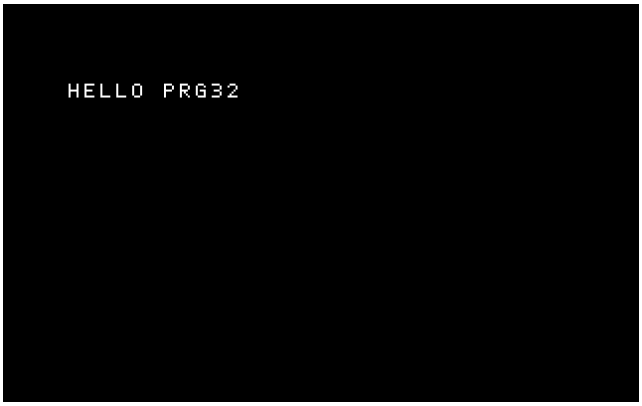


Figure 2.2. Step 2, built and run on PRG32.

The rule for passing arguments is simple: the first argument goes in `a0`, the second in `a1`,

and so on. `li` (*load immediate*) puts a number in a register; `la` (*load address*) puts there the address of a label. The string lives in the section `.rodata`, *read-only data*, and the directive `.asciz` stores its characters followed by a byte of value zero that marks its end.

2.2.3 Why save `ra`? The stack frame

The first and the last three lines of `first_draw` are new, and they are the price of calling another function. The instruction `call` stores in `ra` the address to come back to. But `first_draw` was itself called by the firmware, and `ra` already held the address of *its* return: the first `call` would overwrite it, and the final `ret` would jump to the wrong place.

A function that calls another one therefore saves `ra` first. It lowers `sp` by 16 to make room on the *stack*, a region of memory that each function may use for its own temporary needs, and stores `ra` at `12(sp)`, that is, 12 bytes above the address in `sp` (Figure 2.3). These lines are the *prologue* of the function. After the last call it undoes everything in reverse order, in the *epilogue*: it loads `ra` back, raises `sp` by 16, and returns.

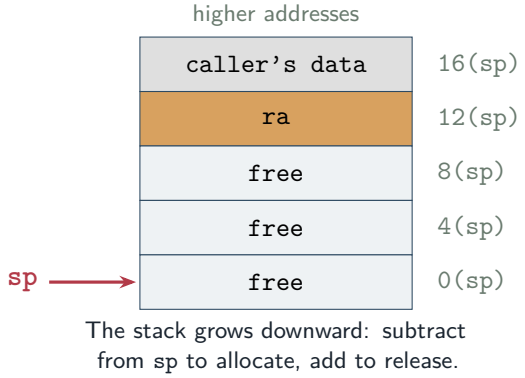


Figure 2.3. Stack memory after the prologue of `first_draw`.

The classic first bug

Forgetting the pair `sw ra`/`lw ra` is the classic first bug: the function returns to the wrong place, and the program crashes or freezes long after the line that is at fault.

The convention requires `sp` to stay aligned to a multiple of 16 bytes across calls, which is why the frame is 16 bytes although only 4 are used. The stack is treated formally in Chapters 15 and 19.

2.2.4 Step 3: a value that survives between frames

Registers do not keep their values between one call of `update` and the next: the firmware uses them in the meantime. Whatever must last has to live in memory (Listing 2.4).

```
first_update:
    # A leaf function: it calls nothing,
    # so it needs no stack frame.
    la   t0, counter      # t0 = ADDRESS of counter
    lw   t1, 0(t0)        # t1 = VALUE stored there   (load)
    addi t1, t1, 1        # one tick per frame        (modify)
    sw   t1, 0(t0)        # write it back             (store)
```

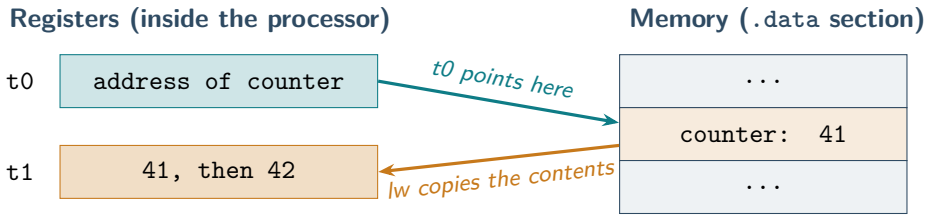
```
8       ret
9
10  .section .data
11  counter: .word 0
```

Listing 2.4. Step 3: the new `first_update` and its variable.

`counter` is a label: it names a location in memory, in the section `.data`, which holds one 32-bit word, initially zero. `la` puts the *address* of that location in `t0`. `lw` reads the *value* at that address into `t1`; `sw` writes it back. Because the processor computes only in registers, every change to a variable has the same three movements: *load, modify, store*. The function is a *leaf*: it calls nothing, so it needs no stack frame.

Address or value? The crux of memory

A label names a *location*. `la` gives the location; `lw` gives the contents (Figure 2.4). This is how every variable of every program ultimately works. Using an address where a value belongs, or the reverse, is one of the most common beginner's errors: comment every register as ADDRESS or VALUE.




```
la   t0, counter    # t0 = WHERE the value lives
lw   t1, 0(t0)      # t1 = WHAT is stored there
addi t1, t1, 1      # change the copy in the register
sw   t1, 0(t0)      # memory still says 41 until this line
```

Figure 2.4. The register `t0` holds an address; `t1` holds a value.

Nothing visible changes after this step: the counter ticks, but nobody draws it. How could one prove that it is ticking? With the debugger of PRG32-QT, or with the next step.

2.2.5 Step 4: make the counter visible

Eight lines added to `first_draw`, after the call that writes the text, draw a green bar whose width is the counter (Listing 2.5, Figure 2.5). The constant `GREEN` is defined at the top of the file by `.equ GREEN, 0x07e0`.

```
1       la   t0, counter
2       lw   t1, 0(t0)
3       andi a2, t1, 255      # width = counter modulo 256
4       li   a0, 32           # x
5       li   a1, 60           # y
6       li   a3, 8            # height
7       li   a4, GREEN        # .equ GREEN, 0x07e0
```

```
8       call prg32_gfx_rect
```

Listing 2.5. Step 4: the lines added to `first_draw`.



Figure 2.5. Step 4: the bar grows from 0 to 255 pixels, then starts again.

The function `prg32_gfx_rect` takes five arguments: x, y, width, height and colour. The width is computed by `andi` with 255, which keeps only the low eight bits of the counter: the bar grows from 0 to 255 pixels and then starts again. Three future chapters hide in these eight lines: the equality $255 =$ `0xFF` (positional notation), `andi` as a bit mask (Boolean algebra), and `0x07E0` as a colour (the encoding of information).

The complete program is Listing 2.6.

```
1  # Assembly Lab 1: hello, state, and a counter you can see.
2  .option norelax
3  .equ BLACK, 0x0000
4  .equ WHITE, 0xffff
5  .equ GREEN, 0x07e0
6
7  .section .text
8  .global first_init
9  .global first_update
10 .global first_draw
11
12 first_init:
13     ret
14
15 first_update:
16     # A leaf function: it calls nothing, so it needs no stack frame.
17     la   t0, counter           # t0 = ADDRESS of counter
18     lw   t1, 0(t0)             # t1 = VALUE stored there      (load)
19     addi t1, t1, 1             # one tick per frame           (modify)
20     sw   t1, 0(t0)             # write it back                (store)
21     ret
22
23 first_draw:
24     addi sp, sp, -16           # prologue: make a 16-byte frame
25     sw   ra, 12(sp)            # save the return address
26
27     li   a0, BLACK
28     call prg32_gfx_clear
```

```
29
30      li   a0, 32                # x
31      li   a1, 40                # y
32      la   a2, title             # address of the string
33      li   a3, WHITE             # foreground
34      li   a4, BLACK             # background
35      call prg32_gfx_text8
36
37      la   t0, counter
38      lw   t1, 0(t0)
39      andi a2, t1, 255           # width = counter modulo 256
40      li   a0, 32                # x
41      li   a1, 60                # y
42      li   a3, 8                 # height
43      li   a4, GREEN
44      call prg32_gfx_rect
45
46      lw   ra, 12(sp)            # epilogue: restore ra
47      addi sp, sp, 16            # release the frame
48      ret
49
50  .section .rodata
51  title:
52      .asciz "HELLO PRG32"
53
54  .section .data
55  counter: .word 0
```

Listing 2.6. The complete first cartridge (`step4_bar.S`).

2.3 Reading what you wrote

2.3.1 The instructions just used

Table 2.2 orders by family the instructions met so far. In all of them the result goes in the *first* register written. The base integer instruction set of RISC-V, RV32I, has about forty instructions: a quarter of them have already been used.

Table 2.2. The instructions used in this chapter.

Family	Instruction	What it does	Example
Arithmetic	`add`, `sub`	combine two registers	`add t0, t1, t2`
Logic	`and`, `or`, `xor`	bitwise operation on two registers	`and t0, t1, t2`
Immediate	`addi`, `andi`, `ori`, `xori`	one register and one small constant	`addi t1, t1, 1`
Memory	`lw`, `sw`	load or store a 32-bit word	`lw t1, 0(t0)`
Control	`call`, `ret`	go to a function and come back	`call prg32_gfx_rect`

2.3.2 Pseudo-instructions

Several of the mnemonics used are not instructions of the processor at all. They are *pseudo-instructions*: shortcuts that the assembler expands into one or two real instructions (Ta-

ble 2.3). A pseudo-instruction is not a separate hardware operation. The register `zero` is what makes many of them possible: a small instruction set, reused with care.

Table 2.3. Pseudo-instructions and their expansions.

You write	The assembler emits	Meaning
`li t0, 42`	`addi t0, zero, 42`	load a constant
`mv t0, t1`	`addi t0, t1, 0`	copy a register
`not t0, t1`	`xori t0, t1, -1`	bitwise complement
`la t0, counter`	`auipc t0, ...` + `addi t0, t0, ...`	load the address of a label
`ret`	`jalr zero, 0(ra)`	jump to the address in `ra`
`beqz t0, skip`	`beq t0, zero, skip`	branch if `t0` is zero

2.3.3 One instruction is one 32-bit number

The processor does not read text. The assembler translates each line into one 32-bit word, and it is that word that is stored in the cartridge and executed. Figure 2.6 dissects `addi t1, t1, 1`: twelve bits hold the constant, five bits name the source register (`t1` is `x6`), three bits select the operation, five bits name the destination register and seven bits, the *opcode*, say what kind of instruction this is. Written in hexadecimal the thirty-two bits are `0x00130313`.

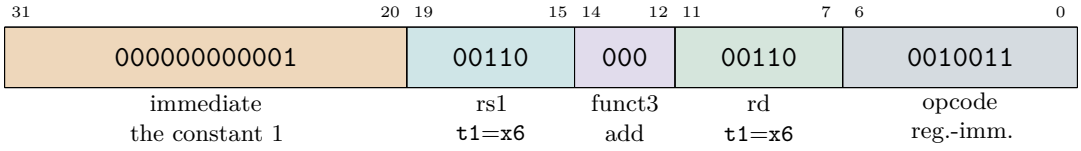


Figure 2.6. `addi t1, t1, 1` is the number `0x00130313`.

Concept

Assembly is a readable spelling of machine code. An instruction is data: the processor reads its fields with logic circuits, which will be designed in the chapters on logic and on the processor.

2.3.4 PRG32-QT: a microscope for registers

PRG32-QT runs the same cartridge on the desktop one instruction at a time, stops between any two instructions and shows every register.

Step through `first_update`

Load step 3 in debug mode, place a breakpoint on `first_update` and press **Step** four times. Before each press, say what `t0` and `t1` will contain; after it, check.

2.3.5 Trace it by hand

The same exercise can, and should, be done on paper. Table 2.4 follows `first_update` through a few frames. At frame 256 the counter keeps growing, but its low eight bits return to zero, because $256 = 1\,0000\,0000_2$: the bar starts again. Tracing by hand is the most valuable debugging skill taught in this book.

Table 2.4. Three frames, and one more, of `first_update`.

Frame	`lw: t1` becomes	`addi: t1` becomes	`sw: counter` becomes	Bar width
1	0	1	1	1
2	1	2	2	2
256	255	256	256	0
257	256	257	257	1

2.4 What comes next

The program of this chapter has raised more questions than it has answered. Table 2.5 is the forward map of the book: each later chapter opens by pointing at one of its rows.

Table 2.5. Questions that the first program has already raised.

You saw	The question	Answered in
`0x07e0`, `0xffff`, 255	Why do programmers write numbers in hexadecimal?	Chapters 4, 5
a position below zero shown as `0xFFFFFFFF`	How does a register hold a negative number?	Chapter 5
a ball that moves less than a pixel per frame	How are fractions represented without a floating-point unit?	Chapter 6
`andi a2, t1, 255`	What does AND do to thirty-two bits at once?	Chapters 8, 9
`addi t1, t1, 1`	Which circuit actually adds two numbers?	Chapter 9
a pressed button is a 1	What is a bit made of, physically?	Chapter 10
`counter` survives between frames	How does a circuit remember a bit?	Chapters 13, 15
`0x00130313`	How does the processor decode and execute it?	Chapters 18, 19
three functions and a rectangle	Can a whole game be made out of this?	Verm1: Chapters 7, 12, 17, 22

2.5 The program of this chapter

The listings of this chapter are parts of one program, `code/l02.S` in the repository of the course, printed in full in Listing 2.7. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 2.7 is the frame that the test wrote.

On the screen.
HELLO PRG32 and, under it, a green bar that grows by one pixel per frame and starts again at 256.

What to press.
Nothing: the program runs by itself.

```
1 # ARC12 Computer Architecture - Lecture 02: First Contact: RISC-V Assembly on
      PRG32
```



Figure 2.7. `l02.S` on PRG32-QT.

```
# l02.S: the four steps of the lecture in one file
#
# On the screen: HELLO PRG32 and, under it, a green bar that grows by one pixel per
# frame and starts again at 256.
# Build: python3 -m prg32 cartridge build l02.S --portable \
#            --entry-prefix first --name first --out first.prg32
.option norelax

.equ BLACK, 0x0000
.equ WHITE, 0xffff
.equ GREEN, 0x07e0

.section .text
.global first_init
.global first_update
.global first_draw

first_init:                 # step 1: a function that returns
    ret

first_update:               # step 3: a value that survives between frames
    # A leaf function: it calls nothing, so it needs no stack frame.
    la   t0, counter        # t0 = ADDRESS of counter
    lw   t1, 0(t0)          # t1 = VALUE stored there   (load)
    addi t1, t1, 1          # one tick per frame        (modify)
    sw   t1, 0(t0)          # write it back             (store)
    ret

first_draw:                 # step 2: say hello
    addi sp, sp, -16        # prologue: a 16-byte frame
    sw   ra, 12(sp)         # save the return address
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 32             # x
    li   a1, 40             # y
    la   a2, title          # address of the string
    li   a3, WHITE          # foreground
    li   a4, BLACK          # background
    call prg32_gfx_text8

```

```
    la   t0, counter          # step 4: make the counter visible
    lw   t1, 0(t0)
    andi a2, t1, 255          # width = counter modulo 256
    li   a0, 32               # x
    li   a1, 60               # y
    li   a3, 8                # height
    li   a4, GREEN
    call prg32_gfx_rect

    lw   ra, 12(sp)           # epilogue: restore ra
    addi sp, sp, 16           # release the frame
    ret

.section .rodata
title:
    .asciz "HELLO PRG32"

.section .data
counter: .word 0
```

Listing 2.7. The complete program of the chapter, `code/l02.S`.

To run it on PRG32-QT the commands are those of Listing 2.8; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l02.S --portable \
    --entry-prefix first --name l02 --out l02.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l02.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 2.8. Building `l02.S` and running it on PRG32-QT.

2.6 Check your understanding

Q2.1 After `la t0, counter` and `lw t1, 0(t0)`, what does `t0` hold?

A. The value of the counter
B. The address of the counter
C. Zero, because `la` clears the register
D. The return address

Q2.2 A function that calls `prg32_gfx_clear` must save one register before the call. Which one?

A. `a0`, because it carries the argument

B. `sp`, because the stack moves
C. `ra`, because `call` overwrites it
D. `t0`, because it is temporary

Q2.3 `t1` holds 300. What does `andi a2, t1, 255` leave in `a2`?

A. 300
B. 255
C. 45
D. 44

2.7 Exercises

Exercise 2.1 (on PRG32)

1. Change the title string and its colour to yellow (`0xffe0`); run it again.
2. Make the bar grow twice as fast, then four times as fast, changing one instruction only.
3. Make the bar shrink instead of grow. What happens when the counter passes zero?
4. In PRG32-QT, single-step `first_update` and write down `t0` and `t1` after each instruction.
5. Draw a second bar, in red, whose width is the counter modulo 64.

Expected result. In the third task the counter goes from 0 to `0xFFFFFFFF`: with `andi 255` the bar jumps to 255 and then shrinks. Keep the observation: Chapter 5 explains it.

2.8 Glossary

English	Italiano
register	registro
load / store	caricare / memorizzare
address	indirizzo
immediate	immediato (costante)
branch	salto condizionato
label	etichetta
stack	pila (stack)
stack pointer	puntatore alla pila
return address	indirizzo di ritorno
calling convention	convenzione di chiamata
machine code	codice macchina
cartridge	cartuccia

2.9 Summary and reading

A RISC-V processor is thirty-two registers and a small set of instructions that compute, move and jump. A PRG32 game is three functions; arguments travel in `a0` to `a7`; a function that calls another saves `ra`. Variables live in memory: `la` for the address, `lw` and `sw` for the value. An instruction is a 32-bit number; assembly is its readable spelling.

Reading. *Playing with RISC-V*, chapter 5, "First Contact" (Montella et al., 2026b); Patterson and Hennessy (2020), chapter 2; the tutorial `docs/learn/tutorial_ascii_game.md` of the PRG32 repository.

Part II

Information

CHAPTER 3

History of Computing

"printf("Hello, world!");"
— Brian Kernighan and Dennis Ritchie, *The C Programming Language*, 1978

In this chapter

By the end of this chapter you will be able to

- define informatics and say why it is not the science of computers;
- place the key machines and people on a timeline, from the abacus to the stored-program computer;
- name the four units of the von Neumann architecture and find them in the machine programmed in Chapter 2;
- explain what RISC means and why RISC-V is an open instruction set;
- describe where informatics sits between science, engineering and the economy.

The reader has already run a RISC-V program. This chapter tells where that machine comes from. It is not a chronicle: of the thousands of machines and people that the history of computing could list, it keeps those that added an idea which is still inside the ESP32-C6 on the breadboard. The thread to follow is that, at every stage, somebody found a new way of doing one of two things: *representing* information in a physical object, or *prescribing* a sequence of operations to be carried out without further thought.

3.1 What is informatics?

Definition 3.1 (Informatics)

Informatics is the applied science that deals with the treatment of information by means of automated procedures.

The Italian word *informatica* comes from the French *informatique*, a contraction of *information automatique*. English says *computer science* and, increasingly, *informatics*. The tool of the discipline is called *elaboratore* or *calcolatore* in Italian, *ordinateur* in French, *computer* in English. The English name of the discipline has the defect of putting the tool in the place of the subject, a defect that a sentence attributed to Edsger W. Dijkstra corrects:

Computer science is no more about computers than astronomy is about telescopes.

The machine is the instrument, not the subject. In this book the instrument is studied closely precisely so that it stops being a mystery.

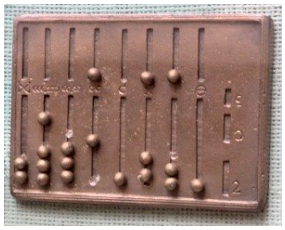
The discipline has two souls. *Theoretical informatics* studies the foundations of information and computation at the logical level: what can be computed, and at what cost. Its central object is the *algorithm*, a finite set of operations that describes the transformation of a datum A into a result B. *Applied informatics* studies the enabling technologies and the information systems built with them. It is pervasive, a driver of economic development and of the whole sector of information and communication technologies, a STEM discipline at the heart of an industrial revolution; and it has a social cost that must be managed, the *digital divide* between those who have access to these technologies and those who do not.

3.2 Before electricity

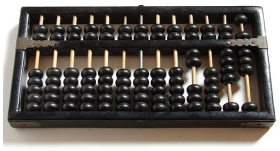
For three thousand years people have built machines to help them calculate.

3.2.1 Origins: the abacus and the Antikythera mechanism

Babylonians, Chinese, Greeks and Romans all used counting frames (Figure 3.1). In an abacus a number *is* a position of beads, and an addition *is* a movement of the fingers: the device is a physical representation of data and of operations. The Antikythera mechanism, recovered from a shipwreck and built between the second and the first century before Christ (most estimates fall between about 205 and 60 BC), is the oldest known geared calculator: its dials computed astronomical positions and eclipses, and in it a number is the angle of a gear.



(a) Roman hand abacus



(b) Chinese *suanpan*



(c) Antikythera mechanism



(d) One of its plates

Figure 3.1. Origins: a number becomes a position of beads or the angle of a gear.

Concept

Representation comes before calculation. Before anything can be computed, something physical must stand for a number.

3.2.2 The medieval Islamic world

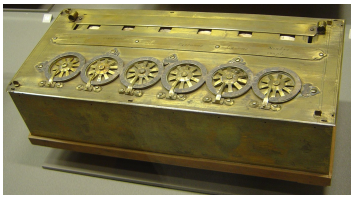
Between the ninth and the thirteenth century the scholars of the Islamic world preserved the science of antiquity and added to it three things that concern this story. The first is a family of astronomical instruments, the mechanical astrolabe (Figure 3.2a) and the torquetum, which are analogue computers for the positions of the stars. The second is

cryptanalysis: in the ninth century al-Kindi described frequency analysis, the method of breaking a cipher by counting how often each symbol occurs. The third is the programmable machine: the automatic flute player of the Banū Mūsā brothers, and the weight-driven clocks and automata of al-Jazari (1206), whose behaviour could be changed by rearranging pegs.

The very word *algorithm* comes from the name of the mathematician al-Khwārizmī, as *algebra* comes from the title of his book.



(a) A brass astrolabe



(b) A Pascaline



(c) Leibniz's reckoner

Figure 3.2. Instruments and calculators before 1700.

3.2.3 From 1600 to 1800: mechanical calculators and logic

The seventeenth century built the first mechanical calculators. Napier's tables of logarithms (1614) turned multiplications into additions. Blaise Pascal's *Pascaline* (1642) added and subtracted, and its real invention was the automatic *carry*: each wheel is one decimal digit (Figure 3.2b), and a wheel that passes from 9 to 0 pushes its neighbour forward by one step, exactly as the full adder of Chapter 11 will do with bits. Leibniz's stepped reckoner (1673) performed the four operations.

The same Leibniz described binary arithmetic in 1703, and a century and a half later George Boole, in *The Laws of Thought* (1854), showed that logic can be treated as an algebra (Boole, 1854): the subject of Chapters 8 and 9.

Meanwhile a machine that had nothing to do with numbers introduced the stored program. In Jacquard's loom (1804) the pattern of the cloth is recorded on a chain of punched cards: change the cards, and the same loom weaves a different pattern.

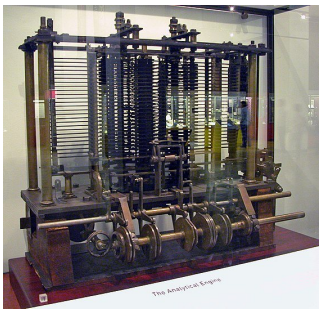
3.2.4 The 1800s: Charles Babbage and Ada Lovelace

Charles Babbage put the two lines together (Figure 3.3). His *Difference Engine* computed mathematical tables automatically. His *Analytical Engine*, designed but never completed, was a general-purpose computer in everything but technology: it had a *mill*, the arithmetic unit; a control unit; a *store*, the memory; and an output device. Programs and data were to be supplied on punched cards, borrowed from the Jacquard loom.

Ada Lovelace devised a method for programming the Analytical Engine. Her notes of 1843 contain what is regarded as the first published program, for computing the Bernoulli numbers, and she is regarded as the first programmer in history.



(a) Difference Engine



(b) Analytical Engine



(c) Punched cards

Figure 3.3. The machines of Charles Babbage.

3.3 The twentieth century

The twentieth century produced the idea of a universal machine, and then the machines that made it real. The theory came first.

3.3.1 Turing, Shannon and Nakajima

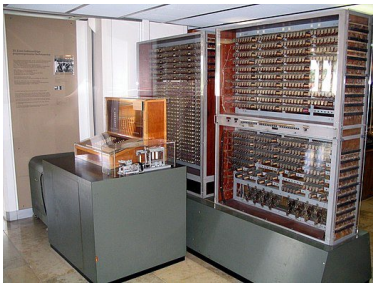
In 1936 Alan Turing defined an abstract universal machine, and with it the notion of what can be computed (Turing, 1936). During the war he worked at Bletchley Park on breaking the German Enigma cipher (Figure 3.4a).

In the same years Akira Nakajima in Japan and Claude Shannon in the United States founded the theory of switching circuits: Boolean algebra describes circuits made of switches (Shannon, 1938). It is the principle on which all digital design rests, and the bridge between the logic of Chapters 8–9 and the circuits of Chapters 10–11.

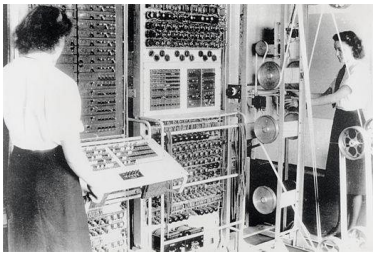
In Germany Konrad Zuse built the Z3 (1941), the first working computer controlled by a program, which was shown in 1998 to be Turing-complete; its successor, the Z4, was one of the first computers to be sold commercially.



(a) An Enigma machine



(b) Zuse Z3 (replica)



(c) Colossus

Figure 3.4. Machines of the 1940s.

3.3.2 John von Neumann: the stored-program computer

In 1945 the report on the EDVAC, signed by John von Neumann, described a hardware architecture for programmable digital computers (von Neumann, 1945). Its heart is one idea.

The stored program

Program and data share the same memory: *a program is data.* Instructions are numbers kept in memory, fetched and executed one after another.

The architecture has four essential units (Figure 3.5): a *control unit*, which governs the sequence of operations; an *arithmetic logic unit* (ALU), with its accumulator register; a *memory*; and devices for *input* and *output*.

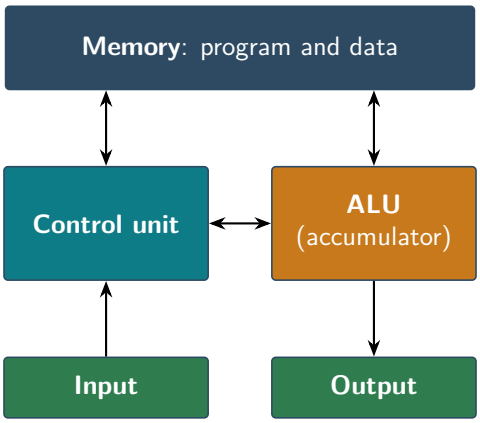


Figure 3.5. The four units of the von Neumann architecture.

Back to the Machine

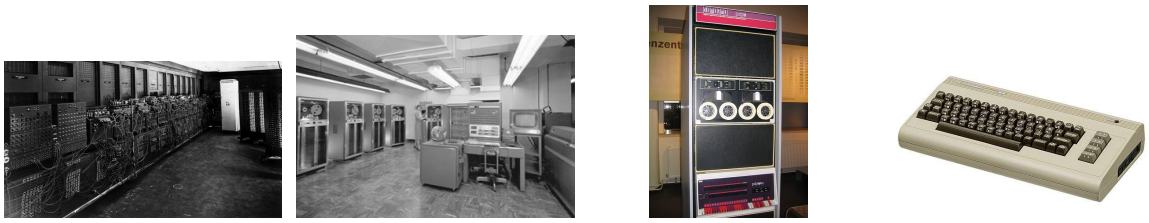
The ESP32-C6 is still a von Neumann machine, and its four units were all at work in Chapter 2. The registers and the ALU executed `addi t1, t1, 1`; the control unit decided that the next instruction was `sw`; memory held the variable `counter` *and the instructions themselves*, among them the number `0x00130313`; the joystick and the display are input and output.

3.3.3 Eight decades of computers

Since then each decade has made the computer smaller, cheaper and closer to people (Table 3.1, Figure 3.6); none of them has changed the stored-program idea.

Table 3.1. Eight decades of computers.

Decade	Era	What changed
1940s	Wartime	ENIAC (USA) and Colossus (UK), 1943–45
1950s	Military and government	EDVAC, 1949: the stored program
1960s	Mainframe computing	one machine, many users
1970s	Minicomputers	a computer per department
1980s	Personal computing	a computer per person
1990s	The Internet	computers connect
2000s	Smartphones and tablets	a computer in the pocket
2010s	Cloud computing	computing as a service
2020s	Internet of Things and AI	computers everywhere



(a) ENIAC **(b)** A mainframe **(c)** A minicomputer **(d)** Commodore 64

Figure 3.6. From a room to a desk in forty years.

3.4 From home computers to RISC-V

3.4.1 Microprocessors: from CISC to RISC to an open standard

The processor of the PRG32 kit is the way it is because of a sequence of decisions taken over fifty years (Table 3.2). The first microprocessors grew by accretion: each generation added instructions, often complicated ones, to make the work of compilers and of assembly programmers easier. The result is called a *complex instruction set computer* (CISC), and the x86 family, born in 1978, still carries that inheritance in today's personal computers.

At the beginning of the 1980s two university projects, at Berkeley and at Stanford, measured what programs actually do, and found that a processor with few, simple and regular instructions can be made to run faster than one with many complicated ones. This is the idea of the *reduced instruction set computer* (RISC), for which David Patterson and John Hennessy received the Turing Award in 2017. ARM applied it to low power, and ARM cores are today in nearly every telephone. In 2010 the fifth RISC design of Berkeley became something new: an instruction set published as an *open standard*.

Table 3.2. Microprocessors: from CISC to RISC to an open standard.

Year	Milestone	Why it matters
1971	Intel 4004	the first commercial microprocessor: 2 300 transistors on one chip
1975	MOS 6502	a cheap 8-bit CPU: Apple II, Commodore 64, the home-computer era
1978	Intel 8086	the start of x86, a complex instruction set (CISC) still in today's PCs
1980–82	Berkeley RISC and Stanford MIPS	few, simple, regular instructions run faster: the RISC idea
1985	ARM1	RISC for low power: today in nearly every telephone
2010	RISC-V starts at UC Berkeley	the fifth Berkeley RISC design becomes an open instruction set
2015–20	RISC-V Foundation, then RISC-V International	a shared specification, governed from Switzerland
2023–25	European Chips Act; the DARE project	RISC-V as a strategy for European technological sovereignty

Definition 3.2 (RISC-V)

RISC-V is an open standard *instruction set architecture*: the specification of the instructions, of the registers and of their encoding is public, and anyone may design, build and sell a processor that implements it without paying a licence for the architecture.

RISC-V is therefore not a processor, and it was not the first RISC: Berkeley RISC, MIPS and ARM came earlier. What distinguishes it from x86 and from ARM is openness.

3.4.2 Moore's law in one number

The Intel 4004 of 1971 had 2 300 transistors. A laptop processor of today has tens of thousands of millions: a growth by a factor of about ten million. For fifty years the number of transistors on a chip has doubled about every two years, an observation made by Gordon Moore in 1965 and revised in 1975 (Moore, 1965). It is an observation about an industry, not a law of nature, and its physical limits are now close. Progress comes increasingly from better architectures, from specialised accelerators and from open designs, which is why architecture, the subject of this book, matters more than ever. Chapter 24 takes up the story from here.

3.4.3 The home-computer spirit, on a modern open CPU

The home computers of the 1980s had a quality that later machines lost: one person could understand all of one. PRG32 was designed to give that quality back, on the architecture that students will meet in industry (Table 3.3). The screen has the same size, and the same immediacy, forty years apart.

Table 3.3. A Commodore 64 and PRG32.

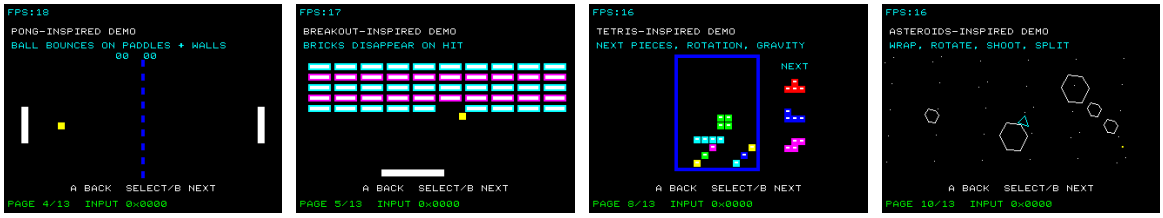
	Commodore 64 (1982), MOS 6510	**PRG32 on ESP32-C6, RISC-V**
Word size	8 bits	32 bits
Registers	A, X, Y (plus stack pointer, status, PC)	32 general-purpose registers, `x0` to `x31`
Clock	about 1 MHz	160 MHz
Address space	64 KB	4 GB
Instruction set	56 instructions, proprietary	about 40 base instructions plus extensions, open standard
Screen	320 by 200 pixels, 16 colours	320 by 200 viewport, 256-entry RGB565 palette
Programmed by	typing on the machine itself	building a cartridge and uploading it

The 6502 is still a pleasure to explore, for instance with the simulator at https://skilldrick.github.io/easy6502.

3.4.4 Retro games as specimens

A biologist studies a fruit fly not because flies are important but because a fly is small enough to be understood completely and rich enough to show how heredity works. A retro game plays the same part here (Figure 3.7): it is small enough to be held in one's head and

rich enough to exercise state, input, arithmetic, decisions and a loop, which is to say the whole course. The four games of the figure are example cartridges of the PRG32 repository (`examples/games`).



(a) Pong **(b)** Breakout **(c)** Tetris **(d)** Asteroids

Figure 3.7. Retro games as specimens.

3.5 Informatics as an applied science

Where does informatics sit among the sciences? An example from the Department in which this book was written makes the answer concrete (Figure 3.8). A storm forms over the sea: a natural phenomenon. Physics and mathematics, the basic sciences, describe it with equations. Informatics, an applied science, turns the equations into algorithms and simulations. Engineering builds the supercomputers, the networks and the sensors that run them reliably. And the economy receives forecasts and services that reach people and create value. The weather and marine forecasts of https://meteo.uniparthenope.it are computed every day in this way, on the Department's own high-performance systems.

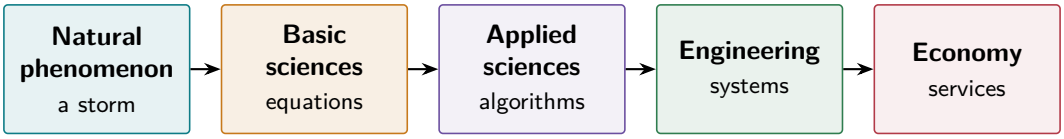
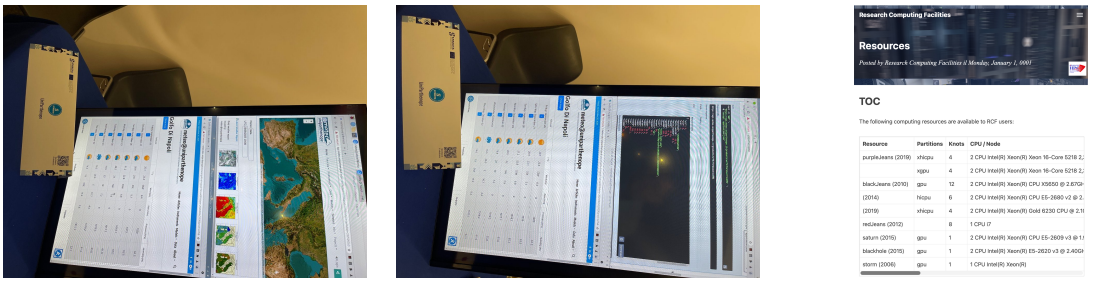


Figure 3.8. Where informatics sits.

The same ideas that are practised in this book on a microcontroller at 160 MHz scale up to clusters (Figure 3.9): registers, memory, instructions, only many more of them.



(a) A teaching cluster **(b)** A parallel simulation **(c)** rcf.uniparthenope.it

Figure 3.9. High-performance computing at Parthenope: mini PCs with GPUs on a 10 Gbit network, and the Research Computing Facilities.

3.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l03.S` in the repository of the course, printed in full in Listing 3.1. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 3.10 is the frame that the test wrote.

On the screen.
A row of 32 lamps showing 0x00130313, the machine code of addi t1, t1, 1, and a green square: the word read is the one expected.

What to press.
Nothing: the program runs by itself.

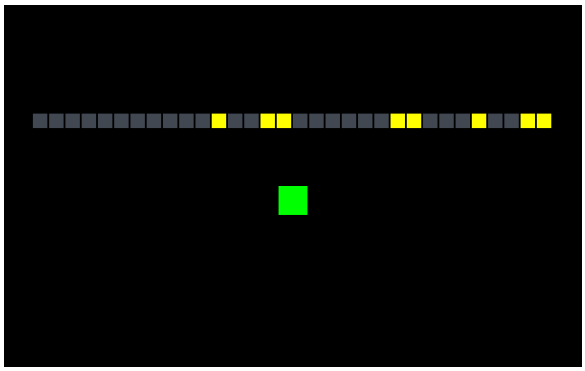


Figure 3.10. `l03.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 03: History of Computing
# l03.S: the stored program: a program that reads one of its own instructions as data
#
# On the screen: a row of 32 lamps showing 0x00130313, the machine code of addi t1,
# t1, 1, and a green square: the word read is the one expected.
# Build: python3 -m prg32 cartridge build l03.S --portable \
#            --entry-prefix l03 --name l03 --out l03.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800

.section .text
.global l03_init
.global l03_update
.global l03_draw

l03_init:
    ret

l03_update:
    ret
```

```
26
27  # One instruction of the program.  For von Neumann's machine it is a number in
28  # the same memory as the data: the program can load it like any other word.
29  .option push
30  .option norvc               # keep it 32 bits long: no compressed form
31  specimen:
32      addi t1, t1, 1
33  .option pop
34      ret
35
36  l03_draw:
37      addi sp, sp, -16
38      sw   ra, 12(sp)
39      sw   s0, 8(sp)
40      li   a0, BLACK
41      call prg32_gfx_clear
42      la   t0, specimen       # the ADDRESS of an instruction ...
43      lw   s0, 0(t0)          # ... and the instruction, read as DATA
44      mv   a0, s0
45      li   a1, 60
46      call lamps32            # show its 32 bits
47      li   t0, 0x00130313     # what the assembler should have produced
48      li   a4, RED
49      bne  s0, t0, 1f
50      li   a4, GREEN          # equal: instructions are numbers
51  1:  li   a0, 152
52      li   a1, 100
53      li   a2, 16
54      li   a3, 16
55      call prg32_gfx_rect
56      lw   s0, 8(sp)
57      lw   ra, 12(sp)
58      addi sp, sp, 16
59      ret
60
61  # lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
62  #   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
63  lamps32:
64      addi sp, sp, -32
65      sw   ra, 28(sp)
66      sw   s0, 24(sp)
67      sw   s1, 20(sp)
68      sw   s2, 16(sp)
69      mv   s0, a0             # the word
70      mv   s1, a1             # y
71      li   s2, 0              # lamp number, 0 on the left
72  1:  li   t0, 31
73      sub  t0, t0, s2         # lamp i shows bit 31 - i
74      srl  t1, s0, t0         # bring that bit down to bit 0
75      andi t1, t1, 1          # keep it, clear all the others
76      li   a4, GREY           # lamp off
77      beqz t1, 2f
78      li   a4, YELLOW         # lamp on
79  2:  slli a0, s2, 3
80      add  a0, a0, s2         # 9 * i
81      addi a0, a0, 16         # x = 16 + 9 * i
```

```
82      mv   a1, s1
83      li   a2, 8
84      li   a3, 8
85      call prg32_gfx_rect
86      addi s2, s2, 1
87      li   t0, 32
88      blt  s2, t0, 1b
89      lw   s2, 16(sp)
90      lw   s1, 20(sp)
91      lw   s0, 24(sp)
92      lw   ra, 28(sp)
93      addi sp, sp, 32
94      ret
```

Listing 3.1. The complete program of the chapter, `code/l03.S`.

To run it on PRG32-QT the commands are those of Listing 3.2; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l03.S --portable \
    --entry-prefix l03 --name l03 --out l03.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l03.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 3.2. Building `l03.S` and running it on PRG32-QT.

3.7 Check your understanding

Q3.1 Which idea is the heart of the von Neumann architecture?

A. The processor has exactly four registers
B. Program and data are stored in the same memory
C. Every instruction takes one clock cycle
D. Input and output are optional

Q3.2 What makes RISC-V different from x86 and ARM?

A. It is the fastest instruction set
B. It was the first RISC architecture
C. Its instruction set is an open standard, free to implement
D. It works only on microcontrollers

Q3.3 Who is regarded as the first programmer in history?

A. Charles Babbage

B. Alan Turing
C. Ada Lovelace
D. Konrad Zuse

3.8 Exercises

Exercise 3.1 (paper first, then PRG32)

1. Draw the four units of the von Neumann architecture and write, inside each, one thing of the program of Chapter 2 that lives or happens there.
2. For each of the following, say whether it is a new way of *representing* information or a new way of *prescribing* operations: the abacus, the Jacquard loom, the Pascaline, Boole's algebra, the punched cards of the Analytical Engine, the stored program.
3. Download two of the four games of Figure 3.7 from the Store or build them from `examples/games`. With the counters of the debugger, find how many instructions each executes per frame, and compare the two games.

Expected result. Registers and ALU: `addi` was executed there. Memory: the counter, and the instructions themselves. Control unit: the fetch of each instruction. Input/output: the bar on the screen. The stored program belongs to both classes: a program is itself represented information.

3.9 Glossary

English	Italiano
computer science / informatics	informatica
computer	calcolatore, elaboratore
punched card	scheda perforata
loom	telaio
mainframe	mainframe, grande elaboratore
stored program	programma memorizzato
control unit	unità di controllo
arithmetic logic unit	unità aritmetico-logica
instruction set	insieme (set) di istruzioni
high-performance computing	calcolo ad alte prestazioni

3.10 Conclusions and reading

Computing is older than electronics: representation and procedure came first. The stored-program computer of 1945 is still the machine that is programmed today. RISC made processors simple enough to be understood whole; RISC-V made them open. Informatics is pervasive: digital information belongs to every field, data are its currency, and algorithms turn data into decisions.

Reading. Patterson and Hennessy (2020), chapter 1; *Playing with RISC-V*, chapter 1, "Why a Game, Why RISC-V, Why Now" (Montella et al., 2026b); Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*; for the history in depth, Ceruzzi (2003).

CHAPTER 4

Representing Digital Information

"printf("Hello, world!");"
— Brian Kernighan and Dennis Ritchie, *The C Programming Language*, 1978

In this chapter

By the end of this chapter you will be able to

- explain the difference between analog and digital information;
- read a pattern of bits as a natural number, in binary and in hexadecimal;
- decode a PRG32 colour such as `0x07E0` into its red, green and blue fields;
- say how characters and strings are stored in memory, in ASCII and in UTF-8;
- describe how an array lives in memory and how assembly reaches one element;
- tell a deterministic algorithm from a probabilistic one, and place machine learning among them.

In Chapter 2 the reader typed `0x07e0` and obtained green, typed 255 and obtained a bar that starts again, typed `.asciz` and obtained a greeting, typed `.word` and obtained a counter. This chapter gives those things their names. Its thesis fits in one sentence, which the rest of the chapter illustrates four times over: *everything in the machine is a pattern of bits, and the meaning of a pattern is a convention.* The two chapters that follow treat numbers formally.

4.1 Information

4.1.1 The flow of data

Every use of a computer has the same shape. *Data* enter from the world: a key, a sensor, a file. A *program*, the embodiment of an algorithm, transforms them. *Results* leave: a picture, a sound, a number. And somebody receives *knowledge*, that is, information on which it is possible to act. On PRG32 the four stages are, in order, the function `prg32_input_read`, the programmer's `update`, the programmer's `draw`, and the player who reacts to the screen.

The last stage has, until now, required a person. Artificial intelligence, to which the end of this chapter returns, is an intelligence of its own kind: powerful pattern recognition, without self-awareness and without subjective experience.

4.1.2 Analog and digital information

Definition 4.1 (Analog and digital)

An *analog* quantity varies continuously: a voltage, a sound pressure, the hands of a watch. A *digital* quantity is a sequence of symbols taken from a finite alphabet: the digits of a display.

To *digitise* an analog quantity means to do two things to it (Figure 4.1): to *sample* it in time, reading its value at regular instants, and to *quantise* it in value, replacing each reading by the nearest of a finite number of levels.

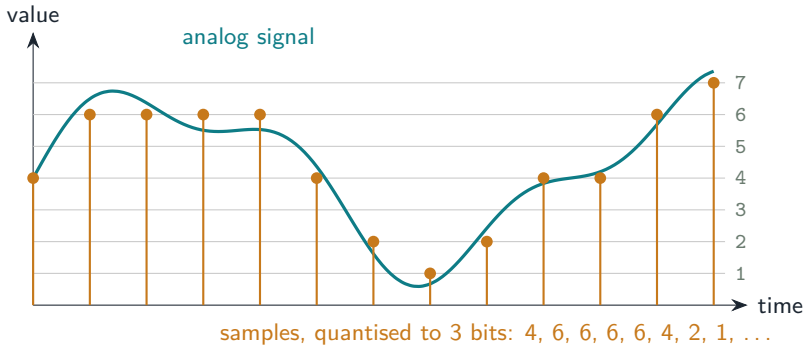


Figure 4.1. Digitising: sampling in time and quantising in value.

Something is lost in both steps, and something decisive is gained. A finite number of levels can be *counted*, *copied exactly* and *checked for errors*; a continuous quantity can only be measured, and every copy degrades it. With two levels the symbol is the *bit*, which is 0 or 1. Two levels are easy to build reliably with transistors, which are off or on (Chapter 10), and that is why computers are binary. Boolean algebra (Chapter 8) is the mathematics of two levels.

4.1.3 Bit, byte, word

Bits are handled in groups of standard sizes (Table 4.1). The fundamental fact about them is a count.

Theorem 4.1 (Number of patterns)

With n bits it is possible to write exactly 2^n distinct patterns.

Proof. By induction on n. One bit has $2 = 2^1$ patterns. If n bits have 2^n patterns, each of them can be followed by a 0 or by a 1, which gives $2 \cdot 2^n = 2^{n+1}$ patterns of $n + 1$ bits. □

Concept

The meaning of a pattern is a convention, not a property of the bits. What a pattern means is not written in it: it is an agreement between whoever writes it and whoever reads it.

Table 4.1. Units of information.

Unit	Size	Patterns	Where it was met
bit	1 binary digit	2	one button of the joystick
nibble	4 bits	16	one hexadecimal digit
byte	8 bits	256	one ASCII character; the width of the bar (`andi 255`)
halfword	16 bits	65 536	one RGB565 colour
word	32 bits	4 294 967 296	one RISC-V register; one instruction; `.word`
KiB, MiB, GiB	2^{10}, 2^{20}, 2^{30} bytes	–	memory sizes: 1 KiB = 1024 bytes

The size of a word depends on the architecture: on RV32 a word is 32 bits by definition.

4.1.4 Counting in base 2 and in base 16

Table 4.2 lists the sixteen patterns of four bits with their values in decimal and their hexadecimal digits. The four columns of a binary number weigh 8, 4, 2 and 1. This table should be learnt by heart: it is the multiplication table of this book.

Table 4.2. The sixteen patterns of four bits.

Dec	Binary	Hex	Dec	Binary	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

4.1.5 Positional notation: what `11011001` means

In a *positional* system the value of a digit depends on its position: each position weighs a power of the base. Base ten uses powers of ten, base two powers of two, base sixteen powers of sixteen. The rule is the same; only the base changes. The byte `11011001` is read as follows.

weight	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	128	64	32	16	8	4	2	1
bit	**1**	**1**	0	**1**	**1**	0	0	**1**
contributes	128	64	0	16	8	0	0	1

$$128 + 64 + 16 + 8 + 1 = 217$$

Chapter 5 gives the general formula and the algorithms of conversion. Here the aim is only to read a byte.

4.1.6 Why programmers write hexadecimal

One hexadecimal digit is exactly four bits, because $16 = 2^4$. Converting between binary and hexadecimal is therefore a table lookup, digit by digit, with no arithmetic:

$$\underbrace{0000}_{0}\ \underbrace{0111}_{7}\ \underbrace{1110}_{E}\ \underbrace{0000}_{0} \quad \Longrightarrow \quad \texttt{0x07E0}.$$

A 32-bit register is eight hexadecimal digits: `0x00130313` is readable, thirty-two zeros and ones are not. The prefix `0x` marks hexadecimal in assembly and in C, and the debugger of PRG32-QT shows the registers in this way.

The decimal value of `0x07E0` is $7 \cdot 256 + 14 \cdot 16 = 2016$, and it is of no use whatever: what matters in this number is the *bit pattern*, as the next section shows.

4.1.7 A colour is a number: RGB565

On PRG32 a colour is sixteen bits divided into three *fields*: five bits of red, six of green, five of blue (Figure 4.2). The format is called RGB565. The constant `.equ GREEN, 0x07e0` is the pattern in which green is at its maximum, 63 of 63, and red and blue are at zero. The eye is most sensitive to green, which is why green receives the extra bit.

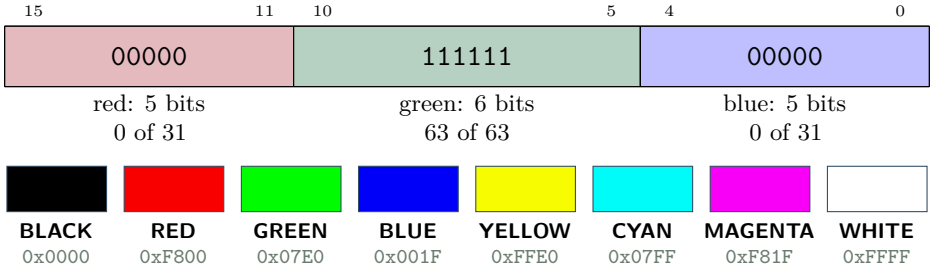


Figure 4.2. RGB565: `0x07E0` is green. Below, the colour constants of `prg32.h`.

Sixteen bits, three fields. The same sixteen bits could be a number, two characters or half an instruction: the meaning is in how the program uses them.

4.1.8 Numbers: naturals, integers, reals

Numbers themselves are represented by three different conventions, which the next two chapters develop.

- **Natural numbers** are the positional pattern itself: n bits cover the interval from 0 to $2^n - 1$.
- **Signed integers** are written in *two's complement*: a 32-bit register covers about -2 thousand million to $+2$ thousand million (Chapter 5).
- **Real numbers** are written in *fixed point* or in the *floating point* of the standard IEEE 754, where the value is $(-1)^s \times 1.f \times 2^{e-bias}$ (Chapter 6). Single precision uses 32 bits, gives about 7 decimal digits and reaches about 3.4×10^{38}; double precision uses 64 bits, gives about 16 digits and reaches about 1.8×10^{308}.

4.2 Text

Characters are numbers too.

4.2.1 ASCII: one character, one byte

The ASCII code (1963) assigns a number between 0 and 127, seven bits, to each of 128 characters: the letters of English, the digits, punctuation and some control codes (Table 4.3).

Table 4.3. Some ASCII characters.

Character	Decimal	Hex	Binary	Note
`A`	65	`0x41`	`0100 0001`	capital letters start at 65
`a`	97	`0x61`	`0110 0001`	the same as `A` with one more bit: $97 - 65 = 32$
`0`	48	`0x30`	`0011 0000`	the character zero is not the number zero
space	32	`0x20`	`0010 0000`	a character like any other
newline	10	`0x0A`	`0000 1010`	a control character
NUL	0	`0x00`	`0000 0000`	marks the end of a C string

The table was designed with care. Upper and lower case differ only in bit 5, so that `xori` with `0x20` switches the case of a letter: a first taste of Boolean algebra applied to text. And the digit characters are consecutive, so that subtracting 48 from the character of a digit gives the value of the digit.

4.2.2 Strings: a sequence of characters in memory

A *string* is a sequence of characters stored in consecutive bytes. The only question is how the reader of the string knows where it ends, and there are two classical answers (Figure 4.3). The *C string* marks the end with a byte of value zero. The *Pascal string* puts the length first.

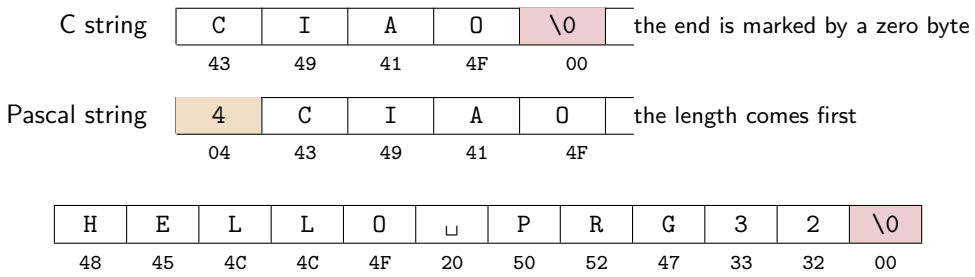


Figure 4.3. Strings in memory; under each cell, the ASCII code in hexadecimal. Below: `.asciz "HELLO PRG32"`.

The `z` of `.asciz` means *zero-terminated*: for the eleven characters of the string of Chapter 2 the assembler stores twelve bytes. The function `prg32_gfx_text8` receives in `a2` only the address of the first byte, and reads until it finds the zero. If the zero were missing, the function would go on reading whatever follows in memory.

4.2.3 Unicode and UTF-8: every writing system

One hundred and twenty-eight characters are enough for English and for nothing else. *Unicode* assigns a number, called a *code point*, to every character of every script; *UTF-8* stores that number in one to four bytes (Table 4.4). The encoding is compatible with ASCII: a plain ASCII text is already valid UTF-8. It is the encoding of the web and of the source files that the reader is writing.

Table 4.4. Unicode code points and their UTF-8 encoding.

Character	Code point	UTF-8 bytes	Length
A	U+0041	`41`	1 byte: identical to ASCII
è	U+00E8	`C3 A8`	2 bytes
€	U+20AC	`E2 82 AC`	3 bytes
the ideogram for “sun”, “day”	U+65E5	`E6 97 A5`	3 bytes
the “grinning face” emoji	U+1F600	`F0 9F 98 80`	4 bytes

Watch Out

In UTF-8 the length of a string in bytes is not its length in characters.

4.2.4 Byte order: little-endian

A word is four bytes, and memory is addressed byte by byte. In which order do the four bytes of a word go? The directive `.word 0x12345678` can be stored in two ways (Figure 4.4).

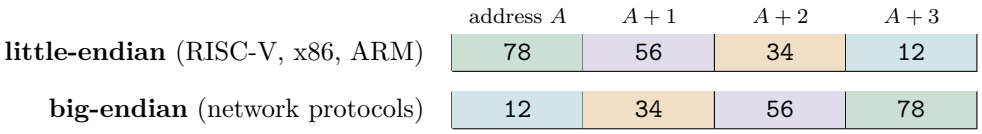


Figure 4.4. The two byte orders of the word `0x12345678`.

Definition 4.2 (Little-endian and big-endian)

In *little-endian* order the least significant byte of a word is stored at the lowest address; in *big-endian* order, the most significant byte.

RISC-V is little-endian: this is what `lw` and `sw` do. The byte order is noticed in two circumstances: when memory is examined in a debugger, and when data are exchanged with another machine. The names come from *Gulliver's Travels*, where two peoples go to war over the end from which an egg should be broken.

4.3 Data structures

From single values one passes to organised collections. Five ways of organising data recur in all of computing.

Array.
Elements of the same type, one after another, reached by index: 32, 11, 34, 22.

List.
Elements linked one to the next: cheap to insert and to remove, slow to reach by position.

Stack.
Last in, first out: *push* adds on top, *pop* removes from the top. The frame that saved `ra` in Chapter 2 was on a stack.

Dictionary.
Pairs of key and value: `NA` gives `NAPOLI`, `PA` gives `PALERMO`, `MI` gives `MILANO`.

Tree.
A hierarchy: an animal walks or swims, has lungs or gills, two legs or four.

In the machine all of them are the same thing: bytes in memory, plus a rule for finding the next element.

4.3.1 An array in memory, an element in a register

For the array the rule is arithmetic (Listing 4.1). An array is a base address followed by elements of equal size; element i of an array of words is at address $base + 4i$, because a word is four bytes.

```
.section .data
scores: .word 32, 11, 34, 22      # four 32-bit words, 16 bytes

.section .text
    la   t0, scores       # t0 = ADDRESS of element 0
    li   t1, 2            # index i = 2
    slli t1, t1, 2        # offset = i * 4 bytes (shift left by 2)
    add  t0, t0, t1       # ADDRESS of element i
    lw   a0, 0(t0)        # a0 = VALUE of scores[2] = 34
```

Listing 4.1. Reading `scores[2]`.

The instruction `slli` shifts the bits of a register to the left; a shift by two places multiplies by four, as a shift by one place in decimal multiplies by ten. (The property is proved in Chapter 5.) The expression `scores[i]` of a high-level language is exactly these four instructions.

Watch Out

What does `lw` read if $i = 4$? Whatever follows the array in memory. The machine does not check bounds: the programmer does.

4.4 Data and algorithms

The chapter closes by widening the view, from the bits of one program to the data of the world and to the procedures that work on them.

4.4.1 Data, big data and data science

Definition 4.3 (Big data)

Big data are data that exceed the limits of traditional database tools.

They are structured and unstructured: the economy, geolocation, time series, social networks. Their size is measured in petabytes and zettabytes, that is, thousands of millions of

terabytes. They come from four kinds of source: *sensor data* (the environment, GPS positions, the Internet of Things: every device measures something); *transactions* (purchases at the supermarket, credit cards, online orders); *public data* (administrative records, territory, consumption: the open data of institutions); and *social media* (preferences, politics, sentiment: what people write and share). *Data science* is the analysis of large quantities of information, and the technologies that extract knowledge and value from data are parallel computing and GPUs.

Definition 4.4 (Data mining)

Data mining is the set of techniques and methodologies for extracting information or knowledge from large quantities of data.

Among its techniques are decision trees and classification, rule learning and Bayesian methods, neural networks and pattern recognition.

4.4.2 GPGPU: general-purpose computing on graphics processors

A graphics processor (GPU) has thousands of simple cores built to do the same operation on many data at once. The sum of N items, which takes N steps one after another, needs only $\log_2 N$ parallel steps (Figure 4.5): the items are added in pairs, then the partial sums in pairs, and so on. The same hardware that draws video games now trains neural networks. Under every core there are still registers, instructions and bits; Chapter 24 returns to the subject.

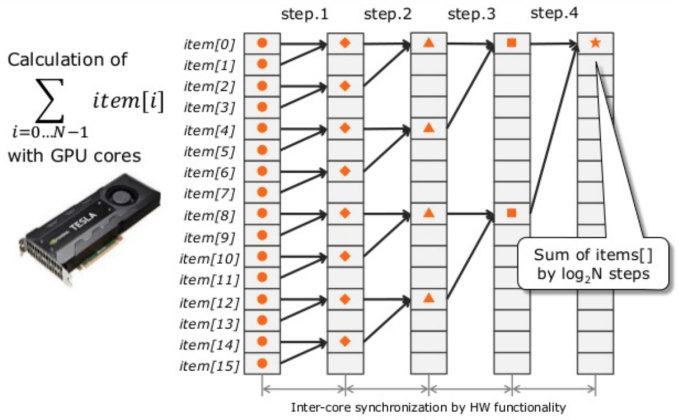


Figure 4.5. A parallel sum on the cores of a GPU: $\log_2 N$ steps.

4.4.3 Algorithms and computability

A problem is *computable* if it can be solved by an algorithm. The *Turing machine* is an ideal computer with a program, expressed mathematically through sets, relations and functions. The *von Neumann machine* is the computing architecture that is able to execute a program which realises an algorithm for the Turing machine. A problem that no algorithm on a Turing machine can solve cannot be solved by any computer: this statement, in its informal version, is the Church–Turing thesis. Not every problem has an algorithm; the classic example of a problem that is not computable is the *halting problem*, that of deciding whether an arbitrary program will eventually stop.

Definition 4.5 (Algorithm)

An algorithm is a procedure with five properties. It is **atomic**: its steps are elementary and cannot be broken down any further. It is **unambiguous**: each step is interpreted in one direct way by the executor, human or machine. It is **finite**: it has a finite number of steps and a finite quantity of input data. It is **terminating**: its execution ends after a finite time. It is **effective**: its execution leads to a single, well-defined result.

A RISC-V instruction is the model of an atomic and unambiguous step: its meaning is fixed by the specification, bit by bit.

4.4.4 Deterministic and non-deterministic algorithms

An algorithm is *deterministic* if for every instruction there is, with the same input, exactly one next step: the path of the algorithm exists and is unique, $y = f(x)$. Converting a unit of measure is deterministic; so is `first_update`, which always adds one.

An algorithm is *non-deterministic* if, with the same input, it can behave differently at each execution. *Probabilistic* algorithms use a generator of random numbers; *concurrent* algorithms can depend on timing (the so-called race conditions). A game that places an enemy by calling `prg32_random_number` is non-deterministic.

Listing 4.2 shows two probabilistic algorithms in pseudo-code. The first tests whether a number is prime: the answer "composite" is always right, the answer "probably prime" is right with overwhelming probability. Trading a tiny chance of error for a huge gain in speed is how real cryptography tests primes. (The Fermat test shown is the simplest; real libraries use the Miller–Rabin test, which closes its loopholes.) The second is a random walk, which gives a different result at every run: simulations average many of them.

```
is_probably_prime(n):
    repeat 30 times:
        a = random(2, n - 2)
        if a^(n-1) mod n != 1:
            return COMPOSITE          # certain
    return PROBABLY PRIME             # almost certain

random_walk():
    while t < t_final:
        r = random(-1, 1)
        x = x + r * u                 # a different path every run
        y = y + r * v
        t = t + 1
```

Listing 4.2. Two probabilistic algorithms.

4.4.5 Machine learning

Machine learning is a method of data analysis that automates the construction of an analytical model: the computer finds hidden patterns without being explicitly programmed where to look. In a classical program the function $y = f(x)$ is written by a person; in machine learning, f is learned from examples. Today's generative models are the same idea at an

enormous scale: a large language model predicts the next piece of text from thousands of millions of learned numbers.

Those numbers are bits in memory, multiplied and added by processors: the subject of this book. And a model can be wrong with great confidence, so that the fundamentals are needed to check it. This is the reason for the rule stated in Chapter 0: the examination verifies that the fundamentals are in the student's head.

4.5 The program of this chapter

The listings of this chapter are parts of one program, `code/l04.S` in the repository of the course, printed in full in Listing 4.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 4.6 is the frame that the test wrote.

On the screen.
Four horizontal bars, one per element of the array (32, 11, 34, 22, four pixels per unit); element 2, the one the listing loads, is yellow.

What to press.
Nothing: the program runs by itself.



Figure 4.6. `l04.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 04: Representing Digital Information
# l04.S: an array in memory, an element in a register
#
# On the screen: four horizontal bars, one per element of the array (32, 11,
    34, 22,
# four pixels per unit); element 2, the one the listing loads, is yellow.
# Build: python3 -m prg32 cartridge build l04.S --portable \
#            --entry-prefix l04 --name l04 --out l04.prg32
.option norelax

.equ BLACK,  0x0000
.equ WHITE,  0xffff
.equ YELLOW, 0xffe0

.section .text
.global l04_init
.global l04_update
.global l04_draw
```

```
18
19 l04_init:
20     ret
21
22 l04_update:
23     ret
24
25 l04_draw:
26     addi sp, sp, -16
27     sw   ra, 12(sp)
28     sw   s0, 8(sp)
29     li   a0, BLACK
30     call prg32_gfx_clear
31     li   s0, 0              # index i, 0 to 3
32 1:  la   t0, scores         # t0 = ADDRESS of element 0
33     mv   t1, s0             # index i
34     slli t1, t1, 2          # offset = i * 4 bytes (shift left by 2)
35     add  t0, t0, t1         # ADDRESS of element i
36     lw   a2, 0(t0)          # VALUE of scores[i]
37     slli a2, a2, 2          # width: four pixels per unit
38     li   a4, WHITE
39     li   t0, 2
40     bne  s0, t0, 2f
41     li   a4, YELLOW         # scores[2] = 34: the element of the slide
42 2:  li   a0, 20             # x
43     slli a1, s0, 5
44     addi a1, a1, 30         # y = 30 + 32 * i
45     li   a3, 16             # height
46     call prg32_gfx_rect
47     addi s0, s0, 1
48     li   t0, 4
49     blt  s0, t0, 1b
50     lw   s0, 8(sp)
51     lw   ra, 12(sp)
52     addi sp, sp, 16
53     ret
54
55 .section .data
56 scores: .word 32, 11, 34, 22     # four 32-bit words, 16 bytes
```

Listing 4.3. The complete program of the chapter, `code/l04.S`.

To run it on PRG32-QT the commands are those of Listing 4.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l04.S --portable \
    --entry-prefix l04 --name l04 --out l04.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l04.prg32 \
    --url http://localhost:8080 --slot cart0
```

```
# 4. look, press, compare with the previous slide
```

Listing 4.4. Building `104.S` and running it on PRG32-QT.

4.6 Check your understanding

Q4.1 How many different values can one byte represent?

A. 8
B. 16
C. 255
D. 256

Q4.2 The PRG32 colour `0xF81F` is:

A. red
B. magenta
C. cyan
D. yellow

Q4.3 How many bytes does `.asciz "CIAO"` occupy in memory?

A. 3
B. 4
C. 5
D. 8

Q4.4 On RISC-V, `.word 0x0000ABCD` is stored at address A. Which byte is at address A?

A. `0x00`
B. `0xAB`
C. `0xCD`
D. `0xDC`

4.7 Exercises

Exercise 4.1 (on PRG32)

1. Change the bar of Chapter 2 to magenta, then to a dark grey of your choice: compute the RGB565 value by hand first.
2. Print your name with `prg32_gfx_text8`. How many bytes does the string occupy?
3. Declare an array of four colours with `.word` and draw four bars, one per colour, reading each colour with `lw`.
4. In PRG32-QT open the memory view on your string and on your array: find the terminating zero and check the byte order.

Expected result. Four bars in four colours; in the memory view the string ends with `00` and each `.word` shows its least significant byte first. A dark grey, for comparison: red 8 of 31, green 16 of 63, blue 8 of 31 is `01000 010000 01000`, that is, `0x4208`.

4.8 Glossary

English	**Italiano**
bit / byte / word	bit / byte / parola
positional notation	notazione posizionale
hexadecimal	esadecimale
sampling	campionamento
quantisation	quantizzazione
byte order (endianness)	ordinamento dei byte
string	stringa
array	vettore (array)
linked list	lista concatenata
stack	pila
tree	albero
machine learning	apprendimento automatico

4.9 Conclusions and reading

Digital information is a pattern of bits plus a convention for reading it. The same sixteen bits can be a number, a colour or two characters: the program decides. Hexadecimal is a compact spelling of binary, four bits per digit. Text is numbers (ASCII, UTF-8); structures are bytes plus a rule for finding the next element. Data are the currency of this realm, and algorithms turn data into information fit for decisions.

Reading. Patterson and Hennessy (2020), sections 2.4 and 2.9; Charles Petzold, *Code*, the chapters on bits, bytes and text; for machine learning, Sebastian Raschka, *Machine Learning with Python*.

CHAPTER 5

Number Representation: Integers

"I may count for little, it's true, said the One to the Zero, but what are you worth? Nothing: nothing at all."

— Trilussa, *Nummeri*, 1944

In this chapter

By the end of this chapter you will be able to

- convert integers and fractions between bases 2, 8, 10 and 16;
- say which natural numbers fit in n bits and recognise an overflow;
- encode and decode signed integers in sign and magnitude and in two's complement;
- add, subtract and negate in two's complement, and detect overflow;
- explain why $0 - 1$ shows as `0xFFFFFFFF` in a RISC-V register;
- choose the right RISC-V instruction for signed and for unsigned data.

5.1 Warm-up on the machine

A marker stands on the screen, at position 2 of a horizontal line. The program reads the joystick and, while **LEFT** is pressed, subtracts one from the position at every frame (Listing 5.1); the function `draw` prints the position in hexadecimal with `prg32_console_hex32`.

```
steer_update:
    ...
    call prg32_input_read      # a0 = button mask
    mv   t0, a0

    la   t1, marker_x
    lw   t2, 0(t1)             # t2 = marker_x, starts at 2
    andi t3, t0, BTN_LEFT      # is LEFT pressed?
    beqz t3, 1f
    addi t2, t2, -1            # one step to the left
1:  sw   t2, 0(t1)
    ...
# draw prints marker_x with prg32_console_hex32
```

Listing 5.1. Hold **LEFT** and watch the number (adapted from the laboratory on state and input of the PRG32 textbook).

Try It Yourself

Hold **LEFT**. The value shown goes `00000002`, `00000001`, `00000000` … and then `FFFFFFFF`, and the marker vanishes. (On the board and in QEMU the console is drawn on the screen; PRG32-QT collects it in its log.) Is it a bug? Is `0xFFFFFFFF` the number 4 294 967 295 or the number -1?

The answer is: both. It depends on how the bits are read, and this chapter gives the rules.

5.2 Bases and conversions

5.2.1 Positional systems

Definition 5.1 (Positional representation)

Let $b \geq 2$ be an integer, the *base*. A string of digits $d_{n-1}\, d_{n-2} \cdots d_1\, d_0$, with $0 \leq d_i \leq b-1$ for every i, represents in base b the natural number

$$N \;=\; \sum_{i=0}^{n-1} d_i\, b^{\,i} \;=\; d_{n-1} b^{n-1} + \cdots + d_1 b + d_0. \tag{5.1}$$

The digit d_{n-1} is the *most significant*, d_0 the *least significant*.

Four bases are used in computing (Table 5.1). Every natural number has exactly one representation in each base, apart from zeros added on the left.

Table 5.1. The four bases of computing.

Base b	Digits	Name	Thirteen is written
2	0, 1	binary	`1101`
8	0, 1, …, 7	octal	`15`
10	0, 1, …, 9	decimal	`13`
16	0, …, 9, A, …, F	hexadecimal	`D`

5.2.2 Bases that are powers of two share the same bits

If each digit is written in pure binary, the bits of a number are the same in every base $b = 2^k$: it is enough to group them from the right, k at a time, padding with zeros on the left. For the number 127:

$$\texttt{01111111}_2 \;=\; \underbrace{\texttt{0111}}_{7}\,\underbrace{\texttt{1111}}_{F} \;=\; \texttt{0x7F} \qquad \text{and} \qquad \texttt{01111111}_2 \;=\; \underbrace{\texttt{01}}_{1}\,\underbrace{\texttt{111}}_{7}\,\underbrace{\texttt{111}}_{7} \;=\; 177_8.$$

The reason is that in (5.1) with $b = 2$ the terms can be collected k at a time, and each group is a digit multiplied by a power of 2^k.

5.2.3 Converting an integer: repeated division

To write a natural number N in base r: divide N by r; the remainder is the next digit, starting from the least significant; repeat on the quotient until it is zero. The remainders are then read from the last to the first (Table 5.2).

Table 5.2. The number 127 converted by repeated division.

$r = 8$

N	$N/8$	$N \bmod 8$
127	15	7
15	1	7
1	0	1

$127 = 177_8$

$r = 16$

N	$N/16$	$N \bmod 16$
127	7	$15 = \mathrm{F}$
7	0	7

$127 = \texttt{0x7F}$

$r = 2$

N	$N/2$	$N \bmod 2$
127	63	1
63	31	1
31	15	1
15	7	1
7	3	1
3	1	1
1	0	1

$127 = 1111111_2$

The method is correct because, by (5.1), $N = r\,(d_{n-1}r^{\,n-2} + \cdots + d_1) + d_0$: the remainder of the division of N by r is d_0, and the quotient is the number whose digits are $d_{n-1}\cdots d_1$, to which the same argument applies. In C the two operations are `/` and `%`; in RISC-V, with the M extension, they are `divu` and `remu`, and Chapter 22 uses exactly this algorithm to print a score.

5.2.4 Converting a fraction: repeated multiplication

A number $0 \le N < 1$ is written in base r as $N = d_{-1}r^{-1} + d_{-2}r^{-2} + \cdots$. Multiplying by r gives $d_{-1} + (d_{-2}r^{-1} + \cdots)$: the integer part of the product is the next digit, starting from the most significant, and the procedure is repeated on the fractional part (Table 5.3).

Table 5.3. Two fractions converted to base 2.

0.875: exact

Multiply	Fraction	Digit
0.875×2	0.75	1
0.75×2	0.5	1
0.5×2	0	1

$0.875 = 0.111_2$

0.8: periodic

Multiply	Fraction	Digit
0.8×2	0.6	1
0.6×2	0.2	1
0.2×2	0.4	0
0.4×2	0.8	0
0.8×2	0.6	1 …

$0.8 = 0.\overline{1100}_2$

Watch Out

A fraction that is finite in base 10 can be periodic in base 2. With a finite number of bits it must be cut: this is where the approximation errors of real numbers come from (Chapter 6).

5.3 Natural numbers and overflow

5.3.1 Encoding strategies

Definition 5.2 (Representation)

A *representation* is a law $X = R(x)$ that associates a number x with a string of bits X. Here x is the number that is meant; X is the pattern stored in the register.

The number of bits depends on how many numbers are to be represented; it is usually a multiple of 8, typically 16, 32 or 64. The law R is chosen according to the kind of number, natural, signed or rational, and a good law is one that makes arithmetic operations and comparisons simple to build as circuits.

5.3.2 Natural numbers on n bits

With n bits there are 2^n patterns, and the natural choice is to associate each natural number from 0 to $2^n - 1$ with the string of bits that represents it in the binary positional system. With $n = 4$ the interval is $[0, 15]$. The sizes handled by RISC-V are in Table 5.4.

Table 5.4. Natural numbers on 8, 16 and 32 bits.

Bits	Range	In RISC-V	Maximum
8	$0 \ldots 2^8 - 1$	byte (`lbu`)	255
16	$0 \ldots 2^{16} - 1$	halfword (`lhu`)	65 535
32	$0 \ldots 2^{32} - 1$	word, register	4 294 967 295

The number 4 294 967 295 is the reading of `0xFFFFFFFF` as a natural number: the first half of the answer to the warm-up.

5.3.3 Properties of binary strings

Three properties follow at once from (5.1).

1. *Extension on the left with zeros does not change the value*: $5 = 101_2 = 00101_2$.
2. *Multiplying by 2^k is a shift to the left by k positions*: $00101_2 \times 2 = 01010_2$, five becomes ten. Each digit d_i moves to position $i + k$, where it weighs 2^k times as much.
3. *Integer division by 2^k is a shift to the right by k positions*: $00101_2/2 = 00010_2$, five becomes two, and the remainder is lost.

Shifts are much cheaper than multiplications, and compilers use them whenever they can (Listing 5.2). The instruction `slli t1, t1, 2` of Chapter 4 multiplied an index by four in this way. Shifts are also the basis of fixed-point arithmetic (Chapter 6).

```
# multiply by 4
slli t0, t0, 2

# divide by 2 (unsigned)
srli t0, t0, 1

```

```
# x * 10 = x*8 + x*2
slli t1, t0, 3
slli t2, t0, 1
add  t0, t1, t2
```

Listing 5.2. Shifts as multiplications and divisions.

5.3.4 Overflow

Definition 5.3 (Overflow)

An *overflow* occurs when the result of an operation is not contained in the finite subset of numbers that the available bits can represent.

Registers have a finite size, and there is a finite number of them. The sets of numbers are infinite, and only a finite subset of each can be represented. The arithmetic operators are closed on the whole set, not on the finite subset. With 7 bits the interval $[0, 127]$ is encoded: 100 and 38 are representable, their sum 138 is not.

5.3.5 Arithmetic on four bits

Binary arithmetic follows the same column-by-column rules as decimal arithmetic, with the single table $0+0=0$, $0+1=1$, $1+1=0$ with a carry of 1.

Sum		Sum with overflow		Difference		Product	
6 +	0110 +	14 +	1110 +	11 −	1011 −	5 x	0101 x
8 =	1000 =	3 =	0011 =	5 =	0101 =	3 =	0011 =
14	1110	17	1 0001	6	0110	15	000 1111

In the second sum a fifth bit is needed: the four-bit register keeps `0001` and the carry is lost. Processors signal a lost carry in different ways: many have a *carry flag*; RISC-V has none and lets the program compute it with a comparison, as Section 5.5 shows. A product of two numbers of n bits needs up to $2n$ bits: on RV32 the instruction `mul` gives the low 32 bits and `mulhu` the high 32.

5.4 Signed integers

There are three ways of writing a minus sign with bits. Two are presented here and the third, with a variant, in Chapter 6.

5.4.1 Sign and magnitude

The most significant bit is the sign (1 for negative) and the other $n-1$ bits are the magnitude:

$$X = |x| + 2^{n-1}s, \qquad s = \begin{cases} 1 & x < 0 \\ 0 & x \geq 0. \end{cases}$$

The range goes from $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$. Zero has two representations, $+0$ and -0, so that only $2^n - 1$ numbers are represented. And arithmetic must work separately on sign and magnitude: to add two numbers of opposite sign one must find the larger magnitude,

subtract the smaller from it, and give the result the sign of the operand with the larger magnitude. The representation is simple for people and awkward for circuits. It survives in the mantissa of floating-point numbers (Chapter 6).

5.4.2 Two's complement

Definition 5.4 (Two's complement)

On n bits, the *two's complement* representation of an integer x with $-2^{n-1} \le x \le 2^{n-1}-1$ is the natural number

$$X = \begin{cases} x & \text{if } 0 \le x < 2^{n-1}, \\ 2^n - |x| & \text{if } -2^{n-1} \le x < 0, \end{cases}$$

written in binary on n bits. Equivalently, $X = x \bmod 2^n$.

The range goes from -2^{n-1} to $+2^{n-1}-1$, and zero has a single representation. The most significant bit is 1 if and only if the number is negative, because the negative numbers occupy the patterns from 2^{n-1} to 2^n-1. This is the representation used by every modern processor, RISC-V included: a 32-bit register read as a signed integer covers $-2^{31} \ldots 2^{31}-1$, about minus two thousand million to plus two thousand million. The range is asymmetric, with one more negative number than positive ones, and this has a consequence that will appear shortly.

5.4.3 Four bits, three readings

Table 5.5 reads each of the sixteen patterns of four bits in the three ways seen so far. When the most significant bit is 0 the three readings agree; when it is 1 they differ.

Table 5.5. Four bits, three readings.

Bits	Unsigned	Sign-mag.	Two's c.	Bits	Unsigned	Sign-mag.	Two's c.
0000	0	+0	0	1000	8	−0	−8
0001	1	+1	+1	1001	9	−1	−7
0010	2	+2	+2	1010	10	−2	−6
0011	3	+3	+3	1011	11	−3	−5
0100	4	+4	+4	1100	12	−4	−4
0101	5	+5	+5	1101	13	−5	−3
0110	6	+6	+6	1110	14	−6	−2
0111	7	+7	+7	1111	15	−7	−1

Concept

The bits do not say which column is the right one. The program does, by choosing its instructions.

5.4.4 The number wheel

With n bits arithmetic is *modulo* 2^n: the number line closes into a circle (Figure 5.1). Adding 1 moves one step clockwise, whatever the reading. The two readings differ only in the place where the wheel "breaks". In the unsigned reading it breaks between `1111` and `0000`, and crossing that point is a carry: $15+1=0$. In the two's complement reading it breaks between `0111` and `1000`, and crossing that point is a signed overflow: $7+1=-8$. One step back from `0000` is `1111`: that is why $0-1$ gives all ones.

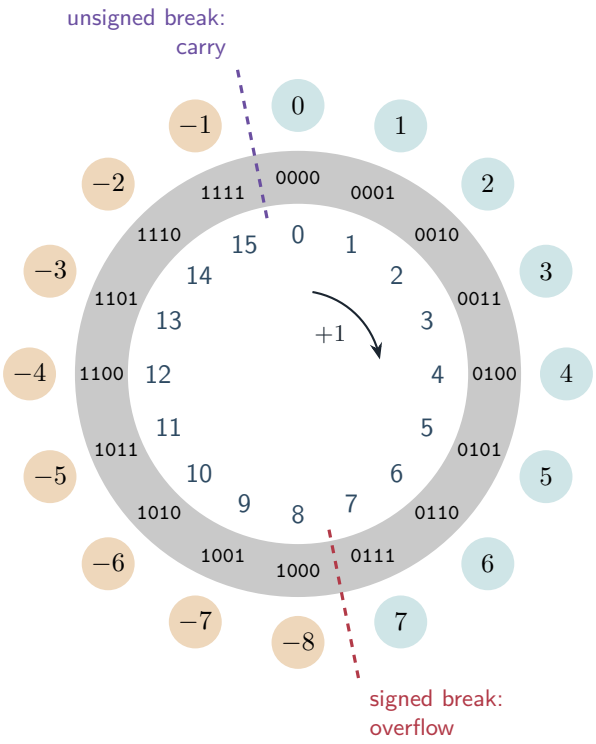


Figure 5.1. The number wheel for four bits. Inner numbers: unsigned value; ring: bits; outer discs: two's complement value.

The same adder serves both readings; only the place where the wheel breaks changes.

5.4.5 Two's complement arithmetic

Theorem 5.1 (Addition in two's complement)

Let x, y and $x+y$ all lie in $[-2^{n-1}, 2^{n-1}-1]$, and let X, Y be the representations of x and y. Then the representation of $x+y$ is $(X+Y) \bmod 2^n$: the sum of the two patterns as natural numbers, with the carry out of the most significant bit discarded.

Proof. $X \equiv x$ and $Y \equiv y \pmod{2^n}$ by definition, hence $X+Y \equiv x+y \pmod{2^n}$; and the representation of $x+y$ is, by definition, $(x+y) \bmod 2^n$. □

Addition therefore needs no knowledge of the signs: the adder for natural numbers already gives the right answer.

Opposite signs		Two negatives		Product	
2 +	0010 +	-2 +	1110 +	2 x	0010 x
-6 =	1010 =	-3 =	1101 =	-3 =	1101 =
-4	1100	-5	1 1011	-6	001 1010

The same argument holds for multiplication: $XY \equiv xy \pmod{2^n}$, so the low n bits of a product are correct whatever the signs. In the third example $2 \times 13 = 26 = 1\,1010_2$, and the low four bits `1010` read -6. This is why RISC-V has a single `mul` for signed and unsigned operands.

5.4.6 Negating a number

By definition $-x$ is represented by $2^n - X$. Two methods compute it without a subtraction.

Method 1: complement every bit, then add 1.
$6 =$ `0110`, complement `1001`, plus one: `1010` $= -6$. It works because the complement of every bit of X is the number $(2^n - 1) - X$, a subtraction from all ones in which no borrow can occur; adding 1 gives $2^n - X$.

Method 2: scan from the right.
Starting from the right, copy the bits up to and including the first 1, and complement all the others. $5 =$ `0101` gives `1011` $= -5$; $1 =$ `0001` gives `1111` $= -1$. It is method 1 with the addition carried out mentally: the final $+1$ propagates through the trailing ones of the complement, which are the trailing zeros of the original.

Negating twice returns the original number, with one exception: the most negative number, -2^{n-1}, has no positive partner, and its negation overflows and returns the number itself.

```
# neg t0, t1 is a pseudo-instruction:
sub  t0, zero, t1

# the same thing by hand:
not  t0, t1        # complement
addi t0, t0, 1     # plus one
```

Listing 5.3. Negation in RISC-V.

Subtraction is addition

$y - x = y + (2^n - x) \pmod{2^n}$. An adder and a complementer are the only components needed to build addition, subtraction, multiplication and division.

This is the engineering reason for two's complement: one circuit, the adder of Chapter 11, does all the work. In the RISC-V datapath `sub` is `add` with the second operand complemented and a carry-in of 1.

5.4.7 Detecting overflow

Unsigned, 8 bits		Signed, 8 bits	
128 +	1000 0000 +	127 +	0111 1111 +
130 =	1000 0010 =	127 =	0111 1111 =
258	1 0000 0010	254	1111 1110

In the first sum there is a carry out, and the register holds 2, the sum modulo 256. In the second there is no carry out, but the result reads -2.

Theorem 5.2 (Overflow conditions)

In an addition on n bits:

1. an *unsigned* overflow occurs if and only if there is a carry out of the most significant bit;
2. a *signed* overflow occurs if and only if two operands with the same sign give a sum with the opposite sign: two positives give a negative, or two negatives give a positive.

The sum of two numbers of opposite sign never overflows.

The last statement holds because such a sum lies between the two operands, both of which are representable. The two conditions are independent: the same addition can trigger one, both or neither. In terms of circuits the signed condition is equivalent to this: the carry *into* the most significant bit differs from the carry *out* of it.

5.5 Back to the machine

5.5.1 The warm-up, explained

The variable `marker_x` held 0. The instruction `addi t2, t2, -1` moved one step back on a wheel with 2^{32} positions, and the register now holds thirty-two ones: `0xFFFFFFFF`. Read as unsigned it is 4 294 967 295; read in two's complement it is -1. The adder did the same thing in both cases. The function `prg32_console_hex32` prints the pattern, not an interpretation: it is neither right nor wrong. And the marker disappears because `prg32_gfx_rect` is asked to draw at an enormous x, or at -1: off the screen either way (Figure 5.2). It is not a bug of the processor; it is a missing decision in the program.

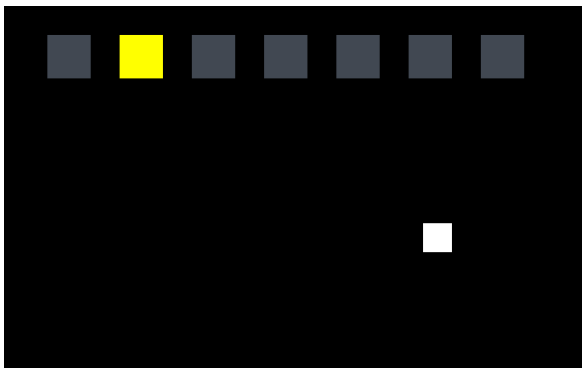


Figure 5.2. The laboratory program: seven lamps for the bits of the buttons and a marker.

5.5.2 Signed or unsigned? The instruction chooses

RISC-V registers have no type. The same 32 bits are signed or unsigned only in the eye of the instruction that reads them (Table 5.6).

Table 5.6. Signed and unsigned instructions.

Operation	Signed	Unsigned	Why they differ
add, subtract	`add`, `addi`, `sub`	the same	two's complement: one adder for both
multiply (low word)	`mul`	the same	the low 32 bits do not depend on the sign
compare	`slt`, `slti`	`sltu`, `sltiu`	is `0xFFFFFFFF` less than 0? yes if signed
branch	`blt`, `bge`	`bltu`, `bgeu`	the same question, as a jump
shift right	`sra`, `srai`	`srl`, `srli`	copy the sign bit, or insert zeros
load a byte	`lb`	`lbu`	sign extension or zero extension
divide, remainder	`div`, `rem`	`divu`, `remu`	magnitudes are read differently

5.5.3 Keeping the marker on the screen

The decision that was missing is taken in Listing 5.4: after moving, the position is *clamped* to the interval $[0, 304]$.

```
    andi t3, t0, BTN_LEFT
    beqz t3, 1f
    addi t2, t2, -SPEED
1:  andi t3, t0, BTN_RIGHT
    beqz t3, 2f
    addi t2, t2, SPEED
2:
    bgez t2, 3f             # x < 0 ?  clamp to 0
    li   t2, 0
3:  li   t3, MAX_X          # .equ MAX_X, 304
    ble  t2, t3, 4f         # x > 304 ?  clamp to 304
    mv   t2, t3
4:
    sw   t2, 0(t1)          # one store, after all adjustments
```

Listing 5.4. Clamping the position (laboratory on state and input, step 3).

`bgez` reads `t2` as a signed number: it tests the sign bit. `ble` is a signed comparison too: -1 is less than 304, whereas with `bleu` the pattern `0xFFFFFFFF` would be far greater. The constant `-SPEED` is an immediate: twelve bits in two's complement, from -2048 to $+2047$.

Concept

A game does not detect overflow after the fact: it keeps its values inside a range where overflow cannot happen.

5.5.4 Detecting overflow without flags

RISC-V has no carry flag and no overflow flag: fewer hidden states, a simpler processor. When a program does need to know, it asks with a comparison (Listing 5.5). For unsigned

numbers, a sum that has wrapped round is always smaller than either operand. For signed numbers, adding a negative number must make the sum smaller, and adding a non-negative one must not. On the wheel, `sltu` detects the crossing of the top and the signed test the crossing of the bottom.

```
# unsigned: did the sum wrap around?
    add  t2, t0, t1        # t2 = t0 + t1   (modulo 2^32)
    sltu t3, t2, t0        # t3 = 1 if the sum is smaller than
                           #        an operand: a carry was lost

# signed: did the sign come out wrong?
    add  t2, t0, t1
    slti t3, t1, 0         # t3 = 1 if t1 is negative
    slt  t4, t2, t0        # t4 = 1 if the sum is less than t0
    bne  t3, t4, overflow  # they must agree
```

Listing 5.5. The two overflow tests suggested by the RISC-V specification.

5.5.5 Sign extension: the same number on more bits

Theorem 5.3 (Sign extension)

A number in two's complement on n bits is written on $m > n$ bits by copying its sign bit into all the new positions on the left.

Proof. If $x \geq 0$ the sign bit is 0 and the statement is the extension with zeros of natural numbers. If $x < 0$ its pattern on n bits is $X = 2^n - |x|$ and the pattern wanted is $2^m - |x| = X + (2^m - 2^n)$; the number $2^m - 2^n$ is exactly $m - n$ ones followed by n zeros. □

-1 on 4 bits	`1111`	`= -1`
-1 on 8 bits	`1111 1111`	`= -1`
$+5$ on 4 bits	`0101`	`= +5`
$+5$ on 8 bits	`0000 0101`	`= +5`

Extending with zeros would turn `1111` (-1) into `0000 1111` ($+15$). The instruction `addi t2, t2, -1` stores -1 in twelve bits, and the processor sign-extends it to thirty-two before adding. `lb` sign-extends the byte that it loads, `lbu` fills with zeros: the choice depends on what the byte means.

5.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l05.S` in the repository of the course, printed in full in Listing 5.6. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 5.3 is the frame that the test wrote.

On the screen.

The 32 bits of marker_x as lamps. It starts at 2 (one lamp); each frame with LEFT held subtracts 1: after 2, 1 and 0 every lamp lights up, 0xFFFFFFFF, which is -1.

What to press.
Hold LEFT for a few frames.

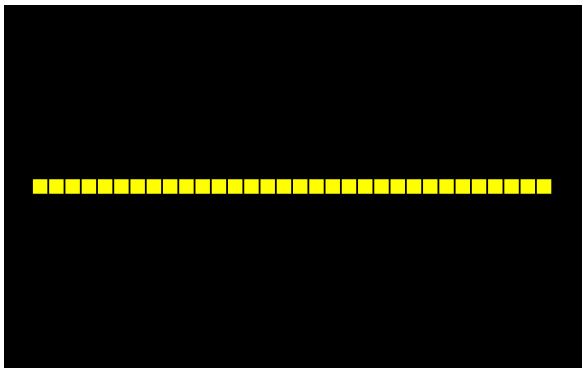


Figure 5.3. `l05.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 05: Number Representation: Integers
# l05.S: hold LEFT and watch a number go below zero
#
# On the screen: the 32 bits of marker_x as lamps. It starts at 2 (one lamp); each
# frame with LEFT held subtracts 1: after 2, 1 and 0 every lamp lights up, 0xFFFFFFFF,
# which is -1.
# Build: python3 -m prg32 cartridge build l05.S --portable \
#            --entry-prefix l05 --name l05 --out l05.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ BTN_LEFT, 1

.section .text
.global l05_init
.global l05_update
.global l05_draw

l05_init:
    ret

l05_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read      # a0 = button mask
    mv   t0, a0

    la   t1, marker_x
    lw   t2, 0(t1)             # t2 = marker_x, starts at 2
    andi t3, t0, BTN_LEFT      # is LEFT pressed?
    beqz t3, 1f
    addi t2, t2, -1            # one step to the left
1:  sw   t2, 0(t1)
    lw   ra, 12(sp)
```

```
37      addi sp, sp, 16
38      ret
39
40  l05_draw:
41      addi sp, sp, -16
42      sw   ra, 12(sp)
43      li   a0, BLACK
44      call prg32_gfx_clear
45      call prg32_console_clear   # the number in hexadecimal, on the console
46      la   t0, marker_x          # (the board and QEMU show it on the screen,
47      lw   a0, 0(t0)             #  PRG32-QT in its log)
48      call prg32_console_hex32
49      la   t0, marker_x          # and the same number, bit by bit
50      lw   a0, 0(t0)
51      li   a1, 96
52      call lamps32
53      lw   ra, 12(sp)
54      addi sp, sp, 16
55      ret
56
57  # lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
58  #   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
59  lamps32:
60      addi sp, sp, -32
61      sw   ra, 28(sp)
62      sw   s0, 24(sp)
63      sw   s1, 20(sp)
64      sw   s2, 16(sp)
65      mv   s0, a0              # the word
66      mv   s1, a1              # y
67      li   s2, 0               # lamp number, 0 on the left
68  1:  li   t0, 31
69      sub  t0, t0, s2          # lamp i shows bit 31 - i
70      srl  t1, s0, t0          # bring that bit down to bit 0
71      andi t1, t1, 1           # keep it, clear all the others
72      li   a4, GREY            # lamp off
73      beqz t1, 2f
74      li   a4, YELLOW          # lamp on
75  2:  slli a0, s2, 3
76      add  a0, a0, s2          # 9 * i
77      addi a0, a0, 16          # x = 16 + 9 * i
78      mv   a1, s1
79      li   a2, 8
80      li   a3, 8
81      call prg32_gfx_rect
82      addi s2, s2, 1
83      li   t0, 32
84      blt  s2, t0, 1b
85      lw   s2, 16(sp)
86      lw   s1, 20(sp)
87      lw   s0, 24(sp)
88      lw   ra, 28(sp)
89      addi sp, sp, 32
90      ret
91
92  .section .data
```

```
93 marker_x: .word 2
```

Listing 5.6. The complete program of the chapter, `code/l05.S`.

To run it on PRG32-QT the commands are those of Listing 5.7; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --out l05.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l05.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 5.7. Building `l05.S` and running it on PRG32-QT.

5.7 Check your understanding

Q5.1 Which number does the 8-bit pattern `1111 1100` represent in two's complement?

A. 252
B. -4
C. -3
D. -124

Q5.2 On 8 bits, which addition produces a signed overflow?

A. $100 + 27$
B. $100 + 28$
C. $-100 + 28$
D. $-1 + 1$

Q5.3 `t0` holds `0xFFFFFFFF` and `t1` holds 1. After `slt t2, t0, t1` and `sltu t3, t0, t1`:

A. `t2` $= 1$, `t3` $= 1$
B. `t2` $= 0$, `t3` $= 0$
C. `t2` $= 1$, `t3` $= 0$
D. `t2` $= 0$, `t3` $= 1$

Q5.4 How is 0.625 written in binary?

A. 0.101
B. 0.110
C. 0.625
D. 0.011

5.8 Exercises

Exercise 5.1 (paper first, then PRG32)

1. Convert 2026 to binary, octal and hexadecimal; convert 0.1 to binary up to eight fractional bits.
2. Write -100 on 8 bits in sign and magnitude and in two's complement.
3. Compute on 8 bits in two's complement: $90+60$, $-90-60$, $90-60$. Which overflow?
4. In the clamp code replace `bgez` and `ble` with their unsigned versions, predict the behaviour, then run it.
5. Write an `update` that adds `0x10000000` to a counter at each frame and lights a red rectangle when the signed sum overflows.

Expected result. On paper: $2026 = \texttt{0x7EA} = 111\,1110\,1010_2$. On PRG32: the red rectangle flashes once every sixteen frames, when the counter passes from `0x7...` to `0x8...`.

5.9 Glossary

English	**Italiano**
base (radix)	base
natural number	numero naturale
signed integer	numero relativo (intero con segno)
most significant bit	bit più significativo
remainder	resto
shift	scorrimento (shift)
two's complement	complemento a due (alla base)
sign and magnitude	segno e modulo
overflow	overflow (traboccamento)
carry / borrow	riporto / prestito
sign extension	estensione del segno
to clamp	limitare a un intervallo

5.10 Conclusions and reading

Every base has its advantages: base 2 for circuits, base 16 for people who read bits. n bits represent 2^n numbers: which ones depends on the encoding law. Two's complement turns subtraction into addition, so that one adder does everything. Overflow is the price of finite registers: it is detected, or designed away. In RISC-V the bits have no type: the instruction chooses between signed and unsigned.

Reading. Patterson and Hennessy (2020), sections 2.4 and 3.1–3.2; *Playing with RISC-V*, chapter 6, "State and Input" (Montella et al., 2026b); Stallings (2021), the chapters on number systems and on computer arithmetic.

CHAPTER 6

Number Representation: Real Numbers

"I may count for little, it's true, said the One to the Zero, but what are you worth? Nothing: nothing at all."
— Trilussa, *Nummeri*, 1944

In this chapter

By the end of this chapter you will be able to

- encode signed integers in one's complement and in excess-k, and say what each is good for;
- represent a fractional quantity in fixed point and compute with it using integer instructions;
- explain mantissa, exponent and normalisation in floating point;
- encode and decode a number in IEEE 754 single precision;
- recognise zero, denormalised numbers, infinity and NaN;
- say why the ESP32-C6 has no floating-point instructions and what PRG32 games do instead.

6.1 Warm-up on the machine

A ball must cross the screen slowly. One pixel per frame is too fast; zero pixels per frame is not a movement. The ball needs something in between: 0.375 pixels per frame. But the processor of the kit is RV32IMAC: it has integer instructions only, and no floating point at all. Listing 6.1 solves the problem with a trick.

```
.equ SPEED_Q8, 96            # 96 / 256 = 0.375 pixel per frame

ball_update:
    la   t0, ball_x_q8
    lw   t1, 0(t0)           # position, in 1/256 of a pixel
    addi t1, t1, SPEED_Q8    # add a fraction of a pixel
    sw   t1, 0(t0)
    ret

ball_draw:
    ...
```

```
12      la   t0, ball_x_q8
13      lw   t1, 0(t0)
14      srai a0, t1, 8          # x in whole pixels: drop 8 bits
15      ...
16      call prg32_gfx_rect
```

Listing 6.1. Three-eighths of a pixel per frame, with integers only.

The position is stored *multiplied by 256*, and is divided by 256 only when the ball is drawn. In units of $1/256$ of a pixel the speed is $0.375 \times 256 = 96$, an honest integer.

Try It Yourself

Run the program with `SPEED_Q8` equal to 96, then 16, then 1.

The trick has a name, *fixed point*. This chapter says where it comes from and when it is not enough, in which case the general answer, *floating point*, takes over. Before either, two more encodings of the signed integers are needed, because floating point is built from them.

6.2 More encodings for signed integers

6.2.1 One's complement

Definition 6.1 (One's complement)

On n bits, a non-negative number $0 \le x < 2^{n-1}$ is represented by itself, $X = x$; a negative number is represented by $X = 2^n - 1 - |x|$.

The range goes from $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$. Zero has two representations, `0000` and `1111`, so that $2^n - 1$ numbers are represented, as in sign and magnitude. The reason for using it is that negation is trivial: since $2^n - 1$ is a string of ones, subtracting from it complements every bit,

$$-x \;=\; \texttt{not}\ x.$$

Thus $6 = \texttt{0110}$ and $-6 = \texttt{1001}$; $5 = \texttt{0101}$ and $-5 = \texttt{1010}$; $1 = \texttt{0001}$ and $-1 = \texttt{1110}$. The bitwise `not` of RISC-V computes exactly this.

Arithmetic in one's complement works modulo $2^n - 1$ rather than modulo 2^n, and this costs a correction.

No correction		Two negatives		Positive result	
3 +	0011 +	-2 +	1101 +	5 +	0101 +
-4 =	1011 =	-3 =	1100 =	-2 =	1101 =
-1	1110		1 1001 + 1 =		1 0010 + 1 =
		-5	1010	3	0011

Whenever there is a carry out of the most significant bit, 1 must be added to the result: the *end-around carry*. The correction is needed when both operands are negative, or when the operands have opposite signs and the sum is positive. An adder and a complementer are still enough; but the extra step and the double zero made two's complement win.

6.2.2 Excess-k (biased) representation

Definition 6.2 (Excess-k)

In the *excess-k* representation every number is shifted upward by a constant k, the *bias*: $X = x + k$. Typically $k = 2^{n-1}$.

On four bits with $k = 8$ the pattern `0000` means -8, `1000` means 0 and `1111` means $+7$. The decisive property is that the order is preserved: if $x_1 > x_2$ then $X_1 > X_2$, so that signed numbers are compared as if they were naturals. The price is paid in arithmetic: a sum must be corrected by subtracting k, since $X_1 + X_2 = (x_1 + x_2) + 2k$, and multiplication and division are much more awkward than with the complements. Excess-k is therefore preferred where only additions and comparisons are needed, and that is exactly the situation of the exponent of a floating-point number.

6.2.3 Four bits, five readings

Table 6.1 completes the table of Chapter 5. Sign and magnitude and one's complement have two zeros. Two's complement and excess-8 cover the same range, -8 to $+7$, with a single zero; they differ only in the most significant bit, which is inverted. Only excess-k keeps the patterns in the same order as the numbers.

Table 6.1. Four bits, five readings.

Bits	Uns.	S-M	1's	2's	Exc-8	Bits	Uns.	S-M	1's	2's	Exc-8
`0000`	0	+0	+0	0	−8	`1000`	8	−0	−7	−8	0
`0001`	1	+1	+1	+1	−7	`1001`	9	−1	−6	−7	+1
`0010`	2	+2	+2	+2	−6	`1010`	10	−2	−5	−6	+2
`0011`	3	+3	+3	+3	−5	`1011`	11	−3	−4	−5	+3
`0100`	4	+4	+4	+4	−4	`1100`	12	−4	−3	−4	+4
`0101`	5	+5	+5	+5	−3	`1101`	13	−5	−2	−3	+5
`0110`	6	+6	+6	+6	−2	`1110`	14	−6	−1	−2	+6
`0111`	7	+7	+7	+7	−1	`1111`	15	−7	−0	−1	+7

6.3 Fixed point

6.3.1 Numbers in fixed point

In fixed point the integer part and the fractional part of a number are stored side by side in one word. The position of the point is fixed and implicit: it is not stored, the program knows it.

Definition 6.3 (Fixed point)

With f fractional bits a number x is stored as the integer $X = x \cdot 2^f$ (rounded). The *resolution* is 2^{-f}. The notation Qm.f denotes a format with m integer bits and f fractional bits.

Usually the whole word is in two's complement. Fixed point is how games, audio and control software handle fractions on processors without a floating-point unit. It is also how money should be stored: as an integer number of cents.

6.3.2 The ball, bit by bit: Q24.8

The ball of the warm-up is in the format Q24.8 (Figure 6.1). When its variable holds `0x00002A60` = 10848, the ball is at 10848/256 = 42.375 pixels: the upper 24 bits, `0x00002A`, are the 42 whole pixels and the lower 8, `0x60`, are 96/256 = 0.375.

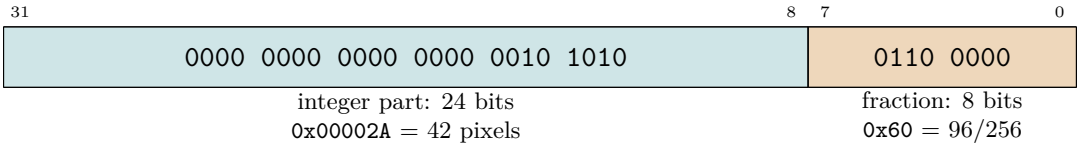


Figure 6.1. `ball_x_q8 = 0x00002A60`: the ball is at 42.375 pixels.

After one more frame the value is `0x2AC0` (42.75), then `0x2B20` (43.125): the ball crosses the boundary of a pixel every two or three frames, and on average advances by 0.375.

Arithmetic in fixed point is integer arithmetic with a little care (Table 6.2). Sums and differences need none, because the points of the two operands are aligned. A product of two numbers scaled by 2^f is scaled by 2^{2f}, and must be shifted back by f. The shift that returns to whole pixels is the *arithmetic* one, `srai`, which keeps the sign (Chapter 5).

Table 6.2. Operations in fixed point.

Operation	In fixed point	RISC-V
add, subtract	as integers: the points are aligned	`add`, `sub`, `addi`
to whole pixels	shift right by f, keeping the sign	`srai a0, t1, 8`
multiply	multiply, then shift right by f	`mul`, then `srai`

6.3.3 Rational numbers: the approximation error

The real numbers are dense: even in a limited interval, not every number can be represented with a finite number of bits. Each number is therefore associated with the nearest representable one. In Q24.8 the number 0.1 becomes 26/256 = 0.1015625, because $0.1 \times 256 = 25.6$ is rounded to 26: an error of about 1.6%.

The approximation error is at most half the resolution: 1/512 with eight fractional bits. *Underflow* is the case of a number close to zero, but not zero, that is represented by zero: with a speed of 0.001 pixels per frame the constant would be rounded to 0, and the ball would never move.

Concept

In fixed point the *absolute* error is the same everywhere on the number line. The resolution of eight fractional bits is 1/256: nothing smaller can be told apart from zero.

6.4 Floating point

Fixed point spends all its bits on one fixed range. Floating point spends some of them to say *where* the range is: it trades range for precision.

6.4.1 Real numbers in floating point

Definition 6.4 (Floating point)

A real number x is represented in floating point by a pair (m, e) such that

$$x = m \cdot b^{e},$$

where m is the *mantissa*, e the *exponent* and b the base.

The point "floats" because the exponent says where it is. This is the scientific notation of the physics laboratory. Infinitely many pairs represent the same number,

$$346.09801 \;=\; (346.09801,\, 0) \;=\; (346098.01,\, -3) \;=\; (0.034609801,\, 4),$$

and *normalisation* chooses a single representative: the point goes immediately after the first significant digit, $(3.4609801,\, 2)$.

6.4.2 The range of a small decimal format

Consider a format in base 10 with four digits for the mantissa and two for the exponent (Figure 6.2). It represents the numbers from -9.999×10^{99} to -1.000×10^{-99}, zero, and the numbers from $+1.000 \times 10^{-99}$ to $+9.999 \times 10^{99}$. An *overflow* occurs when the magnitude is too large for the exponent; an *underflow* when it is too small, and the number collapses to zero.

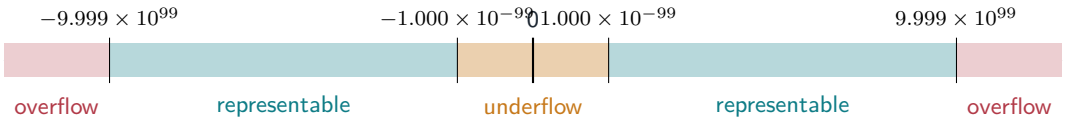


Figure 6.2. A decimal format with four digits of mantissa and two of exponent (not to scale).

6.4.3 Approximation: absolute and relative error

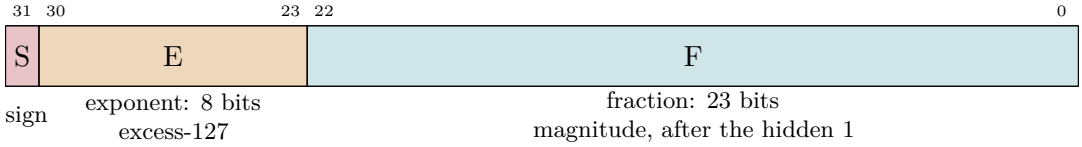
In such a format the absolute error is not constant. Near zero the distance between two consecutive numbers is $1.001 \times 10^{-99} - 1.000 \times 10^{-99} = 0.001 \times 10^{-99}$; near the upper limit it is $9.999 \times 10^{99} - 9.998 \times 10^{99} = 0.001 \times 10^{99}$. The absolute error is small on small numbers and large on large numbers. What is constant along the whole axis is the *relative* error, because the number of significant digits, four, is the same everywhere.

Concept

Fixed point: constant absolute error, relative error that explodes near zero. Floating point: constant relative error, absolute error that grows with the number.

6.4.4 IEEE 754 single precision

Until 1985 every manufacturer had its own floating-point format, and programs gave different results on different machines. The standard IEEE 754 (1985, revised in 2008 and 2019) put an end to this (IEEE, 2019). Its single-precision format divides 32 bits into three fields (Figure 6.3).



$$\text{value} \;=\; (-1)^{S} \times 1.F \times 2^{\,E-127}$$

Figure 6.3. IEEE 754 single precision.

- The mantissa is in sign and magnitude: one sign bit and 23 bits of magnitude.
- A normalised binary mantissa always starts with 1. That bit is *hidden*: it is not stored, and one bit of precision is gained.
- The exponent is in excess-127 (not 128): the true exponents go from -126 to $+127$. Thanks to the excess encoding, and to the position of the exponent before the fraction, two positive floating-point numbers can be compared as if they were naturals.
- The patterns $E = 0$ and $E = 255$ are reserved: zero and denormalised numbers, infinity and NaN.

Example 6.1 (-5.125 in IEEE 754)

1. Convert the two parts: $5 = 101_2$ and $0.125 = 1/8 = 0.001_2$, so $5.125 = 101.001_2$.
2. Normalise: $101.001_2 = 1.01001_2 \times 2^2$.
3. Sign $S = 1$; exponent $E = 2 + 127 = 129 = \texttt{1000 0001}$.
4. Fraction $F = \texttt{01001}$ followed by zeros: the leading 1 is hidden.

$$\underbrace{\texttt{1}}_{S}\ \underbrace{\texttt{1000 0001}}_{E}\ \underbrace{\texttt{0100 1000 0000 0000 0000 000}}_{F}$$

Regrouped in nibbles, `1100 0000 1010 0100 0000 ...` is `0xC0A40000`.

6.4.5 IEEE 754 at a glance

Table 6.3 compares the two formats in common use, and Table 6.4 lists the meaning of all the patterns.

Denormalised (or subnormal) numbers fill the gap of underflow around zero with a precision that decreases gradually. The division $1/0$ gives infinity; $0/0$ and the square root of -1 give NaN. NaN is not equal to anything, itself included.

6.4.6 Floating point is not the real numbers

A floating-point number is a rational number chosen from a finite set, and arithmetic on such numbers obeys laws that are not those of the real field (Goldberg, 1991).

- 0.1 has no finite binary expansion: it is stored approximately, like 0.8 in Chapter 5.

Table 6.3. IEEE 754 single and double precision.

	Single precision	**Double precision**
Total bits	32	64
Sign bits	1	1
Exponent bits	8	11
Fraction bits	23	52
Exponent encoding	excess-127	excess-1023
Exponent range	$-126 \ldots +127$	$-1022 \ldots +1023$
Smallest normalised	$2^{-126} \approx 1.2 \times 10^{-38}$	$2^{-1022} \approx 2.2 \times 10^{-308}$
Largest	about $2^{128} \approx 3.4 \times 10^{38}$	about $2^{1024} \approx 1.8 \times 10^{308}$
Smallest denormalised	about 1.4×10^{-45}	about 4.9×10^{-324}
Decimal digits	about 7	about 16

Table 6.4. IEEE 754: what the patterns mean.

Kind	**Sign**	**Exponent** E	**Fraction** F	**Value**
Normalised	0 or 1	$0 < E < \max$	any pattern	$(-1)^S \times 1.F \times 2^{E-bias}$
Denormalised	0 or 1	all zeros	not zero	$(-1)^S \times 0.F \times 2^{1-bias}$
Zero	0 or 1	all zeros	all zeros	$+0$ or -0
Infinity	0 or 1	all ones	all zeros	$+\infty$ or $-\infty$
Not a Number	0 or 1	all ones	not zero	NaN: an invalid result

- Two floating-point results should never be tested for equality: their difference is compared with a tolerance (Listing 6.2).
- Addition is not associative: $(a + b) + c$ can differ from $a + (b + c)$.
- Adding a very small number to a very large one can leave the large one unchanged.
- For money and for the positions of a game, scaled integers are often the right tool.

```
double a = 0.1 + 0.2;

// a == 0.3 ?   false
// a = 0.30000000000000004

// better:
// fabs(a - 0.3) < 1e-9
```

Listing 6.2. The most famous sum in computing.

6.4.7 Beyond single and double

The choice of a representation is an engineering decision that is still being made today. *Half precision*, added to the standard in 2008, has sixteen bits: 1 of sign, 5 of exponent, 10 of fraction; it gives about three decimal digits in half the memory of single precision. It matters because neural networks hold thousands of millions of numbers: halving each one halves memory, bandwidth and energy. Many models go further and are stored, by *quantisation*, as 8-bit integers with a scale factor: fixed point again, at industrial scale. Bits are a budget; range, precision and cost pull in different directions, on a GPU as on a Q24.8 ball.

6.5 Back to the machine

6.5.1 Floating point on RISC-V, and on the kit

RISC-V is modular: a small mandatory base plus optional extensions (Table 6.5). A RISC-V processor with the F and D extensions has thirty-two more registers, `f0` to `f31`, for floating-point values, and instructions such as `fadd.s`, `fmul.s`, `flw` and `fsw` that work on IEEE 754 numbers; `fcvt.w.s` converts to an integer and `fcvt.s.w` from an integer. Such processors are found in laptops, servers and supercomputers.

The ESP32-C6 is RV32IMAC: it has no F and no D, and therefore no floating-point instructions at all. A C program can still use the type `float`: the compiler calls software routines such as `__addsf3`, which implement exactly the algorithm of this chapter with shifts, masks and integer additions, at a cost of tens to hundreds of integer instructions per operation. PRG32 games use integers and fixed point: fast, exact enough, and visible in the debugger.

Table 6.5. Reading the name RV32IMAC.

Letter	Extension	What it adds	ESP32-C6
RV32I	base integer set	32-bit registers, add, logic, load and store, branches	yes
M	multiply and divide	`mul`, `mulh`, `div`, `rem` and their unsigned forms	yes
A	atomic	instructions for safe sharing between tasks	yes
C	compressed	16-bit encodings of the most common instructions	yes
F	single precision	`f0` to `f31` and IEEE 754 single-precision instructions	no
D	double precision	IEEE 754 double-precision instructions	no

A microcontroller picks the extensions that it needs. This is one reason why the same architecture scales from a chip that costs two euros to a supercomputer.

6.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l06.S` in the repository of the course, printed in full in Listing 6.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 6.4 is the frame that the test wrote.

On the screen.
A white square that crosses the screen slowly from the left: one pixel every 2.67 frames.

What to press.
Nothing: the program runs by itself.

```
# ARC12 Computer Architecture - Lecture 06: Number Representation: Real Numbers
# l06.S: three-eighths of a pixel per frame, with integers only
#
# On the screen: a white square that crosses the screen slowly from the left: one
# pixel every 2.67 frames.
# Build: python3 -m prg32 cartridge build l06.S --portable \
#            --entry-prefix l06 --name l06 --out l06.prg32
.option norelax

```



Figure 6.4. `l06.S` on PRG32-QT.

```
10 .equ SPEED_Q8, 96            # 96 / 256 = 0.375 pixel per frame
11
12 .section .text
13 .global l06_init
14 .global l06_update
15 .global l06_draw
16
17 l06_init:
18     ret
19
20 l06_update:
21 ball_update:
22     la   t0, ball_x_q8
23     lw   t1, 0(t0)           # position, in 1/256 of a pixel
24     addi t1, t1, SPEED_Q8    # add a fraction of a pixel
25     sw   t1, 0(t0)
26     ret
27
28 l06_draw:
29 ball_draw:
30     addi sp, sp, -16
31     sw   ra, 12(sp)
32     li   a0, 0
33     call prg32_gfx_clear
34     la   t0, ball_x_q8
35     lw   t1, 0(t0)
36     srai a0, t1, 8           # x in whole pixels: drop 8 bits
37     li   a1, 100
38     li   a2, 8
39     li   a3, 8
40     li   a4, 0xffff
41     call prg32_gfx_rect
42     lw   ra, 12(sp)
43     addi sp, sp, 16
44     ret
45
46 .section .data
47 ball_x_q8: .word 0           # Q24.8: 24 bits of pixels, 8 bits of fraction
```

Listing 6.3. The complete program of the chapter, `code/l06.S`.

To run it on PRG32-QT the commands are those of Listing 6.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l06.S --portable \
    --entry-prefix l06 --name l06 --out l06.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l06.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 6.4. Building `l06.S` and running it on PRG32-QT.

6.7 Check your understanding

Q6.1 Which signed encoding keeps the bit patterns in the same order as the numbers?

A. Sign and magnitude
B. One's complement
C. Two's complement
D. Excess-k

Q6.2 In Q24.8 fixed point, which real number does the word `0x00000180` represent?

A. 384
B. 1.5
C. 1.8
D. 0.375

Q6.3 Which real number is the IEEE 754 single-precision pattern `0x40400000`?

A. 2.0
B. 3.0
C. 4.0
D. 0.75

Q6.4 In IEEE 754, an exponent field of all ones with a fraction of all zeros means:

A. zero
B. a denormalised number
C. infinity
D. NaN

6.8 Exercises

Exercise 6.1 (paper first, then PRG32)

1. Write -6 on four bits in sign and magnitude, one's complement, two's complement and excess-8.
2. Encode 10.5 and 0.15625 in IEEE 754 single precision; give the result in hexadecimal.
3. Decode `0xBF800000`.
4. Run the ball with `SPEED_Q8` equal to 96, 16 and 1. After how many frames does it move by one pixel in each case?
5. Give the ball a vertical speed too, and make it bounce: negate the speed with `neg` when it reaches an edge.

Expected result. On paper: `1110`, `1001`, `1010`, `0010`; `0x41280000` and `0x3E200000`; -1.0. On PRG32: the ball advances one pixel every 2.67, 16 and 256 frames; with the bounce it moves diagonally and stays on the screen.

6.9 Glossary

English	**Italiano**
one's complement	complemento a uno (alla base diminuita)
excess-k (biased)	eccesso-k
fixed point	virgola fissa
floating point	virgola mobile
single / double precision	singola / doppia precisione
rounding	arrotondamento
mantissa / fraction	mantissa
exponent	esponente
normalisation	normalizzazione
hidden bit	bit nascosto
denormalised	denormalizzato
Not a Number (NaN)	non un numero (NaN)

6.10 Conclusions and reading

Integers have several representations: 8, 16, 32 or 64 bits, signed or unsigned, complements or excess. Fixed point stores a fraction as a scaled integer: constant absolute error, the speed of integers. Floating point stores a mantissa and an exponent: a huge range, constant relative error. IEEE 754 is the universal standard for floating point, with special patterns for zero, infinity and NaN. On a processor without the F extension, fixed point is the natural choice.

Reading. Patterson and Hennessy (2020), section 3.5; Goldberg (1991); *Playing with RISC-V*, the appendix on the runtime interface, for playfields and parallax in Q8 (Montella et al., 2026b).

The first block is complete. The next chapter puts its four lectures to work in a real program: the first part of Verm1.

CHAPTER 7

Verm1, Part 1: A Head That Crosses the World

"The journey of a thousand miles begins with a single step."
— Laozi, *Tao Te Ching*

In this chapter

By the end of this chapter you will have

- a cartridge of your own, `verm1.prg32`, built from one assembly file that you typed;
- a title, a grid of 40 by 24 cells, and the head of the worm drawn in one of them;
- the head moving by itself, one cell at a time, at a speed that you control in fixed point;
- a world with no edges: past the last column the head comes back from the first;
- the habit of the working loop: edit, build, run, look, and stop the program in the debugger.

This is the first tutorial of the project. It uses only what Chapters 2 to 6 have taught, and it is meant to be followed at the keyboard: the reader types the program. Everything printed in the listings is read from the reference file `verm1_part1.S`, which is built and run automatically; the whole file is in Appendix D.

7.1 The project

7.1.1 Verm1: a tribute to Snake

Verm1, read *vermone*, "the big worm", is a worm that moves on a grid, eats, grows, and must not bite itself. The game has a long history: it appeared as *Blockade*, an arcade machine by Gremlin, in 1976, and as *Snake* on the Nokia 6110 in 1997, written by Taneli Armanto, it became one of the most played games ever. Verm1 is a tribute, not a copy: the name, the rules of detail and the look are its own. It is small enough to be written from scratch in assembly and rich enough to need every topic of the course, and the reader writes every line of it: there is no engine, and no library beyond the calls of PRG32. The reference implementation that the tutorial chapters follow, and against which a reader can

always compare a program of their own, was contributed by Giuseppe Coviello (Appendix D). Figure 7.1 shows the destination.

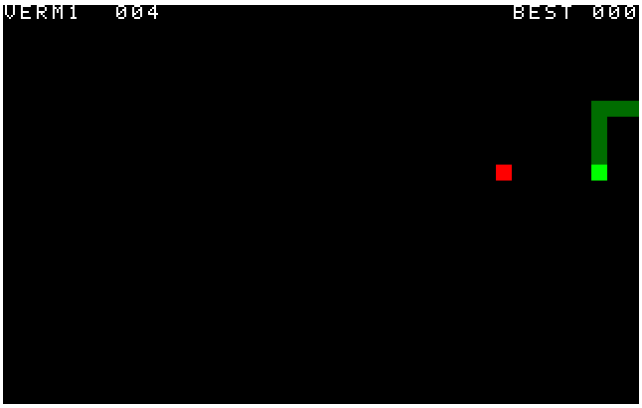


Figure 7.1. Verm1 at the end of the book (Part 4).

7.1.2 The plan

Table 7.1. Four parts and a publication.

Chapter	You add	It uses Chapters
7 Part 1	a grid, a head, automatic movement, wrap-around	3 to 6: information and numbers
12 Part 2	steering with the joystick, food, a score	8 to 11: logic and circuits
17 Part 3	a body that grows, collisions, title and game-over screens	13 to 16: state and memory
22 Part 4	fast drawing, a score in digits, levels, sound	18 to 21: the machine
23 The Store	a bundle with manifest, icon and credits	all of them

Each part ends with a program that runs (Table 7.1).

Watch Out

Never start a part with a program that does not run. If yours is broken, start from the reference file of the previous part.

Part 1 takes the following from the chapters read so far.

From	Idea	Where it appears
Chapter 2	registers, `la`/`lw`/`sw`, `call` and the stack frame	everywhere
Chapter 3	retro games as specimens	the choice of the game
Chapter 4	colours as RGB565 numbers; strings ending with zero	`GREEN`, the title
Chapter 5	shifts as multiplications; comparisons; wrap-around	cell to pixel; the edge of the world
Chapter 6	fixed point: a fraction as a scaled integer	the speed of the worm

7.2 Step 0: the working loop

Make a folder `~/verm1` and keep in it one file, `verm1.S`, which will grow from part to part. The two commands of Listing 7.1 build the cartridge and send it to PRG32-QT; they are the same in every tutorial. PRG32-QT listens on port 8080 and shows its address on the Setup screen: use that address if `localhost` does not answer. The same cartridge runs on QEMU (`qemu upload`, then `qemu run`) and on the board (`--url http://192.168.4.1`).

```
# 1. build the cartridge (from your PRG32 folder, toolchain active)
python3 -m prg32 cartridge build ~/verm1/verm1.S --portable \
    --entry-prefix verm1 --name verm1 --out ~/verm1/verm1.prg32

# 2. send it to PRG32-QT, running on the same computer
python3 -m prg32 esp32c6 upload-and-run ~/verm1/verm1.prg32 \
    --url http://localhost:8080 --slot cart0

# or: in PRG32-QT, import the file verm1.prg32 and choose Run Cartridge
```

Listing 7.1. Build and run: edit, build, run, look.

7.3 Step 1: three functions that return

```
1  # verm1.S - my Verm1
2  .option norelax
3
4  .section .text
5  .global verm1_init
6  .global verm1_update
7  .global verm1_draw
8
9  verm1_init:
10     ret
11
12 verm1_update:
13     ret
14
15 verm1_draw:
16     ret
```

Listing 7.2. Step 1 **(new)**: the skeleton.

This is the skeleton of Chapter 2 with another prefix: `verm1` is the name passed to `--entry-prefix`. `init` runs once; `update` and `draw` run once per frame.

Run it

Type it, build it, run it: a black screen is the right answer. If the build fails here the problem is the installation, not the program: solve it before going on. From now on, add one thing, run, look. Never two things at once.

7.4 Step 2: a grid and a head

7.4.1 Constants: colours and the grid

Replace the first lines of the skeleton with those of Listing 7.3.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40              # 320 / 8 cells across
.equ ROWS,  24              # 192 / 8 cells down
.equ TOP,   8               # the top 8 pixels are the title line
.equ SPEED_Q8, 40           # 40/256 of a cell per frame (Q24.8)

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
```

Listing 7.3. Step 2 **(changed)**: the head of the file.

Colours are RGB565 numbers: `0x07e0` is pure green. The screen is 320 pixels across and a cell is 8 pixels wide, which makes 40 columns. The top 8 pixels hold the title, and 192 are left: 24 rows. The directive `.equ` gives a name to a number and uses no memory. The constant `SPEED_Q8` is used in step 3.

7.4.2 The world of Verm1

Concept

The worm does not live in pixels: it lives in cells. A position is a column, 0 to 39, and a row, 0 to 23.

To draw a cell its coordinates are turned into pixels (Figure 7.2): $x = column \times 8$, a shift to the left by 3, and $y = row \times 8 + 8$, because the title line comes first. The cell $(0, 0)$ is at pixel $(0, 8)$; the cell $(39, 23)$ is at $(312, 192)$. Thinking in cells makes every later rule simple: moving is adding 1.

7.4.3 Variables and a string

Put Listing 7.4 at the end of the file. The section `.rodata` holds constants: the title ends with a zero byte. The section `.data` holds variables, one 32-bit word each. Their initial values are only a start: `init` will set them again.

```
.section .rodata
title:   .asciz "VERM1"

.section .data
head_x:  .word 5                # column of the head, 0 .. COLS-1
head_y:  .word 12               # row of the head, 0 .. ROWS-1
```

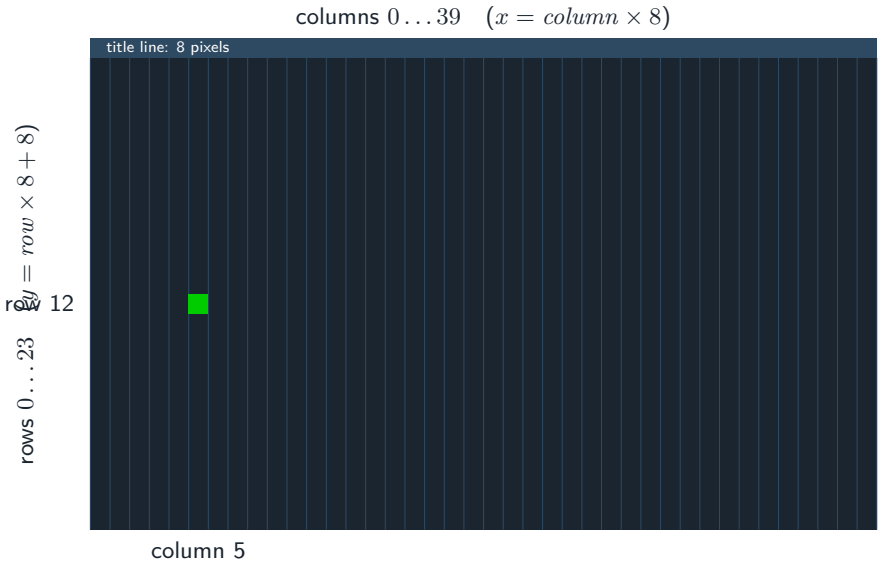


Figure 7.2. The world of Verm1: 40 by 24 cells of 8 by 8 pixels. The head starts in column 5 of row 12.

```
7 step_q8: .word 0                  # fraction of a step, in 1/256
```

Listing 7.4. Step 2 **(new)**: data, at the end of the file.

7.4.4 `init`: put the head in its place

```
1  verm1_init:
2      la   t0, head_x
3      li   t1, 5
4      sw   t1, 0(t0)               # the head starts in column 5 ...
5      la   t0, head_y
6      li   t1, 12
7      sw   t1, 0(t0)               # ... of row 12
8      la   t0, step_q8
9      sw   zero, 0(t0)             # no fraction of a step yet
10     ret
```

Listing 7.5. Step 2 **(changed)**: `verm1_init`.

`la` takes an *address*, `sw` stores a *value* there. The function sets the start explicitly: a program should never trust what memory contained. `zero` is the register that always reads 0. The function is a leaf: it makes no calls, so it needs no stack frame.

7.4.5 `draw`: title and head

```
1  verm1_draw:
2      addi sp, sp, -16
3      sw   ra, 12(sp)
4      li   a0, BLACK
5      call prg32_gfx_clear
6      li   a0, 0                   # x
```

```
7       li   a1, 0                  # y
8       la   a2, title              # address of the string
9       li   a3, WHITE
10      li   a4, BLACK
11      call prg32_gfx_text8
12      la   t0, head_x
13      lw   a0, 0(t0)
14      slli a0, a0, 3              # x = column * 8
15      la   t0, head_y
16      lw   a1, 0(t0)
17      slli a1, a1, 3
18      addi a1, a1, TOP            # y = row * 8 + 8
19      li   a2, CELL               # width
20      li   a3, CELL               # height
21      li   a4, GREEN
22      call prg32_gfx_rect
23      lw   ra, 12(sp)
24      addi sp, sp, 16
25      ret
```

Listing 7.6. Step 2 **(changed)**: `verm1_draw`.

Clear the screen, write the title, draw one rectangle: three calls, with the arguments travelling in `a0` to `a4`, in order. The signatures are `prg32_gfx_text8(x, y, string, foreground, background)` and `prg32_gfx_rect(x, y, width, height, colour)`. `slli` by 3 multiplies by 8: from cell to pixel. Since `draw` makes calls, it saves `ra` in a stack frame.

Run it: a head on the grid

Expected now: the word `VERM1` at the top left and a green square standing still in column 5, row 12 (Figure 7.3 shows it later, when it has begun to move).

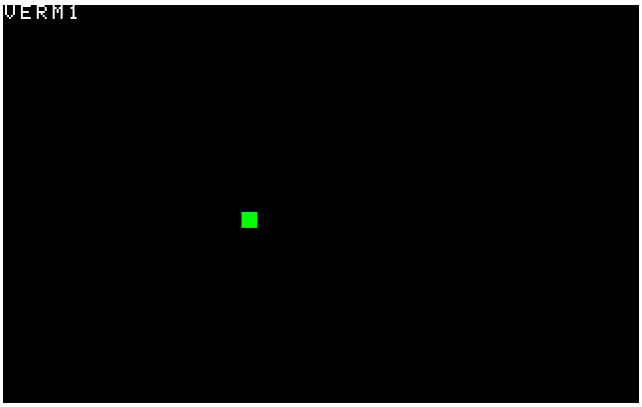


Figure 7.3. The title and the head. This picture was taken by the automatic tests 64 frames into the finished Part 1: the head is in column 15.

7.5 Step 3: movement

7.5.1 How fast should a worm go?

`update` runs about thirty times per second. One cell per frame would cross the screen in little more than a second: unplayable. What is wanted is a fraction of a cell per frame, and the processor of the kit has only integers. The answer is the fixed point of Chapter 6, in the format Q24.8: the program keeps a fraction in units of 1/256 of a cell, and takes a step each time a whole cell has accumulated. With `SPEED_Q8` = 40 the worm advances $40/256 \approx 0.156$ cells per frame: about one step every 6 frames.

This is the ball of Chapter 6 turned into a timer. The remainder is not thrown away, so the average speed is exact: 256 frames give exactly 40 steps.

7.5.2 `update`: accumulate, step, wrap

```
verm1_update:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 2f             # less than one whole cell: wait
    sub  t1, t1, t2             # one cell done: keep the remainder
    sw   t1, 0(t0)

    la   t0, head_x
    lw   t3, 0(t0)
    addi t3, t3, 1              # one cell to the right
    li   t4, COLS
    blt  t3, t4, 1f
    li   t3, 0                  # past the right edge: back to column 0
1:  sw   t3, 0(t0)
    ret
2:  sw   t1, 0(t0)
    ret
```

Listing 7.7. Step 3 **(changed)**: `verm1_update`.

The speed is added to the fraction; below 256, the function waits. At 256 it subtracts 256, keeps the rest, and moves the head one cell to the right. Column 40 does not exist: past the right edge the head goes back to column 0. The function has two exits, and each stores what it has changed. `blt` compares signed numbers, which is right here because every value is small and positive. The local labels `1:` and `2:` are referred to as `1f` and `2f`: "the next 1, or 2, *forward*".

7.5.3 Trace it by hand

The steps come every 6 or 7 frames (Table 7.2): the remainder carries the difference, as a leap year does. After 64 frames the sum is $64 \times 40 = 2560 = 10 \times 256$: ten steps, and the head is in column 15, exactly as in Figure 7.3. Fill in frames 8 to 12 as an exercise.

Table 7.2. A trace of `verm1_update`.

Frame	`step_q8` before	+ 40	Whole cell?	`step_q8` after	`head_x`
1	0	40	no	40	5
2	40	80	no	80	5
6	200	240	no	240	5
7	240	280	yes: 280 − 256	24	6
13	224	264	yes: 264 − 256	8	7
64	–	–	–	0	15

Run it: the head crosses the world

1. Build and run: the head moves to the right, about five cells per second.
2. Wait for the right edge: the head leaves at column 39 and returns at column 0.
3. Change `SPEED_Q8` to 8, then to 128, then to 256: predict the speed before you run.
4. Change the starting row in `init`, and the colour of the head.

Expected result. The head crosses the forty columns in 256 frames, about eight and a half seconds at thirty frames per second, for ever. With `SPEED_Q8` = 256 it moves one cell per frame; larger values still give one step per frame, because `update` takes at most one step: a limit worth discovering.

7.6 Step 4: look inside with PRG32-QT

The debugger is part of the method, not an emergency tool.

Stop the program and read the machine

1. In PRG32-QT open *Settings* and enable *Run cartridges in RISC-V debug mode*.
2. Run your cartridge: the game is on the left, the debugger on the right.
3. Press **Update**: the listing jumps to `verm1_update`. Click in the gutter to put a breakpoint on its first instruction.
4. The program stops there at the next frame. Press **Step** and watch the registers change, one instruction at a time.
5. After the two instructions of `la`, `t0` holds an address: type it in the *Memory* field to see `step_q8` in memory.
6. Press **Resume**, and watch the bytes of `step_q8` grow by 40 (`0x28`) at every stop.

Resume ignores the current breakpoint once, so that each press runs one whole frame.

7.6.1 What the listing shows

The debugger does not show the source file: it shows the code as the assembler left it (Listing 7.8).

```
verm1_update:
  auipc   t0, 0x0          # la t0, step_q8   is two instructions:
  addi    t0, t0, ...      #   t0 = address of step_q8
```

```
lw      t1, 0(t0)
addi    t1, t1, 40       # SPEED_Q8 is gone: only the number is left
addi    t2, zero, 256    # li t2, 256
blt     t1, t2, ...
sub     t1, t1, t2
sw      t1, 0(t0)
...
c.addi  t3, 1            # addi t3, t3, 1 in a 16-bit form
...
c.jr    ra               # ret
```

Listing 7.8. `verm1_update`, disassembled (abridged; the exact offsets depend on the build).

- Names are gone: labels have become addresses, and constants have become numbers.
- Pseudo-instructions are expanded: `la` is `auipc` plus `addi`, `li` is `addi` from `zero`, `ret` is a jump to `ra`.
- Some instructions start with `c.`: they are the *compressed*, 16-bit forms of the same instructions (the C of RV32IMAC).
- The cartridge is loaded at address `0x40800000`: every address shown starts there.

Instruction formats and compressed instructions are the subject of Chapter 19.

7.6.2 Reading memory: little-endian, live

The *Memory* view shows 128 bytes from the address typed, in hexadecimal (Figure 7.4). The variable `step_q8` is a word: its four bytes come lowest byte first, so that with `step_q8` $= 240$ one reads `F0 00 00 00`. In the file the order of the variables is `head_x`, `head_y`, `step_q8`, four bytes each: if `step_q8` is at address A, `head_x` is at $A - 8$.

The debugger only observes. To change the program: edit, build, run.

7.7 Check your understanding

Q7.1 With `SPEED_Q8` $= 64$, how many frames pass between two steps of the head?

A. 2
B. 4
C. 64
D. 256

Q7.2 Which pixel is the top-left corner of the cell in column 10, row 3?

A. $(10, 3)$
B. $(80, 24)$
C. $(80, 32)$
D. $(88, 32)$

Q7.3 Why does `verm1_draw` save `ra` while `verm1_update` does not?

A. `draw` is longer

B. `draw` calls other functions, which overwrite `ra`
C. `update` uses no registers
D. `ra` is saved automatically in `update`

7.8 Make it yours

Exercise 7.1 (on PRG32)

1. Make the head move to the left: which instruction changes, and which comparison detects the edge?
2. Make it move downwards instead, wrapping from row 23 to row 0.
3. Draw a second cell, in a darker green, one column behind the head: a first hint of a body.
4. Write your own name next to the title, in a colour that you computed by hand in RGB565.

Expected result. Each change is a few lines. Keep a copy of the working Part 1 before experimenting: Part 2 starts from it. (Hints: to move left, `addi t3, t3, -1`, then `bgez t3` to skip the wrap, else `li t3, COLS-1`: the comparison is on the sign. A darker green is `0x03e0`.)

7.9 Where we are, and what comes next

The reader has a cartridge that runs, built from a file typed and understood line by line. It has state in memory, a rule that changes the state at every frame, and a function that draws it: the shape of every game. What is missing is the player: the head goes where it wants. Chapters 8 to 11 give the tools: bits and masks to read the joystick, Boolean rules, tables that select. Part 2 adds steering, food and a score.

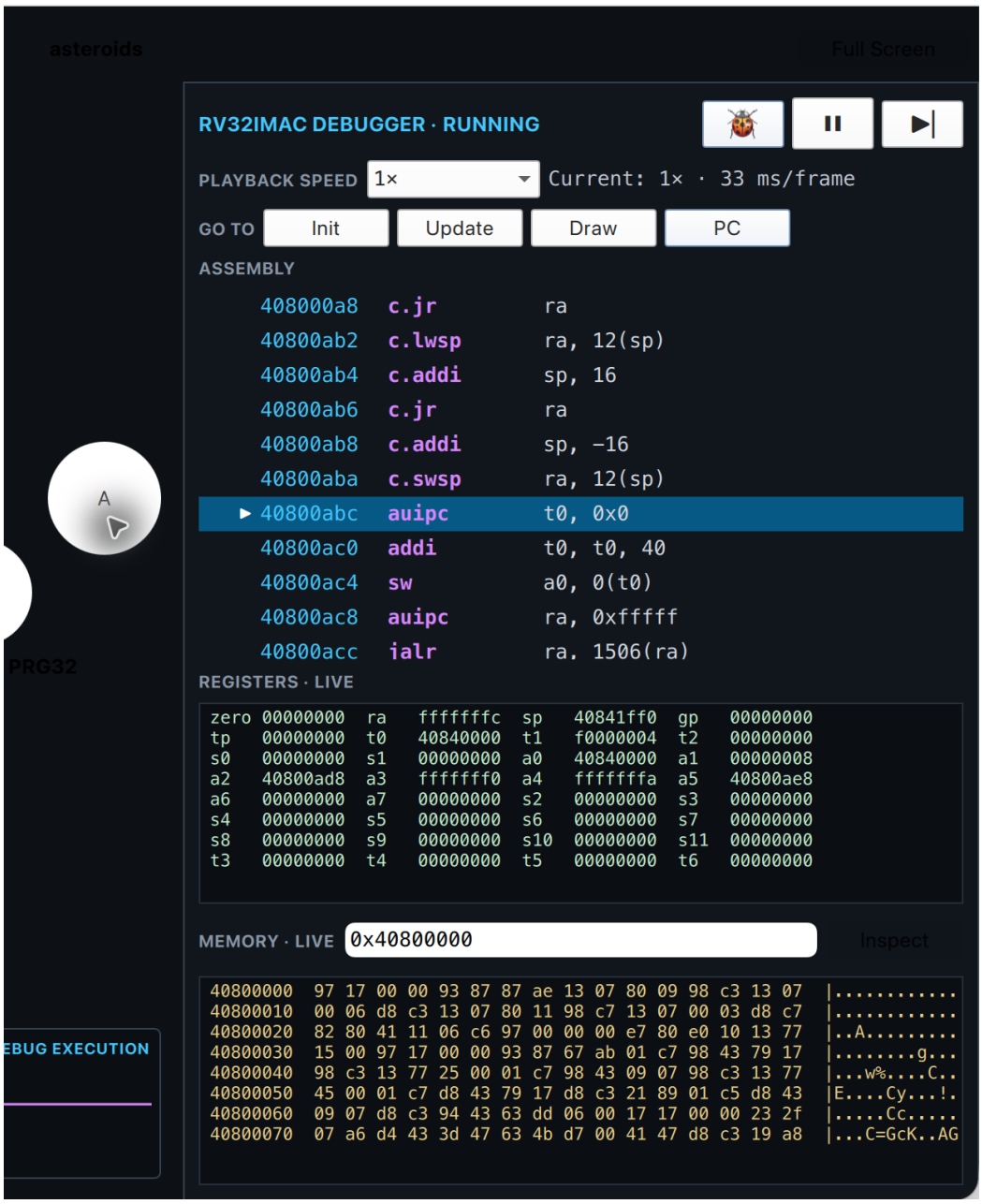


Figure 7.4. PRG32-QT: listing, registers, memory.

Part III

Logic

CHAPTER 8

Boolean Algebra: Operators and Properties

"Oh dear! Oh dear! I shall be too late!"
— Lewis Carroll, *Alice's Adventures in Wonderland*, 1865

In this chapter

By the end of this chapter you will be able to

- describe a computer as processor, memory and input/output, connected by a bus;
- explain the difference between the analysis and the synthesis of a logic circuit;
- write the truth tables of AND, OR, NOT, XOR, NAND and NOR and recognise their symbols;
- apply the properties of Boolean algebra, De Morgan's laws included, to simplify an expression;
- use `and`, `or`, `xor` and `not` on RISC-V to test, set, clear and toggle single bits.

8.1 Warm-up on the machine

The reader has been doing Boolean algebra since the first days. Listing 8.1 is the way in which every PRG32 game asks whether a direction is pressed.

```
    call prg32_input_read    # a0 = all the buttons, one bit each
    mv   t0, a0              # keep our own copy

    andi t3, t0, 0x01        # keep only bit 0: LEFT
    beqz t3, 1f              # zero: LEFT is not pressed
    addi t2, t2, -2          # move left
1:
    andi t3, t0, 0x02        # keep only bit 1: RIGHT
    beqz t3, 2f
    addi t2, t2, 2           # move right
2:
```

Listing 8.1. Is **LEFT** pressed?

The function `prg32_input_read` returns all the buttons at once, one bit per button (Table 8.1). Holding **RIGHT** and **A** together gives `0x12` = $0001\,0010_2$. The instruction `andi` with `0x01` keeps bit 0 and forces every other bit to zero, so that the result is different from zero exactly when **LEFT** is pressed. The constant used with `andi` is called a *mask*.

Table 8.1. One bit per button (the constants `PRG32_BTN_*` of `prg32.h`).

Button	LEFT	RIGHT	UP	DOWN	A	B	SELECT
Bit	0	1	2	3	4	5	6
Mask	`0x01`	`0x02`	`0x04`	`0x08`	`0x10`	`0x20`	`0x40`



Figure 8.1. Seven lamps, one per bit of the button mask: a lamp is lit when its bit is 1.

Every question about the joystick is a Boolean expression over these seven variables. And `andi` is thirty-two AND gates working side by side. What is an AND, exactly?

8.2 The computer as a logic machine

This part of the book descends one level below the instructions.

8.2.1 Three subsystems and a bus

In today's terms the von Neumann model of Chapter 3 is drawn as in Figure 8.2: the units of 1945, connected by shared wires. The *processor* (CPU) contains the control unit, the ALU and the registers, and executes the instructions. The *main memory* holds the instructions and the data. The *input/output* subsystem connects joystick, display, sound, network and storage. The *bus* is the set of wires that joins them: address lines, data lines and control lines. Everything that travels on the bus is a pattern of bits. On the ESP32-C6 the three subsystems and the bus are all inside one chip.

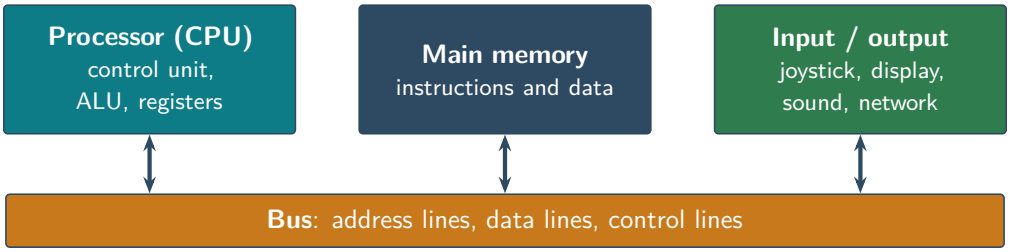


Figure 8.2. The electronic computer: three subsystems and a bus.

8.2.2 The processor

The processing capability of a computer resides in the processor. It can execute a set of elementary actions, the *instructions*: explicit commands that govern the transfer of information inside the computer and between the computer and the I/O devices, and that specify the arithmetic and logic operations to be performed. Main memory holds the input and output data of the computation and the sequence of instructions; a sequence of instructions stored in main memory is a *program*. The processor takes the instructions from memory and executes them one by one: *fetch, execute, repeat*.

An electronic computer is, accordingly, a system that is

numeric:
a digital system, in which all the information handled is expressed as strings of bits;

automatic:
once started it proceeds by itself, instruction after instruction, with no human intervention;

stored-program:
the program is recorded in memory and can be replaced, so that the same machine does different jobs;

electronic:
it is built from electronic circuits that store bits and process them.

8.2.3 Logic circuits

The electronic circuits that store and process bits are called *switching circuits* or *logic circuits*. A logic circuit forms a *switching network*, and designing one is called *logic design*. Two voltage levels on a wire, two states of a switch, two values of a variable: to describe the behaviour of such circuits an algebra is used, a mathematical notation that simplifies both their analysis and their synthesis. The algebra is that of Boole (1854), applied to switches by Nakajima and by Shannon (1938).

Definition 8.1 (Analysis and synthesis)

Analysis: given the description of the *structure* of a circuit (how it is made), find its *behaviour* (what it does). *Synthesis*: given the description of the *behaviour* (what it must do), find a *structure* that realises it (how to build it).

Reading a schematic and deriving its truth table is analysis; so is reading `andi t3, t0, 1` and saying what ends up in `t3`. Passing from the truth table of a sum to an adder is synthesis; so is passing from "move if **LEFT** or **RIGHT** is pressed" to a mask and a branch. Synthesis usually has many solutions, and choosing the cheapest is the problem of *minimisation*, the subject of Chapter 9.

8.3 Boolean algebra

Definition 8.2 (Boolean algebra)

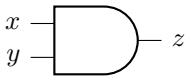
Boolean algebra is the algebra of variables that can take only two values, 0 and 1, with three basic operations:

- AND, the *logical product* or conjunction, written $x \cdot y$ or simply xy;
- OR, the *logical sum* or disjunction, written $x + y$;
- NOT, the *complement* or negation, written $\overline{x}$ (in programs and on a keyboard, `x'`).

8.3.1 The three fundamental operations

An operation is defined by its *truth table*, which lists the output for every combination of the inputs (Figure 8.3). AND is 1 only when both inputs are 1; OR is 0 only when both inputs are 0; NOT inverts. Each operation has a graphical symbol, the *logic gate*, which stands for the circuit that computes it.

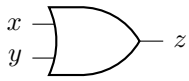
AND (conjunction)



$z = x \cdot y$

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

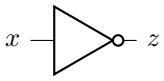
OR (disjunction)



$z = x + y$

x	y	z
0	0	0
0	1	1
1	0	1
1	1	1

NOT (negation)



$z = \overline{x}$

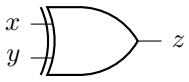
x	z
0	1
1	0

Figure 8.3. The three fundamental operations.

8.3.2 Three more gates

Three other operations are met everywhere (Figure 8.4). XOR, the *exclusive or*, is 1 when the inputs differ: it is the sum bit of an addition, and RISC-V has it as an instruction. NAND and NOR are the complements of AND and OR; the small circle on a symbol means negation.

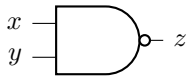
XOR (exclusive or)



$z = \overline{x}y + x\overline{y} = x \oplus y$

x	y	z
0	0	0
0	1	1
1	0	1
1	1	0

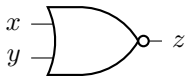
NAND (not and)



$z = \overline{x \cdot y}$

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

NOR (not or)



$z = \overline{x + y}$

x	y	z
0	0	1
0	1	0
1	0	0
1	1	0

Figure 8.4. XOR, NAND and NOR.

8.3.3 Properties

Table 8.2 collects the properties of the algebra. Every property comes in a pair: one form for AND and one for OR.

Table 8.2. Properties of Boolean algebra.

Property	AND form	OR form
Commutative	$x \cdot y = y \cdot x$	$x + y = y + x$
Associative	$(x \cdot y) \cdot z = x \cdot (y \cdot z)$	$(x + y) + z = x + (y + z)$
Idempotence	$x \cdot x = x$	$x + x = x$
Absorption	$x \cdot (x + y) = x$	$x + (x \cdot y) = x$
Distributive	$x \cdot (y + z) = x \cdot y + x \cdot z$	$x + y \cdot z = (x + y) \cdot (x + z)$
Identity element	$x \cdot 1 = x$	$x + 0 = x$
Null element	$x \cdot 0 = 0$	$x + 1 = 1$
Complement	$x \cdot \overline{x} = 0$	$x + \overline{x} = 1$
Involution	$\overline{\overline{x}} = x$	

Because of the associative property, AND and OR extend to any number of operands: $x \cdot y \cdot z$, $x + y + z$.

Watch Out

OR distributes over AND too: $x + yz = (x + y)(x + z)$. This is false for the ordinary sum and product, and it is the property that is most often forgotten.

How is a property proved? Since a variable has only two values, every identity can be verified by trying all the cases: the method of *perfect induction*. Two expressions are *equivalent* if they have the same truth table. Alternatively a property can be derived algebraically from others; absorption, for instance, follows from identity, distributivity and null element:

$$x + xy \;=\; x \cdot 1 + x \cdot y \;=\; x\,(1 + y) \;=\; x \cdot 1 \;=\; x.$$

8.3.4 De Morgan's laws

Theorem 8.1 (De Morgan, 1847)

$$\overline{x \cdot y} = \overline{x} + \overline{y}, \qquad \overline{x + y} = \overline{x} \cdot \overline{y}.$$

The complement of a product is the sum of the complements, and vice versa.

Proof of the first law, by perfect induction: the columns of $\overline{xy}$ and of $\overline{x} + \overline{y}$ are identical.

x	y	xy	$\overline{xy}$	$\overline{x}$	$\overline{y}$	$\overline{x} + \overline{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

The second law is proved in the same way (Exercise 8.1). □

In terms of gates: a NAND is an OR of the negated inputs, and a NOR is an AND of the negated inputs. The laws generalise to any number of variables.

8.3.5 Duality and precedence

Theorem 8.2 (Duality principle)

Every identity of Boolean algebra remains true if AND is exchanged with OR and 0 with 1.

Thus $x \cdot 1 = x$ becomes $x + 0 = x$, and each of De Morgan's laws is the dual of the other. Duality halves the work: it is enough to prove one law of each pair.

As for *precedence*, NOT comes first, then AND, then OR, as with powers, products and sums: $x + y\,\overline{z}$ means $x + (y \cdot (\overline{z}))$. Parentheses change the order exactly as in ordinary algebra.

8.3.6 Simplifying with the algebra

The properties are tools for transforming an expression into an equivalent and cheaper one.

Example 8.1 (Three gates become a wire)

$$\begin{aligned} x\,y + x\,\overline{y} &= x\,(y + \overline{y}) && \text{distributive law, read from right to left} \\ &= x \cdot 1 && \text{complement law} \\ &= x && \text{identity element.} \end{aligned}$$

Two AND gates, an OR gate and an inverter have become a piece of wire.

In the same way, using the distributive law of OR over AND,

$$x + \overline{x}\,y \;=\; (x + \overline{x})(x + y) \;=\; 1 \cdot (x + y) \;=\; x + y, \tag{8.1}$$

an identity sometimes called the *simplification theorem*, which will be used again in Chapter 9.

8.4 Back to the machine

8.4.1 Bitwise instructions: the algebra on a whole register

Each bitwise instruction of RISC-V applies one gate to all thirty-two pairs of bits at once (Table 8.3, where eight bits are shown). The immediate forms `andi`, `ori` and `xori` take a 12-bit constant. `not` is a pseudo-instruction: `xori t0, a, -1`, because -1 is all ones and $x \oplus 1 = \overline{x}$.

8.4.2 Four things to do with a mask

The laws of the identity element and of the null element are exactly what makes a mask work (Table 8.4). With AND, a 1 in the mask lets the bit through and a 0 forces it to 0;

Table 8.3. Bitwise instructions, with $a =$ `1100 1010` and $b =$ `0000 1111`.

Instruction	Operation	Result	In C
`and t0, a, b`	bit by bit AND	`0000 1010`	`a & b`
`or  t0, a, b`	bit by bit OR	`1100 1111`	`a | b`
`xor t0, a, b`	bit by bit XOR	`1100 0101`	`a ^ b`
`not t0, a`	complement every bit	`0011 0101`	`~a`

with OR, a 0 lets the bit through and a 1 forces it to 1; with XOR, a 0 lets the bit through and a 1 inverts it.

Table 8.4. Four uses of a mask (bit 4 is button **A**).

Goal	Law used	Instruction	Other bits
Test a bit	$x \cdot 1 = x$, $x \cdot 0 = 0$	`andi t1, t0, 0x10`	forced to 0 in the result
Set a bit to 1	$x + 1 = 1$, $x + 0 = x$	`ori  t0, t0, 0x10`	unchanged
Clear a bit to 0	$x \cdot 0 = 0$, $x \cdot 1 = x$	`andi t0, t0, ~0x10`	unchanged
Toggle a bit	$x \oplus 1 = \overline{x}$, $x \oplus 0 = x$	`xori t0, t0, 0x10`	unchanged

The mask `~0x10` is `0xFFFFFFEF`, which the assembler encodes as the immediate -17.

8.4.3 De Morgan at the joystick

The worm is "idle" when neither **LEFT** nor **RIGHT** is pressed. Listing 8.2 writes the test in two ways.

```
# "idle" when NEITHER LEFT NOR RIGHT is pressed
    andi t3, t0, 0x03        # 0x03 = LEFT | RIGHT: keep both bits
    beqz t3, idle            # all zero: (LEFT + RIGHT)' is true

# the long way: test each button, the result is the same
    andi t3, t0, 0x01
    bnez t3, moving          # LEFT pressed: not idle
    andi t3, t0, 0x02
    bnez t3, moving          # RIGHT pressed: not idle
    j    idle                # LEFT' . RIGHT'
```

Listing 8.2. $\overline{L + R} = \overline{L} \cdot \overline{R}$, as two programs.

A mask with two bits tests two buttons in one instruction: the result is zero only if both bits are zero, that is, when $\overline{L + R}$ is true. By De Morgan this equals $\overline{L} \cdot \overline{R}$, which is the long way written below it. The algebra says that the two programs are equivalent, and which is cheaper: two instructions instead of five. Minimisation is not only for hardware.

8.4.4 One lamp per bit: shifting and masking

The lamps of Figure 8.1 are drawn by a loop whose body is Listing 8.3. To read bit number n of a word, shift the word to the right by n and mask with 1: it is the standard idiom.

```
    # s0 = bit number, 0 to 6
```

```
    la   t0, buttons
    lw   t1, 0(t0)          # t1 = the button mask
    srl  t1, t1, s0         # bring bit number s0 down to bit 0
    andi t1, t1, 1          # keep it, clear all the others
    li   a4, GREY           # lamp off
    beqz t1, 1f
    li   a4, YELLOW         # lamp on
1:  call prg32_gfx_rect
```

Listing 8.3. Reading bit number `s0` of the button mask.

8.5 The program of this chapter

The listings of this chapter are parts of one program, `code/l08.S` in the repository of the course, printed in full in Listing 8.4. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 8.5 is the frame that the test wrote.

On the screen.
Seven lamps, one per button (LEFT on the left), lit while the button is held; a white marker that LEFT and RIGHT move; a square that is green when neither LEFT nor RIGHT is pressed and red otherwise.

What to press.
LEFT and RIGHT move the marker; try A and B too.

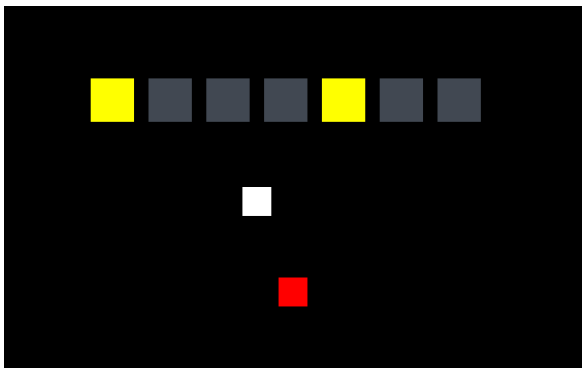


Figure 8.5. `l08.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 08: Elements of Boolean Algebra
# l08.S: one lamp per button, a marker that moves, and De Morgan at the joystick
#
# On the screen: seven lamps, one per button (LEFT on the left), lit while the button
# is held; a white marker that LEFT and RIGHT move; a square that is green when
# neither LEFT nor RIGHT is pressed and red otherwise.
# Build: python3 -m prg32 cartridge build l08.S --portable \
#            --entry-prefix l08 --name l08 --out l08.prg32
.option norelax

.equ BLACK,  0x0000
```

```
12 .equ WHITE,  0xffff
13 .equ GREY,   0x4208
14 .equ YELLOW, 0xffe0
15 .equ GREEN,  0x07e0
16 .equ RED,    0xf800
17
18 .section .text
19 .global l08_init
20 .global l08_update
21 .global l08_draw
22
23 l08_init:
24     ret
25
26 l08_update:
27     addi sp, sp, -16
28     sw   ra, 12(sp)
29     call prg32_input_read    # a0 = all the buttons, one bit each
30     mv   t0, a0              # keep our own copy
31     la   t1, buttons
32     sw   t0, 0(t1)           # remember them for draw
33
34     la   t1, marker_x
35     lw   t2, 0(t1)
36     andi t3, t0, 0x01        # keep only bit 0: LEFT
37     beqz t3, 1f              # zero: LEFT is not pressed
38     addi t2, t2, -2          # move left
39 1:
40     andi t3, t0, 0x02        # keep only bit 1: RIGHT
41     beqz t3, 2f
42     addi t2, t2, 2           # move right
43 2:
44     sw   t2, 0(t1)
45
46 # "idle" when NEITHER LEFT NOR RIGHT is pressed
47     li   t4, 1               # idle = 1
48     andi t3, t0, 0x03        # 0x03 = LEFT | RIGHT: keep both bits
49     beqz t3, 3f              # all zero: (LEFT + RIGHT)' is true
50     li   t4, 0               # idle = 0
51 3:  la   t1, idle
52     sw   t4, 0(t1)
53     lw   ra, 12(sp)
54     addi sp, sp, 16
55     ret
56
57 l08_draw:
58     addi sp, sp, -16
59     sw   ra, 12(sp)
60     sw   s0, 8(sp)
61     li   a0, BLACK
62     call prg32_gfx_clear
63     li   s0, 0               # s0 = bit number, 0 to 6
64 1:  la   t0, buttons
65     lw   t1, 0(t0)           # t1 = the button mask
66     srl  t1, t1, s0          # bring bit number s0 down to bit 0
67     andi t1, t1, 1           # keep it, clear all the others
```

```
    li   a4, GREY            # lamp off
    beqz t1, 2f
    li   a4, YELLOW          # lamp on
2:  slli a0, s0, 5
    addi a0, a0, 48          # x = 48 + 32 * bit
    li   a1, 40
    li   a2, 24
    li   a3, 24
    call prg32_gfx_rect
    addi s0, s0, 1
    li   t0, 7
    blt  s0, t0, 1b

    la   t0, marker_x        # the marker
    lw   a0, 0(t0)
    li   a1, 100
    li   a2, 16
    li   a3, 16
    li   a4, WHITE
    call prg32_gfx_rect

    la   t0, idle            # the result of De Morgan's test
    lw   t1, 0(t0)
    li   a4, RED
    beqz t1, 3f
    li   a4, GREEN
3:  li   a0, 152
    li   a1, 150
    li   a2, 16
    li   a3, 16
    call prg32_gfx_rect
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .data
buttons:  .word 0
marker_x: .word 152
idle:     .word 1
```

Listing 8.4. The complete program of the chapter, `code/l08.S`.

To run it on PRG32-QT the commands are those of Listing 8.5; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l08.S --portable \
    --entry-prefix l08 --name l08 --out l08.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l08.prg32 \
    --url http://localhost:8080 --slot cart0
```

```
# 4. look, press, compare with the previous slide
```

Listing 8.5. Building `108.S` and running it on PRG32-QT.

8.6 Check your understanding

Q8.1 For which inputs is x NAND y equal to 0?

A. $x = 0$, $y = 0$
B. $x = 0$, $y = 1$
C. $x = 1$, $y = 1$
D. never

Q8.2 By De Morgan, $\overline{a \cdot \overline{b}}$ is equal to:

A. $\overline{a} \cdot b$
B. $\overline{a} + b$
C. $a + \overline{b}$
D. $\overline{a} + \overline{b}$

Q8.3 `t0` $=$ `0x0000005A`. What is in `t1` after `andi t1, t0, 0x0F`?

A. `0x0000005A`
B. `0x00000050`
C. `0x0000000A`
D. `0x0000000F`

Q8.4 Which instruction turns bit 5 of `t0` to 1 and leaves every other bit unchanged?

A. `andi t0, t0, 0x20`
B. `ori t0, t0, 0x20`
C. `xori t0, t0, 0x20`
D. `addi t0, t0, 0x20`

8.7 Exercises

Exercise 8.1 (paper first, then PRG32)

1. Prove the second law of De Morgan, $\overline{x + y} = \overline{x} \cdot \overline{y}$, with a truth table.
2. Simplify with the algebra $x\,y + x\,\overline{y}\,z + \overline{x}\,y$, naming the law used at each step.
3. Write the mask and the instructions that test whether **A** and **B** are pressed *together*.
4. Change the lamp program so that the lamps show the complement of the buttons (lit when released).
5. Convert a letter from upper to lower case and back using one bitwise instruction (hint: the ASCII table of Chapter 4).

Expected result. On paper, the second task gives $y\,(x+\overline{x})+x\,\overline{y}\,z = y + x\,\overline{y}\,z = y + x\,z$, the last step by (8.1). For the third: `andi t1, t0, 0x30` and then a comparison *with* `0x30`; testing for non-zero would mean **A** *or* **B**. On PRG32: all seven lamps lit at rest, each one going dark while its button is held. For the letter, `xori` with `0x20` toggles bit 5.

8.8 Glossary

English	**Italiano**
logic circuit	circuito logico
switching network	rete di commutazione
analysis / synthesis	analisi / sintesi
logical product (AND)	prodotto logico (congiunzione)
logical sum (OR)	somma logica (disgiunzione)
involution	involuzione
truth table	tabella di verità
logic gate	porta logica
complement	complemento (negazione)
duality	dualità
mask	maschera
to set / clear / toggle	porre a 1 / azzerare / invertire

8.9 Conclusions and reading

The von Neumann architecture represents a real machine in simplified form: processor, memory, input/output. Boolean algebra represents logic functions that can be implemented with electronic circuits. Three operations, a handful of properties and De Morgan's laws are enough to transform any expression. A bitwise instruction is the algebra applied to thirty-two bits at once, and a mask is the identity law at work.

Reading. Patterson and Hennessy (2020), appendix A, "The Basics of Logic Design"; *Playing with RISC-V*, chapter 6, "State and Input: Branches, Bitmasks, and Boundaries" (Montella et al., 2026b); Bucci (2017), the chapters on Boolean algebra.

CHAPTER 9

Boolean Algebra: Functions, Canonical Forms and Minimisation

"Oh dear! Oh dear! I shall be too late!"
— Lewis Carroll, *Alice's Adventures in Wonderland*, 1865

In this chapter

By the end of this chapter you will be able to

- define literal, clause, factor, minterm and maxterm;
- say how many Boolean functions of n variables exist, and name those of two variables;
- write a function in canonical sum-of-products and product-of-sums form from its truth table;
- minimise a function of up to four variables with a Karnaugh map;
- derive the half adder and the full adder, and explain how thirty-two of them execute `add`.

9.1 Warm-up on the machine

Two registers hold one bit each, 0 or 1. Their sum can be 0, 1 or 2, and therefore needs two bits. Listing 9.1 computes it without any arithmetic instruction.

```
# a0 and a1 hold one bit each: 0 or 1
# goal: compute a0 + a1, which needs two bits (0, 1 or 2)

    xor  t0, a0, a1      # low bit of the sum:  1 when the bits differ
    and  t1, a0, a1      # high bit, the carry: 1 when both are 1

#   a0 a1 | t1 t0
#    0  0 |  0  0        0 + 0 = 0
#    0  1 |  0  1        0 + 1 = 1
#    1  0 |  0  1        1 + 0 = 1
#    1  1 |  1  0        1 + 1 = 2  (binary 10)
```

Listing 9.1. One-bit addition without `add`.

Two logic instructions add two bits. The comment of the listing is a truth table with two outputs, and the pair of gates that it describes is called a *half adder*. Chapter 5 claimed that an adder is all a processor needs for arithmetic: this is its first bit. But how are the gates found when only the table is given? That is the method of this chapter: truth table, canonical form, minimisation.

9.2 Boolean functions

9.2.1 The vocabulary of expressions

Definition 9.1 (Literal, clause, factor)

A *literal* is a Boolean variable, or its complement, as it appears in an expression: a, $\overline{b}$, x. A *clause*, or *product term*, is the logical product of two or more literals: $a\,\overline{b}\,x$. A *factor*, or *sum term*, is the logical sum of two or more literals: $(a + \overline{x})$, $(b + c)$.

Clauses and factors with more than one literal are functions of level 1: one layer of gates. Each occurrence of a variable counts as a literal, and the number of literals measures the cost of an expression.

9.2.2 How many functions of n variables?

Definition 9.2 (Boolean function)

A *Boolean function* of n variables is a map $f : B^n \to B$, with $B = \{0, 1\}$: it associates one binary output with every combination of n binary inputs.

Theorem 9.1 (Number of Boolean functions)

There are exactly 2^{2^n} distinct Boolean functions of n variables.

Proof. There are 2^n combinations of the inputs, and a function is determined by the choice of an output, 0 or 1, for each of them: it is a string of 2^n bits, and there are 2^{2^n} such strings. □

The number is finite, and it grows quickly: 4 functions of one variable, 16 of two, 256 of three, 65 536 of four, and more than four thousand million of five. Boolean functions are isomorphic to digital circuits: every circuit can be written as an expression, and every expression can be built as a circuit.

9.2.3 The sixteen functions of two variables

Table 9.1 lists all the functions of two variables. The index of each function is its column of outputs read as a binary number. With one variable the four functions are the constant 0, x, $\overline{x}$ and the constant 1.

Table 9.1. The sixteen functions of two variables. The columns 00, 01, 10, 11 give the output for each pair of inputs $x\,y$.

f	00	01	10	11	**name**	f	00	01	10	11	**name**
f_0	0	0	0	0	constant 0	f_8	1	0	0	0	**NOR**
f_1	0	0	0	1	**AND**: $x\,y$	f_9	1	0	0	1	**XNOR** (equivalence)
f_2	0	0	1	0	$x\,\overline{y}$ (inhibition)	f_{10}	1	0	1	0	NOT y
f_3	0	0	1	1	x (identity)	f_{11}	1	0	1	1	$x + \overline{y}$ (implication $y \to x$)
f_4	0	1	0	0	$\overline{x}\,y$ (inhibition)	f_{12}	1	1	0	0	NOT x
f_5	0	1	0	1	y (identity)	f_{13}	1	1	0	1	$\overline{x} + y$ (implication $x \to y$)
f_6	0	1	1	0	**XOR**	f_{14}	1	1	1	0	**NAND**
f_7	0	1	1	1	**OR**: $x + y$	f_{15}	1	1	1	1	constant 1

9.2.4 Functionally complete sets

Any Boolean function can be realised using only AND, OR and NOT; for example $x \oplus y = \overline{x}\,y + x\,\overline{y}$. (Section 9.3 proves the general statement by construction.) The set $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}\}$ is therefore called *functionally complete*.

By De Morgan, AND can be built from OR and NOT, and OR from AND and NOT: the sets $\{\mathrm{AND}, \mathrm{NOT}\}$ and $\{\mathrm{OR}, \mathrm{NOT}\}$ are complete too. And a single gate is enough.

Theorem 9.2 (Universal gates)

NAND alone is functionally complete, and so is NOR alone.

Proof for NAND, writing $x \uparrow y = \overline{xy}$:

$$\overline{x} = x \uparrow x, \qquad x\,y = (x \uparrow y) \uparrow (x \uparrow y), \qquad x + y = (x \uparrow x) \uparrow (y \uparrow y),$$

the last by De Morgan. The proof for NOR is dual. □

A single type of gate is enough to build any digital circuit, a whole processor included. Real chips are largely built from NAND and NOR because these are the gates that need the fewest transistors (Chapter 10).

9.2.5 Truth tables

A Boolean function can be defined by its truth table, which gives the value of the dependent variable for every combination of the independent variables. A function of n variables is defined on 2^n points, and its table has 2^n rows, listed in counting order: 000, 001, 010, … The function F of Table 9.2, which is 1 on rows 3, 4, 5, 6 and 7, will be followed through the whole chapter.

9.2.6 The half adder

The *half adder* adds two bits A and B and produces a sum bit S and a carry C. In binary $1 + 1$ gives sum 0 and carry 1; from the table, S is 1 when the inputs differ and C is 1 when both are 1 (Figure 9.1):

$$S = A \oplus B, \qquad C = A \cdot B.$$

Table 9.2. The function F, with the minterm or the maxterm of each row.

row	A	B	C	F	**minterm** ($F = 1$)	**maxterm** ($F = 0$)
0	0	0	0	0		$M_0 = A + B + C$
1	0	0	1	0		$M_1 = A + B + \overline{C}$
2	0	1	0	0		$M_2 = A + \overline{B} + C$
3	0	1	1	1	$m_3 = \overline{A}\,B\,C$	
4	1	0	0	1	$m_4 = A\,\overline{B}\,\overline{C}$	
5	1	0	1	1	$m_5 = A\,\overline{B}\,C$	
6	1	1	0	1	$m_6 = A\,B\,\overline{C}$	
7	1	1	1	1	$m_7 = A\,B\,C$	

The circuit is exactly the two instructions of the warm-up. It is "half" an adder because it has no carry input: it can only add the least significant bits of two numbers.

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

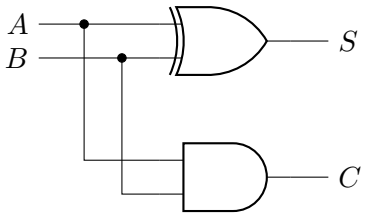


Figure 9.1. The half adder.

9.2.7 The full adder

The *full adder* adds two bits and a carry in, C_{in}, and produces a sum bit and a carry out, C_{out} (Figure 9.2). S is 1 when an odd number of inputs is 1; C_{out} is 1 when at least two inputs are 1:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = A\,B + C_{in}\,(A \oplus B).$$

The circuit is two half adders and an OR.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

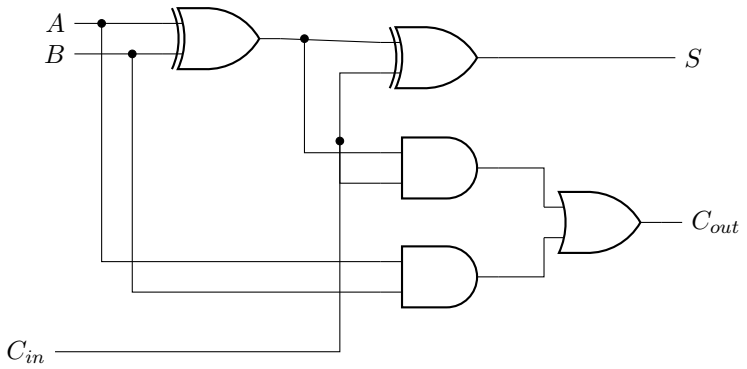


Figure 9.2. The full adder: two half adders and an OR. Wires that cross without a dot are not connected.

9.3 Canonical forms

Canonical forms are a systematic way from the table to an expression.

9.3.1 Elementary forms: functions of level 2

Definition 9.3 (Sum of products and product of sums)

An expression of *type P*, or *sum of products* (SOP), is a logical sum of clauses, such as $y = a\,\overline{b} + \overline{a}\,d + b\,\overline{d}\,x$. It is built as a layer of AND gates feeding one OR gate. An expression of *type S*, or *product of sums* (POS), is a logical product of factors, such as $y = (a + \overline{b})(\overline{a} + d)(b + \overline{d} + x)$. It is built as a layer of OR gates feeding one AND gate.

9.3.2 Minterms and maxterms

Definition 9.4 (Minterm and maxterm)

A *minterm* of n variables is a product that contains one literal of every variable. A *maxterm* is a sum that contains one literal of every variable.

A minterm is 1 for exactly one row of the truth table, and a maxterm is 0 for exactly one row. The minterm m_i of row i contains a variable complemented where the row has a 0; the maxterm M_i contains a variable complemented where the row has a 1 (Table 9.2). By De Morgan, $M_i = \overline{m_i}$.

9.3.3 From the truth table to the canonical forms

Theorem 9.3 (Canonical forms)

Every Boolean function is equal to
1. the sum of the minterms of the rows where it is 1 (*canonical form P*);
2. the product of the maxterms of the rows where it is 0 (*canonical form S*).

Each of the two forms is unique, apart from the order of the terms.

Proof. On a row where $f = 1$, the minterm of that row is 1 and so is the sum; on a row where $f = 0$, all the minterms of the sum are 0. The second statement is the dual. □

For the function F:

$$F = \overline{A}BC + A\overline{B}\,\overline{C} + A\overline{B}C + AB\overline{C} + ABC \;=\; \sum m(3,4,5,6,7),$$
$$F = (A + B + C)\,(A + B + \overline{C})\,(A + \overline{B} + C) \;=\; \prod M(0,1,2).$$

The theorem also proves that AND, OR and NOT are functionally complete. It is synthesis by recipe: always possible, rarely economical.

9.3.4 Algebraic minimisation of F

The canonical form P of F has 5 clauses and 15 literals. The algebra of Chapter 8 reduces it:

$$
\begin{aligned}
F &= \overline{A}BC + A\overline{B}\,\overline{C} + A\overline{B}C + AB\overline{C} + ABC \\
&= \overline{A}BC + A\left(\overline{B}\,\overline{C} + \overline{B}C + B\overline{C} + BC\right) && \text{distributive: collect } A \\
&= \overline{A}BC + A\left(\overline{B}\left(\overline{C} + C\right) + B\left(\overline{C} + C\right)\right) && \text{distributive, twice} \\
&= \overline{A}BC + A\left(\overline{B} + B\right) && \text{complement: } \overline{C} + C = 1 \\
&= \overline{A}BC + A && \text{complement: } \overline{B} + B = 1 \\
&= A + BC && \text{simplification: } x + \overline{x}\,y = x + y.
\end{aligned}
$$

Fifteen literals have become three: from five AND gates with three inputs and an OR with five, to one AND and one OR. It works, but each step needs an idea. What is wanted is a method.

9.3.5 The goal of minimisation

The goal is to find, among all the possible two-level forms of type P, a *minimal* elementary form. Fewer and smaller gates mean lower cost, less power and a faster circuit. Three methods are classical.

- **Karnaugh maps**: graphical, by groups of adjacent cells of maximum area; practical up to four or five variables (Karnaugh, 1953).
- **Quine's method**: algebraic, by systematic comparison of the terms.
- **McCluskey's method**: tabular, suited to a program, and the basis of today's logic-synthesis tools. The Quine–McCluskey procedure is exact but exponential; industrial tools use heuristics.

9.4 Karnaugh maps

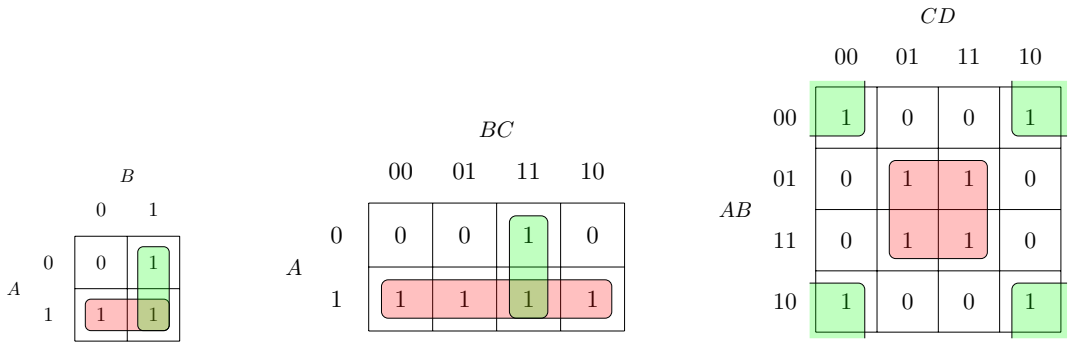
9.4.1 The idea

A Karnaugh map is the truth table of a function of n variables redrawn on a grid of 2^n cells, one cell per row. The cells are ordered so that two neighbours differ in one variable only: hence the order 00, 01, 11, 10 of the columns, which is a Gray code (Chapter 11), and hence the fact that in a map m_3 comes before m_2. The left and right edges are adjacent too: the map wraps round.

Graphical adjacency is thus logical adjacency: two adjacent 1s are two minterms that differ in one variable, and by $x\,y + x\,\overline{y} = x$ they merge into a clause with one literal fewer.

The rule of the map

Group the 1s in rectangles of 1, 2, 4 or 8 cells, as large as possible, until every 1 is covered. A group of 2^k cells eliminates k variables: the clause of a group contains only the variables that do not change inside it.



(a) $F = A + B$ **(b)** $F = A + BC$ **(c)** $F = BD + \overline{B}\,\overline{D}$

Figure 9.3. Karnaugh maps of two, three and four variables.

9.4.2 Two, three and four variables

Two variables (Figure 9.3a). The function is 1 for $AB = 01$, 10 and 11; its canonical form $\overline{A}B + A\overline{B} + AB$ has six literals. One group is the row $A = 1$: inside it B changes, so B disappears and the clause is A. The other is the column $B = 1$: A changes, and the clause is B. Hence $F = A + B$: the function is simply OR. The cell AB belongs to both groups; using a 1 twice is the idempotence law, $x + x = x$.

Three variables (Figure 9.3b). This is the function F of the chapter: a 1 in the cells of minterms 3, 4, 5, 6 and 7. A group of four covers the whole row $A = 1$: B and C both change, and the clause is A. A group of two covers the column $BC = 11$: A changes, and the clause is BC. Every 1 is covered and no group can be made larger: $F = A + BC$, the same result as the algebra, found by eye in seconds.

Four variables (Figure 9.3c). The function $F = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$ has eight minterms and thirty-two literals. A group of four in the centre has $B = 1$ and $D = 1$ throughout: the clause is BD. The four corners are also one group, because the map wraps round both horizontally and vertically, as if it were drawn on a torus: there $B = 0$ and $D = 0$, and the clause is $\overline{B}\,\overline{D}$. The result has four literals, and can be recognised: F is the XNOR of B and D.

The practical rule is: always take the largest groups first, then cover what is left.

9.4.3 Don't-care conditions

Sometimes a combination of the inputs can never occur, or the output does not matter when it does. A digit in binary-coded decimal, for example, uses the patterns 0000 to 1001, and the six patterns 1010 to 1111 never appear. Those cells are marked in the map with a dash (or, in many texts, with a cross): *don't care*. A don't-care cell may be treated as a 1 when it makes a group larger, and as a 0 otherwise. More freedom means larger groups and a cheaper circuit. The classic exercise is the decoder for a seven-segment display, where each segment is a function of four variables with six don't-cares.

9.5 Back to the machine

9.5.1 Thirty-two full adders make an `add`

The instruction `add t2, t0, t1` feeds the two registers into thirty-two full adders, one per bit position (Figure 9.4). The carry out of each position is the carry in of the next: a *ripple-carry adder*, in which the carry ripples leftwards as it does in a sum done by hand.

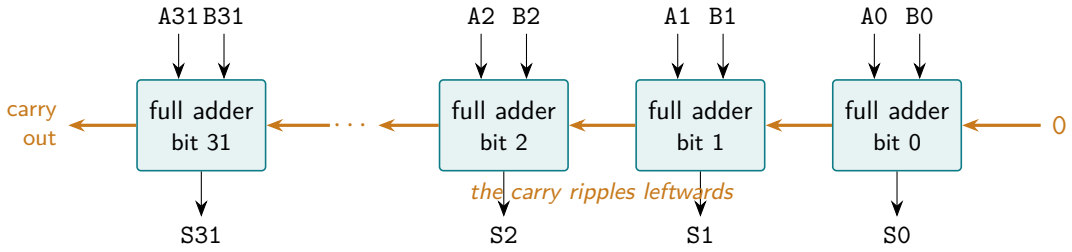


Figure 9.4. A ripple-carry adder of 32 bits.

The carry out of bit 31 is the unsigned overflow of Chapter 5, which RISC-V simply drops. Real processors use faster schemes for the carry, such as carry look-ahead (Chapter 11), but the function computed is the same.

9.5.2 A full adder, one gate per instruction

```
# a0 = A, a1 = B, a2 = Cin   (each 0 or 1)
# result: a3 = S, a4 = Cout

    xor  t0, a0, a1      # t0   = A xor B
    xor  a3, t0, a2      # S    = A xor B xor Cin
    and  t1, a0, a1      # t1   = A . B
    and  t2, t0, a2      # t2   = (A xor B) . Cin
    or   a4, t1, t2      # Cout = A . B + (A xor B) . Cin
```

Listing 9.2. The full adder as a program.

Five instructions, five gates: Listing 9.2 is the schematic of Figure 9.2, read from left to right. Single-stepping it in PRG32-QT and filling in the truth table is *analysis*; deriving these gates from the table with the method of this chapter was *synthesis*. And with whole registers as inputs, the same five lines work on all thirty-two bit positions at once.

9.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l09.S` in the repository of the course, printed in full in Listing 9.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 9.5 is the frame that the test wrote.

On the screen.
Eight rows, one per combination of A, B and Cin (000 at the top): three lamps for the inputs on the left, then S and Cout on the right. It is the truth table of the full adder.

What to press.
Nothing: the program runs by itself.

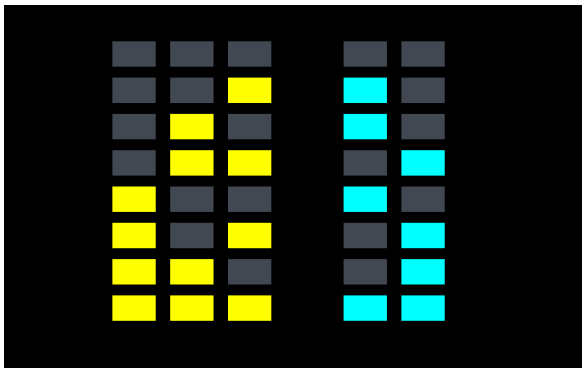


Figure 9.5. `l09.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 09: Elements of Boolean Algebra
# l09.S: a full adder, one gate per instruction, on its eight input combinations
#
# On the screen: eight rows, one per combination of A, B and Cin (000 at the top):
# three lamps for the inputs on the left, then S and Cout on the right. It is the
# truth table of the full adder.
# Build: python3 -m prg32 cartridge build l09.S --portable \
#            --entry-prefix l09 --name l09 --out l09.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ CYAN,   0x07ff

.section .text
.global l09_init
.global l09_update
.global l09_draw

l09_init:
    ret

l09_update:
    ret

# full_adder: a0 = A, a1 = B, a2 = Cin (each 0 or 1); result: a3 = S, a4 = Cout
full_adder:
    xor  t0, a0, a1      # t0   = A xor B
    xor  a3, t0, a2      # S    = A xor B xor Cin
    and  t1, a0, a1      # t1   = A . B
    and  t2, t0, a2      # t2   = (A xor B) . Cin
    or   a4, t1, t2      # Cout = A . B + (A xor B) . Cin
    ret

# lamp: a0 = column, a1 = row, a2 = bit (0 or 1), a3 = colour when the bit is 1
lamp:
```

```
    li   a4, GREY
    beqz a2, 1f
    mv   a4, a3
1:  slli a0, a0, 5
    addi a0, a0, 60          # x = 60 + 32 * column
    slli a1, a1, 4
    slli t0, a1, 1
    srli a1, a1, 1
    add  a1, a1, t0          # 40 * row ...
    srli a1, a1, 1           # ... / 2 = 20 * row
    addi a1, a1, 20          # y = 20 + 20 * row
    li   a2, 24
    li   a3, 14
    tail prg32_gfx_rect      # a tail call: rect returns to our caller

l09_draw:
    addi sp, sp, -32
    sw   ra, 28(sp)
    sw   s0, 24(sp)
    sw   s1, 20(sp)
    sw   s2, 16(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   s0, 0               # row = the three inputs as a number, 0 to 7
1:  srli a0, s0, 2
    andi a0, a0, 1           # A   = bit 2 of the row
    srli a1, s0, 1
    andi a1, a1, 1           # B   = bit 1
    andi a2, s0, 1           # Cin = bit 0
    call full_adder
    mv   s1, a3              # keep S and Cout: they are in a registers,
    mv   s2, a4              # which the next calls destroy

    li   a0, 0               # column 0: A
    mv   a1, s0
    srli a2, s0, 2
    andi a2, a2, 1
    li   a3, YELLOW
    call lamp
    li   a0, 1               # column 1: B
    mv   a1, s0
    srli a2, s0, 1
    andi a2, a2, 1
    li   a3, YELLOW
    call lamp
    li   a0, 2               # column 2: Cin
    mv   a1, s0
    andi a2, s0, 1
    li   a3, YELLOW
    call lamp
    li   a0, 4               # column 4: S
    mv   a1, s0
    mv   a2, s1
    li   a3, CYAN
    call lamp
    li   a0, 5               # column 5: Cout
```

```
94      mv   a1, s0
95      mv   a2, s2
96      li   a3, CYAN
97      call lamp
98
99      addi s0, s0, 1
100     li   t0, 8
101     blt  s0, t0, 1b
102     lw   s2, 16(sp)
103     lw   s1, 20(sp)
104     lw   s0, 24(sp)
105     lw   ra, 28(sp)
106     addi sp, sp, 32
107     ret
```

Listing 9.3. The complete program of the chapter, `code/l09.S`.

To run it on PRG32-QT the commands are those of Listing 9.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l09.S --portable \
    --entry-prefix l09 --name l09 --out l09.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l09.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 9.4. Building `l09.S` and running it on PRG32-QT.

9.7 Check your understanding

Q9.1 How many distinct Boolean functions of three variables are there?

A. 8
B. 9
C. 64
D. 256

Q9.2 Which minterm of three variables A, B, C corresponds to row 5 of the truth table?

A. $\overline{A}\,B\,\overline{C}$
B. $A\,\overline{B}\,C$
C. $A + \overline{B} + C$
D. $A\,B\,\overline{C}$

Q9.3 In a Karnaugh map, a rectangular group of four adjacent 1s eliminates:

A. one variable
B. two variables
C. three variables
D. four variables

Q9.4 In a full adder, the carry out is 1 when:

A. exactly one input is 1
B. an odd number of inputs is 1
C. at least two inputs are 1
D. all three inputs are 1 only

9.8 Exercises

Exercise 9.1 (paper first, then PRG32)

1. Write the canonical forms P and S of the outputs S and C_{out} of the full adder.
2. Minimise C_{out} with a Karnaugh map of three variables. How many gates are saved?
3. Minimise $F = \sum m(0, 1, 2, 5, 8, 9, 10)$ with a map of four variables.
4. Build NOT, AND and OR using NAND only, and check each with a truth table.
5. Program the full adder on PRG32 and display S and C_{out} as lamps for the eight combinations of the inputs.

Expected result. On paper: the canonical form of C_{out} has four minterms (3, 5, 6, 7) and the map gives three clauses of two literals, $C_{out} = AB + AC_{in} + BC_{in}$, the *majority* function. The third task gives $F = \overline{B}\,\overline{D} + \overline{B}\,\overline{C} + \overline{A}\,\overline{C}\,D$. On PRG32: eight pairs of lamps that reproduce the truth table of the full adder.

9.9 Glossary

English	Italiano
literal	letterale
clause, product term	clausola, termine prodotto
factor, sum term	fattore, termine somma
minterm / maxterm	mintermine / maxtermine
half adder / full adder	semisommatore / sommatore completo
carry in / carry out	riporto entrante / uscente
canonical form	forma canonica
sum of products (SOP)	somma di prodotti (tipo P)
product of sums (POS)	prodotto di somme (tipo S)
Karnaugh map	mappa di Karnaugh
don't care	condizione di indifferenza
ripple-carry adder	sommatore a propagazione del riporto

9.10 Conclusions and reading

A Boolean function is a truth table, and a truth table always gives a canonical expression. Minimisation is the process that optimises a logic network in order to simplify its imple-

mentation. Karnaugh maps turn the algebra into a picture: group the 1s, read the clauses. The half adder and the full adder are the first real circuits of this book, and thirty-two full adders in a row are what executes the `add` typed in Chapter 2.

Reading. Patterson and Hennessy (2020), appendix A; Bucci (2017), the chapters on combinational networks; Charles Petzold, *Code*, the chapters on logic gates and on binary adding machines.

CHAPTER 10

Electronics: What a Bit Is Made Of

"Any sufficiently advanced technology is indistinguishable from magic."
— Arthur C. Clarke, *Profiles of the Future*, 1973

In this chapter

By the end of this chapter you will be able to

- use voltage, current, resistance and power, and apply Ohm's law to a simple circuit;
- size the resistor of an LED and compute a voltage divider;
- explain what a transistor is and how a MOS transistor works as a switch;
- say how voltages become logic values, and what a noise margin is;
- draw the CMOS inverter, NAND and NOR, and count their transistors;
- explain pull-up resistors, floating inputs and why a gate takes time and energy to switch.

The gates of the last two chapters were symbols on paper. This chapter gives them a physical body, before the next one composes them into machines. No previous knowledge of physics is assumed. The numbers quoted for the kit come from the datasheet of the ESP32-C6 (Espressif Systems, 2025) and from the hardware documentation of PRG32.

10.1 Warm-up on the machine

```
    call prg32_input_read    # a0 = one bit per button
    andi t3, a0, 0x01        # bit 0: LEFT
    beqz t3, 1f              # 0: not pressed
    ...                      # 1: pressed
1:

# on the kit:
#   LEFT is a switch between pin GPIO18 and ground
#   pressed  -> the pin is at 0 volts
#   released -> the pin is at 3.3 volts
```

Listing 10.1. One button, one bit.

Since the first chapters a pressed button has been a 1 in a register (Listing 10.1). On the board the button is a piece of metal that joins a pin of the chip to ground. Pressed means 0 volts on the pin, and yet the program reads 1. What is a volt? Why is the released pin at 3.3 V? And who decides that 0 V is a bit? The chapter follows the button from the finger to the bit, and its last section answers the three questions.

10.2 Electricity in four quantities

10.2.1 Voltage, current, resistance, power

Electric charge is carried by electrons, and four quantities describe what it does in a circuit (Table 10.1). A comparison with water in pipes helps the intuition.

Table 10.1. Four quantities.

Quantity	Symbol, unit	Water analogy	On the kit
voltage	V, volt (V)	the pressure that pushes the water	3.3 V between the supply rail and ground
current	I, ampere (A)	the flow: litres per second	about 10 mA through a lit LED
resistance	R, ohm (Ω)	a narrow pipe that hinders the flow	45 kΩ: the pull-up inside each input pin
power	P, watt (W)	the work the flow can do each second	at most 2.5 W from a USB 2.0 port (5 V, 0.5 A)

A voltage is always measured *between two points*. When one says that "the pin is at 3.3 V" one means with respect to *ground* (GND), the common reference of the whole circuit, which is at 0 V by definition. The usual prefixes are milli (m, 10^{-3}), micro (μ, 10^{-6}), nano (n, 10^{-9}), pico (p, 10^{-12}), kilo (k, 10^{3}) and mega (M, 10^{6}).

10.2.2 Ohm's law, and the resistor of an LED

Theorem 10.1 (Ohm's law, 1827)

The current through a resistor is proportional to the voltage across it:

$$V = R \cdot I.$$

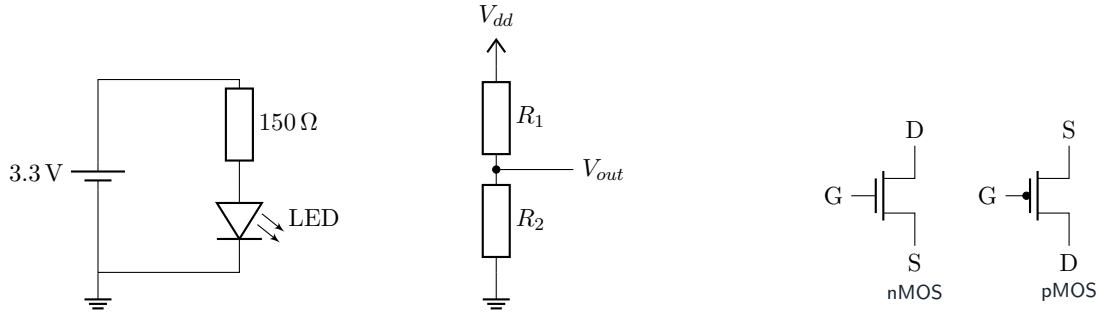
A circuit is a closed loop: the current leaves the source, crosses the components and returns. An LED (Figure 10.1a) lights when current flows through it, and when it does there is a drop of about 2 V across it; without a resistor in series the current would be far too large. The resistor takes what is left, $3.3 - 2 = 1.3$ V. For a current of 10 mA,

$$R = \frac{1.3\ \text{V}}{0.010\ \text{A}} = 130\ \Omega,$$

and the nearest common value is 150 Ω, which gives $1.3/150 = 8.7$ mA. The forward voltage of an LED depends on its colour: about 1.8–2.2 V for red, about 3 V for blue and white.

10.2.3 Series, parallel and the voltage divider

Two resistors are in *series* when the same current crosses both: the resistances add, $R = R_1 + R_2$. They are in *parallel* when the same voltage is across both: the current splits, and



(a) An LED and its resistor **(b)** A voltage divider **(c)** The two MOS transistors

Figure 10.1. Three elementary circuits.

the total resistance, $1/R = 1/R_1 + 1/R_2$, is smaller than either.

Two resistors in series divide the voltage (Figure 10.1b):

$$V_{out} = V_{dd} \frac{R_2}{R_1 + R_2}. \tag{10.1}$$

Equal resistors give half the supply; a much smaller R_2 pulls V_{out} close to ground. The formula follows from Ohm's law, since the current $I = V_{dd}/(R_1 + R_2)$ crosses R_2, and from Kirchhoff's rules: the currents that enter a node add up to those that leave it, and the voltages round a loop add up to zero. The divider is the key to the circuit of the button, where R_2 is a switch: a resistance that is either infinite or nearly zero.

10.2.4 Power and energy

Power is voltage times current, $P = V \cdot I$, in watts; in a resistor it becomes heat, $P = R \cdot I^2$. *Energy* is power times time, in joules; a battery stores energy. The LED above consumes $3.3\ \text{V} \times 8.7\ \text{mA} = 29\ \text{mW}$, of which 11 mW heat the resistor. Every transistor that switches inside a processor dissipates a little energy, and with thousands of millions of them heat is the limit of modern chips: a laptop processor dissipates tens of watts in a few square centimetres.

10.2.5 The capacitor: why signals take time

A *capacitor* stores electric charge as a tank stores water; its capacitance is measured in farads (F). Its voltage cannot jump: the capacitor must be filled, or emptied, through a resistance, and the time that this takes is of the order of the product

$$\tau = R \cdot C,$$

the *time constant*, after which the voltage has covered about 63% of the way. An input pin of the kit has a capacitance of about 2 pF and rises through its pull-up of 45 kΩ in about $45\ \text{k}\Omega \times 2\ \text{pF} = 90\ \text{ns}$ (the wiring and the button add more, so the real figure is larger).

Every wire and every input of a transistor is a small capacitor. This is where the propagation delay of a gate comes from: resistance times capacitance is a time, and it is the reason why nothing in a circuit is instantaneous. A capacitor that is charged or empty is also a way of storing a bit: the DRAM cell of Chapter 15.

10.3 Semiconductors

10.3.1 Conductors, insulators, semiconductors

Conductors, such as copper, let charge flow freely; insulators, such as glass, do not. *Semiconductors*, above all silicon, lie in between, and their conduction can be controlled. The control is obtained by *doping*: adding a few atoms of another element. Silicon of type n has extra electrons, which are negative carriers of charge; silicon of type p has missing electrons, called *holes*, which behave as positive carriers. Where a p region meets an n region there is a *junction*, the building block of every device. A chip is a slice of silicon with thousands of millions of doped regions, connected by layers of metal. Silicon is the second most abundant element in the Earth's crust, the element of sand, and a whole industry rests on purifying it and doping it with extreme precision.

10.3.2 The diode and the LED

A single junction makes a *diode*, a one-way valve for current. In the forward direction, from p to n, it conducts once the voltage exceeds a threshold, about 0.7 V for silicon; in the reverse direction it blocks. A light-emitting diode gives off light when it conducts; its colour depends on the material, and so does its threshold. Diodes turn alternating current into direct current in every power supply. LEDs are everywhere in a computer: indicators, the backlight of the display of Chapter 20, optical links; and the laser of the optical disks of Chapter 16 is a diode too.

10.3.3 The transistor: a switch with no moving parts

A *transistor* is a device with three terminals in which a small signal on one controls the current between the other two. Used as an amplifier it made the radio portable; used as a switch it made the computer (Table 10.2). Vacuum tubes and relays did the same job in the first computers of Chapter 3, a million times larger and slower.

Table 10.2. The transistor in six dates.

Year	Who	What
1947	Bardeen, Brattain and Shockley, Bell Labs	the first transistor: it replaces the vacuum tube
1958–59	Jack Kilby; Robert Noyce	the integrated circuit: several transistors on one piece of semiconductor
1959–60	Atalla and Kahng, Bell Labs	the MOS transistor: small, simple to build, almost no control current
1963	Wanlass and Sah, Fairchild	CMOS: pairs of complementary MOS transistors that draw power only when switching
1971	Intel 4004	2 300 MOS transistors: a processor on a chip
today	the whole industry	tens of thousands of millions of CMOS transistors on a chip

10.3.4 The MOS transistor as a switch

The name MOS lists the three layers of the device: metal, oxide, semiconductor. Its three terminals are the *gate*, which controls whether current can pass, and the *source* and the

drain, between which it passes (Figure 10.1c). The gate is insulated: it draws practically no current (at most 50 nA on a pin of the ESP32-C6), only the tiny charge of its capacitance.

Definition 10.1 (nMOS and pMOS)

An *nMOS* transistor conducts (the switch is closed) when its gate is high, and blocks when it is low. A *pMOS* transistor conducts when its gate is low, and blocks when it is high.

The two types are complementary: what closes one opens the other. The small circle on the gate of the pMOS means, as on logic symbols, "active when low". One more fact decides how they are used: an nMOS passes a good 0, and is therefore used to connect to ground; a pMOS passes a good 1, and is used to connect to the supply.

10.4 From volts to bits

10.4.1 Logic levels: when is a voltage a 1?

A digital circuit uses two *ranges* of voltage: low is 0, high is 1 (Figure 10.2). An output guarantees clean levels: on the kit, above 2.64 V for a 1 and below 0.33 V for a 0. An input accepts wider ranges: above 2.475 V it reads 1, below 0.825 V it reads 0. (With $V_{DD} = 3.3$ V the datasheet gives $V_{OH} \geq 0.8\,V_{DD}$, $V_{OL} \leq 0.1\,V_{DD}$, $V_{IH} \geq 0.75\,V_{DD}$, $V_{IL} \leq 0.25\,V_{DD}$.)

Definition 10.2 (Noise margin)

The *noise margin* is the difference between the level that an output guarantees and the level that an input requires. A disturbance smaller than the noise margin cannot change the bit.

On the kit the margins are $2.64 - 2.475 = 0.165$ V for the high level and $0.825 - 0.33 = 0.495$ V for the low level. Between the two input thresholds the value is undefined: signals must cross that band quickly and never stay there.

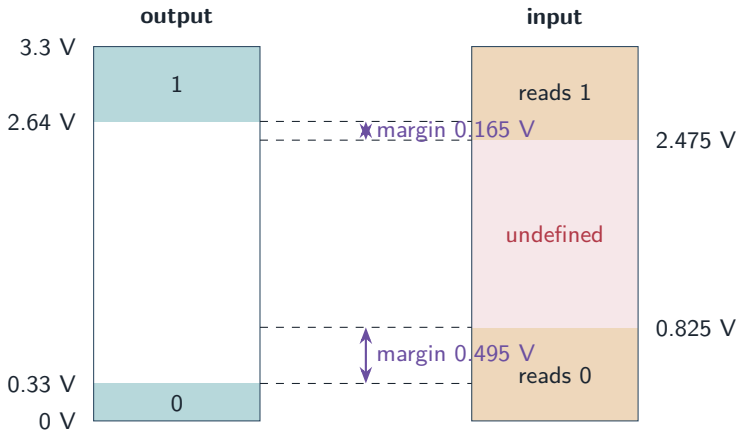


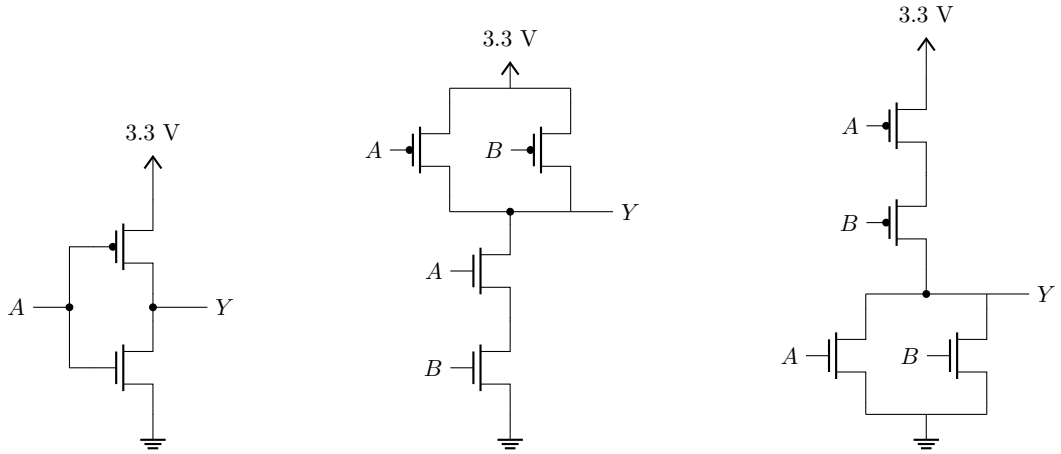
Figure 10.2. Logic levels of the ESP32-C6 with a supply of 3.3 V.

10.4.2 Why digital?

In an *analog* circuit the exact value of the voltage is the information. Every stage adds a little noise and distortion, which accumulate, and copies get worse at every generation. Analog circuits remain indispensable at the edges of a system: sensors, radio, sound. In a *digital* circuit only the range matters, high or low. Every gate restores the levels, so that noise does not accumulate; copies are perfect, and errors can be detected and corrected (Chapter 11). And two levels are easy to tell apart with transistors that are fast, tiny and cheap. It is the story of Chapter 4, seen from the wire.

10.4.3 The CMOS inverter: a NOT gate from two transistors

The simplest gate is made of one pMOS towards the supply and one nMOS towards ground, driven by the same input (Figure 10.3a).



(a) NOT: $Y = \overline{A}$ **(b)** NAND: $Y = \overline{A \cdot B}$ **(c)** NOR: $Y = \overline{A + B}$

Figure 10.3. Three CMOS gates. In the NAND and in the NOR the gates of the transistors marked with the same letter are connected together.

A	**pMOS**	**nMOS**	Y
0	on	off	1: joined to 3.3 V
1	off	on	0: joined to ground

Exactly one of the two transistors conducts: the output is always connected to one rail, never to both. With the input still, no current flows from the supply to ground, so that CMOS (*complementary* MOS) consumes almost nothing at rest. And the output is a fresh, clean level whatever the quality of the input: the gate *regenerates* the signal.

10.4.4 NAND and NOR in CMOS

Series is AND, parallel is OR

Two switches in series conduct when the first *and* the second are closed; two switches in parallel conduct when the first *or* the second is closed. It is the idea of Nakajima and Shannon.

In the NAND (Figure 10.3b) two nMOS in series reach ground only if A and B are both 1, and two pMOS in parallel reach the supply if either is 0. The NOR (Figure 10.3c) is the dual: parallel becomes series and series becomes parallel, exactly as De Morgan's laws exchange AND and OR. In every CMOS gate the network of pMOS that pulls the output up and the network of nMOS that pulls it down are each other's dual, in the sense of the duality principle of Chapter 8. Each of the two gates has four transistors; the reader should check the four combinations of the inputs on the drawings.

10.4.5 What gates cost, in transistors

Table 10.3. The cost of some circuits in conventional static CMOS; the counts vary with the style of design.

Circuit	Transistors	Remark
NOT	2	the CMOS inverter
NAND, NOR (2 inputs)	4	the natural gates of CMOS
AND, OR (2 inputs)	6	a NAND or a NOR followed by an inverter
XOR (2 inputs)	about 8 to 12	depending on the design
full adder	about 28	the classic static CMOS adder
SRAM cell, one bit	6	two inverters in a loop and two access transistors
D flip-flop	about 20	two latches, master and slave

An AND costs more than a NAND (Table 10.3): in CMOS the inverting gates come first. This is why NAND is the universal gate of Chapter 9 in practice as well as in theory. Minimising a function means saving transistors, and therefore area, energy and time. The adder behind the instruction `add`, thirty-two full adders, is about 900 transistors.

10.4.6 Switching takes time and energy

The output of a gate drives the inputs of the next gates, which are small capacitors. To change level the output must charge or discharge them through the resistance of its transistors: this is the *propagation delay* of the gate. And every change of level costs energy: the power spent in switching is about

$$P_{dyn} \approx C \cdot V^2 \cdot f,$$

where C is the capacitance switched, V the supply voltage and f the frequency of switching. Lowering the supply voltage saves the most, because it enters squared: it is the reason why chips moved from 5 V to 3.3 V and to about 1 V inside. Raising the clock frequency raises the consumption in proportion: speed is paid for in heat. The propagation delay returns in Chapter 11 as the critical path, and in Chapter 13 as the reason for the clock.

10.4.7 Scales of integration

The ESP32-C6 of the kit is a system on a chip in a package of a few millimetres (Table 10.4). The chips of the 74 series reappear in the next chapters as the historical form of each building block.

10.5 Back to the machine

Table 10.4. Scales of integration.

Scale	Transistors per chip	Examples
SSI, small	up to about 100	a few gates: the 74 series, such as four NANDs in one package
MSI, medium	hundreds	decoders, multiplexers, counters, registers: the machines of Chapters 11 to 14
LSI, large	thousands	the first microprocessors and memories: Intel 4004
VLSI, very large	hundreds of thousands and more	the processors of the 1980s and 1990s
today	tens of thousands of millions	systems on a chip: processor, memory, radio and peripherals together

10.5.1 Pull-up: an input that is never left floating

An input connected to nothing is *floating*: it picks up noise and reads 0 or 1 at random. A *pull-up resistor* ties the pin to 3.3 V (Figure 10.4). With the button released no current flows, and the pin reads a clean 1. With the button pressed the switch joins the pin to ground: 0 V, a clean 0, while a current of 3.3 V/45 kΩ = 73 μA flows through the resistor, too small to matter. The circuit is the voltage divider of (10.1) with a lower resistor that is either infinite or zero.

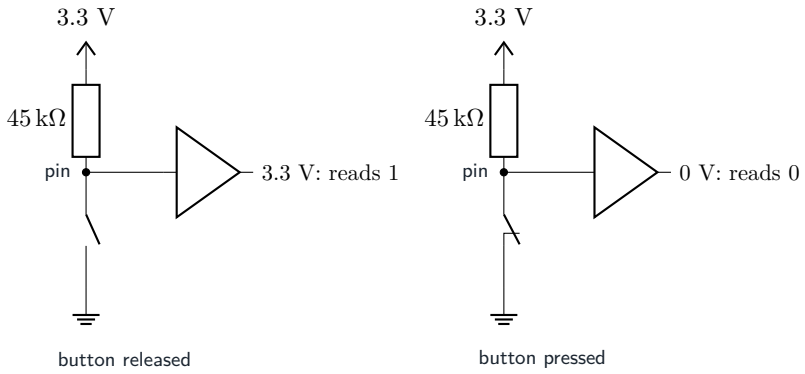


Figure 10.4. The button of the kit with its pull-up resistor. The triangle is the input buffer of the pin.

The firmware enables the internal pull-up of every button pin, reads the pins and inverts the bits, so that in the program pressed is 1. A signal whose active state is the low level is said to use *active-low* logic.

10.5.2 From the finger to the bit

The questions of the warm-up now have answers, and Table 10.5 puts them in order. Some of its steps belong to chapters still to come.

The contacts of a real switch *bounce* for a few milliseconds when they close. Reading the buttons once per frame, every few tens of milliseconds, hides the bounce (Chapter 21).

Table 10.5. From the finger to the bit.

Step	What happens	Where it is explained
1	the finger closes the switch between GPIO18 and ground	this chapter: the circuit
2	the pin falls from 3.3 V to 0 V, in well under a microsecond	this chapter: $R \cdot C$
3	the input buffer sees a voltage below 0.825 V: a logic 0	this chapter: logic levels
4	the level is sampled into a flip-flop of the input register	Chapter 13
5	the firmware reads that register with a load instruction	Chapter 15: memory-mapped I/O
6	it inverts the bit and packs it with the others into `a0`	Chapters 8 and 21
7	the program tests it: `andi t3, a0, 0x01`	Chapter 2

10.5.3 The kit as a circuit

The kit of Figure 1.2 can now be read as a circuit. A *breadboard* has holes joined in rows underneath, so that components connect without soldering. Two long buses carry the supply, 3.3 V and ground; a separate bus carries 5 V for the audio amplifiers. Every part shares the same ground: without a common reference the voltages of two parts mean nothing to each other. The buttons go from a pin to ground; the display and the amplifiers receive digital signals from other pins. An output pin can light an LED through a resistor; an input pin only listens.

Watch Out

A pin of the ESP32-C6 can supply some tens of milliamperes: an LED needs its series resistor. Never join a pin that is driven high directly to ground.

10.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l10.S` in the repository of the course, printed in full in Listing 10.2. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 10.5 is the frame that the test wrote.

On the screen.
A large square, grey while LEFT is released and yellow while it is pressed. On the kit LEFT is a switch between GPIO18 and ground.

What to press.
Press and release LEFT.

```
# ARC12 Computer Architecture - Lecture 10: Introduction to Electronics and
    Digital Electronics
# l10.S: one button, one bit: from the finger to a register
#
# On the screen: a large square, grey while LEFT is released and yellow while
    it is
# pressed. On the kit LEFT is a switch between GPIO18 and ground.
```

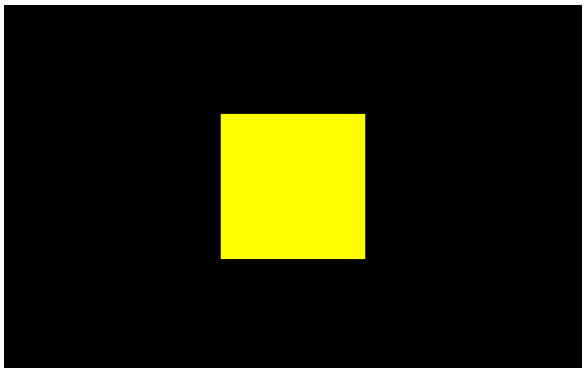


Figure 10.5. `l10.S` on PRG32-QT.

```
# Build: python3 -m prg32 cartridge build l10.S --portable \
#            --entry-prefix l10 --name l10 --out l10.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0

.section .text
.global l10_init
.global l10_update
.global l10_draw

l10_init:
    ret

l10_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read    # a0 = one bit per button
    andi t3, a0, 0x01        # bit 0: LEFT
    la   t0, pressed
    sw   t3, 0(t0)           # 0: not pressed, 1: pressed
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
# on the kit:
#   LEFT is a switch between pin GPIO18 and ground
#   pressed  -> the pin is at 0 volts
#   released -> the pin is at 3.3 volts
# the runtime inverts the level: in the program, pressed is 1

l10_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    la   t0, pressed
    lw   t1, 0(t0)
    li   a4, GREY
    beqz t1, 1f
```

```
47      li   a4, YELLOW
48  1:  li   a0, 120
49      li   a1, 60
50      li   a2, 80
51      li   a3, 80
52      call prg32_gfx_rect
53      lw   ra, 12(sp)
54      addi sp, sp, 16
55      ret
56
57  .section .data
58  pressed: .word 0
```

Listing 10.2. The complete program of the chapter, `code/l10.S`.

To run it on PRG32-QT the commands are those of Listing 10.3; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l10.S --portable \
    --entry-prefix l10 --name l10 --out l10.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l10.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 10.3. Building `l10.S` and running it on PRG32-QT.

10.7 Check your understanding

Q10.1 An LED drops 2 V and must carry 5 mA from a 3.3 V pin. Which resistor is needed?

A. 26 Ω
B. 260 Ω
C. 400 Ω
D. 660 Ω

Q10.2 In a CMOS inverter with the input at 0:

A. both transistors conduct
B. the pMOS conducts and the output is 1
C. the nMOS conducts and the output is 0
D. neither conducts and the output floats

Q10.3 How many transistors does a two-input AND need in static CMOS?

A. 2
B. 4
C. 6
D. 8

Q10.4 Why does the button input of the kit need a pull-up resistor?

A. To limit the current of the LED
B. To make the pin read a defined 1 when the button is released
C. To make the button faster
D. To protect the chip from 5 V

10.8 Exercises

Exercise 10.1 (paper first, then the kit)

1. A blue LED drops 3.0 V: which resistor gives 2 mA from 3.3 V? What power does the resistor dissipate?
2. A divider with $R_1 = 10$ kΩ and $R_2 = 20$ kΩ on 3.3 V: compute V_{out}. Is it read as 0, as 1, or is it undefined, by an input of the kit?
3. Draw a three-input NAND in CMOS and count its transistors.
4. Check the drawing of the NOR for the four combinations of the inputs, marking which transistors conduct.
5. On PRG32, run the lamps cartridge and bridge a button pin to ground with a wire: explain what you see, and what would happen with the pull-up disabled.

Expected result. On paper: 150 Ω and 0.6 mW; $V_{out} = 2.2$ V, in the undefined band; six transistors, three pMOS in parallel and three nMOS in series. On the kit: the lamp lights exactly as if the button were pressed.

10.9 Glossary

English	Italiano
voltage	tensione
current	corrente
resistor / resistance	resistore / resistenza
capacitor	condensatore
ground	massa
semiconductor, doping	semiconduttore, drogaggio
gate, source, drain	gate, source, drain
logic level	livello logico
noise margin	margine di rumore
pull-up resistor	resistenza di pull-up
floating input	ingresso flottante
propagation delay	ritardo di propagazione
integrated circuit	circuito integrato
breadboard	basetta sperimentale (breadboard)

10.10 Conclusions and reading

Voltage pushes, current flows, resistance hinders: $V = R \cdot I$ describes most of what a digital designer needs. A MOS transistor is a switch controlled by a voltage, and nMOS and pMOS are complementary. A bit is a range of voltages; noise margins and regeneration are what make digital circuits reliable. CMOS gates are pairs of networks of switches, and the inverting gates NAND and NOR are the cheapest. Switching costs time ($R \cdot C$) and energy ($C \cdot V^2 \cdot f$): the physical limits behind every clock.

Reading. Charles Petzold, *Code*, the chapters from the flashlight to logic gates; Patterson and Hennessy (2020), sections 1.5 and 1.7 and appendix A; `docs/hardware/hardware.md` in the PRG32 repository.

CHAPTER 11

Combinational Machines

"... it's not my fault, the carrots touched the peas..."
— Alan Turing as a boy, in the film *The Imitation Game*, 2014

In this chapter

By the end of this chapter you will be able to

- tell a combinational machine from a sequential one;
- compute how many bits a fixed-length code needs, and recognise incomplete and redundant codes;
- design a decoder and an encoder, and compose small decoders into larger ones;
- explain what a multiplexer and a demultiplexer do, and build them from gates;
- build a comparator and a parity network, and say what a parity check can and cannot detect;
- point to each of these machines inside a RISC-V processor.

11.1 Warm-up on the machine

```
# a0 = selector, 0 to 3      result: a4 = the selected colour

    la   t0, palette         # four words: the data inputs
    slli t1, a0, 2           # selector * 4 bytes
    add  t0, t0, t1          # address of the selected word
    lw   a4, 0(t0)           # the selected input reaches the output

.section .rodata
# red, green, blue, yellow
palette: .word 0xf800, 0x07e0, 0x001f, 0xffe0
```

Listing 11.1. Choose one of four colours.

Listing 11.1 is the array access of Chapter 4, used as a selection. The same selector always gives the same colour: the result depends only on the present inputs, and nothing is remembered from one use to the next. In hardware this is a *multiplexer*: four data inputs, two selection bits, one output. A processor contains dozens of them, and makes the selection

with gates in well under a nanosecond, where the program takes four instructions. How are they built?

11.2 Combinational machines

11.2.1 A taxonomy of digital circuits

Definition 11.1 (Combinational and sequential circuits)

In a *combinational* circuit the value of the outputs at a given instant depends only on the value of the inputs at that same instant. In a *sequential* circuit the outputs depend on the present inputs and on the past history of the inputs.

A combinational circuit has no memory and no feedback loops, and is completely described by a truth table: adders, decoders, multiplexers and the ALU are combinational. A sequential circuit needs the concepts of *time* and of *state*, and is described by a state table or a state diagram: latches, flip-flops, registers, counters and the processor as a whole are sequential. They are the subject of Chapter 13.

11.2.2 Designing a combinational machine

The design follows the method of Chapter 9 (Figure 11.1): from a specification in words to a truth table, with one row per combination of the inputs and with the don't-cares; from the table to minimal expressions, one output at a time; from the expressions to a network of gates, or of standard blocks. Analysis runs the other way, from the schematic to the truth table. For more than four or five inputs nobody draws maps: designers compose standard blocks, which is why this chapter is a catalogue of them.

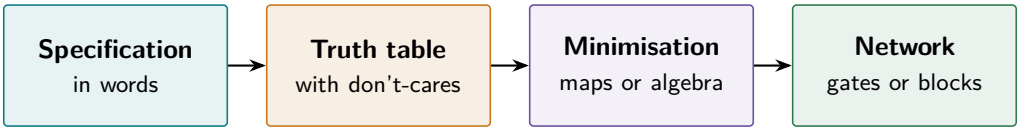


Figure 11.1. The synthesis of a combinational machine.

11.2.3 Real gates take time

A gate does not answer instantly: its output settles after a propagation delay (Chapter 10). The delay of a network is that of its slowest path from an input to an output, the *critical path*. While the signals travel, the outputs can be temporarily wrong.

Concept

A combinational machine is correct only after it has settled. Something must say when to look at its outputs: that something is the clock, and it is where sequential machines begin.

In the ripple-carry adder of Chapter 9 the critical path crosses all thirty-two full adders, because the last carry depends on the first. Faster adders, such as the *carry look-ahead* adder, compute the carries with additional gates arranged in a few levels: they spend area

to shorten the critical path. The function computed is the same. The clock period of a processor must be longer than its longest critical path.

11.3 Codes

Before building machines that treat codes, it must be clear what a code is.

11.3.1 The concept of information

A piece of information has three aspects. The *attribute* is the meaning associated with it: for example, the mark of the Computer Architecture exam. The *type* is the set of elements among which the value is chosen: the marks from 18 to 30, and 30 with honours. The *value* is the element actually taken: 30. To give information is to choose one element in a defined set.

The *cardinality* N of the type is the number of its elements, here $N = 14$; a type with more elements carries more information. The same value can be written in many ways: 3, III, three; and A, B, C, D can stand for 30 with honours, 30, 29, 28. "30 with honours" is not a number: it is a fourteenth element of the type, and how to write it is a problem of encoding. Finally, types are *simple* when they are atomic, such as integers and characters, and *structured* when they combine simple ones: a position is a latitude and a longitude.

11.3.2 Alphabets and strings

Definition 11.2 (Alphabet and string)

An *alphabet* is a finite, non-empty set of symbols. A *string*, or word, over an alphabet is a finite sequence of its symbols. The *length* $|x|$ of a string x is the number of its symbols; the string of length 0 is the empty string. The set of all the strings over an alphabet S, the empty one included, is written S^*.

The binary alphabet is $\{0, 1\}$, the decimal one $\{0, \ldots, 9\}$; `01000010` is a binary string of length 8. An infinite sequence $010101\ldots$ is not a string, and does not belong to $\{0, 1\}^*$.

11.3.3 Codes of fixed length

Definition 11.3 (Code)

A *code* maps every element of a source type T, of cardinality N, to a string of symbols of a code alphabet E with k symbols, the *code word*. In a code of *fixed length* every code word has the same length m.

Theorem 11.1 (Minimum length)

With an alphabet of k symbols there are k^m words of length m. A fixed-length code for N elements exists if and only if $k^m \geq N$, so that the minimum length is

$$m = \lceil \log_k N \rceil, \qquad \text{and in binary} \qquad m = \lceil \log_2 N \rceil.$$

The fourteen marks of the exam need 4 bits; the 32 registers of RISC-V need exactly 5, and those five bits are the fields `rd`, `rs1` and `rs2` of an instruction. A code is *incomplete* when N is not a power of k: then $k^m - N$ words are left unassigned. The fourteen marks leave $16 - 14 = 2$ words unassigned.

11.3.4 An incomplete code: binary-coded decimal

The ten decimal digits need $\lceil \log_2 10 \rceil = 4$ bits, and six of the sixteen words are never used (Table 11.1). In the code BCD 8421, where each digit is written as its value in binary, the unused words are 1010 to 1111. Unused words become don't-care conditions when a circuit is designed.

Table 11.1. Two codes for the decimal digits.

Digit	BCD 8421	Excess-3	Digit	BCD 8421	Excess-3
0	0000	0011	5	0101	1000
1	0001	0100	6	0110	1001
2	0010	0101	7	0111	1010
3	0011	0110	8	1000	1011
4	0100	0111	9	1001	1100

11.3.5 Redundant codes

Definition 11.4 (Redundant code)

A code is *redundant* when it uses more symbols than strictly necessary: its length is greater than the minimum m.

In a redundant code only some of the possible words are legal; the others cannot be produced by the encoder. If a word arrives that is not legal, something has altered it on the way: the error is *detected*. Redundancy is the price paid to detect, and sometimes to correct, errors.

Element	Code 1	Code 2
A	00	000
B	01	011
C	10	101
D	11	110

Code 1 is not redundant: every word of two bits is legal, so that an altered bit turns one element into another without anyone noticing. Code 2 adds a third bit that makes the number of 1s even. The words 001, 010, 100 and 111 are not legal, and a single altered bit is always detected. Code 2 is Code 1 plus a bit of *even parity*; the machines that compute it come later in the chapter.

11.3.6 Codes of variable length

In a code of *variable length* the length of the code word depends on the element: frequent elements receive short words. Such a code must have the *prefix property*: no code word is the beginning of a longer one, so that the reader always knows where a word ends. It pays

when the elements do not all have the same probability: less space in memory, less time in transmission. Morse code and Huffman coding are the classic examples, and the reader has already met two more.

- **UTF-8**: one to four bytes per character (Chapter 4).
- **RISC-V instructions on the kit**: with the C extension each instruction is 16 or 32 bits long. An instruction whose two least significant bits are `11` is 32 bits long; any other pattern marks an instruction of 16 bits. The length is written in the code word itself.

11.4 Decoders and encoders

11.4.1 The decoded representation

At the opposite extreme from the compact code there is the *decoded* representation, also called *one-hot*: a code in which the length of the word equals the cardinality of the set, $m = N$, and every code word has exactly one bit at 1. There is one line per element: `0010` means that the third of four elements is the one chosen. The representation is simple to use, because the active line directly selects something, but costly in wires. The rule of use is: a compact code inside a word, a decoded representation where a choice must be acted upon. The seven lamps of Chapter 8 are a decoded representation of a bit number.

11.4.2 The decoder

Definition 11.5 (Decoder)

A *decoder* receives a code word of m bits and asserts the one output line, out of $N = 2^m$, that corresponds to it.

Each output is a minterm of the inputs: a decoder generates all of them (Figure 11.2). With an *enable* input EN, no output is asserted when EN = 0.

B	A	U_0	U_1	U_2	U_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$U_0 = \overline{B}\,\overline{A} \quad U_1 = \overline{B}\,A$$
$$U_2 = B\,\overline{A} \quad U_3 = B\,A$$

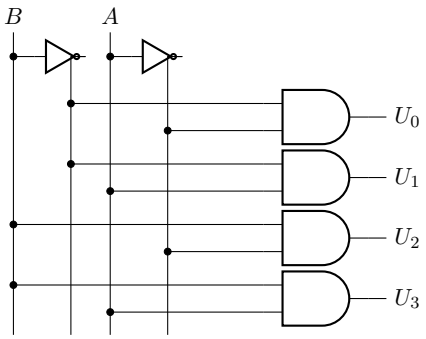


Figure 11.2. The 2-to-4 decoder (A is the least significant bit).

11.4.3 Composing decoders

Large decoders are built from small ones. Figure 11.3 builds a 4-to-16 decoder from five 2-to-4 decoders with enable: the first decodes the two high bits and enables one of the four others; the enabled one decodes the two low bits, which go to all four, and asserts one of

its four lines. Five small decoders select one line of sixteen. Standard parts existed for this: the SN74139 holds two 2-to-4 decoders with enable in one package. (In the real part the enable and the outputs are active-low, a detail omitted here.)

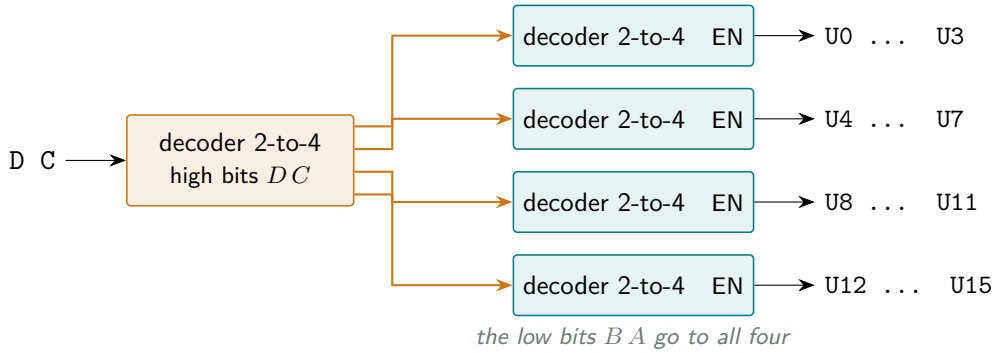


Figure 11.3. Modular composition: a 4-to-16 decoder from five 2-to-4 decoders.

11.4.4 In the processor: which register is written?

Back to the Machine

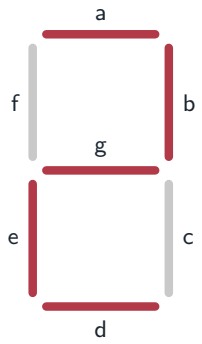
The instruction `addi t1, t1, 1`, the number `0x00130313` of Chapter 2, contains the field `rd` = `00110`: a code word of five bits, because there are thirty-two registers and $\lceil \log_2 32 \rceil = 5$. A 5-to-32 decoder turns it into thirty-two *write-enable* lines, of which only that of `x6`, that is `t1`, is asserted: only that register receives the result. The write enable of `x0` is simply not connected, and `x0` always reads zero. Two more selections, on `rs1` and `rs2`, choose which registers are read: they are made by multiplexers.

11.4.5 The encoder

The *encoder* is the inverse machine: it receives a decoded representation, N lines of which one is asserted, and outputs the corresponding code word of $m = \lceil \log_2 N \rceil$ bits. Each output bit is the OR of the input lines whose number has a 1 in that position; for $N = 4$, $C_0 = U_1 + U_3$ and $C_1 = U_2 + U_3$. If several lines can be asserted together, a *priority encoder* is needed, which outputs the number of the asserted line with the highest priority. A keypad with one wire per key becomes, in this way, a compact key number; and priority encoders will appear again with interrupts (Chapter 19), where several devices ask for attention and one number reaches the processor.

11.4.6 Transcoders: the seven-segment display

A *transcoder* turns each word of a code C_1 into the corresponding word of a code C_2 defined on the same set. The classic example passes from BCD to the seven lines that light the segments a to g of a numeric display (Figure 11.4); each segment is an LED switched by a digital signal. The transcoder has seven outputs: seven functions of four variables to synthesise, all with the same six don't-cares.



digit	BCD	a	b	c	d	e	f	g
0	0000	1	1	1	1	1	1	0
1	0001	0	1	1	0	0	0	0
2	0010	1	1	0	1	1	0	1
3	0011	1	1	1	1	0	0	1
4	0100	0	1	1	0	0	1	1
5	0101	1	0	1	1	0	1	1
6	0110	1	0	1	1	1	1	1
7	0111	1	1	1	0	0	0	0
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1	0	1	1

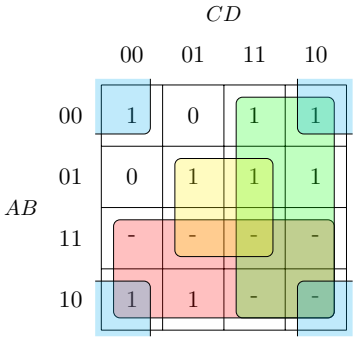


Figure 11.4. The transcoder from BCD to seven segments: the digit 2 lights a, b, g, e and d. Right: the map of segment a, with the BCD digit on $A\,B\,C\,D$ and A most significant.

Example 11.1 (Synthesising segment a)

Segment a is off only for the digits 1 and 4. The words 1010 to 1111 never occur, and are marked as don't-cares in the map of Figure 11.4. Using those cells as 1s the groups become large: the two lower rows give A; the two right-hand columns give C; the central square gives BD; the four corners give $\overline{B}\,\overline{D}$:

$$a = A + C + B\,D + \overline{B}\,\overline{D}.$$

Without the don't-cares the groups would be smaller and the expression longer. As a check, the digit 0 (0000) is covered by the corners and the digit 9 (1001) by A. Six more maps give the segments b to g, and a complete transcoder.

11.5 Multiplexers and demultiplexers

11.5.1 The multiplexer

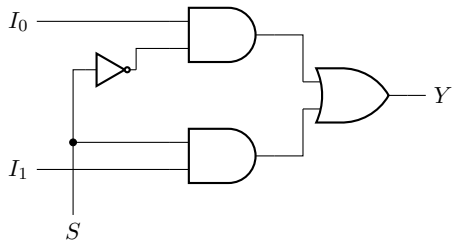
Definition 11.6 (Multiplexer)

A *multiplexer* is a combinational circuit with 2^n data inputs, n selection lines and one output. It selects one of the 2^n input signals and transfers it to the output.

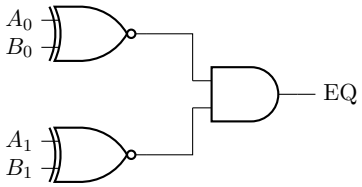
Each data input is multiplied by one minterm of the selection lines, and an OR collects the results (Figure 11.5a):

$$\begin{aligned} &\text{2 inputs:} && Y = \overline{S}\,I_0 + S\,I_1,\\ &\text{4 inputs:} && Y = \overline{S_1}\,\overline{S_0}\,I_0 + \overline{S_1}\,S_0\,I_1 + S_1\,\overline{S_0}\,I_2 + S_1\,S_0\,I_3. \end{aligned}$$

In other words, a decoder on the selection lines enables one AND gate: the datum is chosen by an *address*, not by a decoded line. A multiplexer can also implement any function of its selection variables: it is enough to put the truth table on the data inputs.



(a) A multiplexer with two inputs



(b) A comparator for words of two bits

Figure 11.5. Two machines built from gates.

11.5.2 The demultiplexer

The *demultiplexer* is the inverse machine: one data input D, n selection lines, 2^n outputs. It routes the input to the output chosen by the selection lines, and the other outputs stay at 0:

$$F_i = D \cdot U_i,$$

where U_i are the outputs of a decoder on the selection lines. With D fixed at 1 a demultiplexer is exactly a decoder, in which D plays the part of the enable. A multiplexer followed by a demultiplexer carries many signals over one wire, one at a time: the idea behind every shared line.

11.5.3 In the processor: register or constant?

Back to the Machine

The second operand of the ALU is either the value of the register `rs2` or the constant written in the instruction, sign-extended (Figure 11.6). A multiplexer of two inputs, thirty-two bits wide, chooses: its selection line is 0 for `add` and 1 for `addi`. In the datapath of Patterson and Hennessy (2020) the signal is called `ALUSrc`. A gate applied to every bit of a word under a common control is called a *word gate*.

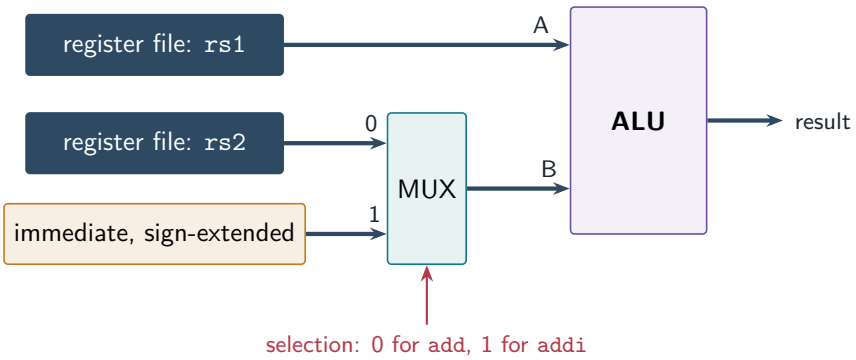


Figure 11.6. A multiplexer chooses the second operand of the ALU.

11.6 Comparing and checking

11.6.1 The comparator

A *comparator* has two data A and B in input and one Boolean output EQ that is 1 when $A = B$. Two bits are equal when their XNOR is 1; two words are equal when all their pairs of bits are equal, an AND of XNORs (Figure 11.5b):

$$\text{EQ} = \overline{(A_0 \oplus B_0)} \cdot \overline{(A_1 \oplus B_1)} \cdots$$

In the processor this is `beq` and `bne`: the two registers are compared bit by bit, and the branch is taken or not according to EQ. Many datapaths reuse the ALU instead: they subtract and test whether the result is zero. Comparisons of order (`blt`, `slt`) need the subtraction and the sign rules of Chapter 5.

11.6.2 Parity

When data travel from a source to a destination, redundant codes allow errors to be checked. The simplest check is a single *parity bit*: the legal code words have, for example, an even number of bits at 1.

Definition 11.7 (Parity network)

A *parity network* computes the parity of the n bits at its input,

$$p = x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1},$$

which is 1 when the number of 1s is odd.

XOR is addition modulo 2, which is why a chain of XORs counts the 1s modulo 2. The network is needed twice (Figure 11.7): at the source, to compute the bit to be added to a non-redundant code, and at the destination, to verify the overall parity of the redundant code on arrival.

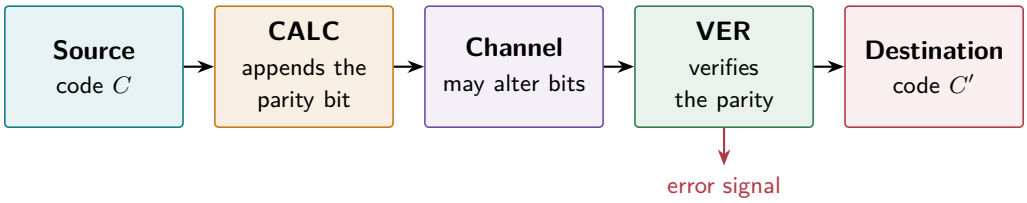


Figure 11.7. Transmission with a parity check.

Theorem 11.2 (Power of a parity bit)

A single parity bit detects every alteration of an odd number of bits. It does not detect the alteration of an even number of bits, and it cannot correct any error.

Proof. Each altered bit changes the number of 1s by one, and therefore inverts the parity: an odd number of inversions changes it, an even number leaves it as it was. And the check gives one bit of information, right or wrong, which is not enough to say *which* bit was altered. □

The code C' that reaches the destination is thus *probably* equal to C if no error was signalled.

Stronger codes add more check bits: the codes of Hamming (1950), with several parity bits over overlapping groups of bits, locate and correct a single error.

11.6.3 A parity network in seven instructions

```
# a0 = one byte          result: a0 = 1 if the number of 1 bits is odd

    srli t0, a0, 4
    xor  a0, a0, t0      # fold eight bits onto four
    srli t0, a0, 2
    xor  a0, a0, t0      # fold four bits onto two
    srli t0, a0, 1
    xor  a0, a0, t0      # fold two bits onto one
    andi a0, a0, 1       # keep the parity bit
```

Listing 11.2. The parity of a byte.

Each `xor` of Listing 11.2 combines the upper half of the bits with the lower half: three levels for eight bits, the software version of a tree of seven XOR gates. The value computed, appended to the byte, makes its total parity even. Trace it with `0x53` $= 0101\,0011_2$, which has four 1s and gives 0, and with `0x13` $= 0001\,0011_2$, which has three and gives 1.

11.7 The arithmetic logic unit

All the blocks of this chapter work together in one machine.

Definition 11.8 (Arithmetic logic unit)

The *arithmetic logic unit* (ALU) is a combinational machine that receives two operands and a code of operation, and produces the result of the operation selected.

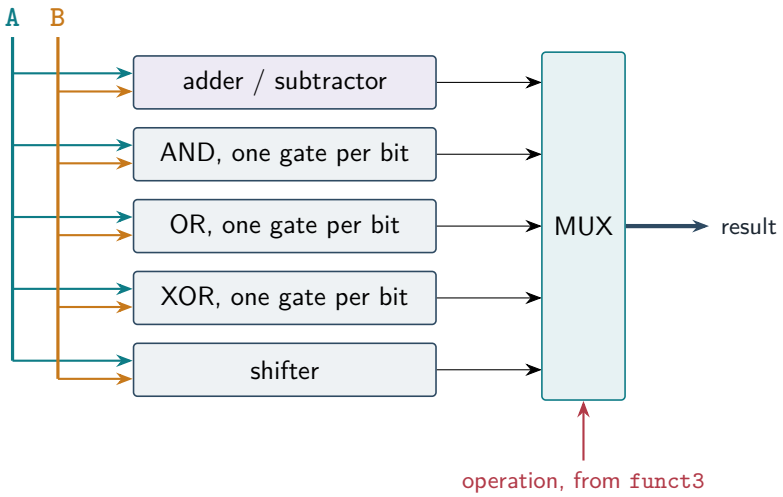


Figure 11.8. The ALU is a set of machines, all at work on the same two operands of 32 bits, and a multiplexer that chooses one result.

The structure of Figure 11.8 is disarmingly simple: *every* unit computes its result at every

instruction, and a multiplexer lets one through. The adder is the ripple-carry adder of Chapter 9, with the trick of the complement and of the carry-in of Chapter 5 for the subtraction; comparisons (`slt`) reuse the subtractor.

The selection is written in the instruction (Table 11.2): the field `funct3`, bits 14 to 12, is a compact code of three bits, eight code words, which a small decoder turns into the selection lines of the multiplexer of the ALU.

Table 11.2. RISC-V instructions and the ALU.

Instruction	funct3	ALU operation	Unit that produces the result
`add`, `sub`	`000`	sum or difference	adder / subtractor (one more bit, in `funct7`, chooses)
`sll`	`001`	shift left	shifter
`slt`, `sltu`	`010`, `011`	set if less than	subtractor, then the sign rules
`xor`	`100`	bitwise XOR	XOR gates
`srl`, `sra`	`101`	shift right	shifter (`funct7` chooses logical or arithmetic)
`or`	`110`	bitwise OR	OR gates
`and`	`111`	bitwise AND	AND gates

11.8 The program of this chapter

The listings of this chapter are parts of one program, `code/l11.S` in the repository of the course, printed in full in Listing 11.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 11.9 is the frame that the test wrote.

On the screen.
Four horizontal bars, red, green, blue and yellow from the top: bar k is drawn in the colour that the selector k chooses in the table.

What to press.
Nothing: the program runs by itself.



Figure 11.9. `l11.S` on PRG32-QT.

```
1 # ARC12 Computer Architecture - Lecture 11: Combinational Machines
2 # l11.S: a multiplexer in software: a selector chooses one of four colours
3 #
```

```
# On the screen: four horizontal bars, red, green, blue and yellow from the top
    : bar k
# is drawn in the colour that the selector k chooses in the table.
# Build: python3 -m prg32 cartridge build l11.S --portable \
#            --entry-prefix l11 --name l11 --out l11.prg32
.option norelax

.section .text
.global l11_init
.global l11_update
.global l11_draw

l11_init:
    ret

l11_update:
    ret

l11_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, 0
    call prg32_gfx_clear
    li   s0, 0               # the selector, 0 to 3
bars:
    mv   a0, s0              # a0 = selector, 0 to 3
    la   t0, palette         # four words: the data inputs
    slli t1, a0, 2           # selector * 4 bytes
    add  t0, t0, t1          # address of the selected word
    lw   a4, 0(t0)           # the selected input reaches the output
    li   a0, 20
    slli a1, s0, 5
    addi a1, a1, 20          # y = 20 + 32 * selector
    li   a2, 200
    li   a3, 16
    call prg32_gfx_rect
    addi s0, s0, 1
    li   t0, 4
    blt  s0, t0, bars
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata
# red, green, blue, yellow
palette: .word 0xf800, 0x07e0, 0x001f, 0xffe0
```

Listing 11.3. The complete program of the chapter, `code/l11.S`.

To run it on PRG32-QT the commands are those of Listing 11.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)
```

```
# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l11.S --portable \
    --entry-prefix l11 --name l11 --out l11.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l11.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 11.4. Building `l11.S` and running it on PRG32-QT.

11.9 Check your understanding

Q11.1 How many bits does a fixed-length binary code need for the 26 letters of the alphabet?

A. 4
B. 5
C. 6
D. 26

Q11.2 A decoder has 3 input lines and no enable. How many outputs does it have, and how many are 1 at any time?

A. 3 outputs, one at 1
B. 6 outputs, two at 1
C. 8 outputs, one at 1
D. 8 outputs, three at 1

Q11.3 A multiplexer with 8 data inputs needs how many selection lines?

A. 2
B. 3
C. 4
D. 8

Q11.4 A word with one even-parity bit is received with two bits altered. The parity check:

A. detects the error and corrects it
B. detects the error but cannot correct it
C. does not detect the error
D. signals an error only if both bits were 1

11.10 Exercises

Exercise 11.1 (paper first, then PRG32)

1. Synthesise segment g of the seven-segment transcoder with a Karnaugh map, using the don't-cares.
2. Build a 3-to-8 decoder from two 2-to-4 decoders with enable and one inverter.
3. Implement $F = \sum m(1, 2, 4, 7)$ with one multiplexer of 8 inputs. Which function is it?
4. Write a comparator in assembly: `a0` $= 1$ if `a0` and `a1` are equal, using `xor` and `sltiu` only.
5. Draw a seven-segment digit on PRG32: a table of ten bytes, one per digit, and seven rectangles lit by testing its bits.

Expected result. The third task puts 0, 1, 1, 0, 1, 0, 0, 1 on the data inputs: it is the XOR of three inputs, the sum bit of the full adder. The fourth is `xor a0, a0, a1` followed by `sltiu a0, a0, 1` (the pseudo-instruction `seqz`). On PRG32: a large digit that counts from 0 to 9, one step per second, drawn from seven rectangles; it is a software transcoder followed by seven bit tests, and its table is the truth table of Figure 11.4, one row per byte.

11.11 Glossary

English	**Italiano**
combinational network	rete combinatoria
propagation delay	ritardo di propagazione
cardinality	cardinalità
code word	parola codice
fixed / variable length	lunghezza fissa / variabile
redundant code	codice ridondante
decoder / encoder	decodificatore / codificatore
transcoder	transcodificatore
multiplexer / demultiplexer	multiplexer / demultiplexer
enable	abilitazione
comparator	comparatore
parity bit	bit di parità

11.12 Conclusions and reading

Information is an attribute, a type and a value; a code writes the value as a string over an alphabet. Fixed-length codes need $\lceil \log_2 N \rceil$ bits; redundant codes spend more to detect errors. Encoding, decoding and transcoding are done by combinational machines. Multiplexers select a source, demultiplexers a destination; comparators and parity networks check. A RISC-V processor uses all of them: decoders on the fields of the instruction, multiplexers on the operands, and an ALU that is itself built from these blocks.

What is missing here is memory. Before the sequential machines of Chapter 13, the next chapter puts the logic of this part to work in the second part of Verm1.

Reading. Patterson and Hennessy (2020), appendix A and section 4.1; Bucci (2017), the chapters on combinational networks; Stallings (2021), the chapter on digital logic.

CHAPTER 12

Verm1, Part 2: Steering, Food and a Score

"A game is a series of interesting choices."
— Sid Meier

In this chapter

By the end of this chapter you will have

- a worm's head that you steer with the joystick, in four directions;
- a rule that forbids the U-turn, written as one XOR and one comparison;
- movement taken from two small tables instead of four branches;
- food that appears at a random cell, is eaten, and appears again;
- a score, shown as marks in the title line, and a sound when the worm eats.

In this second tutorial the player enters the game. Every idea used has already been seen as a circuit in Chapters 8 to 11; here the same ideas are instructions (Table 12.1). The reference file is `verm1_part2.S` (Appendix D).

Table 12.1. What Part 2 takes from the chapters on logic.

From	Idea	Where it appears
Chapter 8	AND as a mask: keep one bit, clear the others	one button out of the word of buttons
Chapter 8	truth tables: a code for each case	the four directions in two bits
Chapter 9	XOR is 1 where two bits differ	the U-turn rule; the test for equality
Chapter 10	a button is a switch to ground with a pull-up	why "pressed" arrives as 1 only after the runtime inverts it
Chapter 11	comparator, multiplexer, ROM as a table	head on the food; `dx` and `dy`

12.1 Start from a Part 1 that runs

Open your `verm1.S` of Part 1, build it and run it with the two commands of Listing 7.1: the head must cross the screen. If it does not, copy the reference file `verm1_part1.S` and start from that. The file is changed in three steps and is run after the second and after the third. Listings marked **changed** replace code that the file already contains; listings marked **new**

add code. Keep a copy of each finished part, and keep the debugger of PRG32-QT open: step 4 uses it to watch a decision being taken.

12.2 Step 1: names for bits

12.2.1 Constants: buttons and directions

Add the lines from `RED` and from `BTN_LEFT` to `DIR_UP` to the constants of Part 1 (Listing 12.1). There is a new colour, red, for the food. Each button is one bit of a word, and a mask selects it. A direction is a number from 0 to 3: two bits.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ RED,   0xf800
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01        # one bit per button
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ DIR_RIGHT, 0           # a direction is a 2-bit code:
.equ DIR_DOWN,  1           #   bit 0 = vertical
.equ DIR_LEFT,  2           #   bit 1 = backwards
.equ DIR_UP,    3

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
```

Listing 12.1. Step 1 **(changed)**: constants.

12.2.2 A code chosen with care

The codes of the four directions are not arbitrary (Table 12.2). Bit 0 says “vertical” and bit 1 says “backwards”, so that a direction and its opposite differ in bit 1 and only there: the XOR of a code with its opposite is always $10_2 = 2$.

Table 12.2. The four directions. On the screen y grows downwards: DOWN is $+1$, UP is -1.

Direction	Code	Bit 1	Bit 0	dx	dy	Opposite
RIGHT	0	0	0	+1	0	LEFT = 2
DOWN	1	0	1	0	+1	UP = 3
LEFT	2	1	0	−1	0	RIGHT = 0
UP	3	1	1	0	−1	DOWN = 1

Concept

A good encoding turns a rule into one gate. It is the lesson of Chapter 9, and it is worth more than any trick of programming.

12.2.3 Data: two tables and five new variables

Replace the data of Part 1 with Listing 12.2. The tables `dx` and `dy` have four words each, one per direction; they are constants, and therefore live in `.rodata`. The variable `dir` is the direction of the last step, `want` the one asked for by the player; `score`, `food_x` and `food_y` are used in step 3.

```
.section .rodata
title:   .asciz "VERM1"
dx:      .word  1, 0, -1,  0    # right, down, left, up
dy:      .word  0, 1,  0, -1

.section .data
head_x:  .word 5
head_y:  .word 12
dir:     .word 0                # direction of the last step
want:    .word 0                # direction asked by the player
step_q8: .word 0
score:   .word 0
food_x:  .word 20
food_y:  .word 12
```

Listing 12.2. Step 1 **(changed)**: data.

12.3 Step 2: steering

The plan is: read the buttons, refuse the U-turn, look the movement up in a table.

12.3.1 From finger to bit

The pin of a button reads 0 when the button is pressed: it is active-low (Chapter 10). The function `prg32_input_read` inverts and packs: one word, one bit per button, with 1 = pressed. **UP** and **RIGHT** together give $\texttt{0x04} + \texttt{0x02} = \texttt{0x06}$, and $\texttt{a0} \cdot \texttt{0x04}$ is zero if and only if **UP** is not pressed. The cartridge never sees the electrical level: the runtime gives the same word on the board, on QEMU and on PRG32-QT.

12.3.2 `update`: from buttons to a wish

```
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read       # a0 = one bit per button
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
```

```
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)              # remember it for the next step
5:
```

Listing 12.3. Step 2 **(changed)**: the beginning of `verm1_update`.

Listing 12.3 replaces the beginning of `verm1_update`, which now calls a function and therefore needs a stack frame. `andi` keeps one bit, and zero means "not pressed". Each pressed button overwrites the wish in `t2`; with two buttons pressed the last test wins, so that **UP** beats **LEFT**, which beats **DOWN**, which beats **RIGHT**. Then comes the rule: the XOR of the wish and of the current direction is 2 exactly when they are opposite, and in that case the wish is not stored and the old one survives.

The wish is stored in `want` and becomes the direction only at the next step. In this way two quick presses between one step and the next cannot fold the worm back onto itself.

Table 12.3. The U-turn rule as a truth table: six of the sixteen rows.

dir	want	dir ⊕ want	= 2?	Decision
0 RIGHT	0 RIGHT	00	no	go on
0 RIGHT	1 DOWN	01	no	turn
0 RIGHT	2 LEFT	10	yes	refuse: U-turn
0 RIGHT	3 UP	11	no	turn
1 DOWN	3 UP	10	yes	refuse: U-turn
3 UP	2 LEFT	01	no	turn

The rule is one XOR and one comparison with the constant 2 (Table 12.3). Complete the sixteen rows as an exercise and count the refusals: there are four, one per direction. Without the careful encoding the rule would need four comparisons.

12.3.3 `update`: the timer calls `step`

```
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
```

```
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 12.4. Step 2 **(changed)**: the rest of `verm1_update`.

The fixed-point timer of Part 1 is unchanged (Listing 12.4). The lines that moved `head_x` are gone: moving is now a function of its own, `step`. The local labels are 6 and 7 because the numbers 1 to 5 are used above.

12.3.4 `step`: look the movement up

```
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]: -1, 0 or +1
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
```

Listing 12.5. Step 2 **(new)**: `step`, first half.

The wish becomes the direction, once per step (Listing 12.5). Then the direction selects the movement: a word is 4 bytes, so the offset is the code shifted to the left by 2, and the address of the table plus the offset is the address of the word to load. There is no branch: the code selects, as in a multiplexer. A table in read-only memory addressed by a code is a ROM (Chapter 16). The alternative is four branches; the table is shorter, and changing the rules means changing data, not code.

12.3.5 `step`: move and wrap, both ways

```
    la   t0, head_x
    lw   t4, 0(t0)
    add  t4, t4, t2
    bgez t4, 1f
    li   t4, COLS-1             # off the left edge: wrap
1:  li   t5, COLS
    blt  t4, t5, 2f
    li   t4, 0                  # off the right edge: wrap
2:  sw   t4, 0(t0)              # t4 = new column
    la   t0, head_y
    lw   t6, 0(t0)
```

```
    add  t6, t6, t3
    bgez t6, 3f
    li   t6, ROWS-1
3:  li   t5, ROWS
    blt  t6, t5, 4f
    li   t6, 0
4:  sw   t6, 0(t0)              # t6 = new row
```

Listing 12.6. Step 2 **(new)**: `step`, second half.

In Part 1 only the right edge existed. Now `dx` can be -1 and the column can go below 0: a negative column is a signed number, and `bgez` and `blt` are the signed comparisons of Chapter 5 (Listing 12.6). The same is done for the row, with `ROWS`. The registers `t4` and `t6` keep the new cell for step 3.

For the moment the function is closed with the three lines of Listing 12.7, which restore `ra` and return. Step 3 will put the test for the food before them.

```
    lw   ra, 12(sp)             # temporary end of step:
    addi sp, sp, 16             # Step 3 will replace these three lines
    ret
```

Listing 12.7. Step 2: a temporary ending for `step`.

Run it: you are driving

1. Build and run: the head still starts to the right, from column 5 of row 12.
2. Hold **DOWN**: the head turns at its next step and goes down, wrapping from row 23 to row 0.
3. While it goes right, hold **LEFT**: nothing happens. The U-turn is refused.
4. Hold **DOWN** and then **LEFT**: two turns, and now the head goes left.

Expected result. Holding **DOWN** from the start for 64 frames brings the head to column 5, row 22: ten steps, all downwards. Holding **LEFT** for 64 frames brings it to column 15, row 12, as if nothing had been pressed. (Both were checked on the headless runner of PRG32-QT.)

12.4 Step 3: food, a score, a sound

12.4.1 `init`: a complete start

```
verm1_init:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)
    la   t0, head_y
    li   t1, 12
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)            # moving right
```

```
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 12.8. Step 3 **(changed)**: `verm1_init`.

Every variable now receives its starting value (Listing 12.8). `init` calls a function, to place the first food, and so it needs a stack frame.

12.4.2 `place_food`: a random cell

```
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0
    li   a1, COLS-1
    call prg32_random_number    # a0 = a column, 0 .. 39
    la   t0, food_x
    sw   a0, 0(t0)
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row, 0 .. 23
    la   t0, food_y
    sw   a0, 0(t0)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 12.9. Step 3 **(new)**: `place_food`.

The function `prg32_random_number(min, max)` returns a number between its two arguments, both included; the arguments go in `a0` and `a1` and the result comes back in `a0` (Listing 12.9). The random numbers come from the runtime, and on the board from the hardware generator of the ESP32-C6: two runs of the game are different. The food may fall on the head; Part 3 fixes it.

12.4.3 `step`: is the head on the food?

```
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, t4             # 0 if same column
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, t6             # 0 if same row
    or   t1, t1, t2             # 0 only if both are equal
    bnez t1, 5f                 # not on the food
```

```
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    call eat_sound
5:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 12.10. Step 3 **(changed)**: the end of `step`.

Replace the temporary ending with Listing 12.10. A comparator for equality is a row of XOR gates followed by an OR (Chapter 11); here it is two `xor` and one `or`: each `xor` is 0 when two numbers are equal, and the `or` of the two results is 0 only if both pairs are equal. Eating means one more point, new food and a sound. The registers `t4` and `t6` still hold the new column and row; after the calls they are no longer needed, which is why it is safe to keep them in temporaries.

12.4.4 `eat_sound`: one note

```
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a2, 72                 # note: C5
    li   a3, 255                # volume
    li   a4, 60                 # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 12.11. Step 3 **(new)**: `eat_sound`.

The signature is

```
prg32_audio_note(channel, instrument, note, volume, milliseconds)
```

with five arguments, `a0` to `a4`, then the call. Note 72 is C5 in the MIDI numbering. On PRG32-QT the note is played by the computer, on the board by the amplifier and its speaker. Sound comes last: the game worked without it.

12.4.5 `draw`: the score as marks

```
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
```

```
    la   a2, title
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:
```

Listing 12.12. Step 3 **(changed)**: the beginning of `verm1_draw`.

Listing 12.12 draws one small rectangle per point, 4 pixels apart, in a loop that ends when the counter reaches the score. The counter is kept in `s0`, because it must survive the calls: a `t` register may be destroyed by any call, whereas an `s` register is preserved, provided that the function which uses it saves it and restores it. So `s0` is saved in the frame, with `ra`. The convention, met in Chapter 2, is studied in Chapter 18. Digits arrive in Part 4, when division is available.

12.4.6 `draw`: food and head

```
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    la   t0, head_x
    lw   a0, 0(t0)
    la   t0, head_y
    lw   a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 12.13. Step 3 **(changed)**: the rest of `verm1_draw`.

The same three lines appear twice, for column, row and colour (Listing 12.13); the conversion to pixels is left to a function. The head is drawn last, so that it covers the food.

```
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
```

```
3       sw   ra, 12(sp)
4       slli a0, a0, 3              # x = column * 8
5       slli a1, a1, 3
6       addi a1, a1, TOP            # y = row * 8 + 8
7       mv   a4, a2                 # the colour is the fifth argument
8       li   a2, CELL
9       li   a3, CELL
10      call prg32_gfx_rect
11      lw   ra, 12(sp)
12      addi sp, sp, 16
13      ret
```

Listing 12.14. Step 3 **(new)**: `draw_cell`.

`draw_cell` is the arithmetic of Part 1, written once (Listing 12.14). It is a function with a contract: `a0` = column, `a1` = row, `a2` = colour. The colour arrives in `a2` and must leave in `a4`: it is moved first, because `a2` is about to be overwritten. Writing the contract in a comment is part of the job.

Run it: Verm1, Part 2

A red cell is somewhere on the screen. Steer the head onto it: a note sounds, a mark appears next to `VERM1`, the food moves (Figure 12.1).

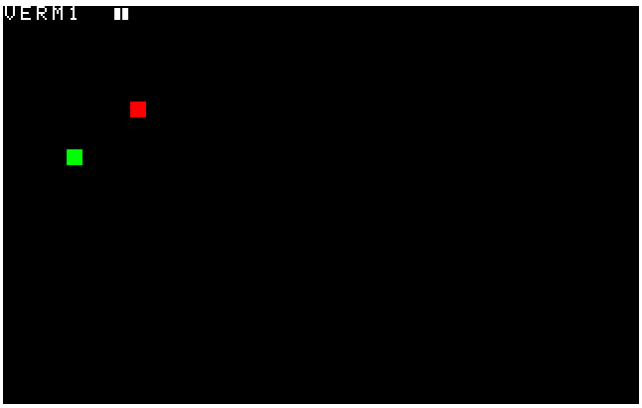


Figure 12.1. The head, the food, and two marks after the title: two points. The screenshot is produced by the tests, in which a scripted player steers the head to the food twice.

12.5 Step 4: watch a decision

In this session PRG32-QT is used as a logic probe.

Catch the U-turn rule at work

1. Run the cartridge in debug mode and press **Update**: the listing shows `verm1_update`.
2. Look at the telemetry while you hold a button: the input mask shows `0x04` for **UP**, `0x01` for **LEFT**.
3. Find the instruction `xor t3, t2, t1` and click in the gutter: a breakpoint.

4. Hold **LEFT** while the head goes right, and press **Resume**: the program stops on the `xor`.
5. Read the registers: `t1` is 0, the direction; `t2` is 2, the wish.
6. Press **Step**: `t3` becomes 2. Step twice more: the branch is taken and the `sw` is skipped.

Lower the speed of the debugger to 0.25x if you need time to press.

Table 12.4 lists what is read in the registers, always with the head going right. Its last row shows the priority of the tests: **UP** is tested last and wins. `a0` is overwritten later in the frame, and must be read at the first instruction after the call.

Table 12.4. What the registers show at the breakpoint.

You hold	`a0`	`t1` (dir)	`t2` (want)	`t3` = `t2`⊕`t1`	Stored?
nothing	`0x00`	0	0	0	yes: 0
DOWN	`0x08`	0	1	1	yes: 1
LEFT	`0x01`	0	2	2	no: U-turn
UP	`0x04`	0	3	3	yes: 3
UP + **RIGHT**	`0x06`	0	3	3	yes: 3

12.5.1 Read the variables in memory

After `la t0, dir` the register `t0` holds the address of `dir`: type it in the *Memory* field. The variables follow each other, 4 bytes each, in the order of the file: from the address of `dir`, at +0 there is `dir`, at +4 `want`, at +8 `step_q8`, at +12 `score`, at +16 `food_x` and at +20 `food_y`; `head_x` and `head_y` are the two words before `dir`. Eat once, pause, and read `score`: `01 00 00 00`, lowest byte first. Then read `food_x` and `food_y` and check them against the screen: they are the column and the row of the red cell.

12.6 Check your understanding

Q12.1 The player holds **DOWN** and **RIGHT** together: `prg32_input_read` returns

A. `0x02`
B. `0x08`
C. `0x0a`
D. `0x10`

Q12.2 The worm goes UP (3). Which wish is refused?

A. RIGHT (0)
B. DOWN (1)
C. LEFT (2)
D. UP (3)

Q12.3 Why `slli t1, t1, 2` before adding to the address of `dx`?

A. to make the code positive

B. each entry of the table is 4 bytes
C. to clear bit 1
D. tables start at address 4

Q12.4 `xor t1, t1, t4` gives zero exactly when

A. `t1` is zero
B. `t4` is zero
C. `t1` and `t4` are equal
D. `t1` and `t4` are opposite

12.7 Make it yours

Exercise 12.1 (on PRG32)

1. Make button **A** a brake: while it is held, the timer does not advance.
2. Give two points for a piece of food eaten in the top row.
3. Draw the head in a different colour for each direction, using a third table of four colours.
4. Change the encoding so that RIGHT = 0, LEFT = 1, DOWN = 2, UP = 3: what does the U-turn rule become?

Expected result. Each change is a few lines. Keep a copy of the working Part 2 before experimenting: Part 3 starts from it. (Hints: for the brake, `andi` with `BTN_A`, `0x10`, at the start of the timer and a branch to the exit. The table of colours is four `.word` in `.rodata`, indexed like `dx`. With the new encoding the opposites differ in bit 0, so the test becomes $\oplus = 1$.)

12.8 Where we are, and what comes next

The player is in the game: a wish, a rule, a movement, a reward. Bits and masks, XOR as a comparator, a table as a selector: logic has been turned into instructions. But the worm has no body, nothing can go wrong, and the game never ends. Chapters 13 to 16 give the tools: state machines, registers, memory as an array, and what survives when the power is switched off. Part 3 adds a body that grows, collisions, a title screen and a game over.

Part IV

State and Memory

CHAPTER 13

Sequential Machines

"Steve, every car has a steering wheel, but nobody tries to claim the steering wheel was their invention."

— Bill Gates, in the film *Pirates of Silicon Valley*, 1999

In this chapter

By the end of this chapter you will be able to

- explain why a sequential circuit needs the concepts of time and of state;
- define a finite-state machine and distinguish Mealy machines from Moore machines;
- describe a machine with a state table and with a state diagram, and go from one to the other;
- explain how an SR latch remembers one bit, and why $S = R = 1$ is not allowed;
- distinguish latches from flip-flops, level triggering from edge triggering;
- recognise registers, counters and the program counter of RISC-V as sequential machines.

13.1 Warm-up on the machine

A game has a title screen, a screen on which one plays, and a screen that says "game over". Listing 13.1 is the part of its `update` that passes from one to another.

```
.equ TITLE, 0
.equ PLAY,  1
.equ OVER,  2

game_update:
    ...
    call prg32_input_read      # a0 = buttons: the INPUT
    la   t0, state
    lw   t1, 0(t0)             # t1 = the present STATE
    andi t2, a0, 0x10          # button A
    li   t3, TITLE
    bne  t1, t3, 1f
    beqz t2, done              # TITLE: stay until A is pressed
```

```
    li   t1, PLAY              # TITLE --A--> PLAY
    j    store
1:  li   t3, OVER
    bne  t1, t3, done          # PLAY is handled elsewhere
    beqz t2, done              # OVER: stay until A is pressed
    li   t1, TITLE             # OVER --A--> TITLE
store:
    sw   t1, 0(t0)             # the next state becomes the present state
done:
    ...
```

Listing 13.1. A game is a state machine.

Pressing **A** starts the game from the title screen and returns to the title from the game-over screen: *the same button has different effects*, and the effect depends on the state. The state is one word in memory that survives from frame to frame. `update` computes the next state from the present state and from the input; `draw` shows a different screen for each state, the output.

The program has a bug that is worth discussing: holding **A** on the game-over screen leads to the title and, one frame later, to the game. To fix it the program must remember the previous value of the button: one more bit of state. The chapter will come back to this.

13.2 Time and state

13.2.1 Sequential circuits

In a combinational circuit the outputs at a given instant depend only on the inputs at that same instant. In a sequential circuit they depend on the inputs at that instant and on the past history of the inputs. The definition implies two concepts that a combinational machine does not have: *time* (when are the signals looked at, and when do things change?) and *state* (what does the circuit remember of the past?). A sequential circuit is a combinational one plus something that remembers. Analysis and synthesis still apply: from the structure to the behaviour, and from a functional specification to a structure.

13.2.2 Time: from continuous to discrete

In any interval a physical signal passes through infinitely many instants, and a voltage through infinitely many values. A binary signal is read, or *sampled*, at a finite number of chosen instants: 0 if the voltage is below a threshold, 1 if it is above (Chapter 10 made the threshold precise). Between two samples the signal may do anything: only the samples count. It is the sampling and quantising of Chapter 4, applied to the wires of the circuit itself. In a digital system time is therefore a sequence of discrete instants t_0, t_1, t_2, ... Who decides when the instants are? A dedicated periodic signal: the clock.

13.2.3 The clock

Definition 13.1 (Clock)

The *clock* is a periodic binary signal that marks the discrete instants of a digital system. Its *period* T is the duration of one cycle; its *frequency* is the number of periods per second, $f = 1/T$; its *duty cycle* is the ratio between the time for which it is high and the period. The *rising edge* is the passage from 0 to 1, the *falling edge* the passage from 1 to 0.

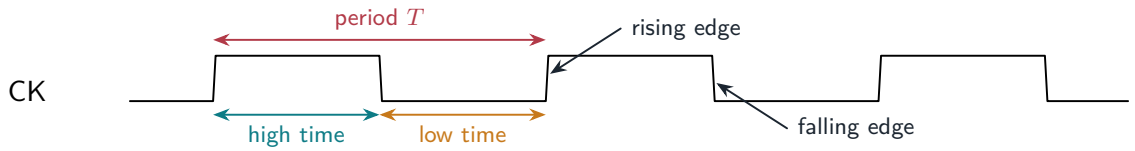


Figure 13.1. The clock.

Synchronous circuits change state on one of the two edges. The ESP32-C6 runs at 160 MHz: one period lasts 6.25 nanoseconds, and every combinational network inside the chip must settle within that time, because the period must exceed the critical path (Chapter 11). In a PRG32 game the frame plays the part of a much slower clock.

13.2.4 The concept of state

Definition 13.2 (State)

The *state* of a sequential circuit is a summary of the whole previous history of its inputs, sufficient to determine, together with the present inputs, the outputs and the next state. It is held by a set of *state variables*.

A *transition function* computes the next state from the present state and the present inputs, and the state is updated at every instant of discrete time, that is, at every clock cycle:

$$S(k) = f\big(S(k-1),\, I(k)\big).$$

To store the state, memory elements called *bistables* are used: latches and flip-flops. In the warm-up there is one state variable, the word `state`; the transition function is `game_update`; the clock is the frame.

13.3 Finite-state machines

13.3.1 The finite-state automaton

The number of states of a sequential network that can be physically built is necessarily finite: with k state variables there are at most 2^k states. The model that describes its behaviour is the finite-state automaton.

Definition 13.3 (Sequential machine)

A *sequential machine*, or finite-state machine, is a quintuple (I, U, S, Ω, w) where
- I is the input alphabet and U the output alphabet, finite sets of symbols;
- S is the finite, non-empty set of states;
- $\Omega : S \times I \to S$ is the *next-state function*: to each pair of state and input corresponds, if specified, one and only one next state;
- w is the *output function*, which generates the output symbol.

13.3.2 Mealy machines and Moore machines

The two classical models differ in the output function (Mealy, 1955; Moore, 1956).

Definition 13.4 (Mealy and Moore)

In a *Mealy machine* the output depends on the state and on the input, $w : S \times I \to U$.
In a *Moore machine* the output is associated with the state alone, $w : S \to U$.

In the diagram of a Mealy machine the output is written on the transitions; the machine can react to an input in the same cycle, and often needs fewer states. In the diagram of a Moore machine the output is written inside the states; it changes only when the state changes, so that no disturbance of the inputs reaches it. Every Mealy machine has an equivalent Moore machine and vice versa, possibly with a different number of states. The game of the warm-up is a Moore machine: each state has its screen.

13.3.3 General architecture of a sequential machine

Every sequential machine has the structure of Figure 13.2: a combinational network computes the next state and the outputs from the inputs and from the present state; a *state register* of k flip-flops holds the present state and, at each clock edge, replaces it with the next one. The warm-up has exactly this shape: `game_update` is the upper box, the word `state` is the register, the frame is the clock.

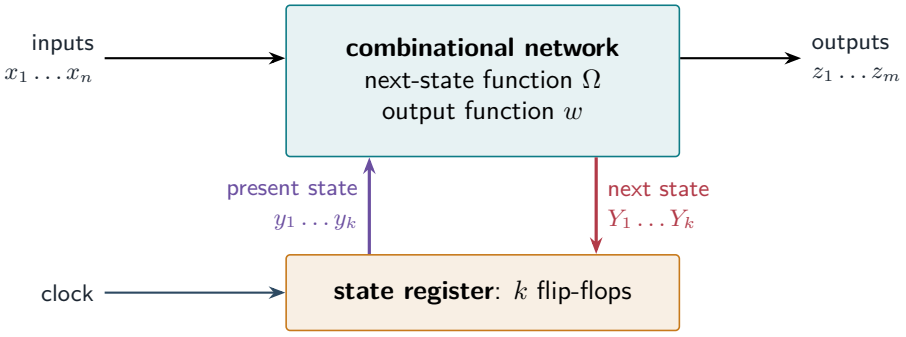


Figure 13.2. The general architecture of a sequential machine.

13.3.4 Synthesis of a sequential network

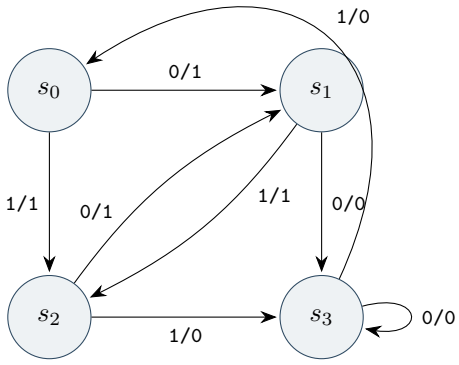
1. Define the sets I, U and S: inputs, outputs and states.

2. Define the next-state function Ω and the output function w, with a state table or a state diagram.
3. Assign a binary code to each state: k state variables for up to 2^k states.
4. Synthesise the two combinational networks that realise Ω and w, with the methods of Chapters 9 and 11.
5. Choose the memory elements: the next-state network depends on the type of bistable, the output network does not.

With D flip-flops the next-state network simply computes the code of the next state, which is why they are the usual choice today.

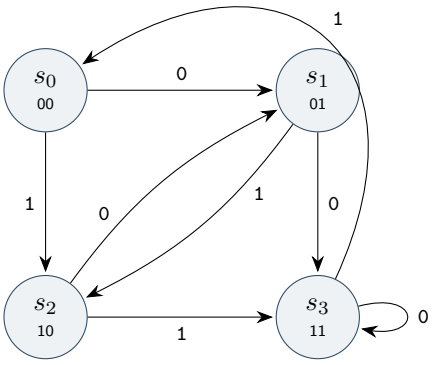
13.3.5 State tables and state diagrams

A machine is described in two equivalent ways. The *state table* has one row per present state and one column per input symbol; each cell holds the next state and, for a Mealy machine, the output. The *state diagram* is a directed graph with one node per state and one arc per transition, labelled with the input and, for a Mealy machine, the output. Figure 13.3 shows the same transitions as a Mealy machine and as a Moore machine in which a two-bit output, here simply the binary code of the state, is attached to each state.



state	input 0	input 1
s_0	s_1/1	s_2/1
s_1	s_3/0	s_2/1
s_2	s_1/1	s_3/0
s_3	s_3/0	s_0/0

(a) A Mealy machine: arcs carry input/output



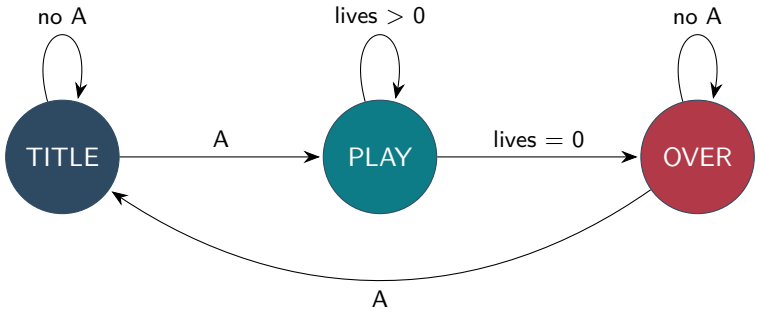
state	input 0	input 1	output
s_0	s_1	s_2	00
s_1	s_3	s_2	01
s_2	s_1	s_3	10
s_3	s_3	s_0	11

(b) A Moore machine: the output is in the state

Figure 13.3. State diagrams and state tables.

13.3.6 The warm-up as a state diagram

The game of the warm-up is the Moore machine of Figure 13.4. It has three states: two bits of state would be enough, and the program uses a whole word.



Present state	Input that changes it	Next state	Output (`draw`)
`TITLE`	**A** pressed	`PLAY`	the title screen
`PLAY`	lives = 0	`OVER`	the playfield
`OVER`	**A** pressed	`TITLE`	the final score

Figure 13.4. The game of the warm-up as a Moore machine.

Concept

Drawing the state diagram *before* writing the code is the single most useful design habit for a game. Chapter 17 applies it to Verm1.

13.4 Latches

13.4.1 Memory elements

The electronic circuits able to store a single bit are essentially of two kinds: latches and flip-flops. Both are *bistable*: they have two stable states, 0 and 1, which they keep as long as nothing changes them; and both have two outputs, Q and $\overline{Q}$, that are always the complement of each other.

Historical Note

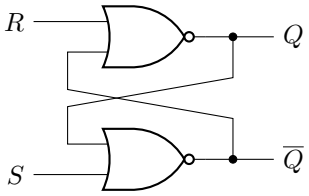
In 1918 Eccles and Jordan built the first electronic flip-flop, with two vacuum tubes. In 1943 the same circuit was at the heart of Colossus. Then came relays, transistors, and today logic gates on a chip. The name “flip-flop” comes from the sound of the electromechanical versions.

13.4.2 The SR latch

The *SR latch* is two NOR gates, each with its output connected to an input of the other (Figure 13.5): a *feedback*. The input S (*set*) brings Q to the stable state 1; the input R (*reset*) brings Q to the stable state 0. With $S = R = 0$ the latch keeps the last value: it remembers which of the two inputs was 1 most recently.

From the circuit, $Q = \overline{R + \overline{Q}}$ and $\overline{Q} = \overline{S + Q}$, hence

$$Q = \overline{R} \cdot (S + Q),$$



S	R	next Q
0	0	Q: hold
0	1	0: reset
1	0	1: set
1	1	not allowed

Figure 13.5. The SR latch built with NOR gates.

where the Q on the right is the value one propagation delay earlier. Because of the feedback Q depends on itself: in a network with loops every variable is a waveform in time, not a single value. Breaking the feedback wire turns the latch into a plain combinational network of two NORs, and that is how it is analysed, case by case. The same latch can be built with two NAND gates and active-low inputs.

13.4.3 The SR latch in time

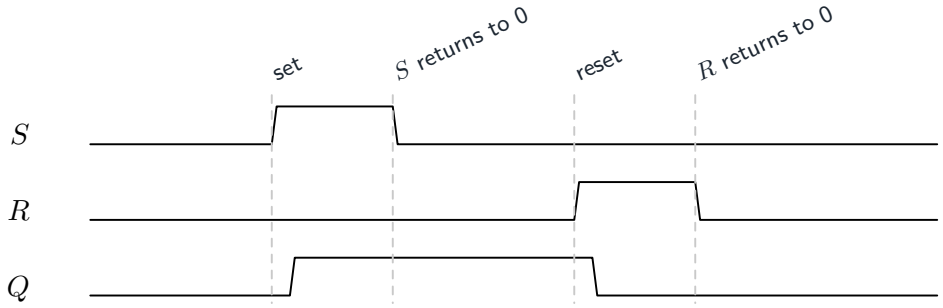


Figure 13.6. A timing diagram of the SR latch.

Figure 13.6 follows the latch in time. At the *set*, S goes to 1 and, after the delay of two gates, Q becomes 1. Then S returns to 0 and nothing changes: the feedback keeps Q at 1. *The latch remembers.* At the *reset*, R goes to 1 and, after the delay, Q becomes 0; when R returns to 0, Q stays there. The latch is a Moore automaton whose output is its state.

13.4.4 Why $S = R = 1$ is not allowed

With both inputs at 1 both NOR gates output 0: Q and $\overline{Q}$ are no longer complementary. Worse, if S and R then return to 0 together the two gates race, and the final state depends on which is faster: the output is unpredictable, and the latch may even oscillate. S and R are called *excitation inputs*, and the designer must guarantee that

$$S \cdot R = 0.$$

The JK and D elements exist to remove this problem.

13.4.5 Synchronous latches: adding a clock

In an *asynchronous* system the outputs can change whenever an input changes. In a *synchronous* system the instant at which the outputs can change is decided by the clock. In a clocked SR latch (Figure 13.7) S and R pass through two AND gates controlled by the clock: with $CK = 0$ both AND gates output 0 and the latch cannot change state; with $CK = 1$ the latch is sensitive to S and R exactly like the asynchronous one. The clock decides *when the inputs are listened to*.

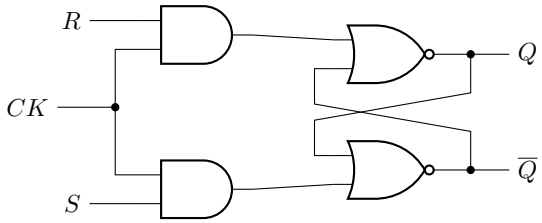


Figure 13.7. A clocked SR latch.

13.4.6 The JK and D latches

Two variants remove the forbidden combination (Table 13.1). In the *JK latch* the outputs are fed back to the input gates, $S = J \cdot \overline{Q}$ and $R = K \cdot Q$; since Q and $\overline{Q}$ are complementary, S and R can never both be 1, and the fourth combination of the inputs acquires a meaning: *toggle*. In the *D latch* (D for *delay*, or *data*) a single input goes to S and, through an inverter, to R, so that only the cases $S = 0$, $R = 1$ and $S = 1$, $R = 0$ are possible: the output simply repeats the input while the clock is active.

Table 13.1. The JK and D latches.

Inputs	JK latch	Input	D latch
$J = 0$, $K = 0$	hold: Q unchanged	$D = 0$	Q becomes 0
$J = 0$, $K = 1$	reset: Q becomes 0	$D = 1$	Q becomes 1
$J = 1$, $K = 0$	set: Q becomes 1		
$J = 1$, $K = 1$	toggle: Q is inverted		

13.4.7 Level triggering and its problem

A latch is *level-triggered*: it is sensitive for the whole time in which the clock is at its active level. While the clock is high a latch is *transparent*: every change of the inputs reaches the outputs, so that during one clock cycle several changes of state are possible. In a machine with feedback, such as that of Figure 13.2, this is fatal: the new state travels round the loop and changes the state again, in the same cycle. A JK latch with $J = K = 1$ keeps toggling as long as the clock stays high.

Concept

A state register must change *at most once per cycle*, and must not let the next state through while it is still being computed. A transparent latch cannot guarantee it.

13.5 Flip-flops

13.5.1 The master-slave flip-flop

The solution is to put two latches in a row and to make them listen in opposite phases of the clock (Figure 13.8). The first, the *master*, is active when $CK = 0$: it follows the input D, but the second, the *slave*, is closed and the output does not move. When the clock rises the master closes, freezing the value that D had at that instant, and the slave opens and

shows it. At no moment are both latches transparent: the output changes once per cycle, on the rising edge.

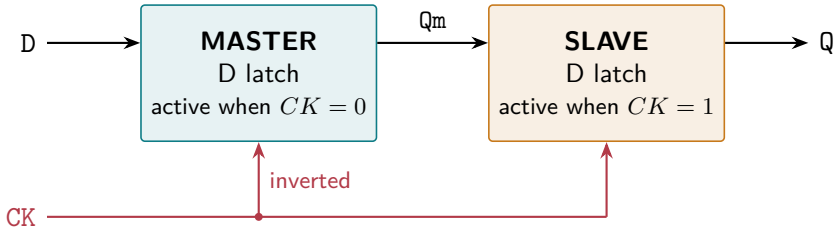


Figure 13.8. A master-slave D flip-flop.

Definition 13.5 (Flip-flop)

A *flip-flop* is an *edge-triggered* bistable: it samples its inputs only on an edge of the clock, and its outputs change only then.

13.5.2 Latch or flip-flop? Watch the output

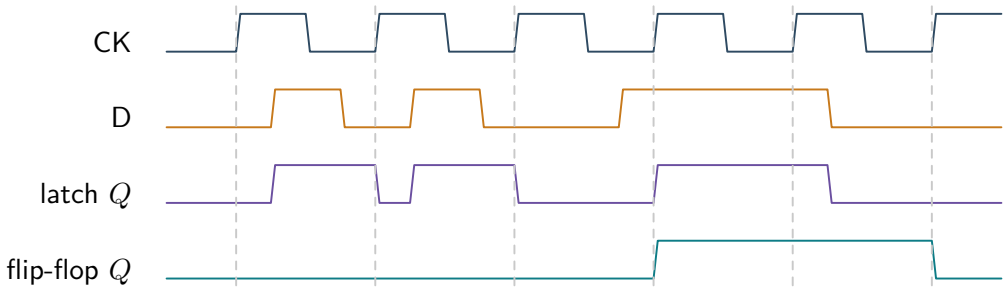


Figure 13.9. The same clock and the same input D, seen by a D latch and by a D flip-flop. The dashed lines are the rising edges.

In Figure 13.9 the D latch follows D for the whole time in which the clock is high, and holds the last value when the clock goes low: its output changes in the middle of a high phase. The D flip-flop looks at D only on the rising edge: short pulses between two edges are ignored. The output of the flip-flop is steady for a whole period, and this is what a state register needs.

13.5.3 Memory elements: a summary

Table 13.2 gives, for each element, the *characteristic equation* that expresses the next state Q^+. A T flip-flop is a JK with $J = K = T$. The D flip-flop is the standard memory element of today's processors.

13.6 Back to the machine

Table 13.2. Memory elements.

Element	Inputs	Next state	Changes when
SR latch ($S\,R = 0$)	S, R	$Q^+ = S + \overline{R}\,Q$	any time an input changes
clocked SR latch	S, R, CK	as above	while the clock is at the active level
D latch	D, CK	$Q^+ = D$	while the clock is at the active level
JK flip-flop	J, K, CK	$Q^+ = J\,\overline{Q} + \overline{K}\,Q$	on the clock edge
D flip-flop	D, CK	$Q^+ = D$	on the clock edge
T flip-flop	T, CK	$Q^+ = T \oplus Q$	on the clock edge

13.6.1 A register is a row of flip-flops

Definition 13.6 (Register)

A *register* is a device able to store a group of bits: one flip-flop per bit, and a common clock.

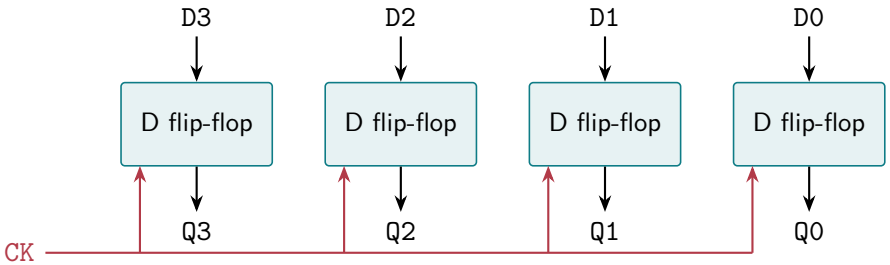


Figure 13.10. A register of four bits.

With a *load-enable* signal the register takes the new value only when the signal is asserted, and otherwise keeps the old one. The thirty-two registers of RISC-V are thirty-two rows of thirty-two flip-flops, and the write-enable lines of the decoder of Chapter 11 are their load enables. The classical part of the 74 series is the 74273: eight D flip-flops, one clock and a clear. Registers, shift registers, buses and three-state buffers are the subject of the next chapter.

13.6.2 The program counter: a register in a loop

The *program counter* (PC) is the register that holds the address of the instruction being executed. Its next value is computed by a small combinational network (Figure 13.11): an adder computes $PC + 4$, because instructions are 4 bytes apart (with the compressed extension the adder adds 2 after an instruction of 16 bits); a multiplexer chooses between that value and the target of a branch; the comparator of Chapter 11 produces, for `beq` and `bne`, the signal that drives the multiplexer. At each clock edge the register takes the value chosen. It is the general architecture of Figure 13.2 in its smallest useful instance.

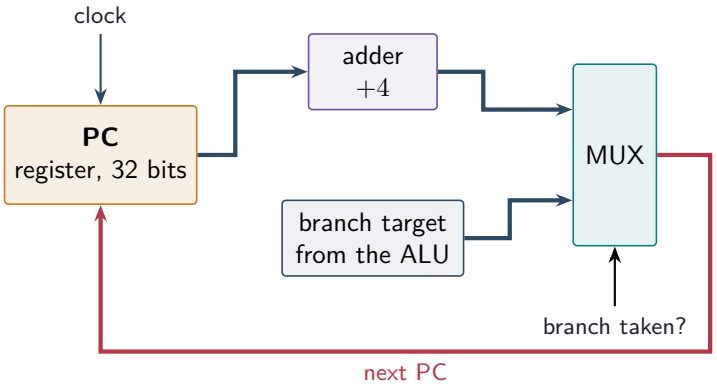


Figure 13.11. The program counter and the network that computes its next value.

13.6.3 Synthesis example: a two-bit counter

Example 13.1 (A counter modulo 4)

States: four, one per value of the count; there is no input, and the output is the state itself (a Moore machine). *State code*: two state variables Q_1Q_0, the count in binary. *State table*: beside each present state, the next one. With D flip-flops the excitation is the next state, $D_1D_0 = Q_1^+Q_0^+$.

present Q_1Q_0	next Q_1Q_0	D_1	D_0
0 0	0 1	0	1
0 1	1 0	1	0
1 0	1 1	1	1
1 1	0 0	0	0

D_0 is 1 in the rows 00 and 10, so $D_0 = \overline{Q_0}$. D_1 is 1 in the rows 01 and 10, so $D_1 = \overline{Q_1}Q_0 + Q_1\overline{Q_0} = Q_1 \oplus Q_0$. Two D flip-flops, one inverter and one XOR gate: a machine that counts 0, 1, 2, 3, 0, ..., one step per clock edge.

The same five steps, with more states and more inputs, design any sequential machine. And the result is worth comparing with the full adder: a counter is an adder that adds 1 to its own register.

13.6.4 The processor is a sequential machine

Back to the Machine

The *state* of a processor is the program counter, the thirty-two registers and the memory. Its *input* is the instruction at the address in the PC, with the data that it reads. Its *next-state network* is made of decoders, multiplexers and the ALU, which compute the new values. At each *clock edge* the registers named by the instruction take their new values.

Executing a program is nothing but a long walk through the states of a very large finite-state machine, one clock edge at a time. Single-stepping in PRG32-QT shows that walk, one state after another.

This is the bridge to Part V: the datapath (the combinational part and the registers) and

the control, which is itself a finite-state machine.

13.7 The program of this chapter

The listings of this chapter are parts of one program, `code/l13.S` in the repository of the course, printed in full in Listing 13.2. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 13.12 is the frame that the test wrote.

On the screen.
One colour per state. Blue is TITLE: press A and the screen turns green, PLAY. (OVER, red, is reached only when a game ends: this warm-up has no game yet. Hold A on the red screen and you meet the bug discussed in the lecture.)

What to press.
Press A.

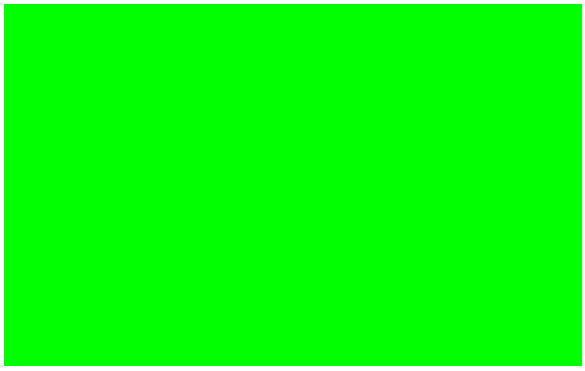


Figure 13.12. `l13.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 13: Sequential Machines
# l13.S: a game is a state machine
#
# On the screen: one colour per state. Blue is TITLE: press A and the screen
    turns
# green, PLAY. (OVER, red, is reached only when a game ends: this warm-up has
    no game
# yet. Hold A on the red screen and you meet the bug discussed in the lecture.)
# Build: python3 -m prg32 cartridge build l13.S --portable \
#            --entry-prefix l13 --name l13 --out l13.prg32
.option norelax

.equ TITLE, 0
.equ PLAY,  1
.equ OVER,  2

.section .text
.global l13_init
.global l13_update
.global l13_draw

l13_init:
    ret
```

```
22
23  l13_update:
24  game_update:
25      addi sp, sp, -16
26      sw   ra, 12(sp)
27      call prg32_input_read       # a0 = buttons: the INPUT
28      la   t0, state
29      lw   t1, 0(t0)              # t1 = the present STATE
30      andi t2, a0, 0x10           # button A
31      li   t3, TITLE
32      bne  t1, t3, 1f
33      beqz t2, done               # TITLE: stay until A is pressed
34      li   t1, PLAY               # TITLE --A--> PLAY
35      j    store
36  1:  li   t3, OVER
37      bne  t1, t3, done           # PLAY is handled elsewhere
38      beqz t2, done               # OVER: stay until A is pressed
39      li   t1, TITLE              # OVER --A--> TITLE
40  store:
41      sw   t1, 0(t0)              # the next state becomes the present state
42  done:
43      lw   ra, 12(sp)
44      addi sp, sp, 16
45      ret
46
47  l13_draw:
48      addi sp, sp, -16
49      sw   ra, 12(sp)
50      la   t0, state
51      lw   t1, 0(t0)
52      la   t0, colours          # the OUTPUT depends on the state only:
53      slli t1, t1, 2            # a Moore machine
54      add  t0, t0, t1
55      lw   a0, 0(t0)
56      call prg32_gfx_clear
57      lw   ra, 12(sp)
58      addi sp, sp, 16
59      ret
60
61  .section .rodata
62  colours: .word 0x001f, 0x07e0, 0xf800     # TITLE blue, PLAY green, OVER red
63
64  .section .data
65  state: .word TITLE
```

Listing 13.2. The complete program of the chapter, `code/l13.S`.

To run it on PRG32-QT the commands are those of Listing 13.3; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l13.S --portable \
    --entry-prefix l13 --name l13 --out l13.prg32
```

```
# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l13.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 13.3. Building `l13.S` and running it on PRG32-QT.

13.8 Check your understanding

Q13.1 In a Moore machine the output depends on:

A. the present input only
B. the present state only
C. the present state and the present input
D. the next state and the input

Q13.2 An SR latch built with NOR gates has $Q = 1$. S and R are both 0. What happens?

A. Q becomes 0
B. Q oscillates
C. Q stays at 1
D. the state is undefined

Q13.3 What distinguishes a D flip-flop from a D latch?

A. The flip-flop has two inputs
B. The flip-flop stores two bits
C. The flip-flop changes only on a clock edge
D. The flip-flop does not need a clock

Q13.4 How many flip-flops are needed for a finite-state machine with 10 states?

A. 3
B. 4
C. 5
D. 10

13.9 Exercises

Exercise 13.1 (paper first, then PRG32)

1. Draw the state diagram of a machine that outputs 1 when the last two inputs were both 1; write it as a Moore and as a Mealy machine.
2. Complete the timing diagram of an SR latch for a sequence of S and R of your choice, marking where it holds.
3. Synthesise a counter modulo 3 (0, 1, 2, 0, …) with two D flip-flops; what happens in the unused state?

4. Add a `PAUSE` state to the game of the warm-up, entered and left with the **SELECT** button, and draw the new diagram first.
5. Fix the warm-up so that **A** must be released and pressed again: store the previous state of the buttons and react only to a change from 0 to 1.

Expected result. The Moore machine of the first task needs three states (no 1 seen, one 1 seen, two or more), the Mealy machine two. For the counter: $D_1 = Q_0$ and $D_0 = \overline{Q_1}\,\overline{Q_0}$, with the state 11 as a don't-care or designed to return to 00. On PRG32: four screens, and a held button that causes exactly one transition. The last task is *edge detection* in software: keep `prev` in memory, compute $pressed_now = buttons \cdot \overline{prev}$, then store the buttons in `prev`. The program then behaves like a flip-flop instead of a latch.

13.10 Glossary

English	**Italiano**
sequential network	rete sequenziale
finite-state machine	automa a stati finiti
state table / diagram	tabella / diagramma degli stati
next-state function	funzione stato prossimo
present / next state	stato presente / prossimo
clock period	periodo di clock
latch / flip-flop	latch / flip-flop (bistabile)
feedback	retroazione
rising / falling edge	fronte di salita / di discesa
level- / edge-triggered	sensibile al livello / al fronte
register	registro
program counter	contatore di programma

13.11 Conclusions and reading

Digital circuits are combinational or sequential; for the sequential ones time and state matter. A sequential machine implements a finite-state automaton, described as a Mealy or as a Moore machine. Latches and flip-flops are the elementary memories: feedback makes a circuit remember. The clock decides when: latches listen during a level, flip-flops only on an edge. Registers, counters, the program counter and the processor itself are sequential machines built around combinational networks.

Reading. Patterson and Hennessy (2020), appendix A (clocks, memory elements, finite-state machines); Bucci (2017), the chapters on sequential networks; *Playing with RISC-V*, chapter 8, the assembly capstone game (Montella et al., 2026b).

CHAPTER 14

Elementary Machines: Registers and Buses

"Roads? Where we're going, we don't need roads."
— Dr. Emmett Brown, *Back to the Future*, Robert Zemeckis, 1985

In this chapter

By the end of this chapter you will be able to

- describe a register as a set of flip-flops with a common clock, and explain its load signal;
- explain what a bus is, and compare dedicated buses with a common bus;
- say why a common bus needs three-state buffers, and what high impedance means;
- describe a shift register and its four combinations of serial and parallel input and output;
- convert a datum from parallel to serial and back, in hardware and in RISC-V assembly.

14.1 Warm-up on the machine

The display and the sound chip of the kit are connected to the processor by a few wires, not by thirty-two. A word must therefore travel one bit after another: a *serial* transfer. Listing 14.1 sends a byte in this way.

```
# a0 = the byte to send, most significant bit first

    li   t0, 8              # eight bits to go
1:  srli t1, a0, 7          # bring bit 7 down to bit 0
    andi t1, t1, 1          # t1 = the bit to put on the wire
    ...                     # drive the data line with t1,
    ...                     # then pulse the clock line
    slli a0, a0, 1          # shift left: the next bit moves up
    addi t0, t0, -1
    bnez t0, 1b
```

Listing 14.1. Send a byte one bit at a time, most significant bit first.

The loop is a shift register in software: shift, take one bit, repeat. (The two lines left out would write to registers of the chip that control its pins, a subject of Chapter 15.) The ESP32-C6 contains hardware that does exactly this, millions of times per second: every pixel of the display of the kit arrives one bit at a time. This chapter builds the machines that do it.

14.2 Word gates and registers

14.2.1 Word gates

A *word* is a vector of n bits, $V = (v_0, v_1, \ldots, v_{n-1})$. A *word gate* acts on all the bits of a word at once: with a control signal a, it computes $a \cdot v_i$ for every i. With $a = 1$ the word passes unchanged; with $a = 0$ every bit is forced to 0. It is n identical gates that share one control input, the building block of every datapath. The multiplexer on words of Chapter 11 is built from word gates, and the bitwise instructions `and`, `or` and `xor` of RISC-V are word gates thirty-two bits wide.

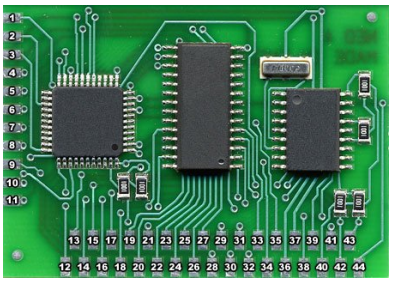
14.2.2 The register

A *register* is a device able to store a group of bits. In its simplest form it is a group of flip-flops with a common synchronisation signal, a clock or an enable (Chapter 13). *Loading* a register means storing the datum present on its inputs, which happens when the *load* signal is active, typically in step with the clock.

The classical example is the 74273, a register of 8 bits in one chip (Figure 14.1a): eight D flip-flops, one clock and one clear, inputs D_0 to D_7 and outputs Q_0 to Q_7. In RISC-V each of the registers `x1` to `x31` is thirty-two such flip-flops.



(a) SN74LS273: eight D flip-flops



(b) Bus lines on a printed circuit board

Figure 14.1. A register and a bus, as objects.

14.3 Buses

14.3.1 Data transfers and the bus

Definition 14.1 (Bus)

A *bus* is a communication channel that lets the components of an electronic system exchange data by transmitting and receiving signals: the set of lines that carries a datum.

The name is a contraction of the Latin *omnibus*, "for all". A bus of n lines carries a word of n bits in one step. The transfer of a datum from a register X to a register Y is the fundamental operation of the architecture of a computer: almost every instruction of Chapter 2 is, at bottom, a sequence of register transfers.

14.3.2 Two technical solutions

With *dedicated buses* there is one set of lines for each possible transfer. Several transfers can happen at the same time; the solution is efficient but expensive, because the wires grow with the square of the number of registers. With a *common bus* a single path collects many potential transfers: each time, one source and one destination are chosen, and a transfer takes two steps, from the source to the bus and from the bus to the destination. The common bus is cheap and widespread; it is less efficient, because transfers cannot be simultaneous. Real processors use intermediate solutions: several buses, each shared by a group of registers.

14.3.3 Parallel, serial and mixed transfers

In a *parallel* transfer the n bits of the word travel together on n lines, in one step. In a *serial* transfer the bits travel one after another on a single line, in n steps. A transfer can also be *mixed*: serial on the byte, for example, with eight lines that carry one byte at a time.

Buses come in very different physical sizes (Figure 14.1b). Inside a chip they are parallel. Between chips and between machines they are today almost always serial: USB, SATA, PCI Express, HDMI and Ethernet are all serial, because at very high speeds it is easier to keep one fast line clean than to keep thirty-two lines in step.

14.3.4 From bus to register, and from register to bus

Figure 14.2 shows the two elementary operations on a common bus. *From bus to register*: the datum is loaded into the register R when the load signal c is active (a *strobe*, usually in step with the clock). *From register to bus*: the content of R reaches the bus through a *buffer* controlled by an enable signal; when the enable is not active the register must leave the bus free for the others. How it does so is the subject of the next section.

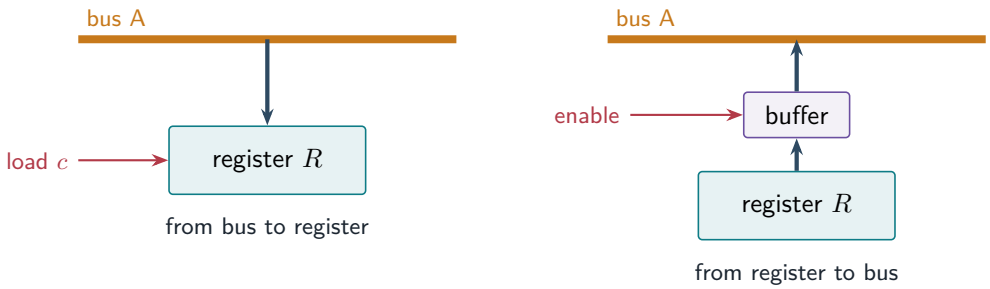


Figure 14.2. The two elementary transfers on a common bus.

14.4 Three-state buffers

14.4.1 The three-state gate

If two registers drove the same wire, one with 0 and one with 1, the result would be a short circuit between the supply and ground.

Definition 14.2 (Three-state output)

A *three-state* (or *tri-state*) output has a third condition besides 0 and 1: *high impedance*, written z. In high impedance the output is electrically disconnected: it neither drives nor loads the wire.

An enable signal a decides: with $a = 1$ the gate works normally, with $a = 0$ its output is z. Electrically, both output transistors of the gate (Chapter 10) are switched off. The table below describes a three-state AND with an active-high enable.

a	x	y	output
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1
0	any	any	z

It is the most widespread technique for connecting several source registers to a common bus.

14.4.2 Enabling a bus

A three-state *word* gate placed between a bus X and a bus Y, with an enable signal a, makes Y reproduce X if a is active; otherwise all the lines of Y are at high impedance, and Y takes the neutral value. One says that Y is *sustained* by a three-state buffer.

Back to the Machine

The GPIO pins of the ESP32-C6 work in this way: each pin can be an output, or can be released and used as an input. On the kit the pins of the joystick are configured as inputs, with their output buffers in high impedance, and the pins of the display as outputs.

14.4.3 Connecting buses: the bus OR

Several source registers are connected to one bus simply by joining their outputs, each through a three-state buffer (Figure 14.3).

The rule of the common bus

Of the n sources that flow into a bus, at most one is enabled at a time: one active signal, and the other $n - 1$ neutral.

Inside modern chips long shared lines are usually replaced by multiplexers, which make the same selection without high-impedance states; between chips, three-state buses are still the rule.

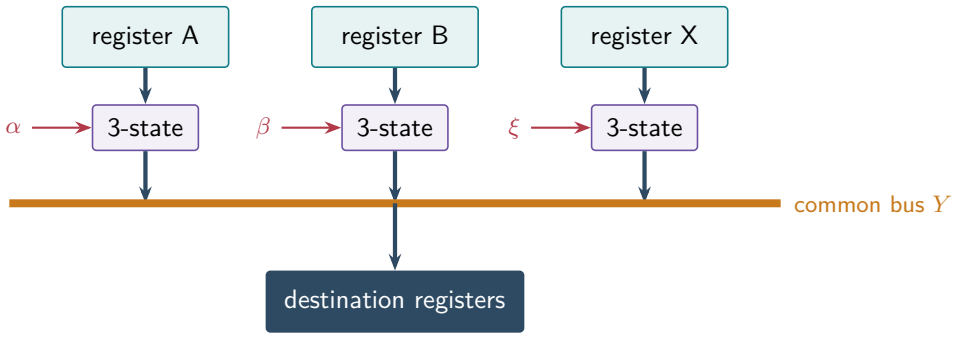


Figure 14.3. Three sources on a common bus, each behind a three-state buffer with its own enable.

14.5 Shift registers

14.5.1 Definition

Definition 14.3 (Shift register)

A *shift register* is a chain of one-bit memory cells connected to each other: at every clock pulse each bit moves from one cell to the adjacent one.

Each cell is a D flip-flop whose input is the output of its neighbour (Figure 14.4). The shift can go in one fixed direction or in a direction chosen by a control line (bidirectional registers). Edge-triggered flip-flops are essential here: with transparent latches a bit would run through the whole chain during one clock pulse.

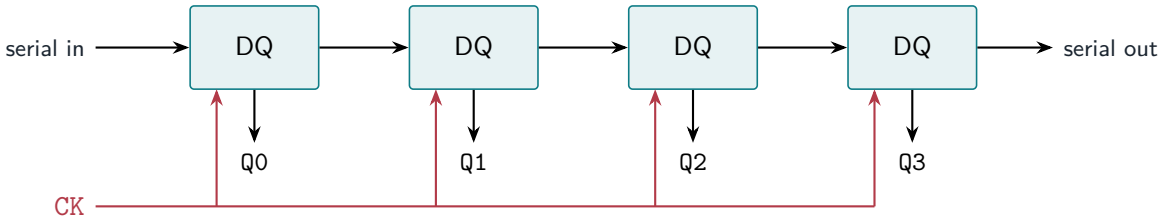


Figure 14.4. A shift register of four bits built with D flip-flops, with serial input and both serial and parallel outputs.

14.5.2 Four kinds of shift register

The input and the output of a shift register can each be serial or parallel, which gives four kinds (Table 14.1). They make the reading and the writing of data easier, because they turn parallel data into serial data and back.

Table 14.1. Four kinds of shift register.

Kind	Name	Use
serial in, serial out	SISO	a delay of n clock pulses; a serial link between two registers
serial in, parallel out	SIPO	conversion from serial to parallel: the receiver
parallel in, serial out	PISO	conversion from parallel to serial: the transmitter
parallel in, parallel out	PIPO	an ordinary register that can also shift

14.5.3 Transfers between registers

In a serial transfer the serial output of one shift register feeds the serial input of another: n clock pulses move a word of n bits, and one wire is enough. In a parallel transfer the parallel outputs of one register feed the parallel inputs of another: one clock pulse moves the whole word, and n wires are needed.

Concept

Moving a word of n bits costs either time or copper: n pulses or n wires. It is the same trade-off as between dedicated and common buses.

14.5.4 Serial to parallel and back

A complete serial link works in five steps (Figure 14.5). The datum of n bits is placed in parallel in a register R. R is a shift register, and the n bits leave it one after another, in n successive steps. A single line carries them to the equipment downstream. There another register is loaded serially in n steps, rebuilding the datum bit by bit. Finally the rebuilt datum is passed on in parallel.



Figure 14.5. The conversion of a datum from parallel to serial and back.

This is the hardware of the warm-up. The display of the kit receives each pixel in this way over a bus called SPI, and the keyboard of Chapter 21 sends each key code in this way. UART, SPI, I2C, I2S and USB all rest on a pair of shift registers, one at each end.

14.5.5 Registers with an address: a first memory

Take four registers of three bits and put a decoder in front of them (Figure 14.6). Two address lines, A_1 and A_0, go to the 2-to-4 decoder, which selects one register. Three data inputs, $I_2I_1I_0$, reach all the registers, but only the selected one is loaded; three data outputs, $O_2O_1O_0$, show the content of the selected register through three-state buffers. Three control lines govern the whole: a *chip select* (CS), which enables the device, a line that chooses between reading and writing, and an *output enable* (OE). The result is a memory of 4 words by 3 bits.

Concept

A decoder in front of a group of registers makes a memory. The next chapter scales this up to the main memory of a computer.

14.6 Back to the machine

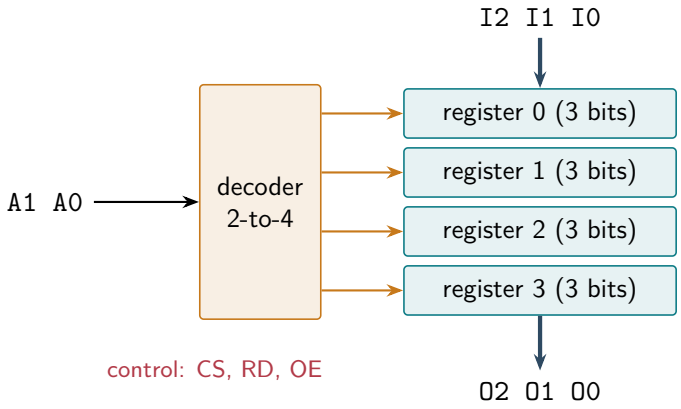


Figure 14.6. Four registers and a decoder: a memory of 4 words by 3 bits.

14.6.1 Buses on the PRG32 kit

The kit contains an example of nearly every kind of bus of this chapter (Table 14.2). The chip select of the display is the CS of the small memory just seen: several devices can share the same serial bus because only the selected one listens.

Table 14.2. Buses on the PRG32 kit (from `docs/hardware` of the repository).

Connection	Kind	Lines	What travels on it
inside the ESP32-C6	parallel	32 data lines, address and control	instructions, data and register transfers
display (ILI9341)	serial, SPI	clock, data out, chip select, data/command	every pixel of every frame, bit by bit
sound amplifier	serial, I2S	bit clock, word select, data	audio samples, 22 050 per second
joystick and buttons	one wire per button	one input pin each	a decoded representation: one line per button
USB to the laptop	serial, differential pair	two data wires	the firmware and the debug console

14.6.2 Shifts in RISC-V: a shift register in one instruction

`slli` and `srli` move all the bits of a register by a given number of positions, filling with zeros; `srai` fills with copies of the sign bit (Chapter 5); `sll`, `srl` and `sra` take the amount from a second register. With them the two conversions are two instructions each (Listing 14.2).

```
# parallel to serial, LSB first
andi t1, a0, 1      # take bit 0
srli a0, a0, 1      # next bit

# serial to parallel, MSB first
slli a1, a1, 1      # make room
or   a1, a1, t1     # insert bit
```

Listing 14.2. Parallel to serial, and serial to parallel.

A true shift register needs n clock pulses to shift by n positions. The processor does not

work in that way: in hardware a shift by any amount in one step is done by a *barrel shifter*, which is made of layers of multiplexers, not of a chain of flip-flops. It is combinational, it does its work within one clock period, and it is one of the units of the ALU of Chapter 11.

14.7 The program of this chapter

The listings of this chapter are parts of one program, `code/l14.S` in the repository of the course, printed in full in Listing 14.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 14.7 is the frame that the test wrote.

On the screen.
Top row: the eight bits of 0xA5 in the order in which they leave, most significant first (lit, dark, lit, dark, dark, lit, dark, lit). Bottom row: the byte rebuilt by the receiver. A green square says that it is equal to the byte sent.

What to press.
Nothing: the program runs by itself.

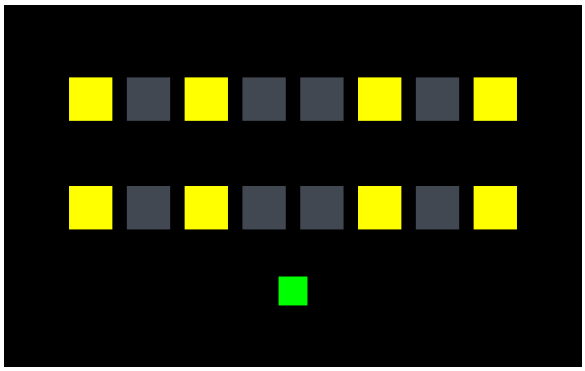


Figure 14.7. `l14.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 14: Elementary Machines: Registers and Buses
# l14.S: a byte sent one bit at a time, and rebuilt at the other end
#
# On the screen: top row: the eight bits of 0xA5 in the order in which they leave,
# most significant first (lit, dark, lit, dark, dark, lit, dark, lit). Bottom row: the
# byte rebuilt by the receiver. A green square says that it is equal to the byte sent.
# Build: python3 -m prg32 cartridge build l14.S --portable \
#            --entry-prefix l14 --name l14 --out l14.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800
.equ BYTE,   0xA5            # the byte to send: 1010 0101

```

```
18 .section .text
19 .global l14_init
20 .global l14_update
21 .global l14_draw
22
23 l14_init:
24     ret
25
26 l14_update:
27     ret
28
29 # lamp: a0 = position 0 to 7, a1 = y, a2 = bit
30 lamp:
31     li   a4, GREY
32     beqz a2, 1f
33     li   a4, YELLOW
34 1:  slli a0, a0, 5
35     addi a0, a0, 36          # x = 36 + 32 * position
36     li   a2, 24
37     li   a3, 24
38     tail prg32_gfx_rect
39
40 l14_draw:
41     addi sp, sp, -32
42     sw   ra, 28(sp)
43     sw   s0, 24(sp)
44     sw   s1, 20(sp)
45     sw   s2, 16(sp)
46     sw   s3, 12(sp)
47     li   a0, BLACK
48     call prg32_gfx_clear
49
50     li   s0, BYTE            # a0 of the slide: the byte to send
51     li   s1, 0               # the receiver's shift register
52     li   s2, 0               # how many bits have gone
53 1:  srli t1, s0, 7           # bring bit 7 down to bit 0
54     andi s3, t1, 1           # s3 = the bit to put on the wire
55     mv   a0, s2              # ... here the wire is a lamp:
56     li   a1, 40              #     position = the order of sending
57     mv   a2, s3
58     call lamp
59     slli s1, s1, 1           # receiver: make room (serial to parallel)
60     or   s1, s1, s3          #           insert the bit
61     slli s0, s0, 1           # sender: shift left, the next bit moves up
62     andi s0, s0, 255         #         (keep eight bits)
63     addi s2, s2, 1
64     li   t0, 8               # eight bits to go
65     blt  s2, t0, 1b
66
67     li   s2, 0               # show the byte that the receiver rebuilt
68 2:  li   t0, 7
69     sub  t0, t0, s2
70     srl  a2, s1, t0
71     andi a2, a2, 1
72     mv   a0, s2
73     li   a1, 100
```

```
    call lamp
    addi s2, s2, 1
    li   t0, 8
    blt  s2, t0, 2b

    li   a4, RED              # is it the byte that was sent?
    li   t0, BYTE
    bne  s1, t0, 3f
    li   a4, GREEN
3:  li   a0, 152
    li   a1, 150
    li   a2, 16
    li   a3, 16
    call prg32_gfx_rect
    lw   s3, 12(sp)
    lw   s2, 16(sp)
    lw   s1, 20(sp)
    lw   s0, 24(sp)
    lw   ra, 28(sp)
    addi sp, sp, 32
    ret
```

Listing 14.3. The complete program of the chapter, `code/l14.S`.

To run it on PRG32-QT the commands are those of Listing 14.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l14.S --portable \
    --entry-prefix l14 --name l14 --out l14.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l14.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 14.4. Building `l14.S` and running it on PRG32-QT.

14.8 Check your understanding

Q14.1 Why does a common bus need three-state buffers?

A. To make transfers faster
B. So that only the enabled source drives the lines
C. To store the datum on the bus
D. To convert serial data to parallel

Q14.2 How many clock pulses does a serial transfer of a 32-bit word need?

A. 1

B. 5
C. 32
D. 64

Q14.3 A memory has 10 address lines and 8 data lines. How many bits does it store?

A. 80
B. 1 024
C. 8 192
D. 10 240

14.9 Exercises

Exercise 14.1 (paper first, then PRG32)

1. Draw a 4-bit serial-in, parallel-out shift register with D flip-flops, and fill in its contents over four clock pulses for the input 1, 0, 1, 1.
2. Three registers share a common bus: write the enable and load signals needed to copy register B into register A.
3. Complete the warm-up: write the loop that receives eight bits from a line and rebuilds the byte.
4. On PRG32, show a byte as eight lamps and shift it to the left by one position every half second.
5. Make the bit that leaves on the left come back in on the right (a rotation): a *ring counter*.

Expected result. The third task is `slli a1, a1, 1` and then `or` with the received bit, eight times. The last is `srli t1, a0, 7`; `andi t1, t1, 1`; `slli a0, a0, 1`; `or a0, a0, t1`; `andi a0, a0, 255`. On PRG32: a pattern of lamps that walks to the left and, in the last task, goes round for ever.

14.10 Glossary

English	Italiano
word gate	porta di parola
register	registro
bus	bus
dedicated / common bus	bus dedicato / comune
load, strobe	caricamento, strobe
three-state (tri-state)	a tre stati (tristate)
high impedance	alta impedenza
shift register	registro a scorrimento
serial / parallel transfer	trasferimento seriale / parallelo
chip select	selezione del chip

14.11 Conclusions and reading

Registers are an evolved form of memory with an elementary way of working. Reading and writing them happens through buses of different kinds. For a common bus to work,

three-state circuits are needed: 0, 1, and disconnected. Data can be transferred serially or in parallel, and shift registers convert between the two. A decoder in front of a group of registers makes a memory: the subject of the next chapter.

Reading. Patterson and Hennessy (2020), appendix A; Bucci (2017), the chapters on registers and buses; `docs/hardware/hardware.md` in the PRG32 repository.

CHAPTER 15

Memory

"... computers and the programs will start thinking and the people will stop."
— Dr. Walter Gibbs, *Tron*, Steven Lisberger, 1982

In this chapter

By the end of this chapter you will be able to

- classify memories as volatile or persistent, on-line or off-line, and place them in the memory hierarchy;
- describe how SRAM and DRAM store a bit, and why they are used for different jobs;
- explain what words, addresses and byte-addressable memory are;
- tell little-endian from big-endian order, and say which one RISC-V uses;
- describe step by step how the processor reads and writes a word;
- explain memory-mapped input/output and what DMA is for.

15.1 Warm-up on the machine

```
    la   t0, value
    lw   t1, 0(t0)       # the whole word        t1 = 0x12345678
    lbu  t2, 0(t0)       # the byte at address A     t2 = 0x78
    lbu  t3, 1(t0)       # the byte at address A+1   t3 = 0x56
    lbu  t4, 3(t0)       # the byte at address A+3   t4 = 0x12

.section .data
value: .word 0x12345678
```

Listing 15.1. The same memory read as a word and as bytes.

In Listing 15.1 a word of thirty-two bits occupies four consecutive addresses: memory is addressed byte by byte. The byte at the lowest address is `0x78`, the least significant: the order is little-endian. `lw` fetches four bytes in one operation, `lbu` fetches one.

Try It Yourself

Run it in PRG32-QT and open the memory view at the address in `t0`: the bytes appear as `78 56 34 12`. The byte order of Chapter 4 has become an experiment.

What is a memory made of, and what happens between the processor and the memory when `lw` runs?

15.2 What is a memory?

Definition 15.1 (Memory)

The *memory* is the element of a computer, or of one of its subsystems, dedicated to the persistence of data and of the instructions of programs.

Memories are the components able to store information: data, programs and results. They are classified along two axes. A memory is *volatile* if the information is kept only as long as the computer is powered, and *persistent* if it is kept even when the computer is switched off. It is *on-line* if the data are always accessible, and *off-line* if the medium must be mounted before the data can be reached.

15.2.1 The memory hierarchy

No technology is at the same time fast, large and cheap. A computer therefore uses several, arranged in a *hierarchy* (Figure 15.1): going up, memories are faster and more expensive per bit; going down, larger and cheaper per bit. Typical access times are below a nanosecond to a few nanoseconds for registers and cache, tens of nanoseconds for DRAM, tens of microseconds for solid-state disks, milliseconds for magnetic disks, seconds to minutes for tape.

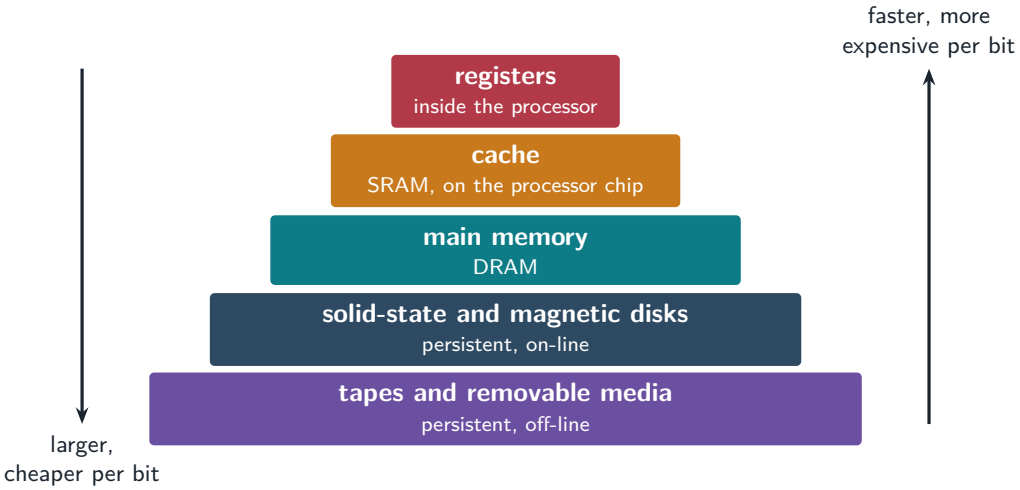
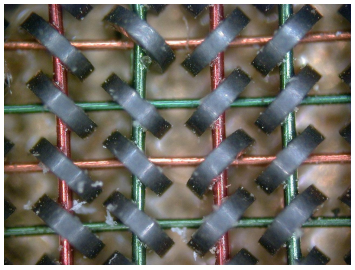


Figure 15.1. The memory hierarchy.

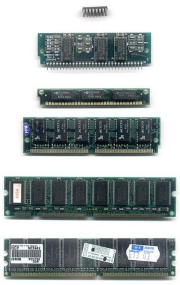
15.3 A little history: rings of ferrite

15.3.1 Magnetic-core memory

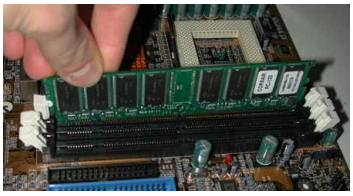
In the first computers, before semiconductor memories became widespread, a bit was the direction of the magnetic field of a small ring of ferrite, a magnetic ceramic (Figure 15.2a). A core memory is static: it keeps its contents without power. The first research was done by An Wang and Way-Dong Woo at Harvard in 1949; at the beginning of the 1950s Jay Forrester at MIT built a core memory for Whirlwind, a computer meant as a flight simulator.



(a) Ferrite cores and their wires



(b) Memory modules of several generations



(c) A module and its slot

Figure 15.2. Memory, yesterday and today.

15.3.2 Reading and writing a core

Each core is threaded by wires. Two address lines, X and Y, select one core: only the core at their crossing receives enough current to change.

Reading is destructive. The selected bit is set to 0; if it contained 1, a pulse appears on a third wire, the *sense* line, and otherwise nothing happens. The datum has been read and lost. Wang's solution was that every read is followed by a write that stores the datum again.

Writing. All the bits start at 0. X and Y select the core, which is set to 1; if the bit must be 0, an *inhibit* line prevents the change. (Sense and inhibit share one wire.) Forrester's contribution was the arrangement that manages a large number of cores with few connections.

The destructive read followed by a rewrite has returned today in DRAM.

15.4 Memory today

15.4.1 SRAM and DRAM

Definition 15.2 (SRAM and DRAM)

In a *static RAM* (SRAM) each bit is held by a bistable circuit, as in a flip-flop. In a *dynamic RAM* (DRAM) each bit is a charge in a tiny capacitor, behind one transistor.

The cells of an SRAM are arranged in a matrix and reached by row and column; each takes six transistors (Chapter 10) and keeps its content as long as it is powered. SRAM is fast and expensive, and is used for cache memories; it is also the main memory of a microcontroller such as the ESP32-C6, which has 512 KB of SRAM and no DRAM at all.

The capacitor of a DRAM cell discharges, and its content must be *refreshed* periodically by a circuit inside the memory. DRAM is cheap and dense, and is the main memory of laptops, telephones and servers. An *SDRAM* is a synchronous DRAM, whose access is governed by a clock; DDR4 and DDR5 are generations of it.

15.4.2 Other memories

A *ferroelectric RAM* (FeRAM) keeps its state without power. *Flash* memory is persistent and is erased in blocks: it is the memory of USB keys, of telephones and of solid-state disks, treated in the next chapter. The program of a microcontroller lives in flash and its variables in SRAM.

Memory modules have changed shape and size through the years (Figure 15.2b): SIMM, SDRAM PC100 and PC133, Rambus and DDR held tens to hundreds of megabytes each; today's DDR5 modules hold 16 to 64 GB.

15.4.3 How to recognise the memory

On the motherboard of a desktop or of a server the memory is in rows of long slots next to the processor, which hold the modules (Figure 15.2c). On a laptop or a telephone it is in chips soldered close to the processor. On the PRG32 kit there is nothing to see: the SRAM is inside the ESP32-C6 itself, and the flash is a small chip beside it.

Two parameters characterise a memory: its *size*, the number of cells, which today is gigabytes on a PC and kilobytes on a microcontroller; and its *access time*, the delay after which the datum is available, tens of nanoseconds for DRAM. For comparison, at 160 MHz one clock period of the ESP32-C6 is 6.25 ns.

15.5 The main memory

15.5.1 Words and addresses

Definition 15.3 (Main memory)

The *main memory* is an array of *words*; a word is a string of m bits, which the processor accesses with a single read or write operation. Each word is identified by an *address*: an integer between 0 and $N-1$, represented on n bits, with $N = 2^n$.

The address is a code word, and the memory decodes it: it is the decoder of Chapter 11 and the small memory of Chapter 14, scaled up. Thirty-two address bits give 2^{32} addresses, 4 GiB. A memory is called *random access* (RAM) when the access time does not depend on the position of the datum. Processors access memory with a parallelism equal to the length of their word: 16, 32 or 64 bits.

15.5.2 Byte-addressable memory

In a *byte-addressable* memory the byte is the smallest unit that has an address (Figure 15.3). A word of 32 bits occupies four consecutive addresses, and its address is that of its first byte. Consecutive words are 4 addresses apart: this is why the program counter advances by 4, and why the index of an array of words is multiplied by 4 (Chapter 4).

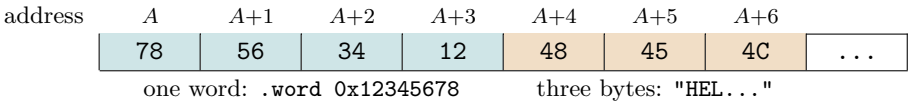


Figure 15.3. A byte-addressable memory: each cell is one byte.

15.5.3 Big-endian and little-endian

The two names say how the bytes of a word are arranged in memory (Figure 15.4). In *little-endian* order the least significant byte goes at the lowest address: it is the order of RISC-V, of x86 and of ARM. In *big-endian* order the most significant byte goes at the lowest address: it is the order of the Motorola 68000, the reference processor of the previous editions of this course, and of the protocols of the Internet.

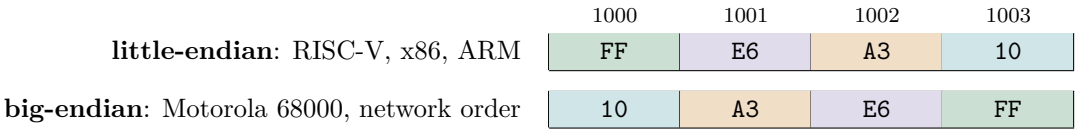


Figure 15.4. The word `0x10A3E6FF` stored at address 1000.

15.5.4 Aligned and unaligned words

A word that starts at an address which is a multiple of its size is *aligned*: multiples of 4 for words of 32 bits. An aligned word is fetched with a single memory access; an unaligned word straddles two memory words, and two accesses are needed. The RISC-V specification leaves the treatment of misaligned accesses to the implementation: a load or a store to a misaligned address is, depending on the processor, done more slowly or refused with an exception. Correct programs keep their data aligned. The assembler aligns the data declared with `.word`, and the directive `.balign 4` forces alignment after a string or a byte (Listing 15.2). Instructions must always be aligned.

```
.section .data
name:   .asciz "PRG32"    # 6 bytes
        .balign 4           # skip 2
score:  .word 0             # aligned
```

Listing 15.2. Forcing the alignment of a word.

15.5.5 Parameters of a memory

The *capacity* of a memory is the number N of its registers and the length in bits of each: 1 MB is 2^{20} registers of 8 bits; "16 kW of 32 bits" is 16×2^{10} registers of 32 bits. The access time is the time needed to carry out a read or a write: the time to select the location, plus the time for the read or the write itself.

Back to the Machine

The ESP32-C6 has 512 KB of SRAM for data, 320 KB of ROM, and an external flash of some megabytes for programs. The menu of PRG32 shows what is left for the programmer: about 98 KB of RAM and 509 KB of flash for cartridges. The main

memory of the chip is that of eight Commodore 64s, or one thirty-thousandth of that of a laptop.

15.6 Processor and memory

15.6.1 The interface

The processor and the memory talk through three buses and two registers (Figure 15.5). The *memory address register* (MA) holds the address and drives the *address bus*. The *memory buffer register* (MB) holds the datum that travels, in either direction, on the *data bus*. The control unit drives the *control lines*, among which are the read enable and the write enable. MA and MB are internal registers of the model of processor of Chapter 18: they are not among the thirty-two registers that a RISC-V programmer sees.

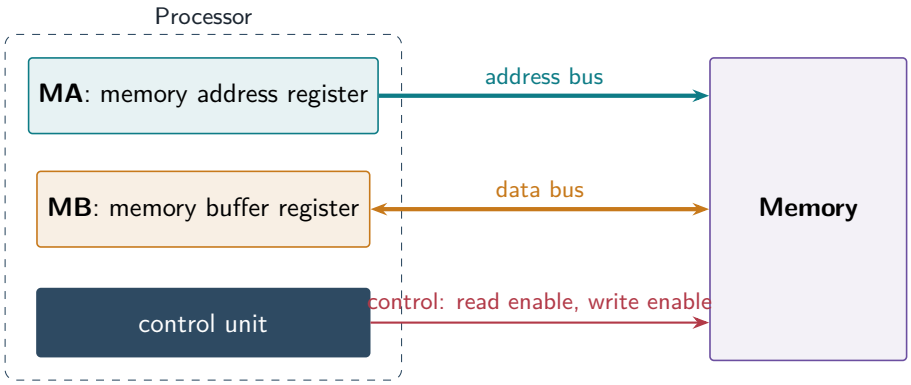


Figure 15.5. The interface between processor and memory.

15.6.2 Fetching a word: `lw t1, 0(t0)`

Table 15.1. The steps of a load.

Step	What happens	Where
1	the address, `t0` + 0, is computed by the ALU and placed in MA	processor
2	MA drives the address bus	address bus
3	the control unit raises the read enable	control lines
4	the memory decodes the address and puts the four bytes of the word on the data bus	memory, data bus
5	the datum on the data bus is latched in MB	processor
6	MB is copied into the destination register `t1`	processor

A store, `sw t1, 0(t0)`, is symmetrical (Table 15.1): the address goes into MA and the datum into MB, the write enable is raised, and the memory stores what it finds on the data bus. The access time is the time that passes between step 2 and step 5.

15.6.3 Synchronising processor and memory

How does the processor know that the memory has finished? With the *asynchronous* technique it waits until the memory signals, on an *acknowledge* line, that the operation is

complete: the processor adapts to the speed of the memory, and the design of the memory becomes independent of that of the processor, at the price of more complexity. With the *synchronous* technique one of the control lines carries the pulses of a clock of fixed frequency, and a read or a write completes within one clock cycle, which must therefore be sized on the speed of the memory. The second technique is simpler, and is the rule inside a single chip such as a microcontroller.

15.7 Memory and input/output

15.7.1 Connecting processor, memory and I/O

The input/output subsystem lets the processor interact with input devices (keyboard, mouse), output devices (screen, printer) and devices that are both (mass storage, network cards, serial ports). It can be connected in two ways.

Definition 15.4 (Memory-mapped I/O)

With *memory-mapped I/O* the memory and the devices share the same bus and the same address space: a register of a device is read and written like a memory location, and no special instructions are needed.

The alternative is *isolated I/O*: two physical buses, or two address spaces, one for memory and one for I/O, and dedicated instructions to reach the devices. The Intel x86 family has both: the instructions `in` and `out` besides memory-mapped devices.

Back to the Machine

Memory-mapped I/O is the RISC-V way: RISC-V has no input/output instructions at all. Writing a word to a peripheral register with `sw` is how a RISC-V program lights a pixel or reads a button.

15.7.2 Where things live on the ESP32-C6

Table 15.2 is a simplified map of the address space of the chip of the kit. One address space serves for everything: the same thirty-two address lines reach ROM, RAM, flash and every device, and the same `lw` and `sw` operate them. (The table is simplified from the technical reference manual (Espressif Systems, 2025), which should be consulted before relying on an address.)

The address `0x40800000`, at which every cartridge is loaded (Chapter 1), is the beginning of the SRAM.

15.7.3 The three component buses, and DMA

The bus that connects processor, memory and I/O is thus made of three buses: the *data bus*, which carries the data; the *address bus*, which carries the address, of memory or of I/O, from or to which the datum is directed; and the *control lines*: read and write enable, acknowledge, and others.

Table 15.2. The address space of the ESP32-C6, simplified.

Range begins with	What is there	Reached by
`0x4000...`	ROM: the code that starts the chip	instruction fetch
`0x4080...`	SRAM: 512 KB for data, stack and cartridges loaded in RAM	`lw`, `sw`, instruction fetch
`0x4200...`	external flash, seen through a cache: firmware and constants	`lw`, instruction fetch
`0x6000...`	peripheral registers: GPIO, SPI, I2S, timers, Wi-Fi	`lw`, `sw`

Definition 15.5 (Direct memory access)

With *direct memory access* (DMA), blocks of data go directly between a device and the memory. The processor starts the transfer and is told when it is over.

If several units share the bus and can access the memory directly, a DMA controller arbitrates among them. On the ESP32-C6 the SPI and I2S peripherals use DMA to send the frame buffer to the display and the audio samples to the amplifier: the processor does not copy the pixels one by one.

15.8 The program of this chapter

The listings of this chapter are parts of one program, `code/l15.S` in the repository of the course, printed in full in Listing 15.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 15.6 is the frame that the test wrote.

On the screen.
Five rows of lamps. The first is the word 0x12345678 read with lw. The other four are the bytes at addresses A, A+1, A+2, A+3 read with lbu: 0x78, 0x56, 0x34, 0x12. The lowest address holds the least significant byte: little-endian.

What to press.
Nothing: the program runs by itself.

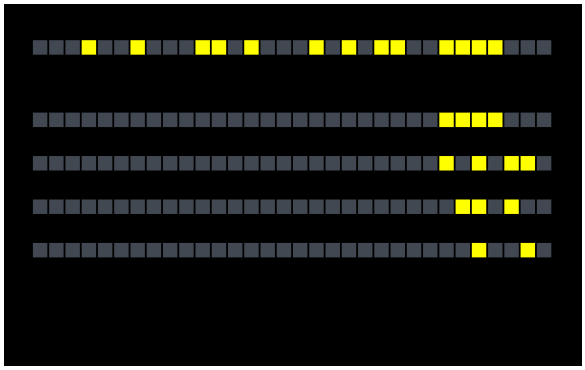


Figure 15.6. `l15.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 15: Memory
# l15.S: the same memory read as a word and as bytes
#
# On the screen: five rows of lamps. The first is the word 0x12345678 read with lw.
# The other four are the bytes at addresses A, A+1, A+2, A+3 read with lbu: 0x78,
# 0x56, 0x34, 0x12. The lowest address holds the least significant byte: little-
# endian.
# Build: python3 -m prg32 cartridge build l15.S --portable \
#            --entry-prefix l15 --name l15 --out l15.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0

.section .text
.global l15_init
.global l15_update
.global l15_draw

l15_init:
    ret

l15_update:
    ret

l15_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    call prg32_console_clear
    la   t0, value
    lw   a0, 0(t0)       # the whole word
    call prg32_console_hex32

    la   t0, value
    lw   a0, 0(t0)       # the whole word: 0x12345678
    li   a1, 20
    call lamps32
    li   s0, 0           # byte number, 0 to 3
1:  la   t0, value
    add  t0, t0, s0      # address A + s0: one byte further each time
    lbu  a0, 0(t0)       # the byte at that address, zero-extended
    slli a1, s0, 4
    slli t1, s0, 3
    add  a1, a1, t1      # 24 * s0
    addi a1, a1, 60      # y = 60 + 24 * byte number
    call lamps32
    addi s0, s0, 1
    li   t0, 4
```

```
    blt  s0, t0, 1b
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

# lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
#   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
lamps32:
    addi sp, sp, -32
    sw   ra, 28(sp)
    sw   s0, 24(sp)
    sw   s1, 20(sp)
    sw   s2, 16(sp)
    mv   s0, a0              # the word
    mv   s1, a1              # y
    li   s2, 0               # lamp number, 0 on the left
1:  li   t0, 31
    sub  t0, t0, s2          # lamp i shows bit 31 - i
    srl  t1, s0, t0          # bring that bit down to bit 0
    andi t1, t1, 1           # keep it, clear all the others
    li   a4, GREY            # lamp off
    beqz t1, 2f
    li   a4, YELLOW          # lamp on
2:  slli a0, s2, 3
    add  a0, a0, s2          # 9 * i
    addi a0, a0, 16          # x = 16 + 9 * i
    mv   a1, s1
    li   a2, 8
    li   a3, 8
    call prg32_gfx_rect
    addi s2, s2, 1
    li   t0, 32
    blt  s2, t0, 1b
    lw   s2, 16(sp)
    lw   s1, 20(sp)
    lw   s0, 24(sp)
    lw   ra, 28(sp)
    addi sp, sp, 32
    ret

.section .data
value: .word 0x12345678
```

Listing 15.3. The complete program of the chapter, `code/l15.S`.

To run it on PRG32-QT the commands are those of Listing 15.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l15.S --portable \
    --entry-prefix l15 --name l15 --out l15.prg32
```

```
# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l15.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 15.4. Building `l15.S` and running it on PRG32-QT.

15.9 Check your understanding

Q15.1 A memory has 16 address lines and is byte-addressable. How large is it?

A. 16 bytes
B. 32 KiB
C. 64 KiB
D. 16 MiB

Q15.2 On RISC-V, `value: .word 0xCAFE0001` is at address A. What does `lbu t0, 0(t0)` read if `t0` $= A$?

A. `0xCA`
B. `0xFE`
C. `0x00`
D. `0x01`

Q15.3 Why must DRAM be refreshed?

A. Because its capacitors lose their charge
B. Because reading is slow
C. Because it is not random access
D. Because the clock is too fast

Q15.4 How does a RISC-V program write to an output device?

A. With the `out` instruction
B. With `sw` to the address of a device register
C. With a special register `x32`
D. It cannot: only the firmware can

15.10 Exercises

Exercise 15.1 (paper first, then PRG32)

1. A memory of 4 KiB words of 16 bits: how many address lines, how many data lines, how many bits in all?
2. Write the bytes stored, in order of address, by `.word 0xDEADBEEF` and by `.asciz "Ciao"` on a little-endian machine.
3. List the steps of `sw t1, 8(t0)` on the processor–memory interface.

4. In PRG32-QT, find your counter variable in the memory view and watch its four bytes change frame by frame.
5. Store a word with `sw` and read it back with four `lbu` instructions; rebuild it with shifts and `or`s and check that it matches.

Expected result. On paper: 12 address lines, 16 data lines, 65 536 bits; `EF BE AD DE` and `43 69 61 6F 00`. On PRG32: the lowest byte of the counter changes at every frame, the next one every 256 frames. In the last task the bytes are shifted by 8, 16 and 24 places and then joined.

15.11 Glossary

English	Italiano
main memory	memoria centrale
volatile / persistent	volatile / persistente
magnetic core	nucleo magnetico (di ferrite)
refresh	rinfresco (refresh)
byte order	ordinamento dei byte
access time	tempo di accesso
aligned word	parola allineata
address / data bus	bus indirizzi / bus dati
read / write enable	abilitazione lettura / scrittura
memory-mapped I/O	I/O mappato in memoria

15.12 Conclusions and reading

Memory is a fundamental component of a computer: a device able to keep the state of data. Different kinds of memory serve different purposes, because no technology is fast, large and cheap together. Data are stored as words of bits, each with an address; RISC-V addresses bytes and orders them little-endian. Access happens through selection (the address bus), reading or writing (the data bus) and control lines. In RISC-V, input and output devices are reached exactly like memory.

Reading. Patterson and Hennessy (2020), sections 2.3 and 5.1–5.2; Stallings (2021), the chapter on internal memory; the technical reference manual of the ESP32-C6, “System and Memory” (Espressif Systems, 2025).

CHAPTER 16

Permanent Memory

"... a datum does not exist unless it has been saved..."
— Prof. Almerico Murli, 1990s

In this chapter

By the end of this chapter you will be able to

- name the functional units of a computer and recognise them on a motherboard and on the PRG32 kit;
- distinguish ROM, PROM, EPROM, EEPROM and flash, and say what firmware is;
- explain why a cache makes a computer faster, and what a cache miss is;
- describe tracks, sectors and blocks of a magnetic disk and the components of its access time;
- explain how an optical disk stores bits, and compare CD and DVD;
- place flash, solid-state disks, hard disks and tapes in a multi-level storage system.

16.1 Warm-up on the machine

```
.section .rodata            # part of the cartridge file
title: .asciz "HIGH SCORE"  # stored in flash: it survives

.section .data              # copied into RAM at start
best:  .word 0              # lives in RAM: it is lost

game_update:
    ...
    la   t0, best
    lw   t1, 0(t0)
    ble  a0, t1, 1f         # new score not better: skip
    sw   a0, 0(t0)          # remember the best score ... in RAM
1:  ...
```

Listing 16.1. Switch the kit off and on again.

A game keeps the best score in a variable (Listing 16.1). After the kit has been switched off and on again the title is still there, but the best score is 0. The reason is in the two section directives. The string is in `.rodata`, which is part of the cartridge file, and the cartridge is a file in *flash* memory, which is non-volatile. The variable is in `.data`, which is copied into RAM at the start and lives in SRAM, which is volatile.

To keep a score for ever, it must be written to a permanent memory. Which ones exist? (On PRG32, persistent scores are a service of the Cartridge Store; the point here is where each section of a program lives.)

16.2 The components of a computer

16.2.1 Functional units

Processor.
Executes the instructions. On the kit: the RISC-V core of the ESP32-C6.

Main memory.
Holds the running programs and their data. On the kit: 512 KB of SRAM inside the chip.

Secondary memory.
Keeps programs and data when the power is off. On the kit: the flash chip that holds the firmware and the cartridges.

Input/output devices.
Video card and monitor, keyboard, mouse, printer, storage. On the kit: display, joystick, amplifier.

Communication lines.
The buses: data and address lines shared by all the units.

A personal computer spreads these units over a motherboard; a microcontroller puts almost all of them in one package.

16.2.2 The motherboard

A *motherboard* (Figure 16.1) carries the sockets for one or more processors and, close to them, the slots for the memory modules; a *chipset* that connects processor, memory and peripherals; expansion buses, today PCI Express slots for video cards and other boards; connections for storage (Serial ATA, M.2) and for the outside world (USB, Ethernet, audio); and a flash chip with the firmware that starts the machine. From an architectural point of view a board of 2002 and one of today have much in common.

16.2.3 Firmware: the BIOS and its heirs

Definition 16.1 (Firmware)

The *firmware* is the basic software of a machine, kept in a non-volatile memory and used by the processor from the moment in which it is switched on.

In a personal computer it was called the BIOS: a program in a non-volatile chip, specific to a given motherboard. Its instructions and data are divided into four parts: the *POST*



Figure 16.1. A dual-processor workstation motherboard.

(power-on self test), which checks that the devices work; the *SETUP*, the configuration of the devices; the *BOOT*, which starts the operating system; and the *DRIVERS*, basic low-level functions for the keyboard, the disks and other devices. Today's PCs use UEFI firmware, which does the same jobs.

Back to the Machine

On the ESP32-C6 a boot program in ROM starts a bootloader in flash, which starts the PRG32 firmware, which loads the cartridge chosen from its *Setup* menu. Each stage finds, checks and starts the next: a chain that goes from the first instruction after reset, through layers of permanent memory, to the first instruction of the reader's `init`.

16.3 Non-volatile memories

16.3.1 From ROM to flash

Semiconductor memories that keep their content without power have evolved towards an ever easier way of writing them (Table 16.1). Reading them takes a time of the order of nanoseconds.

The acronyms are *read-only memory*, *programmable* ROM, *erasable* PROM and *electrically*

Table 16.1. From ROM to flash.

Memory	Written by	Erased	Typical use
ROM	the manufacturer, once and for all	never	start-up code, old game cartridges
PROM	the user, once	never	small production runs
EPROM	the user, many times	all at once, with ultraviolet light through a quartz window (about 15 minutes)	development, BIOS of early PCs
EEPROM	the user, in place	electrically, one byte at a time	configuration data
Flash	the user, in place	electrically, in blocks	firmware, USB keys, memory cards, solid-state disks

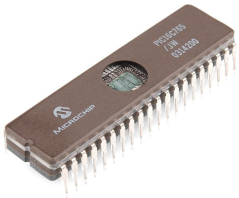
erasable PROM. A ROM is, from the logical point of view, a combinational machine: a table that associates a fixed word with each address, as the tables `dx` and `dy` of Verm1 do.

16.3.2 Flash memory

Flash is the most recent kind of EEPROM: it can be reprogrammed in place, without being removed from the board, and it is today the cheapest semiconductor memory. It has three peculiarities.

- It is erased by *blocks* of bytes, not one byte at a time. Erasing sets all the bits of a block to 1, and writing can only turn 1s into 0s.
- Each block survives a limited number of erase cycles, so that the controller spreads the writes evenly over the blocks.
- Reading is fast; writing and, above all, erasing are slow.

The flash of the kit holds the firmware and the cartridges: uploading a cartridge erases and rewrites some of its blocks. And this is why the best score of the warm-up is not simply written back into the cartridge at every frame: flash is not a place for variables.



(a) An EPROM with its quartz window



(b) A 6 TB hard disk drive



(c) A solid-state disk

Figure 16.2. Three permanent memories.

16.4 The cache

16.4.1 Why a cache?

During the clock cycles needed to read or to write the memory the processor waits; the slower the memory, the more cycles are lost. Why not build memories as fast as the registers of the processor? Because they would have to be on the same chip, since a connection through a bus is slower; and then they could not be very large, or the chip would grow in size and in cost. The compromise is a small, fast memory on the chip of the processor, and one or more larger and slower memories reached through the bus.

16.4.2 How a cache works

Definition 16.2 (Cache)

A *cache* (from the French *cacher*, to hide) is a memory of small capacity but fast, placed between the processor and the main memory, which keeps a copy of part of the content of the latter.

The processor looks in the cache first (Figure 16.3). There is a *hit* if the content of the requested address is there, and it arrives at once. There is a *miss* if it is not, and then it must be copied from the RAM into the cache; a fixed number of neighbouring locations is copied with it. The program does not stop and does not notice: the access is simply slower.

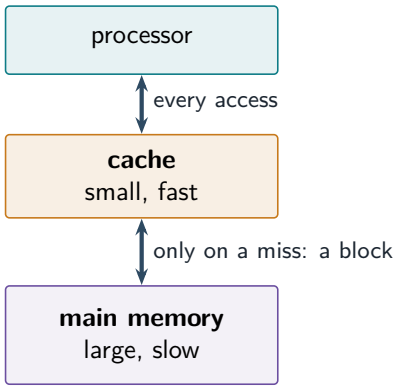


Figure 16.3. A cache between the processor and the main memory.

Why copy the neighbours? Because of an empirical property of programs.

The principle of locality

A program that has just used an address will probably use it again soon (*temporal locality*) and will probably use its neighbours (*spatial locality*). Loops and arrays are the reason.

The more hits, the shorter the average access time. If a fraction h of the accesses are hits, the cache answers in a time t_c and a miss costs in addition the time t_m of the memory, the average is

$$t = h\,t_c + (1-h)\,(t_c + t_m).$$

With $t_c = 1$ ns, $t_m = 50$ ns and 95% of hits, $t = 0.95 \times 1 + 0.05 \times 51 = 3.5$ ns: the system behaves almost as if the whole memory were as fast as the cache.

16.4.3 Cache levels and typical machines

Real processors have several levels of cache, all built with SRAM (Table 16.2). The first level, internal, keeps a continuous flow inside the processor; the second limits the slowdown due to the main memory; the third, of 8 to 32 MB and more, is shared by the cores of the same processor.

Table 16.2. Cache levels on three machines.

	PC of 2002	**PC of 2020**	**ESP32-C6 (the kit)**
Processor	1 core	4 cores	1 RISC-V core, 160 MHz
Cache L1	64 KB	64 KB per core	none for the SRAM
Cache L2	512 KB	512 KB per core	none
Cache L3	none	8 MB, shared by the cores	none
Main memory	256 MB	16 GB	512 KB of SRAM
Storage	40 GB hard disk	1 TB + 1 TB, mirrored	a few MB of flash, read through a small cache

On the ESP32-C6 the SRAM is already as fast as the core needs; only the external flash sits behind a cache.

16.5 Secondary memory

16.5.1 Characteristics

The *secondary memory*, or *mass storage*, keeps all the programs and data of a computer, whereas the main memory holds the programs being executed and the data that they need. Compared with the main memory it is

- non-volatile: the data are not lost when the computer is switched off;
- of large capacity: several orders of magnitude more;
- of low cost per bit: several orders of magnitude less;
- slower: by some orders of magnitude.

Its information is organised in *blocks*, usually of a kilobyte or more, not in single bytes. Some media allow *direct* access to any block; others only *sequential* access.

16.5.2 Magnetic disks: tracks, sectors, blocks

A magnetic disk (Figure 16.2b) is made of platters coated with a magnetisable material and kept spinning at constant speed, and of heads that read and write the information. The surface is divided into concentric *tracks* and into *sectors*, slices of the disk (Figure 16.4). A sector cuts each track into blocks: the block is the smallest addressable unit, identified by the number of its track and of its sector. *Formatting* a disk means dividing its surface into tracks and sectors. Access is direct: the head can be positioned on any block without passing over all the previous ones. Several sectors form a *cluster*; the tracks at the same radius on all the platters form a *cylinder*.

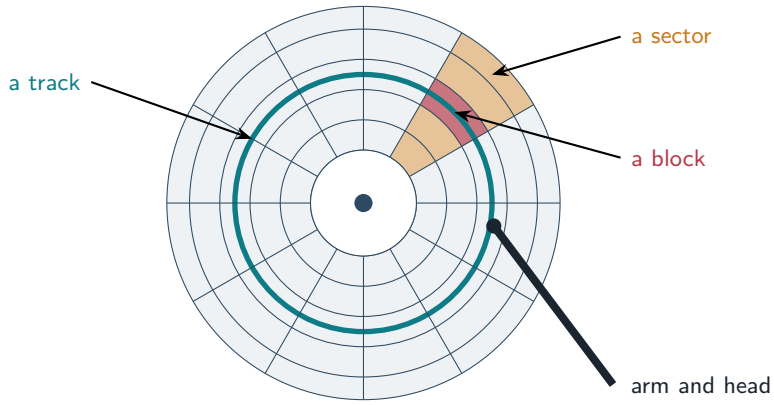


Figure 16.4. The surface of a magnetic disk.

16.5.3 Access time of a magnetic disk

To read or to write a block the head must be brought to the right address; during the transfer the head is still and the disk moves. The access time is the sum of three components.

Seek time.
The time to move the head radially to the right cylinder, and so to the right track.

Latency.
The time to wait for the desired sector to pass under the head; on average, half a revolution.

Transfer time.
The time for the block to pass under the head.

A disk that turns at 7 200 revolutions per minute takes $60/7200 = 8.33$ ms per revolution, and its average latency is about 4.2 ms. Hard disks are sealed against dust, have capacities of terabytes and access times of milliseconds. A millisecond is a million nanoseconds: during one access to a disk a processor at 160 MHz could execute more than a hundred thousand instructions.

16.5.4 Hard disks and solid-state disks

A *hard disk drive* (HDD) has magnetic platters and moving heads; its access time, of milliseconds, is dominated by mechanics; it has the lowest cost per terabyte and is still the choice for large archives and for backups. A *solid-state disk* (SSD) is flash memory with a controller, and has no moving parts; its access time is of tens of microseconds; it is connected through SATA or, faster, through NVMe on PCI Express. It is the standard storage of laptops and telephones, and is the same technology as the flash chip of the kit.

16.6 Optical disks

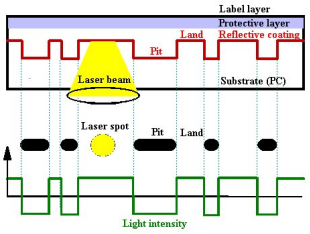
16.6.1 Bits read with light

An optical disk is a rigid disk on which a laser beam of a few milliwatts is projected; the reflection changes between the *pits* and the surface, the *land* (Figure 16.5a).

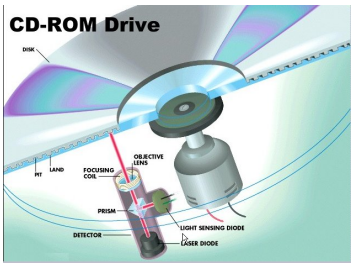
Watch Out

0 and 1 are *not* pit and land. A 1 is a *change*, from pit to land or from land to pit; a 0 is no change.

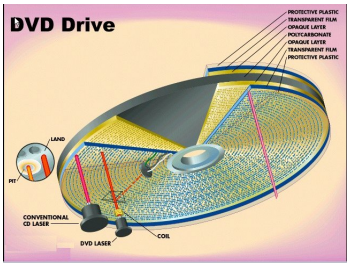
Encoding a 1 as a transition is an idea used in many serial links as well, because it lets the receiver recover the rhythm of the bits from the signal itself. Optical disks are usually written once, because the physical changes are irreversible: they are used to distribute programs and as archives that must not be modified. In a CD-Recordable, two layers of polycarbonate enclose a thin reflective sheet covered by a translucent dye, and recording burns points in the dye; rewritable disks use a material that can change back.



(a) Pits and lands



(b) Inside a CD-ROM drive



(c) Inside a DVD drive

Figure 16.5. Optical storage.

16.6.2 The compact disc

On a compact disc the information is stored on a single spiral track, with constant recording density; the distance between two successive turns of the spiral, the *track pitch*, is 1.6 micrometres. Two ways of turning the disk are possible. With *constant linear velocity* (CLV) the disk turns faster when the head reads near the centre, from 200 to 500 revolutions per minute, so that the track passes under the head always at the same speed (1.2 m/s at the basic speed, set by the bit rate needed to play music). With *constant angular velocity* (CAV) the revolutions are constant and so is the number of bits per sector, with the result that the outer surface is under-used.

16.6.3 DVD and CD compared

The DVD (*digital versatile disk*) has smaller pits and a tighter spiral, and up to two layers and two sides (Table 16.3).

16.7 Storage on several levels

A modern computing system has several levels of permanent memory (Figure 16.6), often managed automatically: the data that are used often migrate towards the fast levels, and the rest sinks towards the cheap ones. It is the idea of the cache, applied to storage; and from a chip of a few megabytes on the kit to rooms of petabytes (Figure 16.7) the hierarchy is the same at every scale.

Table 16.3. DVD and CD.

	DVD	CD
Disk diameter	120 mm	120 mm
Disk thickness	1.2 mm (0.6 mm × 2)	1.2 mm
Surfaces / layers	1 or 2 / 1 or 2	1 / 1
Minimum pit size	0.4 μm	0.83 μm
Track pitch	0.74 μm	1.6 μm
Average transfer rate	4.7 MB/s	0.15 MB/s
Capacity, 1 layer, 1 side	about 5 GB	about 0.68 GB
Capacity, 2 layers, 2 sides	17 GB	–



Figure 16.6. A multi-level permanent memory.

16.8 The program of this chapter

The listings of this chapter are parts of one program, `code/l16.S` in the repository of the course, printed in full in Listing 16.2. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 16.8 is the frame that the test wrote.

On the screen.
The title HIGH SCORE and two bars: the score of this run, which grows while A is held and falls back to zero when A is released, and the best score, which never goes down. Restart the cartridge: the title is still there, the best score is zero again.

What to press.
Hold A, release it, then restart the cartridge.

```
1 # ARC12 Computer Architecture - Lecture 16: Permanent Memory
2 # l16.S: what stays in the file and what lives in RAM
3 #
4 # On the screen: the title HIGH SCORE and two bars: the score of this run,
      which grows
```



Figure 16.7. Disk and tape libraries in a data centre.



Figure 16.8. `l16.S` on PRG32-QT.

```
# while A is held and falls back to zero when A is released, and the best score, which
# never goes down. Restart the cartridge: the title is still there, the best score is
# zero again.
# Build: python3 -m prg32 cartridge build l16.S --portable \
#            --entry-prefix l16 --name l16 --out l16.prg32
.option norelax

.equ BLACK,  0x0000
.equ WHITE,  0xffff
.equ GREEN,  0x07e0
.equ YELLOW, 0xffe0

.section .text
.global l16_init
.global l16_update
.global l16_draw

l16_init:
    ret

l16_update:
game_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read
    andi t2, a0, 0x10        # button A
    la   t0, score
    lw   a0, 0(t0)
    addi a0, a0, 1           # A held: one more point per frame
    bnez t2, 2f
    li   a0, 0               # A released: the run is over
2:  sw   a0, 0(t0)

    la   t0, best
    lw   t1, 0(t0)
    ble  a0, t1, 1f          # new score not better: skip
    sw   a0, 0(t0)           # remember the best score ... in RAM
1:  lw   ra, 12(sp)
    addi sp, sp, 16
```

```
44      ret
45
46  l16_draw:
47      addi sp, sp, -16
48      sw   ra, 12(sp)
49      li   a0, BLACK
50      call prg32_gfx_clear
51      li   a0, 20
52      li   a1, 20
53      la   a2, title          # read from .rodata: it is always there
54      li   a3, WHITE
55      li   a4, BLACK
56      call prg32_gfx_text8
57      la   t0, score
58      lw   a2, 0(t0)
59      andi a2, a2, 255        # width = score, at most 255 pixels
60      li   a0, 20
61      li   a1, 60
62      li   a3, 12
63      li   a4, GREEN
64      call prg32_gfx_rect
65      la   t0, best
66      lw   a2, 0(t0)
67      andi a2, a2, 255
68      li   a0, 20
69      li   a1, 100
70      li   a3, 12
71      li   a4, YELLOW
72      call prg32_gfx_rect
73      lw   ra, 12(sp)
74      addi sp, sp, 16
75      ret
76
77  .section .rodata            # part of the cartridge file
78  title: .asciz "HIGH SCORE"  # stored in flash: it survives
79
80  .section .data              # copied into RAM at start
81  best:  .word 0              # lives in RAM: it is lost
82  score: .word 0
```

Listing 16.2. The complete program of the chapter, `code/l16.S`.

To run it on PRG32-QT the commands are those of Listing 16.3; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l16.S --portable \
    --entry-prefix l16 --name l16 --out l16.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l16.prg32 \
    --url http://localhost:8080 --slot cart0
```

```
# 4. look, press, compare with the previous slide
```

Listing 16.3. Building `116.S` and running it on PRG32-QT.

16.9 Check your understanding

Q16.1 Which memory can be erased electrically, in blocks, without removing it from the board?

A. ROM
B. PROM
C. EPROM
D. Flash

Q16.2 What happens on a cache miss?

A. The program stops with an error
B. The datum is fetched from main memory and copied into the cache, with its neighbours
C. The cache is erased
D. The processor skips the instruction

Q16.3 Which component of the access time of a hard disk depends on the rotation speed?

A. Seek time
B. Latency
C. Formatting time
D. Boot time

Q16.4 On an optical disk, a 1 is encoded as:

A. a pit
B. a land
C. a transition between pit and land
D. a double pit

16.10 Exercises

Exercise 16.1 (paper and PRG32)

1. A disk turns at 5 400 revolutions per minute: what is its average latency?
2. A cache answers in 2 ns and the memory in 80 ns: what is the average access time with 90% of hits? And with 99%?
3. Say where each of these lives on the PRG32 kit: the boot code, the firmware, a cartridge, the variable `counter`, the register `t0`.
4. Find the specifications of the storage of your laptop or telephone: technology, capacity, interface.

5. On PRG32, time two loops that sum a large array: one in order, one that jumps by a large step. Is there a difference, and why might there not be on this chip?

Expected result. Latency: half of $60/5400 = 11.1$ ms, that is, about 5.6 ms. Access time: $0.9 \times 2 + 0.1 \times 82 = 10$ ns, and $0.99 \times 2 + 0.01 \times 82 = 2.8$ ns. The five things live in ROM, in flash, in flash (or in RAM once loaded), in SRAM, and in a register of flip-flops inside the processor. In the last task the SRAM of the ESP32-C6 has no cache in front of it, so that the pattern of access matters little; on a laptop the difference is large.

16.11 Glossary

English	**Italiano**
motherboard	scheda madre
firmware	firmware
non-volatile	non volatile
cache hit / miss	successo / fallimento di cache
mass storage	memoria di massa
track / sector / block	traccia / settore / blocco
seek time	tempo di posizionamento
latency	tempo di latenza
solid-state disk	disco a stato solido
tape library	libreria a nastri

16.12 Conclusions and reading

Non-volatile memories usually hold the basic software of a machine, the firmware. The cache improves performance by keeping a copy of part of the main memory close to the processor, and the same idea, a small fast level in front of a large slow one, is used again in storage systems. Permanent memories can be semiconductor (flash), magnetic or optical, and a modern system has several levels of them, often managed automatically.

Reading. Patterson and Hennessy (2020), sections 5.1–5.4; Stallings (2021), the chapters on cache and on external memory; Tanenbaum and Austin (2013), chapter 2.

The third block is complete. The next chapter uses it in the third part of Verm1: a body, rules and states.

CHAPTER 17

Verm1, Part 3: A Body, Rules and States

"Show me your tables, and I won't usually need your flowcharts; they'll be obvious."
— Fred Brooks, *The Mythical Man-Month*

In this chapter

By the end of this chapter you will have

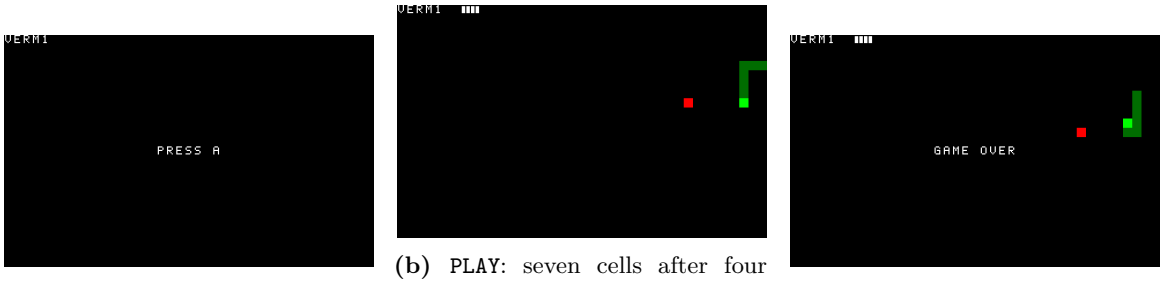
- a worm with a body that follows the head and grows by one cell at every meal;
- a game that can be lost: the worm must not bite itself;
- three screens, a title, the game and a game over, linked by button **A**;
- food that never appears on the worm;
- a best score, kept as long as the cartridge stays in memory.

In this third tutorial memory enters the game: a ring of bytes, a map of the world, a machine with three states. With "data structures" in the place of "tables", the sentence of Brooks quoted above is the motto of the chapter: once the data are designed well, the code is a transcription. Table 17.1 lists what is taken from Chapters 13 to 16. The frame is the clock of the machine: `update` computes the next state from the present state and from the inputs. The reference file is `verm1_part3.S` (Appendix D).

Table 17.1. What Part 3 takes from the chapters on state and memory.

From	Idea	Where it appears
Chapter 13	a finite-state machine: states, inputs, transitions	`TITLE`, `PLAY`, `OVER`
Chapter 13	an edge is a change, not a level	**A** pressed now and not one frame ago
Chapter 14	a register keeps a value for the next clock	`prev_btn`: the buttons of the last frame
Chapter 15	memory is an array of bytes: address = base + index	the body, the map of the world
Chapter 16	what stays in the file and what lives in RAM	`.rodata` and `.data`; the best score

Start from a Part 2 that runs; if yours does not, copy `verm1_part2.S`. The working loop is that of Listing 7.1. In this part the file changes a great deal: build after every *Run it*, not only at the end.



(a) `TITLE` (b) `PLAY`: seven cells after four meals (c) `OVER`

Figure 17.1. Where this chapter is going: three screens, one program.

17.1 The design, before the code

Two data structures and one diagram are designed first.

17.1.1 The body is a ring of 256 entries

The body is kept in two arrays of bytes, `body_x` and `body_y`: entry k holds the cell of one segment. Nothing is ever shifted: a step writes one new entry and moves an index (Figure 17.2). Two numbers describe the worm: `head_i`, the entry of the head, and `length`, the number of entries in use. The tail is the entry

$$\texttt{head_i} - \texttt{length} + 1.$$

After entry 255 comes entry 0, and the wrap is one instruction: the index AND 255. The structure is a *ring*, or circular buffer: the structure behind every first-in, first-out queue.

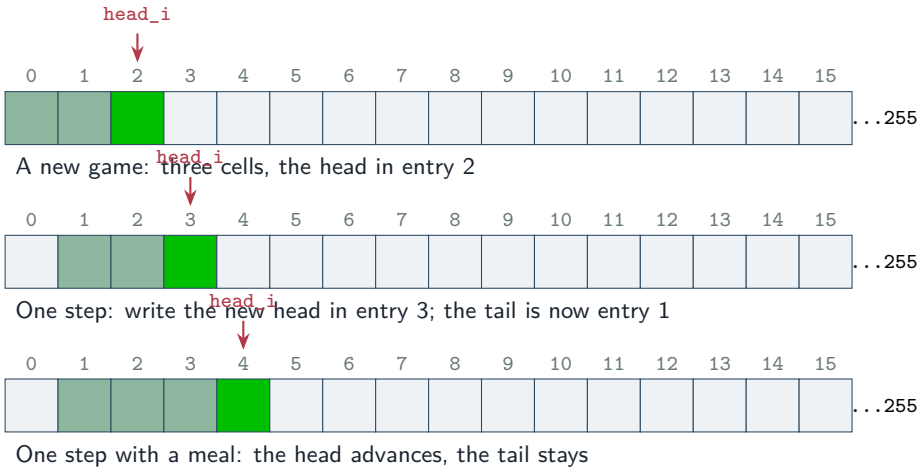


Figure 17.2. The first sixteen entries of the ring in three successive moments.

Moving a worm of 200 cells therefore costs the same as moving one of 3. The size 256 is chosen because it is a power of two, so that the wrap is a mask.

17.1.2 The world is a map of 960 bytes

Does the new head touch the body? Looking at every segment would cost up to 255 comparisons. Instead the program keeps a *map*: one byte per cell, 1 if a segment is there

and 0 if not. The cell in column c of row r is the byte number

$$r \times 40 + c, \qquad \text{from 0 to 959,}$$

and its address is `grid` $+ r \times 40 + c$: a two-dimensional world in a one-dimensional memory, the address computation of Chapter 15. A step sets one byte, that of the new head, and clears one, that of the old tail. The collision test is a single load, whatever the length of the worm.

It is the classic exchange of memory for time. The map and the ring say the same thing in two ways, and the program must keep them consistent: every write to one is paired with a write to the other.

17.1.3 The game is a machine with three states

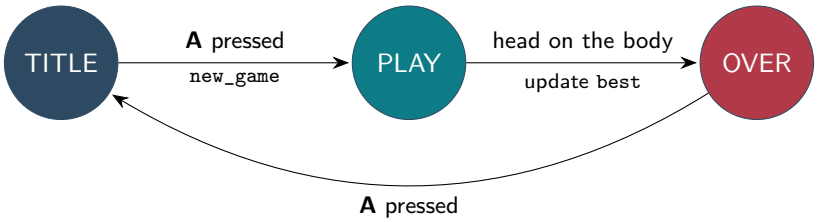


Figure 17.3. The state diagram of Verm1. In every other case the state does not change.

The state is one word in memory (Figure 17.3). `update` looks at the state first and then at the inputs. And "**A** pressed" means: down now, and up one frame ago. Holding **A** must not skip the title: what is wanted is an edge, not a level.

17.2 Step 1: the memory of the game

Replace the constants, the `.rodata` and the `.data` of Part 2 with Listings 17.1, 17.2 and 17.3.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
.equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
.equ DIR_DOWN,  1
.equ DIR_LEFT,  2
.equ DIR_UP,    3
.equ TITLE, 0               # the three states of the game
```

```
.equ PLAY,  1
.equ OVER,  2
```

Listing 17.1. Step 1 **(changed)**: constants.

There is a darker green, `0x03e0`, for the body; `BTN_A` is bit 4 of the word of buttons; the states are numbers, 0, 1 and 2.

```
.section .rodata                # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
```

Listing 17.2. Step 1 **(changed)**: `.rodata`, what stays in the file.

Three strings and the two tables of Part 2: read-only data, to which the program never writes, and which on the board could stay in flash.

```
.section .data                  # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0              # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0              # lost when the power goes
food_x:    .word 0
food_y:    .word 0
head_i:    .word 0              # ring index of the head
length:    .word 0              # number of cells of the worm
body_x:    .space 256           # the ring: one byte per segment
body_y:    .space 256
grid:      .space COLS*ROWS     # one byte per cell: 1 = worm
```

Listing 17.3. Step 1 **(changed)**: `.data`, what lives in RAM.

`head_x` and `head_y` are gone: the head is in the ring. The directive `.space` reserves bytes without naming each one: $256 + 256 + 960$ bytes, the worm and its world. Everything here is lost when the cartridge is unloaded (Chapter 16): `best` survives a game, not a power-off. A best score that survives would need permanent memory, which a portable cartridge does not have: an honest limit.

17.3 Step 2: three states

The machine is built first, with functions that do nothing yet.

17.3.1 `init`: start on the title

Replace `verm1_init` with Listing 17.4, and delete from the file `verm1_update`, `step` and `verm1_draw` of Part 2: the next listings replace them. Keep `place_food`, `eat_sound` and `draw_cell`.

```
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    ret
```

Listing 17.4. Step 2 **(changed)**: `verm1_init`.

`init` only chooses the first state; the game itself is prepared by `new_game`.

17.3.2 `update`: buttons, and their edges

```
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
```

Listing 17.5. Step 2 **(changed)**: `verm1_update`, first half.

In Listing 17.5 `s0` holds the buttons now and `prev_btn` the buttons one frame ago. The variable `prev_btn` is a register in the sense of Chapter 14: it delays its input by one clock, and here the clock is the frame. The expression "now AND NOT before" is 1 only where a press has begun:

$$\texttt{s1} = \texttt{s0} \cdot \overline{\texttt{prev_btn}}.$$

It is the edge detector of Chapter 13: a flip-flop and an AND gate with one inverted input. `s0` and `s1` must survive the calls, and are saved in the frame.

17.3.3 `update`: one branch per state

```
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
```

```
    j    u_done
u_start:
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.6. Step 2 **(changed)**: `verm1_update`, second half.

Listing 17.6 is the transition table, line by line. The state is read first: it decides who looks at the inputs. `TITLE` and `OVER` react only to a new press of **A**; `PLAY` passes the buttons that are held to `play_update`. The transition from `OVER` to `TITLE` only stores the new state; that from `TITLE` to `PLAY` calls `new_game`, which stores `PLAY` itself.

17.3.4 `draw`: what to show in each state

```
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
    call draw_text
    j    2f
1:  call draw_score             # PLAY and OVER: the whole scene
    call draw_worm
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
```

Listing 17.7. Step 2 **(changed)**: `verm1_draw`, first half.

The title line is always drawn (Listing 17.7). In `TITLE` only the invitation `PRESS A` is added, at $x = 132$, which centres 7 characters of 8 pixels; in `PLAY` and `OVER` the score, the worm

and the food. The output depends only on the state and on the memory: `draw` changes nothing. It is a Moore machine.

```
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
    la   a2, game_over
    call draw_text
2:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret

draw_text:                          # a0 = x, a1 = y, a2 = address of the string
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.8. Step 2 **(changed)**: the end of `verm1_draw`, and `draw_text`.

`OVER` adds two words on top of the scene (Listing 17.8). The helper `draw_text` writes white on black: it fills `a3` and `a4` and lets `a0` to `a2` pass through, removing two lines from every call that writes text.

17.3.5 Four functions that do nothing

```
new_game:                           # temporary: only change the state
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    ret

play_update:                        # temporary: nothing moves yet
    ret

draw_score:                         # temporary
    ret

draw_worm:                          # temporary
    ret
```

Listing 17.9. Step 2: temporary stubs.

The stubs of Listing 17.9 make the names exist, so that the program builds; `new_game` only changes the state. Steps 3 and 4 replace them one at a time. It is the method of Chapter 2: start with returns that do nothing, then add one thing at a time.

Run it: a machine with nothing inside

1. Build and run: the title line, and `PRESS A` in the middle of the screen.
2. Press **A**: the invitation disappears. A red cell is in the top-left corner: the food, at column 0 of row 0, because `food_x` and `food_y` start at zero and the stub of `new_game` does not call `place_food`.
3. Nothing moves and the game cannot end: `PLAY` has no rules yet.
4. In the debugger, find the variable `state` and watch it change from 0 to 1 when you press **A**.

17.4 Step 3: a worm in memory

17.4.1 `new_game`: empty the map, lay the worm

Replace the stub of `new_game` with Listings 17.10 and 17.11.

```
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
```

Listing 17.10. Step 3 **(new)**: `new_game`, first half.

The 960 bytes of the map are set to zero, one `sb` at a time. Then three segments are laid in columns 3, 4 and 5 of row 12, and each is written twice: in the ring and in the map. The expression `grid+12*COLS` is computed by the assembler: it is the address of the first byte of row 12. `sb` stores one byte; the index is added to the base address with no shift, because each entry is one byte.

```
    la   t0, head_i
    li   t1, 2
```

```
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.11. Step 3 **(new)**: `new_game`, second half.

`head_i` = 2 and `length` = 3 describe the worm; direction, timer and score start from zero. The state is stored last: the transition from `TITLE` to `PLAY` is complete only when everything is ready.

17.4.2 `draw_worm`: walk the ring backwards

```
draw_worm:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: segment 0 is the head
1:  la   t0, length
    lw   t1, 0(t0)
    bge  s0, t1, 3f
    la   t0, head_i
    lw   t1, 0(t0)
    sub  t1, t1, s0
    andi t1, t1, 255            # ring index of segment i
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, BODY
    bnez s0, 2f
    li   a2, GREEN              # the head is brighter
2:  call draw_cell
    addi s0, s0, 1
    j    1b
3:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
```

```
28      ret
```

Listing 17.12. Step 3 **(new)**: `draw_worm`.

Segment i is the entry (`head_i` $- i$) AND 255 (Listing 17.12). `lbu` loads one byte as an unsigned number, and not `lb`: a column is a number from 0 to 39, never negative. Both would work here, since with `lb` only a byte of 128 or more would become negative (Chapter 5); but `lbu` says what is meant. Segment 0 is the head, in a brighter green.

17.4.3 `draw_score`

```
draw_score:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.13. Step 3 **(new)**: `draw_score`.

This is the loop that was inside `verm1_draw` in Part 2, now a function (Listing 17.13). It uses `s0`, so it saves it and restores it. One stub is left (Listing 17.14): the worm exists, and stands still.

```
play_update:                    # temporary: nothing moves yet
    ret
```

Listing 17.14. Step 3: the last stub.

Run it: a worm that stands still

1. Build and run, press **A**: a worm of three cells in row 12, columns 3 to 5, the head, brighter, on the right.
2. The food is now at a random cell: `new_game` calls `place_food`.
3. In the debugger, break at the start of `draw_worm` and step until `la t0, body_x` has run: `t0` holds the address of `body_x`. Type it in the *Memory* field: the first

bytes are `03 04 05`.

4. Find `grid` + 12 × 40 + 3: three bytes at `01` in a sea of zeros. (`grid` is 512 bytes after `body_x`, and row 12 starts 480 bytes after `grid`.)

17.5 Step 4: rules

One step of the worm has three movements: decide, look, write.

17.5.1 `play_update`: the update of Part 2

Replace the last stub with Listings 17.15 and 17.16. They are the body of the old `verm1_update` under a new name; the buttons arrive in `a0`, so that there is no call to read them.

```
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
```

Listing 17.15. Step 4 **(changed)**: `play_update`, the steering.

```
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
```

```
    addi sp, sp, 16
    ret
```

Listing 17.16. Step 4 **(changed)**: `play_update`, the timer.

17.5.2 `step`: three registers to keep

```
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
```

Listing 17.17. Step 4 **(changed)**: `step`, the beginning.

`step` needs three values that must survive a call to `place_food`: the cell of the new head, in `s0` and `s1`, and a flag in `s2` that is zero when the worm eats. They are kept in `s` registers, and the frame has room for four words: `ra` and the three saved registers (Listing 17.17). The table lookup is that of Part 2.

17.5.3 `step`: where will the head be?

```
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
    add  s0, s0, t2
    bgez s0, 1f
    li   s0, COLS-1             # off the left edge: wrap
1:  li   t4, COLS
    blt  s0, t4, 2f
    li   s0, 0
2:  la   t0, body_y
    add  t0, t0, t5
    lbu  s1, 0(t0)
    add  s1, s1, t3
    bgez s1, 3f
    li   s1, ROWS-1
3:  li   t4, ROWS
    blt  s1, t4, 4f
```

```
    li   s1, 0
4:
```

Listing 17.18. Step 4 **(changed)**: `step`, the new cell.

The present head is the entry `head_i` of the ring: one byte each for column and row, loaded with `lbu`. `dx` and `dy` are added and the result is wrapped as in Part 2 (Listing 17.18). Nothing is written yet: first the program must look.

17.5.4 `step`: eat, or move the tail

```
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, s0
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, s1
    or   s2, t1, t2             # s2 = 0 if the new head is on the food
    la   t0, length
    lw   t1, 0(t0)
    bnez s2, 5f                 # not eating: the tail moves on
    li   t2, 255
    bge  t1, t2, 5f             # the ring is full: no more growth
    addi t1, t1, 1
    sw   t1, 0(t0)              # eating: one cell longer, the tail stays
    j    6f
5:  la   t0, head_i
    lw   t2, 0(t0)
    sub  t2, t2, t1
    addi t2, t2, 1
    andi t2, t2, 255            # ring index of the tail
    la   t0, body_x
    add  t0, t0, t2
    lbu  t3, 0(t0)
    la   t0, body_y
    add  t0, t0, t2
    lbu  t4, 0(t0)
```

Listing 17.19. Step 4 **(changed)**: `step`, the tail.

The comparator of Part 2 leaves its result in `s2` (Listing 17.19). If the worm eats, the length grows by one and the tail stays where it is. If it does not, the tail is found at the entry `head_i` − `length` + 1, and its column and row are put in `t3` and `t4`. At 255 cells the worm stops growing.

Concept

Growing and not moving the tail are the same thing. It is the whole trick of Snake.

17.5.5 `step`: free the tail, test the head

```
    li   t5, COLS
```

```
    mul  t4, t4, t5
    add  t4, t4, t3             # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)            # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0             # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                 # a free cell: go on
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)              # the worm bit itself: PLAY --> OVER
    la   t0, score
    lw   t1, 0(t0)
    la   t0, best
    lw   t2, 0(t0)
    ble  t1, t2, 9f
    sw   t1, 0(t0)              # a new best score
    j    9f
```

Listing 17.20. Step 4 **(new)**: `step`, the collision.

The number of a byte of the map is $row \times 40 + column$: one `mul` and one `add` (Listing 17.20); `mul` belongs to the M extension of RISC-V, which the ESP32-C6 has. The byte of the tail is cleared *first*, and then the byte of the head is tested. The order matters: in this way the head may enter the cell that the tail has just left, and the worm can chase its own tail, as in the original game. If the cell is occupied the state becomes `OVER`, and `best` is updated.

17.5.6 `step`: write the new head

```
7:  li   t1, 1
    sb   t1, 0(t0)              # occupy the new cell
    la   t0, head_i
    lw   t1, 0(t0)
    addi t1, t1, 1
    andi t1, t1, 255            # the ring wraps after entry 255
    sw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    sb   s0, 0(t0)              # write the new head into the ring
    la   t0, body_y
    add  t0, t0, t1
    sb   s1, 0(t0)
    bnez s2, 9f
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    call eat_sound
9:  lw   s2, 0(sp)
    lw   s1, 4(sp)
```

```
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.21. Step 4 **(new)**: the end of `step`.

The cell is marked in the map, while `t0` still points to it; the index `head_i` becomes (`head_i`+ 1) AND 255, and the ring closes; column and row are written in the new entry (Listing 17.21). If `s2` is zero there is a point, new food and a sound. After a collision the function jumps straight to its last label: nothing is written, and the screen shows the worm as it was before the fatal step.

17.5.7 `place_food`: not on the worm

```
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
1:  li   a0, 0
    li   a1, COLS-1
    call prg32_random_number
    mv   s0, a0                 # s0 = a column
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row
    li   t1, COLS
    mul  t2, a0, t1
    add  t2, t2, s0
    la   t0, grid
    add  t0, t0, t2
    lbu  t3, 0(t0)
    bnez t3, 1b                 # on the worm: try again
    la   t0, food_x
    sw   s0, 0(t0)
    la   t0, food_y
    sw   a0, 0(t0)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.22. Step 4 **(changed)**: `place_food`.

A cell is drawn at random, and then the map is consulted: if the cell is occupied another is drawn, until a free one comes (Listing 17.22). The column waits in `s0` during the second call. The loop ends quickly: even with 255 cells out of 960 occupied, the average is fewer than two attempts.

Run it: Verm1, Part 3

The worm follows the head and grows (Figure 17.1b); turning into the body ends the game (Figure 17.1c); **A** returns to the title, and **A** again starts a new game.

17.6 Step 5: look at memory move

Watch the ring turn

1. Run in debug mode, press **A** to start a game, then pause.
2. Press **Update** and put a breakpoint on the first instruction of `step`.
3. **Resume**: the program stops once per step of the worm, not once per frame.
4. Type the address of `body_x` in the *Memory* field: the bytes `03 04 05` of the start, then `06`, `07`, ..., one new byte per step.
5. Go down for a few steps: `body_x` repeats the same column, and `body_y`, 256 bytes further on, counts the rows.
6. Eat, and compare `head_i` and `length` before and after: both grow by one.

The bytes of the old positions are never erased: the ring keeps the whole history of the last 256 steps, and `length` says how much of it is still worm. After 256 steps the writing starts again from the first byte.

17.6.1 Catch the edge of button A

Put a breakpoint on `and s1, s0, t1` in `verm1_update`, hold **A** and press **Resume** three times. Table 17.2 shows what is read: `s1` is `0x10` in one frame only, the frame in which the button goes down. (The test of the reference file holds **A** for 28 frames and checks that exactly one transition happens.)

Table 17.2. The edge of button **A**.

Frame	A is	`s0` (now)	`prev_btn`	`s1` = `s0` AND NOT prev	Effect
n	up	`0x00`	`0x00`	`0x00`	–
$n+1$	down	`0x10`	`0x00`	`0x10`	transition
$n+2$	down	`0x10`	`0x10`	`0x00`	–
$n+3$	up	`0x00`	`0x10`	`0x00`	–

17.6.2 Three questions for the debugger

- *Where is `state`?* After `la t0, state` in `verm1_update`, `t0` holds its address. Watch `00`, `01`, `02` as you play and lose.
- *Is the map consistent with the ring?* Pause, read the head from `body_x` and `body_y`, compute *row* × 40 + *column*, and look at that byte of `grid`.
- *What did the fatal step see?* Break on the `lbu` that follows the second `mul` in `step`: `t1` is 1 exactly once, at the bite. At that moment `s0` and `s1` hold the cell that the head wanted to enter, and the screen shows that it is part of the body.

Concept

A program is right when its memory is right. Learn to read it.

17.7 Check your understanding

Q17.1 `head_i` = 1 and `length` = 4. Which entries of the ring hold the worm?

A. 1, 2, 3, 4
B. 0, 1, 2, 3
C. 254, 255, 0, 1
D. 253, 254, 255, 0

Q17.2 Which byte of `grid` describes the cell in column 7 of row 3?

A. byte 37
B. byte 127
C. byte 283
D. byte 307

Q17.3 The player holds **A** down for two seconds on the title screen. What happens?

A. the game starts and ends at once
B. the game starts once
C. nothing, until **A** is released
D. the states cycle at every frame

Q17.4 Why is the tail's byte cleared before the head's byte is tested?

A. to save an instruction
B. so that the head can enter the cell the tail leaves
C. because `sb` is faster than `lbu`
D. the order does not matter

17.8 Make it yours

Exercise 17.1 (on PRG32)

1. Walls: the worm dies when it leaves the screen instead of wrapping.
2. Start with a worm of five cells; then let the length at the start depend on the best score.
3. Poison: a second, blue cell that makes the worm one cell shorter.
4. On the game-over screen show the best score as marks on the second line.

Expected result. Each change touches one or two functions. Keep a copy of the working Part 3: Part 4 starts from it. (Hints: for the walls, replace each wrap in `step` with a jump to the code that stores `OVER`. For the poison, move the tail twice in one step, clearing both bytes of the map, and never go below length 1.)

17.9 Two functions that did not change

`eat_sound` and `draw_cell` are those of Part 2 and are kept as they are. They are reprinted here so that this chapter contains the whole program: the complete file is the sum of the listings marked new and changed, plus these two.

```
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a2, 72                 # note: C5
    li   a3, 255                # volume
    li   a4, 60                 # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.23. Unchanged: `eat_sound`.

```
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 17.24. Unchanged: `draw_cell`.

17.10 Where we are, and what comes next

Verm1 is now a game: it has a goal, a risk, a beginning and an end. A machine with three states, a ring of bytes, a map: sequential logic and memory written as a program, in about 450 lines of assembly that the reader has written and can explain. Three things are missing: the program redraws the whole screen at every frame, the score is a row of marks, and the speed never changes. The next four chapters open the machine that runs those lines: how the processor executes, calls and division, how video and sound work. Part 4 is the finished game.

Part V

The Machine

CHAPTER 18

The Processor

"One Ring to rule them all…"
— J. R. R. Tolkien, *The Lord of the Rings*, 1954–1955

In this chapter

By the end of this chapter you will be able to

- name the components of a processor: control unit, ALU, registers, links to memory and to input/output;
- distinguish internal registers from machine registers, and general registers from special ones;
- explain how a stack works and how RISC-V implements push and pop;
- compare the accumulator, stack and general-register models on the same computation;
- describe RISC-V as a load-store, general-register processor, and read the name RV32IMAC;
- map every element of the general model onto the processor of the PRG32 kit.

The reader has programmed a processor since Chapter 2. This chapter presents the general model of a processor, of which every real one is a variant, and uses RISC-V as the reference: every abstract element has a name that the reader already knows.

18.1 Warm-up on the machine

```
    la   t0, x
    lw   t1, 0(t0)       # bring x into a register
    la   t0, y
    lw   t2, 0(t0)       # bring y into a register
    add  t3, t1, t2      # compute, between registers
    la   t0, z
    sw   t3, 0(t0)       # take the result back to memory

.section .data
x: .word 5
y: .word 7
```

```
12 z: .word 0
```

Listing 18.1. One line of a high-level language: `z = x + y`.

Seven lines for one sum (Listing 18.1). Memory is touched only by `lw` and `sw`, and the arithmetic happens only between registers: it is the cycle "load, modify, store" of Chapter 2, now seen as a design choice. Other processors do the same job with three instructions, or with one. Why is RISC-V made in this way? To answer, a model of the processor is needed.

18.2 The processor

18.2.1 The computer and its processor

An electronic computer is a numeric, automatic, stored-program system built with electronic circuits (Chapter 8), with three subsystems: processor, main memory and input/output, connected by buses. The processor can execute only a fixed set of instructions, from some tens to a few hundred, more or less complex. The *machine language* is the set of the strings of bits that encode the instructions which the circuits of a processor can execute. Instructions request transfers of information, inside the computer and to or from the I/O devices, and arithmetic and logic operations. The data and the instructions of a program are stored in the main memory.

RV32I, the base of RISC-V, has about forty instructions; with the M, A and C extensions of the kit, about a hundred encodings. The Motorola 68000 had fifty-six types of instruction, with many variants of each.

18.2.2 A reference model

Figure 18.1 is the reference model of a processor. Its elements are the following.

- The *control network*, or control unit, which produces the commands for all the rest.
- The *program counter* PC, which holds the address of the next instruction, and the *decoding register* D, which holds the instruction being executed.
- The *machine registers* $R_0, \ldots, R_{n-1}$.
- The ALU, with its input registers O_1 and O_2 and its output register U.
- The link to memory: the address register MA and the buffer register MB of Chapter 15.
- The link to input/output: an input register RI and an output register RU.
- An *internal bus* that joins them all.

Although they are based on the same logical scheme, and are therefore made of rather similar components, the processors on the market can be very different from one another. Table 18.1 puts the model and RISC-V side by side. The two rows marked "absent" are deliberate simplifications of RISC-V, discussed below and in Chapter 15. The MC68000 of the previous editions of this course had all the elements of the model: eight data registers and eight address registers, a status register with condition codes, and separate circuitry for interrupts.

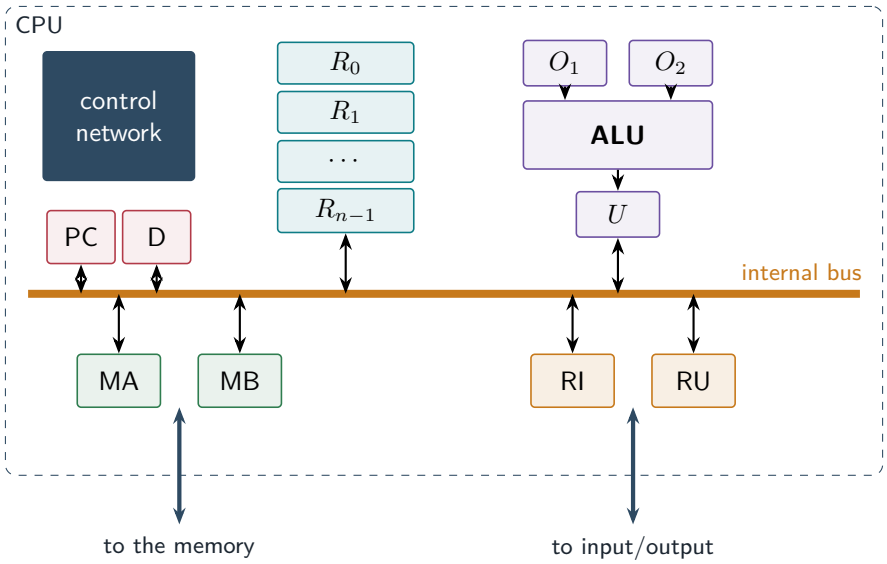


Figure 18.1. The reference model of a processor.

18.2.3 Internal registers and machine registers

Definition 18.1 (Internal and machine registers)

The *internal registers* are not directly visible to the programmer, and are necessary for the processor to work: PC, MA, MB, D, O_1, O_2, U. The *machine registers* define the programming model; they are divided into *general* registers and *special* registers.

A processor handles three kinds of information: instructions, memory addresses and data. The size of MA and of the PC depends on the size of the addresses; the size of the registers of the ALU depends on the size of the data. In RV32 all three are 32 bits wide: one size for registers, addresses and instructions. The "32" of the name is the width of the registers: RV64 has registers and addresses of 64 bits, and the same instructions of 32 bits.

18.2.4 How real processors differ from the model

In general, real processors may have a different and usually richer set of general registers; their machine registers may be specific memory locations; the I/O section may be missing altogether and be implemented by dedicated chips; and very simple arithmetic units can be extended by dedicated chips that improve the ALU, as the floating-point coprocessors 68881 for the 68000 family and 8087 for the 8086 did.

RISC-V on the PRG32 kit has thirty-two general registers in a bank inside the processor; no I/O section, since the peripheral controllers are on the same chip and are reached as memory; optional *extensions* instead of extra chips, of which M adds multiplication and division; and no F extension, so that floating point would be done in software (Chapter 6).

18.2.5 Words or bytes: the length of machine words

In a *word processor* all the machine words have the same length, sized on the longest piece of information; this is awkward with data of intrinsically variable nature. Every RISC-V instruction is 32 bits long (16 in the compressed form). In a *character*, or byte, processor

Table 18.1. The model and RISC-V.

Element of the model	In RISC-V	Seen by the programmer?
PC, program counter	`pc`, 32 bits	indirectly: branches, jumps and `auipc` use it
D, decoding register	the instruction register	no
$R_0 \dots R_{n-1}$, machine registers	`x0`...`x31`, thirty-two registers of 32 bits	yes: `zero`, `ra`, `sp`, `t0`, `a0`, `s0`, ...
O_1, O_2, U, registers of the ALU	the inputs and the output of the ALU	no
MA, MB, link to memory	the address and the datum of a load or store	no
RI, RU, link to input/output	absent: devices are memory-mapped	–
status register with flags	absent: no condition codes	–
special registers	the control and status registers (CSR)	only by privileged software

the length of machine words is variable, a multiple of a byte, which suits the treatment of strings: an Intel x86 instruction takes from 1 to 15 bytes, and an MC68000 instruction from one to five words of 16 bits. A characteristic parameter of a processor is the number of bits of its registers, the parallelism of its internal transfers: 8, 16, 32 or 64.

18.2.6 User state and supervisor state

A processor is in *supervisor state* when it executes the operating system, and may then execute every instruction of its set; it is in *user state* when it executes a user program, and may then execute everything except the *privileged instructions*.

RISC-V defines three *privilege modes*. Machine mode (M), the most privileged and the only mandatory one, runs the firmware. Supervisor mode (S) runs the kernel of an operating system such as Linux. User mode (U) runs applications. The ESP32-C6 implements M and U. The passage from one mode to another happens through traps, the subject of Chapter 19.

18.3 Registers

18.3.1 Kinds of registers

A register can be specialised, or can do several jobs. Specialised registers need specialised operation codes for the same operation on different banks; concentrating several functions in the same registers gives more internal complexity and more flexibility of use. The machine registers, different in number and often grouped in banks, can be organised as an addressable memory (a bank R, of which R_i is a register) or as autonomous registers, each with its own name. Some processors placed their general registers in the first locations of memory: the "page zero" of the 6502.

RISC-V has one bank of thirty-two registers, addressed by 5 bits inside the instruction; the names `t0`, `a0`, `sp` are only conventions.

18.3.2 The functions of registers

Transit of data.
From memory to a register, from register to register, from a register to memory: load, move, store.

Accumulation of results.
Needed for arithmetic and logic: $R := \mathrm{op}\, R$, or $R := R$ op X.

Addressing.
Finding the address of an operand: index registers, base registers.

Stack pointers.
Managing a stack implemented in memory.

Indicators, or flags.
Binary information about events: zero, sign, overflow, carry, illegal addresses or codes. Produced during the execution of an instruction, they can condition the following ones.

In RISC-V the first four functions are all served by the general registers; the fifth does not exist as a register.

18.3.3 The thirty-two registers of RISC-V

Table 18.2. The registers of RISC-V and their conventional roles.

Register	Name	Function	Register	Name	Function
`x0`	`zero`	always 0	`x8`–`x9`	`s0`–`s1`	saved
`x1`	`ra`	return address	`x10`–`x11`	`a0`–`a1`	arguments, results
`x2`	`sp`	stack pointer	`x12`–`x17`	`a2`–`a7`	arguments
`x3`	`gp`	global pointer	`x18`–`x27`	`s2`–`s11`	saved
`x4`	`tp`	thread pointer	`x28`–`x31`	`t3`–`t6`	temporaries
`x5`–`x7`	`t0`–`t2`	temporaries	`pc`	–	program counter

The hardware treats `x1` to `x31` identically, and only `x0` is special (Table 18.2); the program counter is not in the bank. The roles come from the calling convention. The MC68000, instead, separated the data registers D0–D7 from the address registers A0–A7, with A7 as stack pointer.

18.3.4 Flags: the status register, and its absence

Many processors have a *status register*: bits, called flags, accessible from the program. The usual ones are Z (zero), N (negative), V (overflow) and C (carry), sometimes with a parity bit. An instruction sets them, and a later conditional branch tests them. The MC68000 has its condition code register, and x86 its flags register.

RISC-V has no flags. A branch compares two registers and jumps, in one instruction: `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`. The instructions `slt` and `sltu` write the result of a comparison in a general register. Carry and overflow are computed when they are needed (Chapter 5). There is less hidden state, which makes pipelines and interrupts simpler.

18.3.5 Special registers of RISC-V: the CSRs

The special registers of RISC-V are the *control and status registers* (CSR): a separate bank of up to 4 096 registers, read and written only with the instructions `csrrw`, `csrrs` and `csrrc`,

and only by privileged code (Table 18.3). They are the "registers of the interrupts" of the general model, and the subject of Chapter 19.

Table 18.3. Some machine-mode CSRs.

Register	Name	What it holds
`mstatus`	machine status	the global interrupt enable and the previous privilege mode
`mtvec`	trap vector	the address of the routine that handles interrupts and exceptions
`mepc`	exception program counter	the address of the instruction that was interrupted
`mcause`	trap cause	a code that says why the trap happened
`mie`, `mip`	interrupt enable, interrupt pending	one bit per source of interrupt

18.4 The stack

18.4.1 Last in, first out

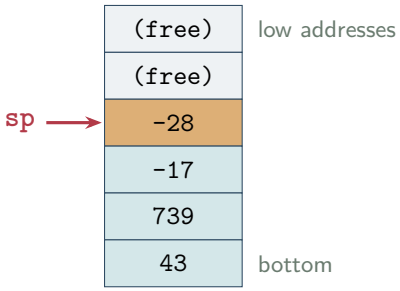
Definition 18.2 (Stack)

A *stack* is an abstract data structure with two operations: *push* stores an element as the last of the pile, and *pop* takes the last element away. The last value pushed is the first one popped (LIFO).

A stack works always and only on its top: there is no need to say where to store or where to fetch. It is used to set aside the contents of registers that must be freed to carry out a task, and to restore them afterwards: this happens in subprograms and in interrupts. It is implemented with a pointer, in memory or in a dedicated register, which indicates at every instant the position of the last element. Operations on a stack should check for overflow on a push and for an empty stack on a pop.

18.4.2 Push and pop in RISC-V

In RISC-V the stack lives in memory, grows towards the low addresses, and the register `sp` points to its top (Figure 18.2). There are no instructions called push and pop: each is two ordinary instructions.




```
# push t0
addi sp, sp, -4       # make room
sw   t0, 0(sp)        # store on the new top

# pop t0
lw   t0, 0(sp)        # read the top
addi sp, sp, 4        # release it
```

Figure 18.2. A stack in memory, and the two operations in RISC-V.

In practice a RISC-V program does not push and pop single values: a *stack frame* is allocated once per function, sixteen bytes at a time, and is filled with ordinary stores, as every function of Verm1 that makes a call has done. Other processors have dedicated forms: on the 68000, `MOVE (SP)+, D0` pops with a post-increment.

18.5 Processor models

Processors are classified according to where the operands of an operation live.

18.5.1 Four architectures

General registers.
The processor has a set of registers that can be used interchangeably.

Accumulator.
All the arithmetic, logic and comparison instructions have an implicit operand: one special register, the accumulator.

Stack.
There are neither accumulators nor general registers, only the stack pointer; the instructions work on the data on top of the stack.

Orthogonal.
A single bank of general registers, some of which serve, when needed, as stack pointer or as other special registers.

18.5.2 `z = x + y` in four processors

Table 18.4. The same computation in four models.

Model	Example	Program	Accesses
accumulator	Motorola 6809, MOS 6502	`LDA x; ADD y; STA z`	3
stack	Java virtual machine, WebAssembly	`PUSH x; PUSH y; ADD; POP z`	3 + stack
general registers, memory operands	MC68000, Intel x86	`MOVE x,D0; ADD y,D0; MOVE D0,z`	3
general registers, load-store	RISC-V	`lw t1,x; lw t2,y; add t3,t1,t2; sw t3,z`	3

The accesses to memory are the same three everywhere (Table 18.4): what changes is how many instructions there are, and how complex each one is. In the register-memory model an instruction such as `ADD y, D0` reads memory *and* computes; in the load-store model every instruction does one thing. Stack machines with the stack in memory are generally slower than general-register machines. (In the table, `lw t1, x` is a shorthand that the assembler accepts for the pair `la` and `lw` of the warm-up.)

18.5.3 RISC-V: a load-store, general-register processor

Load, compute, store

Only loads and stores touch memory; every other instruction works from register to register. It is the rhythm of every RISC-V program, from the first counter to the last game.

The reasons are four. Instructions that work on registers are faster than those that work on memory locations, so that the most used data are kept in registers. An instruction has three operands, two sources and a destination, so that no value is destroyed unless the programmer wants it to be. Each instruction does one simple thing and has a fixed size: it is easy to decode, and easy to execute in a pipeline. This is the meaning of RISC, *reduced instruction set computer*: the forms that go from memory to a register and from a register to memory, which exist in CISC processors, are exactly what RISC gives up.

18.6 The ALU and the whole picture

18.6.1 The arithmetic logic unit

The ALU of Chapter 11, seen from the processor, supervises the arithmetic and logic operations between operands. In the model it has two input registers, O_1 and O_2, and an output register U. It is physically included in the processor in elementary architectures and is an independent apparatus in more complex ones; a widespread solution is a reduced arithmetic unit inside the processor with independent apparatus for the more sophisticated operations. In RISC-V that apparatus is the extensions: M for multiplication and division, F and D for floating point. The operations available depend on the types of data and on their representation: logical values, strings of characters, pointers, and numbers in the encodings of Chapters 5 and 6.

18.6.2 The architecture of an RV32 processor

Figure 18.3 reduces to its blocks the classic single-cycle RISC-V datapath of Patterson and Hennessy (2020). The PC gives the address of the instruction; the instruction is fetched from memory; the decoder separates its fields and the control unit produces the commands; the register file supplies the operands; the ALU computes a result or forms an address; the data memory is read or written; and the result goes back to the register file. The instruction memory and the data memory are the same address space, drawn twice for clarity.

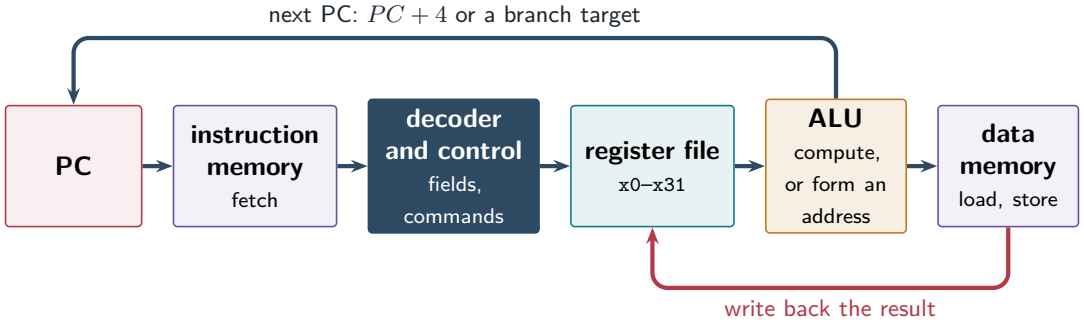


Figure 18.3. The architecture of an RV32 processor, reduced to its blocks.

18.6.3 The processor of the PRG32 kit

Table 18.5. The ESP32-C6 (from the datasheet of Espressif).

Feature	ESP32-C6
Instruction set	RV32IMAC: base integer set, multiply and divide, atomics, compressed instructions
Main core	one 32-bit RISC-V core, up to 160 MHz, four-stage pipeline
Second core	a low-power RISC-V core, up to 20 MHz, for tasks that run while the main core sleeps
Privilege modes	machine and user
Floating point	none: no F or D extension
Memory	512 KB of SRAM, 320 KB of ROM, external flash
On the same chip	Wi-Fi 6, Bluetooth, Thread and Zigbee radios; SPI, I2S, GPIO and timer peripherals

The chip of Table 18.5 is a complete computer on a few square millimetres that costs about two euros: processor, memory and input/output are all there, as in the model with which the chapter began.

18.7 The program of this chapter

The listings of this chapter are parts of one program, `code/l18.S` in the repository of the course, printed in full in Listing 18.2. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 18.4 is the frame that the test wrote.

On the screen.
Three bars, eight pixels per unit: x = 5 (white), y = 7 (white) and z = 12 (green), computed by the seven instructions of the slide.

What to press.
Nothing: the program runs by itself.



Figure 18.4. `l18.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 18: The Processor
# l18.S: z = x + y on a load-store machine
#
```

```
# On the screen: three bars, eight pixels per unit: x = 5 (white), y = 7 (white) and z
# = 12 (green), computed by the seven instructions of the slide.
# Build: python3 -m prg32 cartridge build l18.S --portable \
#            --entry-prefix l18 --name l18 --out l18.prg32
.option norelax

.equ BLACK, 0x0000
.equ WHITE, 0xffff
.equ GREEN, 0x07e0

.section .text
.global l18_init
.global l18_update
.global l18_draw

l18_init:
    ret

l18_update:
    la   t0, x
    lw   t1, 0(t0)        # bring x into a register
    la   t0, y
    lw   t2, 0(t0)        # bring y into a register
    add  t3, t1, t2       # compute, between registers
    la   t0, z
    sw   t3, 0(t0)        # take the result back to memory
    ret

# bar: a0 = ADDRESS of a variable, a1 = y of the bar, a4 = colour
bar:
    lw   a2, 0(a0)        # the VALUE of the variable
    slli a2, a2, 3        # width: eight pixels per unit
    li   a0, 20
    li   a3, 16
    tail prg32_gfx_rect

l18_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    la   a0, x
    li   a1, 40
    li   a4, WHITE
    call bar
    la   a0, y
    li   a1, 72
    li   a4, WHITE
    call bar
    la   a0, z
    li   a1, 120
    li   a4, GREEN
    call bar
    lw   ra, 12(sp)
    addi sp, sp, 16
```

```
59      ret
60
61  .section .data
62  x: .word 5
63  y: .word 7
64  z: .word 0
```

Listing 18.2. The complete program of the chapter, `code/l18.S`.

To run it on PRG32-QT the commands are those of Listing 18.3; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l18.S --portable \
    --entry-prefix l18 --name l18 --out l18.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l18.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 18.3. Building `l18.S` and running it on PRG32-QT.

18.8 Check your understanding

Q18.1 Which register of the general model has no counterpart in RISC-V?

A. the program counter
B. the status register with the flags
C. the stack pointer
D. the instruction register

Q18.2 In RISC-V, which instructions can read or write the main memory?

A. all of them
B. `add` and `sub`
C. only loads and stores
D. only jumps

Q18.3 After `addi sp, sp, -4` and `sw t0, 0(sp)`, what has happened?

A. `t0` was popped from the stack
B. `t0` was pushed on the stack
C. the stack was emptied
D. `sp` was saved in `t0`

Q18.4 In an accumulator machine, the instruction `ADD y` means:

A. $y := y +$ accumulator
B. accumulator := accumulator + $[y]$
C. push y, then add
D. $y :=$ accumulator

18.9 Exercises

Exercise 18.1 (paper first, then PRG32)

1. Write $z = (a + b) - (c + d)$ for an accumulator machine, for a stack machine and for RISC-V; count the instructions and the memory accesses.
2. Write the push and the pop of two registers, `t0` and `t1`, keeping `sp` a multiple of 16 as the calling convention requires.
3. For each register used in your last PRG32 program, say which function it serves: transit, accumulation, addressing, stack.
4. In PRG32-QT, single-step a call and its return: watch `pc`, `ra` and `sp`, and the word saved on the stack.
5. Find in the datasheet of the processor of your telephone or laptop: instruction set, number of cores, clock, privilege levels.

Expected result. Accumulator: `LDA a; ADD b; STA tmp; LDA c; ADD d; STA tmp2; LDA tmp; SUB tmp2; STA z`. Stack: `PUSH a; PUSH b; ADD; PUSH c; PUSH d; ADD; SUB; POP z`. RISC-V: four loads, two `add`, one `sub`, one store. In PRG32-QT: `sp` drops by 16 at the prologue and returns at the epilogue; `ra` holds the address after the call.

18.10 Glossary

English	Italiano
control unit	unità di controllo
program counter	contatore di programma
machine register	registro di macchina
general / special register	registro generico / speciale
instruction set	insieme (set) di istruzioni
accumulator	accumulatore
stack pointer	puntatore allo stack
status register, flags	registro di stato, indicatori
supervisor / user state	stato supervisore / utente
privileged instruction	istruzione privilegiata

18.11 Conclusions and reading

The processor commands and controls, relying on the memory and on the input/output systems. Registers are storage locations that may be physically inside or outside the processor, and the kinds of registers and the way in which they are used determine the model of the processor: accumulator, stack, general registers. The ALU carries out the operations between operands. RISC-V is a load-store, general-register processor with no flags and no I/O instructions: few elements, each one simple.

Reading. Patterson and Hennessy (2020), chapters 2 and 4; the RISC-V instruction set manual, volume I (RISC-V International, 2024c); Stallings (2021), the chapters on the structure and function of the processor. Appendix A is the complete reference.

CHAPTER 19

Instructions and Interrupts

"Why does the CPU never get any work done? Because it gets interrupted constantly."
— Anonymous joke about computer architecture

In this chapter

By the end of this chapter you will be able to

- describe an instruction as an operation code and its operands, and name the kinds of operand;
- decode a RISC-V instruction by hand, recognising its format and fields;
- follow an instruction through the processor cycle: fetch, decode, execute, memory, write back;
- explain what a pipeline is and why it makes a processor faster;
- define interrupts, exceptions and traps, and list their causes;
- describe what RISC-V does when a trap occurs and how the interrupted program resumes.

19.1 Warm-up on the machine

```
address     machine code     what you wrote

0x000       0x0002A303       lw   t1, 0(t0)
0x004       0x00130313       addi t1, t1, 1
0x008       0x0062A023       sw   t1, 0(t0)
0x00C       0x00008067       ret
```

Listing 19.1. Four words in memory.

Listing 19.1 is the function `first_update` of Chapter 2 as it sits in memory: four numbers, four addresses apart. (The `la` that precedes them is omitted, and the encodings are the uncompressed ones of 32 bits: with the C extension the assembler may choose forms of 16 bits for some of them.) Three questions follow. How does the processor know that the first number is a load and the second an addition? Which steps does it take for each of them? And while it runs them, how does it notice that a button was pressed or that a timer has expired?

19.2 Instructions

19.2.1 What an instruction is made of

The typical instructions of a processor are transfers of data between memory and registers, arithmetic and logic operations, instructions for the control of the flow, and transfers of data to and from input/output. (In RISC-V the fourth family does not exist as such, because devices are memory.)

Definition 19.1 (Instruction)

An *instruction* is composed of an *operation code*, the opcode, and of zero or more *operands*. An operand can be a constant (an *immediate*), a memory location, or a register; it can be explicit or implicit.

Instructions can have zero, one, two or three operands: `add` has three, all registers; `ret` has none visible.

19.2.2 The formats of RISC-V instructions

Every RISC-V instruction of the base set is 32 bits long, and is laid out in one of six *formats* (Figure 19.1). The formats were designed so that the same field is always in the same place: the opcode in bits 6 to 0, the destination register `rd` in bits 11 to 7, the source registers `rs1` and `rs2` in bits 19 to 15 and 24 to 20, the field `funct3` in bits 14 to 12. The formats B and J differ from S and U only in how the bits of the immediate are arranged.

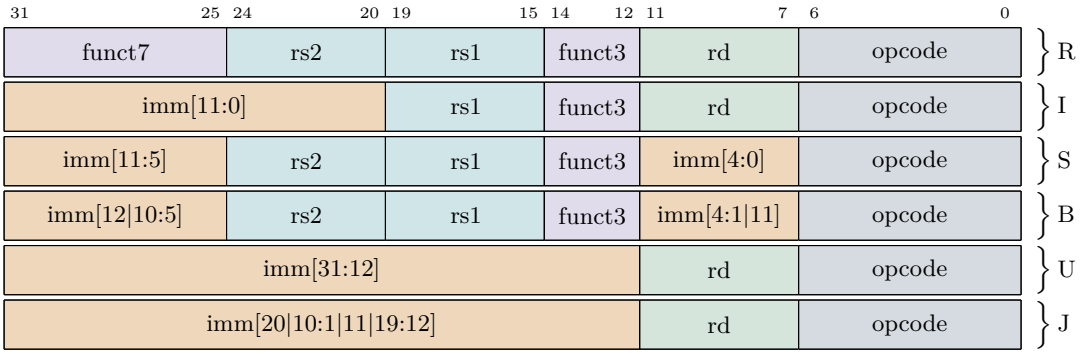


Figure 19.1. The six formats of RISC-V. R: register to register (`add`, `sub`, `and`, `sll`, ...). I: immediates and loads (`addi`, `andi`, `lw`, `jalr`, ...). S: stores. B: branches. U: upper immediates (`lui`, `auipc`). J: jumps (`jal`).

This regularity is the heart of the RISC idea. The processor always knows where to look first, and can read the registers named by an instruction while it is still finding out which instruction it is.

19.2.3 Decoding by hand

The method

Write the eight hexadecimal digits in binary; cut off the last seven bits, the opcode; look up the format; then cut the fields.

Example 19.1 (`0x006283B3`)

`0x006283B3` = $0000\,0000\,0110\,0010\,1000\,0011\,1011\,0011_2$. The last seven bits, `0110011`, are the opcode of the register-to-register operations: the format is R.

0000000	00110	00101	000	00111	0110011
funct7	rs2 = x6 = t1	rs1 = x5 = t0	funct3	rd = x7 = t2	opcode

With `funct3` = 000 and `funct7` = 0000000 the operation is an addition (with `funct7` = 0100000 it would be a subtraction): the instruction is `add t2, t0, t1`.

Example 19.2 (`0x0002A303`)

`0x0002A303` = $0000\,0000\,0000\,0010\,1010\,0011\,0000\,0011_2$. The opcode `0000011` is that of the loads: the format is I.

000000000000	00101	010	00110	0000011
immediate = 0	rs1 = x5 = t0	funct3 = word	rd = x6 = t1	opcode

The instruction is `lw t1, 0(t0)`, the first of the warm-up.

The decoders of Chapter 11 do the same thing in a fraction of a nanosecond. The tables needed to decode any instruction are in Appendix A.

19.2.4 Instructions of fixed or of variable length

In a machine with instructions of *fixed length* all the instructions have the same number of bits, the next instruction is always at the PC plus a constant, and fetch and decode are simple. RISC processors are of this kind: RISC-V uses 32 bits and, with the compressed extension, 16 or 32, told apart by the two lowest bits (Chapter 11). In a machine with instructions of *variable length* the length is known only after the first part has been decoded: programs are compact, and decoding is complex. CISC processors are of this kind: the MC68000 uses one to five words of 16 bits, and Intel x86 one to fifteen bytes. In either case the length of the instruction decides by how much the PC advances.

19.2.5 Addressing modes: where an operand is found

RISC-V has only the first four modes of Table 19.1. A CISC processor such as the MC68000 has a dozen, including memory-indirect and auto-increment modes. The operation $R := M[M[A]]$ becomes in RISC-V `lw t0, 0(t1)` followed by `lw t0, 0(t0)`. The price of rich addressing modes is a further phase in the processor cycle, in which the operands are assembled.

Table 19.1. Addressing modes.

Mode	The operand is	RISC-V example
immediate	a constant inside the instruction	`addi t1, t1, 1`
register	the content of a register	`add t2, t0, t1`
base plus displacement	the memory word at register + constant	`lw t1, 8(sp)`
PC-relative	an address at a distance from the instruction itself	`beq t0, t1, loop`
memory indirect	the memory word whose address is in memory	not available: two loads

19.2.6 Calls, returns and the calling convention

The instructions for the control of the flow deserve one more word, because every function of Verm1 depends on them. `jal` (*jump and link*, format J) jumps to an address relative to the PC and writes in its destination register the address of the following instruction; `jalr` (format I) does the same with a target taken from a register. The pseudo-instruction `call` is a `jal` (or an `auipc` with a `jalr`, for distant targets) whose destination is `ra`; `ret` is `jalr zero, 0(ra)`.

The *calling convention* is the agreement that lets separately written functions call one another (RISC-V International, 2024a). It says that arguments travel in `a0`–`a7` and results in `a0` and `a1`; that a function may destroy the registers `t0`–`t6` and `a0`–`a7`, which are therefore *caller-saved*; that it must return the registers `s0`–`s11` and `sp` as it found them, which are therefore *callee-saved*; and that `sp` is always a multiple of 16. This is why, since Chapter 12, a value that must survive a call has been kept in an `s` register and saved in the frame. The convention is stated in full in Appendix A.

19.3 The processor cycle

19.3.1 The phases of the cycle

After a start-up phase, the *bootstrap*, a processor repeats for ever the same sequence: its fundamental algorithm. In the classic RISC formulation the sequence has five phases (Figure 19.2).

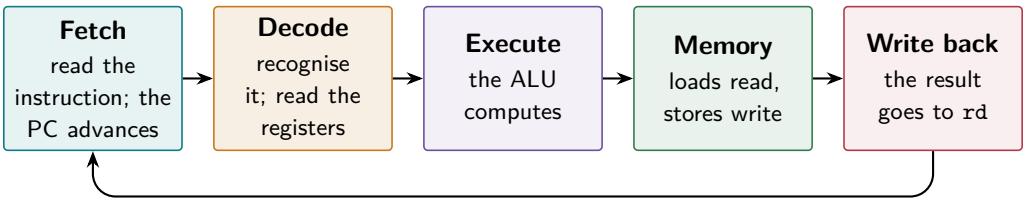


Figure 19.2. The processor cycle.

Fetch.
The instruction at the address in the PC is read from memory; the PC advances.

Decode.
The opcode is recognised; the registers named by the instruction are read.

Execute.
The ALU computes: a result, an address or a comparison.

Memory.
Loads read the memory and stores write it; the other instructions skip this phase.

Write back.
The result is written into the destination register.

19.3.2 The fetch phase, transfer by transfer

In terms of the reference model of Chapter 18, the fetch is four *register transfers* on the internal bus (Table 19.2). In RV32 a word is four bytes and addresses count bytes, so that the PC advances by 4 (by 2 after a compressed instruction).

Table 19.2. The fetch phase.

Step	Register transfer	Meaning
1	$MA := PC$	the address of the instruction goes to the memory address register
2	read enable; $MB := M[MA]$	the memory puts the instruction on the data bus; it is latched in the buffer
3	$D := MB$	the instruction is copied into the decoding register
4	$PC := PC + 4$	the PC now points to the next instruction

19.3.3 The rest of the cycle for two instructions

Table 19.3. The remaining phases for an addition and for a load.

Phase	`add t2, t0, t1`	`lw t1, 0(t0)`
decode	$O_1 := R[\texttt{t0}]$; $O_2 := R[\texttt{t1}]$	$O_1 := R[\texttt{t0}]$; $O_2 := 0$ (the immediate)
execute	$U := O_1 + O_2$	$U := O_1 + O_2$ (an address)
memory	–	$MA := U$; read enable; $MB := M[MA]$
write back	$R[\texttt{t2}] := U$	$R[\texttt{t1}] := MB$

The same ALU adds two registers for `add` and forms an address for `lw` (Table 19.3): only the control signals differ. The control unit, by decoding the opcode, decides which transfers happen. A store ends at the memory phase; a branch ends at the execute phase, by choosing the next PC.

19.3.4 The pipeline

Five phases are five pieces of hardware. While one instruction is executed, the next can be decoded and a third fetched: this is a *pipeline* (Table 19.4).

Each instruction still takes five cycles, but one instruction completes at every clock cycle: the throughput is five times as large. In general, n instructions on a pipeline of k stages complete in $k + n - 1$ cycles. *Superscalar* processors put two or more pipelines side by side, and complete more than one instruction per cycle (Chapter 24). The main core of the ESP32-C6 has a pipeline of four stages. Branches, and dependences between neighbouring instructions, complicate the picture: they are called *hazards*, and are the subject of an advanced course.

Table 19.4. A linear pipeline of five stages: IF instruction fetch, ID decode, EX execute, MEM memory, WB write back.

clock cycle	1	2	3	4	5	6	7	8
instruction 1	IF	ID	EX	MEM	WB			
instruction 2		IF	ID	EX	MEM	WB		
instruction 3			IF	ID	EX	MEM	WB	
instruction 4				IF	ID	EX	MEM	WB

19.4 Interrupts

19.4.1 The problem with the simple cycle

If the processor only ever did fetch, decode and execute, three problems would arise. The interaction with the outside world through the I/O units would be inefficient, because the program would have to keep asking. There would be no way of forcibly removing an application that, by mistake, enters an infinite loop: who takes the processor away from a program stuck in `j loop`? And the operating system would have limited control over the system.

19.4.2 Interrupts, exceptions, traps

Definition 19.2 (Interrupt)

Interrupts are exceptional events, usually asynchronous with respect to the execution of the program in progress, that require the attention of the processor.

The RISC-V specification uses three words precisely (RISC-V International, 2024d). An *interrupt* comes from outside the program, at any moment: a timer, a device. An *exception* is caused by the instruction being executed: an illegal opcode, a forbidden address. A *trap* is the transfer of control to a handler that follows either of them. (Older processors call "traps" the synchronous events and use "interrupt" for everything.)

19.4.3 The real processor cycle

The real cycle therefore has one more step (Figure 19.3): after each instruction the processor checks whether an interrupt is pending and, if so, passes to the handler instead of fetching the next instruction.

19.4.4 Possible causes of interruption

Periodic interrupt.
A timer, for example every 10 ms: to share the processor among programs and to measure the time that each one uses.

I/O interrupt.
A peripheral reports its state, for example that it is ready to receive data, to synchronise with the processor.

Errors in the program.
An instruction that does not exist or is privileged, a forbidden address: synchronous events, the exceptions. (In RISC-V an integer overflow does *not* raise an exception.)

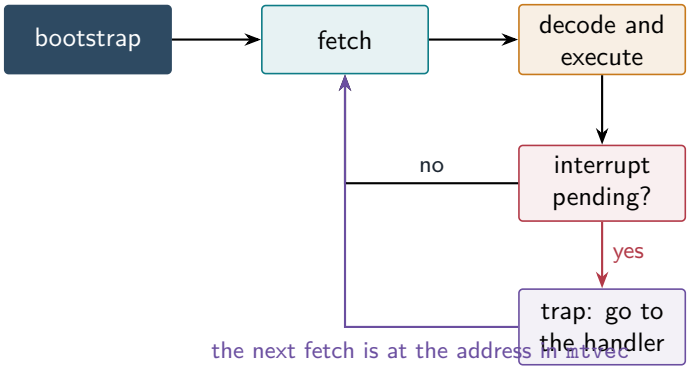


Figure 19.3. The real processor cycle.

Faults.
Failures of the system detected by dedicated probes: power, temperature, memory errors.

Reset.
Brings the whole system back to a known state.

Software interrupt.
Generated by a program that needs a shared resource and asks the operating system to mediate: in RISC-V, the instruction `ecall`, *environment call*, which is synchronous with the program that executes it.

19.4.5 Handling an interrupt

1. Save the context, hardware part: at least the address at which to resume.
2. Identify the cause of the interrupt.
3. Jump to the entry point of the *interrupt service routine* (ISR).
4. Save the context, software part: the registers that the routine is going to use.
5. Serve the interrupt.
6. If the interrupt allows it, restore the context and resume the suspended program from the instruction that follows the last one executed.

The *context* to save and restore is the PC, the status and the registers. The saving can be done partly or wholly in hardware or in software; RISC-V does the minimum in hardware.

19.4.6 Who interrupted? Two solutions

With several devices the processor must identify who interrupted. With *status registers and polling*, each device has a status register: a device that requests an interrupt sets a bit there and raises a common request line; the service routine interrogates the devices in order of priority and serves the first that it finds with its bit high. The scheme is simple to build but takes time, because it also interrogates devices that asked for nothing. With *vectored interrupts* the identity of the source is supplied to the processor at the moment of the request, and the processor jumps directly to the routine that corresponds to it: a very efficient mechanism, in which the number of bits of the vector limits the number of different sources.

The *interrupt latency* is the delay between the instant in which the request line goes high and the instant in which the service routine starts doing useful work; it includes the end of the current instruction, the check and the switch.

19.5 The RISC-V solution

RISC-V solves the whole problem with a few registers and one instruction.

19.5.1 The registers of the trap mechanism

Table 19.5. The machine-mode CSRs of the trap mechanism.

CSR	Written by	Content
`mtvec`	the firmware, once, at start-up	the address of the trap handler, and whether the vectored mode is used
`mstatus`	hardware and firmware	MIE, the global interrupt enable, and MPIE, its previous value
`mie`	the firmware	one enable bit for each source: software, timer, external
`mip`	the hardware	one pending bit for each source
`mepc`	the hardware, at each trap	the address of the interrupted instruction
`mcause`	the hardware, at each trap	one bit that says interrupt or exception, and a code for the cause

An interrupt is taken only if its bit is set in both `mip` and `mie`, and MIE is 1 (Table 19.5).

19.5.2 What the hardware does on a trap

1. `mepc` := the address of the interrupted instruction.
2. `mcause` := the cause: the top bit is 1 for an interrupt and 0 for an exception, and the other bits hold a code.
3. In `mstatus`: MPIE := MIE, then MIE := 0, so that further interrupts are disabled.
4. The processor passes to machine mode.
5. PC := the address in `mtvec`; or, in vectored mode and for interrupts, that address plus 4 times the cause.

At the end of the handler the instruction `mret` undoes it all: MIE := MPIE and PC := `mepc`.

Nothing is pushed on a stack by the hardware: saving the general registers is left to the handler. The "hardware context save" of the general scheme is reduced to two registers and one bit.

19.5.3 The cause of a trap

Table 19.6. Some standard values of `mcause`; chips add their own.

Kind	Code	Cause
interrupt	3	machine software interrupt
interrupt	7	machine timer interrupt
interrupt	11	machine external interrupt: a device, through the interrupt controller
exception	2	illegal instruction
exception	3	breakpoint (`ebreak`): how a debugger stops a program
exception	5, 7	load or store access fault: a forbidden address
exception	8, 11	environment call (`ecall`) from user mode, from machine mode

The same code 3 means different things for interrupts and for exceptions (Table 19.6): the top bit of `mcause` distinguishes them. With `mtvec` in vectored mode, a timer interrupt enters at $base + 4 \times 7$.

19.5.4 A trap handler, in outline

```
trap_entry:
    addi sp, sp, -16
    sw   t0, 0(sp)            # software context: save what we use
    sw   t1, 4(sp)

    csrr t0, mcause           # why are we here?
    bgez t0, exception        # top bit 0: an exception
    ...                       # top bit 1: find the device, serve it

    lw   t0, 0(sp)            # restore the context
    lw   t1, 4(sp)
    addi sp, sp, 16
    mret                      # MIE := MPIE, PC := mepc
```

Listing 19.2. The outline of a trap handler.

The interrupted program must not notice anything: every register that the handler touches is saved and restored (Listing 19.2). Read as a signed number, `mcause` is negative for interrupts, so that `bgez` separates the two families. `mret` is the only way back: it re-enables the interrupts and reloads the PC in one step. The instruction `csrr t0, mcause` is the pseudo-instruction for `csrrs t0, mcause, zero`.

The listing is an outline, not the source of PRG32: a real handler saves all the caller-saved registers, usually on a separate stack reached through the register `mscratch`. The firmware of the kit contains exactly such a routine, and a cartridge is interrupted many times per frame.

19.5.5 Priorities and nesting

To handle *nested* interrupts a priority is established: while a service routine runs, only requests from devices of higher priority are accepted, and a request from a device of high priority, such as the clock, must be accepted even while one of lower priority is being served. Table 19.7 compares the classical solution of the MC68000 with that of RISC-V, where priorities belong to an interrupt controller outside the core (on Espressif chips they go from 1 to 15).

Watch Out

It is sometimes said that "an interrupt is a signal that cannot be masked". The opposite is true: most interrupts can be masked, and only a few cannot.

19.5.6 Interrupts on the PRG32 kit

The three problems of the simple cycle, and their solution, are all visible on the kit.

Table 19.7. Priorities and nesting in two processors.

	Motorola MC68000	**RISC-V**
priority of a request	three lines encode an interrupt priority level, 1 to 7	the interrupt controller gives each source a priority
priority of the processor	three bits in the status register: the mask	a threshold register in the interrupt controller
a request is accepted if	its level is higher than the mask	its priority is higher than the threshold, and it is enabled
while serving it	the mask is raised to that level	interrupts are disabled (MIE = 0) unless the handler re-enables them
cannot be masked	level 7	a non-maskable interrupt, defined by each chip
identity of the source	an 8-bit vector number from the device: 256 vectors	`mcause`, then the controller says which device
return	`RTE`, which pops PC and status from the stack	`mret`, which uses `mepc` and `mstatus`

- A timer interrupts the processor a hundred times per second: the firmware uses it to keep time and to pace the frames.
- The display and sound peripherals interrupt when a block of data has been sent and the next is wanted.
- The Wi-Fi radio interrupts when a packet arrives: a cartridge can be uploaded while a game is running.
- The functions `init`, `update` and `draw` of a cartridge are ordinary functions called by the firmware between one interrupt and the next.
- A cartridge stuck in a loop does not stop the interrupts. PRG32-QT ends the frame with an error ("instruction budget exhausted"); on the board a watchdog reports the stall, and the user presses reset.

In PRG32-QT a breakpoint is the exception with code 3.

19.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l19.S` in the repository of the course, printed in full in Listing 19.3. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 19.4 is the frame that the test wrote.

On the screen.
Four rows of 32 lamps, the machine code of lw t1, 0(t0) / addi t1, t1, 1 / sw t1, 0(t0) / ret, and beside each a green square: the word in memory is the one decoded by hand in the lecture.

What to press.
Nothing: the program runs by itself.

```
# ARC12 Computer Architecture - Lecture 19: Instructions and Interrupts
# l19.S: four words in memory: a function read back as machine code
#
```

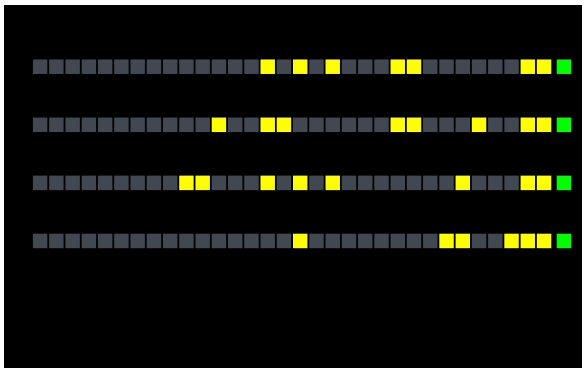


Figure 19.4. `l19.S` on PRG32-QT.

```
# On the screen: four rows of 32 lamps, the machine code of lw t1, 0(t0) / addi t1,
# t1, 1 / sw t1, 0(t0) / ret, and beside each a green square: the word in memory is
# the one decoded by hand in the lecture.
# Build: python3 -m prg32 cartridge build l19.S --portable \
#            --entry-prefix l19 --name l19 --out l19.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800

.section .text
.global l19_init
.global l19_update
.global l19_draw

l19_init:
    ret

l19_update:
    ret

# The function of Lecture 02, kept in its uncompressed 32-bit forms.
.option push
.option norvc
sample:
    lw   t1, 0(t0)          # 0x0002A303
    addi t1, t1, 1          # 0x00130313
    sw   t1, 0(t0)          # 0x0062A023
    ret                     # 0x00008067
.option pop

l19_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
```

```
43      li   a0, BLACK
44      call prg32_gfx_clear
45      li   s0, 0              # which instruction, 0 to 3
46  1:  la   t0, sample
47      slli t1, s0, 2          # instructions are 4 bytes apart
48      add  t0, t0, t1
49      lw   s1, 0(t0)          # the instruction, fetched as a datum
50      mv   a0, s1
51      slli a1, s0, 5
52      addi a1, a1, 30         # y = 30 + 32 * s0
53      call lamps32
54      la   t0, expected       # compare with the encoding worked out by hand
55      slli t1, s0, 2
56      add  t0, t0, t1
57      lw   t2, 0(t0)
58      li   a4, RED
59      bne  s1, t2, 2f
60      li   a4, GREEN
61  2:  li   a0, 306
62      slli a1, s0, 5
63      addi a1, a1, 30
64      li   a2, 8
65      li   a3, 8
66      call prg32_gfx_rect
67      addi s0, s0, 1
68      li   t0, 4
69      blt  s0, t0, 1b
70      lw   s1, 4(sp)
71      lw   s0, 8(sp)
72      lw   ra, 12(sp)
73      addi sp, sp, 16
74      ret
75
76  # lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
77  #   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
78  lamps32:
79      addi sp, sp, -32
80      sw   ra, 28(sp)
81      sw   s0, 24(sp)
82      sw   s1, 20(sp)
83      sw   s2, 16(sp)
84      mv   s0, a0             # the word
85      mv   s1, a1             # y
86      li   s2, 0              # lamp number, 0 on the left
87  1:  li   t0, 31
88      sub  t0, t0, s2         # lamp i shows bit 31 - i
89      srl  t1, s0, t0         # bring that bit down to bit 0
90      andi t1, t1, 1          # keep it, clear all the others
91      li   a4, GREY           # lamp off
92      beqz t1, 2f
93      li   a4, YELLOW         # lamp on
94  2:  slli a0, s2, 3
95      add  a0, a0, s2         # 9 * i
96      addi a0, a0, 16         # x = 16 + 9 * i
97      mv   a1, s1
98      li   a2, 8
```

```
99      li   a3, 8
100     call prg32_gfx_rect
101     addi s2, s2, 1
102     li   t0, 32
103     blt  s2, t0, 1b
104     lw   s2, 16(sp)
105     lw   s1, 20(sp)
106     lw   s0, 24(sp)
107     lw   ra, 28(sp)
108     addi sp, sp, 32
109     ret
110
111 .section .rodata
112 expected: .word 0x0002A303, 0x00130313, 0x0062A023, 0x00008067
```

Listing 19.3. The complete program of the chapter, `code/l19.S`.

To run it on PRG32-QT the commands are those of Listing 19.4; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l19.S --portable \
    --entry-prefix l19 --name l19 --out l19.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l19.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 19.4. Building `l19.S` and running it on PRG32-QT.

19.7 Check your understanding

Q19.1 In which bits of a RISC-V instruction is the opcode?

A. the seven most significant
B. the seven least significant
C. bits 14 to 12
D. it depends on the format

Q19.2 During the fetch of an RV32 instruction that is not compressed, the PC is increased by:

A. 1
B. 2
C. 4
D. 32

Q19.3 Which register holds the address at which an interrupted RISC-V program will resume?

A. `ra`
B. `mtvec`
C. `mepc`
D. `mcause`

Q19.4 Which of these is an exception rather than an interrupt, in RISC-V terms?

A. a timer expires
B. a key is pressed
C. an illegal instruction is fetched
D. a packet arrives from the network

19.8 Exercises

Exercise 19.1 (paper first, then PRG32)

1. Decode `0x00A28293` and `0x40628333` by hand: format, fields, assembly.
2. Encode `sub t0, t1, t2` and `sw a0, 8(sp)`.
3. Write the register transfers of the whole cycle for `sw t1, 0(t0)` and for `beq t0, t1, target`.
4. Draw the pipeline diagram for six instructions on a pipeline of four stages: when does the sixth complete?
5. In PRG32-QT set a breakpoint in `first_update`, step, and compare the machine code shown with your decoding of the warm-up.

Expected result. On paper: `addi t0, t0, 10` (immediate 10, `rs1` = `x5`, `funct3` = 000, `rd` = `x5`, opcode 0010011) and `sub t1, t0, t1` (`funct7` = 0100000, `rs2` = `x6`, `rs1` = `x5`, `rd` = `x6`); `0x407302B3` and `0x00A12423`; the sixth instruction completes at cycle 9. In PRG32-QT: the four words of the warm-up, or their compressed equivalents.

19.9 Glossary

English	Italiano
operation code (opcode)	codice operativo
operand	operando
addressing mode	modo di indirizzamento
fetch / decode / execute	prelievo / decodifica / esecuzione
pipeline stage	stadio della pipeline
context	contesto
interrupt	interruzione
exception / trap	eccezione / trap
interrupt service routine	routine di servizio dell'interruzione
interrupt latency	latenza di interruzione
to mask an interrupt	mascherare un'interruzione
nesting	annidamento

19.10 Conclusions and reading

An instruction is an operation code and zero, one or more operands; in RISC-V it is always 32 bits, in six regular formats. The length of the instructions, fixed or variable, determines how the PC behaves. The processor has a cycle: it fetches an instruction, decodes it, executes it and writes the result. Exceptional events break the cycle: the processor saves where it was, serves the event and resumes. Interrupts can be enabled, masked and ordered by priority; RISC-V does it with a handful of registers and the instruction `mret`.

Reading. Patterson and Hennessy (2020), sections 2.5, 4.1–4.6 and 4.10; the RISC-V instruction set manual, volume II, "Privileged Architecture" (RISC-V International, 2024d); *Playing with RISC-V*, chapter 4, on PRG32-QT and its debugger (Montella et al., 2026b).

CHAPTER 20

Buses and the Video Card

"... nothing is more serious than gaming..."
— attributed to Jean Cocteau (1889–1963)

In this chapter

By the end of this chapter you will be able to

- explain what a peripheral controller is and why devices need synchronisation with the processor;
- characterise a bus by width, speed and transfer rate, and list its advantages and limits;
- describe the job of a video card: frame buffer, resolution, colour depth, output interface;
- compute the memory that a frame needs and the time that it takes to send it over a bus;
- explain what a GPU does and what general-purpose computing on GPUs is.

20.1 Warm-up on the machine

```
    li   a0, 100            # x
    li   a1, 50             # y
    li   a2, 0xf800         # red, in RGB565
    call prg32_gfx_pixel    # ... where does it go?

# what the runtime does, in principle:
    li   t0, 320
    mul  t1, a1, t0         # y * 320
    add  t1, t1, a0         # + x  =  position in the frame
    la   t2, framebuffer
    add  t2, t2, t1         # ADDRESS of that pixel
    sb   t3, 0(t2)          # store its palette index (one byte)
```

Listing 20.1. One pixel, please: the call, and what the runtime does in principle.

The first four lines of Listing 20.1 draw one red pixel. The rest is a sketch of what happens

behind the call (it is a sketch of the principle, not the source of the runtime: `t3` stands for the palette index that the runtime derives from the colour in `a2`).

Concept

The screen is an array in memory, with one element per pixel: the *frame buffer*. Drawing a pixel is a store instruction at the address $base + y \times 320 + x$.

Nothing appears until the frame is sent to the display, which the function `prg32_gfx_present` does. On a personal computer these two jobs, keeping the frame and sending it to the monitor, belong to the video card. The kit has no video card: its processor does everything, which makes every step visible. The chapter follows the pixel from the frame buffer to the display, and then generalises.

20.2 Peripherals and controllers

20.2.1 Input/output devices

Peripherals are used for the interaction between the user and the system. They have limited autonomy: they are controlled and coordinated by the processor. And they work asynchronously with respect to it, so that mechanisms of synchronisation are needed: to know when an input datum will be available, and to know when an output device has finished producing its data. The terminal, the most common instrument of interaction, is made of two independent devices, the keyboard and the video; they are not directly connected, and it is the processor that reproduces on the video what arrives from the keyboard (Chapter 21).

20.2.2 Controllers

Definition 20.1 (Controller)

A *controller* is the unit that interfaces a peripheral to the bus: it receives commands from the processor, drives the device, and holds the registers through which data and state are exchanged.

Peripherals are interfaced to the processor through dedicated controllers (Figure 20.1). On the ESP32-C6 the controllers (SPI, I2S, GPIO, USB) are on the same chip as the processor, and their registers are memory-mapped (Chapter 15).

20.3 Buses

20.3.1 What characterises a bus

A bus (Chapter 14) is a communication channel shared by several units, and its characteristics decisively influence the performance of the whole system. Three numbers describe it.

Width.
The number of bits of the data bus.

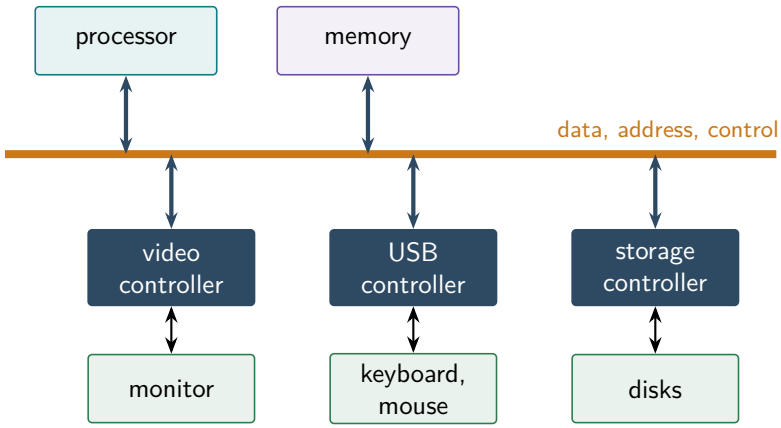


Figure 20.1. Peripherals reach the bus through their controllers.

Speed.
The working frequency of the bus; with the width, it gives the number of bytes per clock cycle.

Transfer rate.
The megabytes transmitted per second: width times frequency.

A bus of 32 bits at 66 MHz, the PCI bus, carries 4 bytes $\times 66 \cdot 10^6 = 264$ MB/s. A bus is also a *standard*: signals, connectors and a protocol on which all the units agree. The *interface* is the hardware module that encodes the data electrically or optically; the *protocol* is the higher-level description of how the dialogue takes place.

20.3.2 Advantages and disadvantages

With a bus, new devices can easily be added; peripherals can be moved from one computer to another that uses the same bus; a single set of wires connects several peripherals; and the cost per byte transmitted is low. On the other hand the bandwidth of the bus limits the speed of transfer; the maximum speed depends on the length of the bus and on the number of devices connected to it; and the bus is a shared resource, on which only one transfer at a time can take place.

20.3.3 Buses through the years

The history of Table 20.1 is the passage from parallel to serial explained in Chapter 14. (The acronyms: ISA, *industry standard architecture*; PCI, *peripheral component interconnect*; AGP, *accelerated graphics port*; IDE, *integrated device electronics*; SCSI, *small computer system interface*; USB, *universal serial bus*.)

20.4 The video card

20.4.1 What a video card does

The function of a *video card* is to send to the monitor the information to be displayed. It translates the representation of the image produced by the processor into a format that the monitor can show. It contains memory of its own, the *video memory*, which holds the frame: the frame buffer. Every time the screen must be redrawn, it reads the digital representation

Table 20.1. Buses through the years.

Bus	Kind	Used for	Today
ISA, EISA, VESA	parallel, 16 to 32 bits	expansion cards of early PCs	disused
PCI	parallel, 32 or 64 bits	expansion cards	replaced by PCI Express
AGP	parallel, dedicated port	the video card alone	replaced by PCI Express
PCI Express	serial, 1 to 16 lanes	video cards, fast storage	current: each generation doubles the speed
IDE (ATA), SCSI	parallel	disks	replaced by Serial ATA and NVMe
USB	serial	almost every external device	current: from 12 Mbit/s to tens of Gbit/s
SPI, I2C, I2S	serial, a few wires	chips on the same board	current: the buses of the PRG32 kit

of the image and sends the monitor the signal for the colour of each pixel. And it has processing capability of its own. On PRG32 the "video card" is a region of RAM plus the SPI peripheral: no separate hardware at all.

20.4.2 Analogue and digital monitors

For a CRT monitor, which has an analogue input, the card converts the digital image into analogue signals; the conversion is done by a component called RAMDAC (*random access memory digital-analog converter*). The VGA output of 1987 was designed for cathode-ray tubes, and suffers from electrical noise, distortion and errors in the placing of the pixels. For an LCD monitor, which has a digital input, the card adapts the colours and the resolution of the image, provides local memory to use as a buffer and supports graphics acceleration, for example in three dimensions; with DVI each pixel of the output corresponds to one pixel of the screen, at its native resolution.

Table 20.2. Video outputs.

Interface	Year	Signal	Notes
VGA	1987	analogue	for CRT monitors; kept for years for compatibility
DVI	1999	digital (and analogue)	introduced with LCD monitors
HDMI	2003	digital	replaces the earlier standards; carries audio too
DisplayPort	2006	digital	video, audio and other data; also over USB-C connectors
SPI to a display controller	–	digital, serial	the PRG32 kit: an ILI9341 panel of 320 by 240 pixels

The ILI9341 of the last row of Table 20.2 is the chip on the display module of the kit: it has its own memory for the frame and drives the liquid crystals (Chapter 21).

20.4.3 Resolution: the size of the pixel grid

The *resolution* determines the definition of the image, and how many pixels must be stored and sent for every frame (Table 20.3). The traditional ratio of 4:3 has given way to the panoramic 16:9.

Table 20.3. Some resolutions.

Resolution	Pixels	Aspect ratio	Where
320×200	64 000	8:5	Commodore 64, VGA games, the PRG32 viewport
640×480	307 200	4:3	VGA
1024×768	786 432	4:3	monitors of the 1990s
1920×1080	2 073 600	16:9	Full HD
3840×2160	8 294 400	16:9	4K

20.4.4 Colour depth: bits per pixel

The *colour depth* is the number of bits used for each pixel (Table 20.4). In True Color every colour is obtained from 256 shades of each of the three additive primaries, red, green and blue. In the mode with 8 bits the value of a pixel is not a colour but an *index* into a table of 256 colours, the *palette*.

Table 20.4. Colour depths.

Depth	Colours	Bytes per pixel	Name
4 bits	16	0.5	standard VGA
8 bits	256	1	256-colour mode: an index into a palette
16 bits	65 536	2	High Color: RGB565
24 bits	16 777 216	3	True Color

PRG32 uses both middle rows: a frame buffer of 8 bits per pixel that holds palette indices, and a palette of 256 colours in RGB565 (Chapter 4).

Theorem 20.1 (Memory of a frame)

A frame of $W \times H$ pixels with a depth of d bits per pixel occupies $W \cdot H \cdot d/8$ bytes.

For instance 640×480 in True Color needs $307\,200 \times 3 = 921\,600$ bytes.

20.4.5 Following the pixel on PRG32

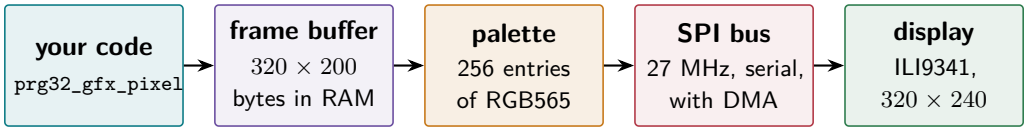


Figure 20.2. The journey of a pixel on the PRG32 kit.

Figure 20.2 is the journey of the pixel of the warm-up, and Table 20.5 gives its numbers (from `docs/hardware/ili9341.md` of the repository).

Table 20.5. The arithmetic of a frame on PRG32.

Quantity	Value
memory for one frame, one byte per pixel	$320 \times 200 \times 1 = 64\,000$ bytes
the same frame at two bytes per pixel	128 000 bytes: a quarter of all the RAM of the chip
bits sent to the display for a whole frame	64 000 pixels $\times$ 16 bits = 1 024 000 bits
time on the bus at 27 MHz	about 38 ms: at most some 26 full frames per second

The last row follows from $1\,024\,000/27\,000\,000 = 37.9$ ms. The platform gives two engineering answers to these numbers: an *indexed* frame buffer, which halves the memory, and the tracking of the rows that have changed, so that only those are sent. Chapter 22 will exploit the second.

20.4.6 Memory and bandwidth: the pressure on video hardware

The memory and the processor time needed by ever more demanding graphics, three-dimensional video games above all, imposed a rapid evolution on video cards. As the definition of the images grows, so does the memory needed to represent a frame, and the flow of data from the memory to the video card risks saturating the capacity of the bus: a 4K frame in True Color at 60 frames per second is $8.3 \cdot 10^6 \times 3 \times 60$ bytes, about 1.5 GB of pixels every second. There were two lines of attack: more and more memory on the card itself, and a wider or dedicated bus, AGP yesterday and PCI Express today. Several cards could even be connected together to share the work; today a single card, or many cards in a server, are the rule.

20.5 GPU and GPGPU

20.5.1 The graphics processing unit

The share of processor time spent computing images grew with the use of graphical interfaces. To relieve the CPU, video cards were given dedicated processors.

Definition 20.2 (GPU)

A *graphics processing unit* (GPU) is a processor specialised in the operations of graphics, which it executes in parallel on many pixels.

A GPU accelerates raster graphics, that is, the copying of blocks of pixels (*blitting*) and the operations on textures, and three-dimensional graphics: transformations, texture mapping, lighting. Programs use it through libraries: OpenGL, Direct3D, and today Vulkan and Metal. A *device driver* adapts the software that runs on the CPU to the primitives offered by the GPU; what is not supported is emulated by the CPU, slowly.

Back to the Machine

On PRG32 there is no GPU: `prg32_gfx_rect` is a loop of store instructions executed by the RISC-V core. This is why the frame rate of a busy cartridge drops, and why the cost of every pixel can be seen.

20.5.2 General-purpose computing on GPUs

Modern GPUs execute the same operation in parallel on whole matrices of pixels, with a very large number of simple cores: hundreds, thousands. *GPGPU* is the use of GPUs for applications of a general kind, not necessarily graphical (Figure 4.5 showed the simplest example, a parallel sum). The programming models are CUDA (*Compute Unified Device Architecture*, by NVIDIA) and OpenCL, an open standard. Scientific simulation, data analysis and, above all today, the training and the use of neural networks run on GPUs; so do the weather and marine models of the Department. Chapter 24 returns to the subject.

20.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l20.S` in the repository of the course, printed in full in Listing 20.2. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 20.3 is the frame that the test wrote.

On the screen.
A single red pixel at x = 100, y = 50 on a black screen. Look closely.

What to press.
Nothing: the program runs by itself.

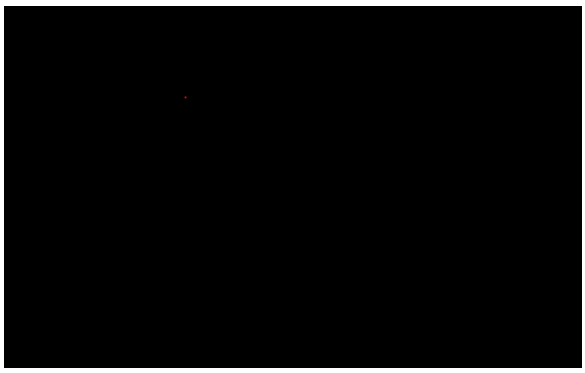


Figure 20.3. `l20.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 20: Buses and the Video Card
# l20.S: one pixel, please
#
# On the screen: a single red pixel at x = 100, y = 50 on a black screen. Look
# closely.
# Build: python3 -m prg32 cartridge build l20.S --portable \
#            --entry-prefix l20 --name l20 --out l20.prg32
.option norelax

```

```
.section .text
.global l20_init
.global l20_update
.global l20_draw

l20_init:
    ret

l20_update:
    ret

l20_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0
    call prg32_gfx_clear
    li   a0, 100             # x
    li   a1, 50              # y
    li   a2, 0xf800          # red, in RGB565
    call prg32_gfx_pixel     # ... where does it go?
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 20.2. The complete program of the chapter, `code/l20.S`.

To run it on PRG32-QT the commands are those of Listing 20.3; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l20.S --portable \
    --entry-prefix l20 --name l20 --out l20.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l20.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 20.3. Building `l20.S` and running it on PRG32-QT.

20.7 Check your understanding

Q20.1 How much memory does a frame of 640×480 pixels need in True Color (24 bits)?

A. 307 200 bytes
B. 614 400 bytes
C. 921 600 bytes
D. 7 372 800 bytes

Q20.2 A 16-bit bus works at 50 MHz, one transfer per cycle. What is its transfer rate?

A. 50 MB/s
B. 100 MB/s
C. 400 MB/s
D. 800 MB/s

Q20.3 In an 8-bit indexed frame buffer, the byte of a pixel contains:

A. its red component
B. its brightness
C. the position of its colour in a palette
D. its x coordinate

Q20.4 What is GPGPU?

A. A faster video output
B. Using graphics processors for non-graphical computation
C. A bus for video cards
D. A kind of video memory

20.8 Exercises

Exercise 20.1 (paper first, then PRG32)

1. Compute the memory for one 4K frame in True Color, and the bytes per second at 60 frames per second.
2. The display bus of PRG32 runs at 27 MHz: how long does it take to send 20 rows of 320 pixels at 16 bits per pixel?
3. Draw a diagonal line pixel by pixel, with `prg32_gfx_pixel` in a loop.
4. Fill the screen once with `prg32_gfx_clear` and once with 64 000 calls to `prg32_gfx_pixel`: compare the frame rates.
5. Change one entry of the palette with `prg32_palette_set` at every frame, and watch every pixel that uses it change at once.

Expected result. On paper: 24 883 200 bytes per frame, about 1.5 GB per second; 102 400 bits, about 3.8 ms. On PRG32: palette cycling animates the screen without redrawing a single pixel. It is the reason why indexed colour was loved by the programmers of games: one store changes thousands of pixels.

20.9 Glossary

English	Italiano
peripheral	periferica
controller	controller (unità di controllo della periferica)
bus width	larghezza del bus
transfer rate	velocità di trasferimento
video card	scheda video
resolution	risoluzione
colour depth	profondità di colore
palette	tavolozza (palette)
video memory	memoria video
device driver	driver di periferica

20.10 Conclusions and reading

Input/output devices are a fundamental component of computers, and each is managed by a controller that interfaces it to the bus. A bus is characterised by width, speed and transfer rate, and its bandwidth is a limit to design round. Video cards make it possible to display graphics and text: a frame buffer, a resolution, a colour depth, an output. Their processors, the GPUs, are also used for high-performance computing.

Reading. Tanenbaum and Austin (2013), chapters 2 and 3 (buses, displays); *Playing with RISC-V*, chapter 7, the graphics game (Montella et al., 2026b); `docs/hardware/ili9341.md` in the PRG32 repository.

CHAPTER 21

Input and Output

"Need input!"
— Number 5, in the film *Short Circuit*, John Badham, 1986

In this chapter

By the end of this chapter you will be able to

- describe how a picture is formed on a CRT and on an LCD, and what a TFT active matrix adds;
- explain the path of a key press: matrix, encoder, shift register, serial interface;
- say what a mouse sends to the computer, and how often;
- describe the stages of laser printing;
- explain how the PRG32 kit reads its joystick, and compare polling, interrupts and DMA.

21.1 Warm-up on the machine

```
steer_update:
    addi sp, sp, -16
    sw   ra, 12(sp)

    sw   s0, 8(sp)

    call prg32_input_read      # a0 = one bit per button
    mv   s0, a0                # keep it in a saved register

    andi t0, s0, 0x10          # button A
    beqz t0, 1f
    li   a0, 0                 # channel
    li   a1, 0                 # instrument
    li   a2, 69                # note: A4, 440 Hz
    li   a3, 255               # volume
    li   a4, 120               # milliseconds
    call prg32_audio_note
1:  andi t0, s0, 0x01          # LEFT: s0 survived the call
    beqz t0, 2f
```

```
    ...                         # move left
2:  ...
```

Listing 21.1. Read the world, once per frame.

The reader has read the joystick since the first chapters (Listing 21.1). Each button is a switch on one input pin of the chip: pressed or released, one bit. The runtime reads the pins, which are memory-mapped registers configured as inputs, and packs them into one word: a decoded representation, in the sense of Chapter 11. And the game asks once per frame: this is called *polling*.

The listing also repeats a lesson of Chapter 19. The mask of the buttons is kept in `s0`, a saved register, and therefore survives the call that plays the note: the marker moves with **LEFT** also while **A** sounds. In `t0` the mask would be lost at the call. (The program was built as a cartridge and run on the headless runner of PRG32-QT.)

A keyboard has a hundred keys, a mouse moves in two dimensions, a display has thousands of pixels: how do they talk to a processor?

21.2 Displays

21.2.1 The video: an introduction

The first displays showed mostly characters, in black and white; today they are graphical and in colour. A graphical display is a matrix of points, the *pixels*, each lit with a different intensity, and an image is composed by switching the pixels on and off. A different density of pixels gives a different resolution; the colours go from 256 (8 bits) to 16 million (24 bits) and beyond. The size is measured in inches along the diagonal. The display of the PRG32 kit has a diagonal of 2.8 inches, 320 by 240 pixels and 65 536 colours.

21.2.2 The cathode-ray tube

Table 21.1. A short history of the cathode-ray tube.

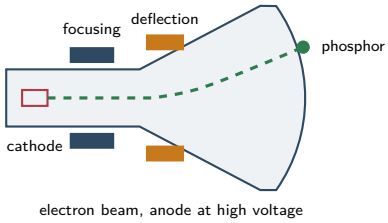
Year	Who	What
early 1870s	William Crookes	the Crookes tube: the first cathode-ray tube, a laboratory instrument
1897	Karl Ferdinand Braun	the Braun tube and the first cathode-ray oscilloscope: the first practical application
1907	Boris Rosing	a CRT displays simple geometric shapes from a received video signal: the root of television
1934	Telefunken	the first television set with a CRT on the market, in Germany
1970s–1990s	the computer industry	the CRT monitor: the display of every terminal and personal computer

In a *cathode-ray tube* (CRT; Table 21.1, Figure 21.1) the cathode is a small metal element heated to incandescence, which emits electrons by the thermionic effect. Inside the tube the electrons are directed into a beam by a high difference of potential between cathode and anode, and other electromagnetic fields focus the beam. The beam, the “electron brush”, is deflected by magnetic fields (the Lorentz force) or by electric fields, and strikes a point on

the inner surface of the screen, which is coated with phosphor and lights up. Sweeping the beam line by line, many times per second, paints the picture.



(a) A compact Macintosh: a 9-inch CRT



(b) The scheme of a CRT



(c) An oscilloscope used as a display

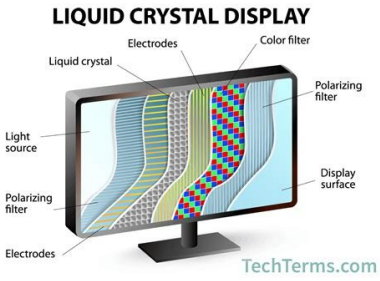
Figure 21.1. The cathode-ray tube.

21.2.3 Liquid crystal displays

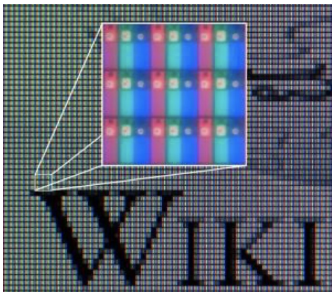
LCD monitors have almost completely replaced the cathode-ray tube: they give a better picture, with less bulk, weight and power, and less strain on the eyes. *Liquid crystals* are viscous organic molecules that flow like a liquid but have a spatial structure. An electric field changes the orientation of the molecules, and with it an optical property of the crystal: the angle of polarisation of the light that crosses it. A crystal is a valve for light, opened and closed by a voltage.

21.2.4 Inside an LCD panel

The screen is made of many cells, often one per pixel, in which the liquid crystals are trapped (Figure 21.2). Each cell has electrical contacts, so that a field can be applied to the liquid that it contains. The cells lie between two polarising filters whose axes are perpendicular to each other, with a light, the *backlight*, behind the rear one. Polarisation decides whether the light passes: when the crystals are not driven the light is rotated and crosses the front polariser; when they are driven it is completely blocked. In a colour screen each cell is divided into three sections with a red, a green and a blue filter: the three fields of the RGB565 colour of Chapter 4 end up here, as three voltages.



(a) The layers of an LCD



(b) Red, green and blue sub-pixels

Figure 21.2. Liquid crystal displays.

21.2.5 Driving the cells: from segments to the active matrix

In calculators and digital watches there is one electrical contact per segment, which becomes impossible as the number of elements grows. A *passive matrix* has one group of contacts

per row and one per column, not per pixel; only one pixel at a time is driven, and the others must remember their state. The consequences are *persistence*, the trails behind moving objects, and crosstalk between neighbouring cells.

High-resolution screens use an *active matrix*. The panel contains a thin film of transistors (TFT, *thin-film transistor*) on a transparent semiconductor layer; the field is applied exactly where it is needed by a transistor in each cell, and the transistor stores the electrical state of its pixel while the others are updated. Each pixel is, in effect, an analogue memory cell. The display of the kit is a TFT panel with an ILI9341 controller. In the more recent OLED displays each pixel emits its own light, and no backlight is needed.

21.3 Keyboard and mouse

21.3.1 The keyboard

The keyboard derives from that of typewriters, and is the main device for the input of characters. Its keys are grouped: alphanumeric keys; special keys such as enter and backspace; arrow keys and function keys. Keyboard and video are not directly connected to each other: it is the processor that reproduces on the video the information typed on the keyboard. The echo of a key on the screen is a program: read a code, draw a character. Today's keyboards connect through USB, or without wires.

21.3.2 The path of a key press

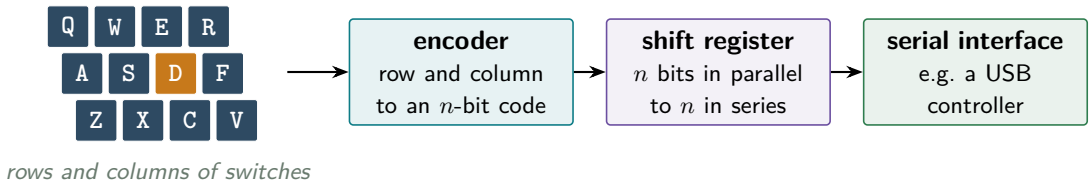


Figure 21.3. The path of a key press.

The keys of a keyboard are switches arranged in rows and columns (Figure 21.3). A key that is pressed joins one row to one column; an *encoder* turns that combination of row and column into a key code of n bits; a *shift register* turns the n bits from parallel to serial; and a *serial interface*, for example a USB controller, sends them to the computer.

Concept

Every block of Figure 21.3 is a machine that the reader has already designed. This is the pay-off of Chapters 11 to 14.

21.3.3 The mouse

The mouse was invented in the 1960s by Douglas Engelbart (the first prototype is of 1964, the patent was filed in 1967 and the famous public demonstration is of 1968): it was the first pointing device in history. It was a wooden box with two wheels underneath and a wire leaving from the back, like the tail of a mouse. The two wheels detected the movements along the horizontal and the vertical axis, and the electronics turned them into electrical signals. In a *mechanical* mouse a ball turns two rollers, each of which drives a counter; in

an *optical* mouse a source of light and a tiny camera measure the movement of the surface underneath.

21.3.4 What a mouse sends

Every time it has moved by a minimum distance, about a hundredth of an inch, called a *mickey*, the mouse sends three bytes: how many units it has moved along x since the last report; the movement along y; and the state of the buttons. The two movements are signed numbers in two's complement (Chapter 5). Modern mice report up to a thousand times per second. Inside, the mouse is two counters, a shift register that puts the three bytes in series, and a serial interface; on the side of the computer the control software, the *device driver*, supervises the communication with the processor.

21.4 Printers

21.4.1 The printer

A printer is an output device that prints information on paper. Its speed is measured in lines per minute, characters per second or pages per minute; its resolution, that is, the quality of the print, in dots per inch. The daisy-wheel and chain printers are now disused; a *dot-matrix* printer strikes an inked ribbon with needles; an *ink-jet* printer throws tiny drops of ink; and then there is the laser printer. A printer is also a small computer: it has its own processor, memory and firmware.

21.4.2 The laser printer

The laser printer derives from the photocopier (Figure 21.4). It works in six stages.

1. A drum is charged electrically, to about 1 000 volts.
2. A laser beam, reflected by a rotating mirror, scans the length of the drum; the beam is modulated so as to produce a pattern of light and dark points.
3. The points struck by the beam lose their charge. After each line the drum turns by a fraction of a degree.
4. The toner is attracted by the points that are still charged: an image of the line is formed.
5. The drum is pressed against the paper and transfers the toner to it; heated rollers fix it permanently.
6. The drum is discharged and cleaned.

The page is first built as a bitmap in the memory of the printer: a frame buffer for paper.

21.5 Back to the machine

21.5.1 The devices of the PRG32 kit

One detail of Table 21.2 was promised in Chapter 10: a switch bounces for a few milliseconds when it is pressed, and sampling it once per frame, every few tens of milliseconds, hides the bounce.

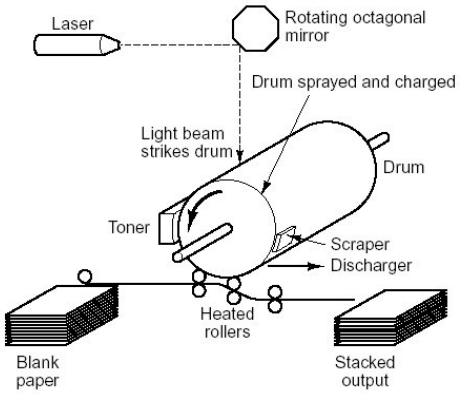


Figure 21.4. The path of the paper in a laser printer.

Table 21.2. The devices of the PRG32 kit.

Device	Direction	How it is connected	How a cartridge uses it
joystick and buttons	input	one switch per input pin (GPIO)	`prg32_input_read`: one bit per button
display, 320 × 240 TFT	output	SPI serial bus, with DMA	the `prg32_gfx_*` calls draw into the frame buffer
amplifier and speaker	output	I2S serial bus	the `prg32_audio_*` calls: notes, samples, music
Wi-Fi radio	input and output	inside the chip	cartridge upload, the Store, multiplayer
USB serial port	input and output	USB	flashing the firmware, debug messages

21.5.2 Three ways to deal with a device

Polling (programmed I/O).
The program asks the device, again and again, whether it has something. It is simple, and it is what a game does once per frame; it is wasteful when nothing happens.

Interrupts.
The device asks for attention when it needs it; the processor suspends what it is doing, serves the device and resumes (Chapter 19).

Direct memory access.
The device exchanges whole blocks of data with the memory by itself; the processor only starts the transfer and is told when it ends (Chapter 15).

Back to the Machine

On the kit all three are at work at the same time: the game polls the joystick, the timer that paces the frames interrupts, and the display and the sound are fed by DMA.

21.6 The program of this chapter

The listings of this chapter are parts of one program, `code/l21.S` in the repository of the course, printed in full in Listing 21.2. The automatic tests of the course material build it

and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 21.5 is the frame that the test wrote.

On the screen.
A yellow square in the middle of the screen. LEFT moves it to the left; A plays a note, and the square keeps moving while the note sounds.

What to press.
LEFT moves the square; A plays a note.

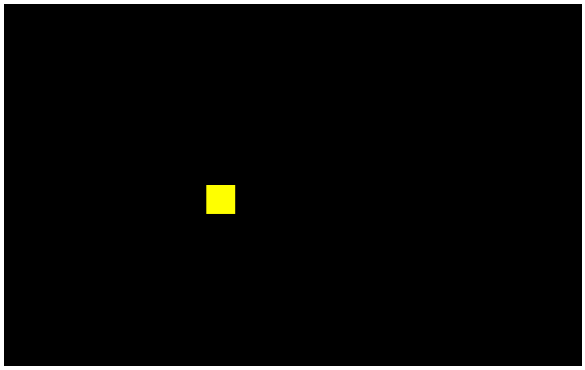


Figure 21.5. `l21.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 21: Input and Output
# l21.S: read the world, once per frame
#
# On the screen: a yellow square in the middle of the screen. LEFT moves it to the
# left; A plays a note, and the square keeps moving while the note sounds.
# Build: python3 -m prg32 cartridge build l21.S --portable \
#            --entry-prefix l21 --name l21 --out l21.prg32
.option norelax

.section .text
.global l21_init
.global l21_update
.global l21_draw

l21_init:
    ret

l21_update:
steer_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)

    call prg32_input_read      # a0 = one bit per button
    mv   s0, a0                # keep it in a saved register

    andi t0, s0, 0x10          # button A
    beqz t0, 1f
    li   a0, 0                 # channel
    li   a1, 0                 # instrument
```

```
    li   a2, 69                # note: A4, 440 Hz
    li   a3, 255               # volume
    li   a4, 120               # milliseconds
    call prg32_audio_note
1:  andi t0, s0, 0x01          # LEFT: s0 survived the call
    beqz t0, 2f
    la   t1, marker_x
    lw   t2, 0(t1)
    addi t2, t2, -2
    sw   t2, 0(t1)
2:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

l21_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0
    call prg32_gfx_clear
    la   t0, marker_x
    lw   a0, 0(t0)
    li   a1, 100
    li   a2, 16
    li   a3, 16
    li   a4, 0xffe0
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .data
marker_x: .word 152
```

Listing 21.2. The complete program of the chapter, `code/l21.S`.

To run it on PRG32-QT the commands are those of Listing 21.3; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l21.S --portable \
    --entry-prefix l21 --name l21 --out l21.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l21.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 21.3. Building `l21.S` and running it on PRG32-QT.

21.7 Check your understanding

Q21.1 What does the encoder of a keyboard controller do?

A. It lights the key that was pressed
B. It turns a row and column combination into a key code
C. It sends the code to the monitor
D. It counts how long a key is held

Q21.2 In a colour LCD, a pixel is made of:

A. one cell that changes colour
B. three cells with red, green and blue filters
C. a phosphor dot hit by electrons
D. a transistor that emits light

Q21.3 Why does a passive-matrix LCD show trails behind moving objects?

A. Its backlight is too weak
B. Its pixels are driven one at a time and must hold their state in between
C. It has too few colours
D. The video card is slow

Q21.4 A PRG32 game reads the joystick once per frame. This technique is called:

A. direct memory access
B. interrupt-driven input
C. polling
D. memory mapping

21.8 Exercises

Exercise 21.1 (paper first, then PRG32)

1. A keyboard has 104 keys: how many bits does the key code need? How many rows and columns could the matrix have?
2. A page of 8 by 11 inches at 600 dots per inch, one bit per dot: how much memory does the printer need for the bitmap?
3. Move a pointer on the PRG32 screen with the joystick, as a mouse would: keep x and y, and clamp them to the screen.
4. Make button **A** act only once per press (edge detection, Chapter 13).
5. Draw a seven-segment score with rectangles, using the transcoder table of Chapter 11.

Expected result. On paper: 7 bits and, for example, 8 rows by 13 columns; $4\,800 \times 6\,600$ dots, that is 31 680 000 bits, about 4 MB. On PRG32: a pointer that stays on the screen, and a button that fires once per press.

21.9 Glossary

English	Italiano
cathode-ray tube	tubo a raggi catodici
liquid crystals	cristalli liquidi
backlight	retroilluminazione
active matrix	matrice attiva
keyboard / key	tastiera / tasto
pointing device	periferica di puntamento
dot-matrix / ink-jet printer	stampante ad aghi / a getto d'inchiostro
drum / toner	tamburo / toner
polling	interrogazione ciclica
device driver	driver di periferica

21.10 Conclusions and reading

Input/output devices are a fundamental component of computers. Display technology has made enormous progress, from the electron beam to a transistor behind every pixel. Keyboard, mouse and printer have changed little in form, and inside they are built from encoders, counters and shift registers. A device reaches the processor through a controller and a bus, and is handled by polling, by interrupts or by DMA. On the PRG32 kit the whole chain, from a switch to a pixel, can be followed instruction by instruction.

Reading. Tanenbaum and Austin (2013), section 2.4 (input/output); Stallings (2021), the chapter on input/output; *Playing with RISC-V*, chapter 6, "State and Input" (Montella et al., 2026b).

The fourth block is complete. The next chapter finishes Verm1, and the one after it publishes the game.

CHAPTER 22

Verm1, Part 4: The Finished Game

"Premature optimization is the root of all evil."
— Donald Knuth, 1974

In this chapter

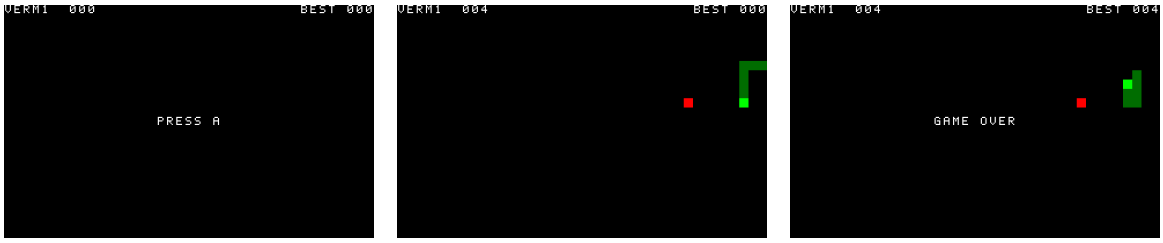
By the end of this chapter you will have

- the score and the best score written in digits, converted by your own division by ten;
- a worm that goes faster as the score grows, up to a limit;
- two sounds: a short note for a meal, a long low one for the end;
- a `draw` function that touches a handful of cells per step instead of the whole screen;
- the finished Verm1, ready to be packaged in Chapter 23.

In this last tutorial on the program the machine itself enters the game. Knuth's sentence is its motto: the game was made to *work* in Part 3, and only now, with a measure in hand, is it made *fast*. Table 22.1 lists what is taken from Chapters 18 to 21; Chapter 4 comes back too, because a digit becomes a character by adding 48, the ASCII code of `0`. The reference file is `verm1_part4.S` (Appendix D).

Table 22.1. What Part 4 takes from the chapters on the machine.

From	Idea	Where it appears
Chapter 18	the calling convention: `a`, `t`, `s` registers, `ra`, the stack	small functions with contracts; leaf functions
Chapter 18	the M extension: `divu` and `remu`, quotient and remainder	the digits of the score
Chapter 19	the processor cycle: every instruction costs time	the measure before and after step 4
Chapter 20	the screen is memory: writing pixels is the cost	draw only what has changed
Chapter 21	devices belong to the runtime: the program asks, the runtime does	`prg32_audio_note`, with a duration



(a) `TITLE`, with the best score **(b)** `PLAY`: the score in digits **(c)** `OVER`

Figure 22.1. Where this chapter is going.

Start from a Part 3 that runs; if yours does not, copy `verm1_part3.S`. The chapter has four steps, and the program runs after each one.

Before you start: measure

Run Part 3 in debug mode and write down what the telemetry shows during a game: retired instructions, frames per second, late frames. The numbers will be needed in step 5.

22.1 Step 1: the score in digits

22.1.1 A number is not its digits

The variable `score` holds a binary number: 137 is `0x89`, one byte. The screen wants three characters, `'1'`, `'3'`, `'7'`, that is, the bytes `0x31`, `0x33`, `0x37`. The conversion is the algorithm of Chapter 5, done by the program instead of by hand: divide by ten, three times.

$$\begin{aligned} 137/10 = 13, &\quad \text{remainder } 7\text{: the units;} \\ 13/10 = 1, &\quad \text{remainder } 3\text{: the tens;} \\ 1/10 = 0, &\quad \text{remainder } 1\text{: the hundreds.} \end{aligned}$$

The digits come out from the right, and are stored from the last character to the first. A digit d becomes a character by adding 48.

22.1.2 `to_decimal`

```
to_decimal:                     # a0 = value 0..999, a1 = address of 3 chars
    li   t0, 10
    remu t1, a0, t0             # the units: the remainder of value / 10
    addi t1, t1, 48             # 48 is the ASCII code of '0'
    sb   t1, 2(a1)
    divu a0, a0, t0
    remu t1, a0, t0             # the tens
    addi t1, t1, 48
    sb   t1, 1(a1)
    divu a0, a0, t0
    remu t1, a0, t0             # the hundreds
    addi t1, t1, 48
    sb   t1, 0(a1)
    ret                         # a leaf function: no stack frame needed
```

Listing 22.1. Step 1 **(new)**: `to_decimal`.

The contract of Listing 22.1 is: `a0` = value, `a1` = where the three characters go. `remu` gives the digit and `divu` what is left: two instructions of the M extension. `sb` stores a byte at the offsets 2, 1 and 0 from `a1`. The function is a leaf that uses only `t` registers: it needs no frame at all. Values above 999 would show only their last three digits, a limit that no player will meet soon. The division instructions are slow, tens of clock cycles each on a small processor; ten of them per step of the worm are nothing.

22.1.3 `draw_score`: two numbers, two texts

```
draw_score:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, score
    lw   a0, 0(t0)
    la   a1, score_txt
    call to_decimal             # the score, as three characters
    li   a0, 56
    li   a1, 0
    la   a2, score_txt
    call draw_text
    la   t0, best
    lw   a0, 0(t0)
    la   a1, best_txt+5         # after the letters "BEST "
    call to_decimal
    li   a0, 256
    li   a1, 0
    la   a2, best_txt
    call draw_text
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.2. Step 1 **(changed)**: `draw_score`.

Replace `draw_score` of Part 3 with Listing 22.2: the marks go. The digits are written into a string in RAM, and then the string is drawn like any other; the address `best_txt+5` skips the word `BEST` and the space, five characters. Text is drawn with its background colour, so that new digits cover the old ones without any clearing.

22.1.4 The variables of Part 4

```
.section .data                  # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0              # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0              # lost when the power goes
```

```
food_x:     .word 0
food_y:     .word 0
head_i:     .word 0              # ring index of the head
length:     .word 0              # number of cells of the worm
body_x:     .space 256           # the ring: one byte per segment
body_y:     .space 256
grid:       .space COLS*ROWS     # one byte per cell: 1 = worm
dirty:      .word 2              # 0 nothing to draw, 1 a step, 2 everything
old_x:      .word 0              # where the head was before the step
old_y:      .word 0
erase_on:   .word 0              # 1 if the tail left a cell
erase_x:    .word 0
erase_y:    .word 0
score_txt:  .asciz "000"         # filled in by to_decimal
best_txt:   .asciz "BEST 000"
```

Listing 22.3. Step 1 **(changed)**: `.data`.

Replace the `.data` of Part 3 with Listing 22.3, and type all of it now. The two strings `score_txt` and `best_txt` are in `.data` and not in `.rodata`, because the program writes into them. The six variables from `dirty` to `erase_y` are not used until step 3.

Run it: digits

1. Build and run, press **A**: the top line reads `VERM1 000` and, on the right, `BEST 000`.
2. Play and eat: `001`, `002`, ...
3. Lose: on the game-over screen `BEST` shows your score.
4. In the debugger find `score_txt` in memory and read its bytes: `30 30 33 00` after three meals.

22.2 Step 2: levels

The speed becomes a function of the score.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_MIN, 32          # 32/256 of a cell per frame at the start
.equ SPEED_MAX, 96          # the worm never goes faster than this
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
.equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
```

```
.equ DIR_DOWN,  1
.equ DIR_LEFT,  2
.equ DIR_UP,    3
.equ TITLE, 0               # the three states of the game
.equ PLAY,  1
.equ OVER,  2
```

Listing 22.4. Step 2 **(changed)**: constants. Only the two `SPEED` lines differ from Part 3.

`SPEED_Q8` is replaced by a minimum and a maximum (Listing 22.4): 32/256, one step every 8 frames, at the start, and 96/256, three steps every 8 frames, at most.

```
    la   t0, score
    lw   t3, 0(t0)
    slli t3, t3, 2
    addi t3, t3, SPEED_MIN      # speed = 32 + 4 * score, in 1/256 of a cell
    li   t4, SPEED_MAX
    ble  t3, t4, 8f
    mv   t3, t4                 # ... but never more than 96
8:  la   t0, step_q8
    lw   t1, 0(t0)
    add  t1, t1, t3             # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.5. Step 2 **(changed)**: the timer at the end of `play_update`.

The speed is $32 + 4 \times \mathit{score}$: a shift and an add (Listing 22.5). Then `ble` and `mv` take the smaller of the speed and 96. It is a *saturation*: the value follows a line and then stays at the limit, like the clamp of Chapter 5. The limit is reached at 16 points (Table 22.2). With fixed point the speed changes by a sixty-fourth of a cell per frame at each meal: no integer speed could be so fine.

Table 22.2. The levels, in numbers. 256/*speed* gives the frames per step.

Score	Speed (1/256 cell/frame)	Frames per step	Cells/s at 30 fps
0	32	8	3.75
4	48	5.3	5.6
8	64	4	7.5
12	80	3.2	9.4
16 and more	96	2.7	11.25

Fractions of a frame are real: at 48/256 the steps come every 5 or 6 frames, 5.33 on average. Compute the row of score 2 as an exercise.

Run it: faster and faster

1. Build and run: the worm starts more slowly than in Part 3, and gains speed at every meal.
2. Count with the debugger the frames between two steps at score 0: eight.
3. Reach 16 points: the speed no longer changes.
4. Try `SPEED_MAX` = 256, if you dare.

22.3 Step 3: `step` tells what it changed

This step adds the sounds, and the notes that `draw` will need in step 4.

22.3.1 `play_note`: one function, any note

```
play_note:                      # a2 = MIDI note, a4 = milliseconds
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a3, 255                # volume
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.6. Step 3 **(changed)**: `play_note` replaces `eat_sound`.

The caller chooses the note and the duration, in `a2` and `a4`; the function fills in the other three arguments (Listing 22.6). The arguments stay in the registers where `prg32_audio_note` expects them: designing a contract round the callee saves moves.

22.3.2 `step`: what it leaves for `draw`

The function `step` of Part 3 is changed in five places.

```
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
    la   t0, old_x
    sw   s0, 0(t0)              # remember where the head was
    add  s0, s0, t2
    bgez s0, 1f
    li   s0, COLS-1             # off the left edge: wrap
1:  li   t4, COLS
    blt  s0, t4, 2f
    li   s0, 0
2:  la   t0, body_y
    add  t0, t0, t5
    lbu  s1, 0(t0)
    la   t0, old_y
```

```
    sw   s1, 0(t0)
    add  s1, s1, t3
    bgez s1, 3f
    li   s1, ROWS-1
3:  li   t4, ROWS
    blt  s1, t4, 4f
    li   s1, 0
4:
```

Listing 22.7. Step 3 **(changed)**: `step` remembers the old head.

Two stores are added (Listing 22.7): `old_x` and `old_y`, the cell that the head is leaving, which `draw` will repaint in the colour of the body.

```
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, s0
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, s1
    or   s2, t1, t2            # s2 = 0 if the new head is on the food
    la   t0, length
    lw   t1, 0(t0)
    bnez s2, 5f                # not eating: the tail moves on
    li   t2, 255
    bge  t1, t2, 5f            # the ring is full: no more growth
    addi t1, t1, 1
    sw   t1, 0(t0)             # eating: one cell longer, the tail stays
    la   t0, erase_on
    sw   zero, 0(t0)           # ... so there is no cell to erase
    j    6f
5:  la   t0, head_i
    lw   t2, 0(t0)
    sub  t2, t2, t1
    addi t2, t2, 1
    andi t2, t2, 255           # ring index of the tail
    la   t0, body_x
    add  t0, t0, t2
    lbu  t3, 0(t0)
    la   t0, body_y
    add  t0, t0, t2
    lbu  t4, 0(t0)
```

Listing 22.8. Step 3 **(changed)**: `step`, when the worm eats.

When the worm eats, the tail stays and there is nothing to erase: `erase_on` $= 0$ says so to `draw` (Listing 22.8).

```
    la   t0, erase_x
    sw   t3, 0(t0)             # remember the cell the tail leaves
    la   t0, erase_y
    sw   t4, 0(t0)
    la   t0, erase_on
    li   t5, 1
    sw   t5, 0(t0)
```

```
    li   t5, COLS
    mul  t4, t4, t5
    add  t4, t4, t3            # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)           # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0            # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                # a free cell: go on
    call game_is_over          # the worm bit itself
    j    9f
```

Listing 22.9. Step 3 **(changed)**: `step` notes the tail, and calls the end.

When it does not eat, the tail leaves a cell, and `step` remembers which: `erase_on` = 1 means that `draw` must paint it black (Listing 22.9). The end of this fragment is shorter than in Part 3, because the collision is now one call, to `game_is_over`.

```
7:  li   t1, 1
    sb   t1, 0(t0)             # occupy the new cell
    la   t0, head_i
    lw   t1, 0(t0)
    addi t1, t1, 1
    andi t1, t1, 255           # the ring wraps after entry 255
    sw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    sb   s0, 0(t0)             # write the new head into the ring
    la   t0, body_y
    add  t0, t0, t1
    sb   s1, 0(t0)
```

Listing 22.10. Step 3 **(changed)**: `step` writes the new head (as in Part 3).

```
    la   t0, dirty
    li   t1, 1
    sw   t1, 0(t0)             # the screen must follow: a few cells
    bnez s2, 9f
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    li   a2, 72                # a short high note
    li   a4, 60
    call play_note
9:  lw   s2, 0(sp)
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
```

```
    addi sp, sp, 16
    ret
```

Listing 22.11. Step 3 **(new)**: the end of `step`.

At the end (Listing 22.11) `dirty` = 1 records that a step has happened and that `draw` has work to do. A meal is a point, new food and a short high note: note 72 for 60 milliseconds, the sound of Part 2 through the new function.

22.3.3 `game_is_over`: the transition, in one place

```
game_is_over:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)              # PLAY --> OVER
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    la   t0, score
    lw   t1, 0(t0)
    la   t0, best
    lw   t2, 0(t0)
    ble  t1, t2, 1f
    sw   t1, 0(t0)              # a new best score
1:  li   a2, 36                 # a long low note
    li   a4, 400
    call play_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.12. Step 3 **(new)**: `game_is_over`.

A transition of the state machine deserves a function of its own (Listing 22.12): the state, the outputs and the sound are in one place. The function stores `OVER`, updates the best score as in Part 3, sets `dirty` = 2 because a new screen must be drawn, and plays a long low note: 36, for 400 milliseconds.

Run it: the sound of defeat

1. Build and run: the game looks exactly as after step 2.
2. A meal sounds as before; biting yourself gives a long low note.
3. In the debugger watch `erase_x` and `erase_y` follow the tail, one step behind.
4. `dirty` stays at 1 or 2: nobody reads it yet.

Expected result. No visible change: the screen is still drawn completely at every frame. The new variables are written and not yet used.

22.4 Step 4: draw only what changed

22.4.1 What does a frame cost?

Part 3 clears 64 000 pixels and redraws the title, the score, the whole worm and the food, thirty times per second. But between two steps nothing on the screen changes, and a step changes very little: at most the cell that the tail has left, which becomes black; the old head, which becomes body; the new head; and perhaps the food and the score.

Concept

Remember what changed, and draw only that. The picture stays on the screen by itself: the frame buffer is memory (Chapter 20).

A worm of 200 cells costs 200 rectangles per frame in Part 3, and three per step in Part 4. This is what the old home computers did, because they had no choice.

22.4.2 `draw`: nothing, a little, or everything

```
verm1_draw:
    la   t0, dirty
    lw   t1, 0(t0)
    beqz t1, 3f                 # nothing changed: nothing to draw
    sw   zero, 0(t0)
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   t2, 2
    beq  t1, t2, 1f
    call draw_changes           # one step: a few cells
    j    2f
1:  call draw_full              # a new screen: everything
2:  lw   ra, 12(sp)
    addi sp, sp, 16
3:  ret
```

Listing 22.13. Step 4 **(changed)**: `verm1_draw`.

The variable `dirty` has three values (Listing 22.13). With 0 the function returns at once, with no frame and no call: the common case, in which nothing has changed, costs five machine instructions. With 1 a step has happened, and the changes are drawn. With 2 there is a new screen, and everything is drawn. The flag is cleared as soon as it is read.

22.4.3 `draw_changes`: three cells and the score

```
draw_changes:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, erase_on
    lw   t1, 0(t0)
    beqz t1, 1f
    la   t0, erase_x
    lw   a0, 0(t0)
    la   t0, erase_y
    lw   a1, 0(t0)
    li   a2, BLACK
```

```
    call draw_cell              # the cell the tail has left
1:  la   t0, old_x
    lw   a0, 0(t0)
    la   t0, old_y
    lw   a1, 0(t0)
    li   a2, BODY
    call draw_cell              # the old head is now body
    call draw_head
    call draw_food
    call draw_score
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.14. Step 4 **(new)**: `draw_changes`.

The order matters (Listing 22.14): first the cell that the tail has left, if any, in black; then the old head, in the colour of the body; then the new head, the food and the score. Since the head may enter the cell that the tail has just left, erasing first and drawing the head afterwards gives the right picture.

22.4.4 `draw_full`: a whole screen

```
draw_full:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    call draw_score
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
    call draw_text
    j    2f
```

Listing 22.15. Step 4 **(new)**: `draw_full`, first half.

```
1:  call draw_worm              # PLAY and OVER: the whole scene
    call draw_food
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
    la   a2, game_over
```

```
    call draw_text
2:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.16. Step 4 **(new)**: `draw_full`, `PLAY` and `OVER`.

`draw_full` is the body of the old `verm1_draw`, and is called once for each change of state (Listings 22.15 and 22.16). The score is drawn on the title screen too.

22.4.5 Three small functions

```
draw_head:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_i
    lw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.17. Step 4 **(new)**: `draw_head`.

```
draw_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.18. Step 4 **(new)**: `draw_food`.

```
draw_text:                      # a0 = x, a1 = y, a2 = address of the string
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   ra, 12(sp)
```

```
    addi sp, sp, 16
    ret
```

Listing 22.19. Step 4: `draw_text`, the helper of Part 3.

22.4.6 Every transition asks for a screen

Three small additions complete the mechanism: wherever the state changes, `dirty` becomes 2.

```
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # draw the whole title screen, once
    ret
```

Listing 22.20. Step 4 **(changed)**: `verm1_init` asks for the first screen.

```
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    j    u_done
u_start:
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.21. Step 4 **(changed)**: `verm1_update`, from `OVER` to `TITLE`.

```
    la   t0, head_i
    li   t1, 2
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.22. Step 4 **(changed)**: the end of `new_game`, from `TITLE` to `PLAY`.

Watch Out

Every transition of the machine asks for a full drawing; every step asks for a partial one. Forgetting one of them is the typical bug of this technique: the screen shows a state that is no longer true.

Run it: the finished Verm1

The same game as after step 3, pixel for pixel (Figures 22.1b and 22.1c). If you see trails or holes, a change was not recorded: use the debugger.

22.5 Step 5: measure

PRG32-QT is now used as an instrument: count before you believe.

Before and after, with numbers

1. Run the finished game in debug mode and start a game.
2. Read the telemetry during ten seconds of play: retired instructions, frames per second, late frames.
3. Compare with the numbers written down for Part 3 at the beginning of the chapter.

4. Let the worm grow to twenty cells in both versions and read again: one number grows with the worm, the other does not.
5. Put a breakpoint on the first instruction of `draw_changes`: it stops once per step, not once per frame.
6. Write the numbers in your notes: they go in the description of the game in Chapter 23.

The numbers are deliberately not printed here: they must be measured. One caution in reading them: the telemetry counts the instructions of the cartridge, not those of the runtime that fills the rectangles, so that the real saving on the board is larger than the count shows; there every pixel also travels over SPI to the display.

22.5.1 Follow a call with the debugger

The calling convention of Chapter 18 can be watched instruction by instruction.

1. Break on `call to_decimal` inside `draw_score`, and read `a0`, `a1`, `ra` and `sp`.
2. Step once: the PC jumps into `to_decimal`, and `ra` now holds the address of the instruction after the call.
3. Step through: only `t` registers and `a0` change, and `sp` never moves. It is a leaf function.
4. At `ret` the PC takes the value of `ra`: the program is back in `draw_score`.
5. Do the same with `call draw_text`: `sp` goes down by 16, `ra` is stored at `12(sp)`, and comes back.

In the listing of the debugger `call` appears as `jal`, or as `auipc` followed by `jalr`, and `ret` as `c.jr ra`.

22.6 Check your understanding

Q22.1 `to_decimal` receives `a0` = 205. Which bytes does it store, from offset 0 to 2?

A. `02 00 05`
B. `32 30 35`
C. `35 30 32`
D. `cd 00 00`

Q22.2 At which score does the worm take one step every 4 frames?

A. 4
B. 8
C. 16
D. 32

Q22.3 `dirty` is 0 when `verm1_draw` is called. How many machine instructions does it execute?

A. 1
B. 5

C. about 50
D. thousands

Q22.4 Why can `to_decimal` do without a stack frame?

A. it is short
B. it calls no function and uses only `a` and `t` registers
C. it has no arguments
D. frames are optional

22.7 Make it yours

Exercise 22.1 (on PRG32)

1. A pause: button **B** stops the game and writes `PAUSE`; **B** again resumes. A fourth state.
2. A melody on the title screen: a table of notes in `.rodata`, one note every 15 frames.
3. Bonus food: every fifth piece is yellow, is worth three points, and disappears after 100 frames.
4. Your own colours with `prg32_palette_set`, or your own rules: this is your game now.

Expected result. The version published in Chapter 23 should have something that is yours. Change the title string, too. (Hints: the pause is a state `PAUSED` with transitions on the edge of **B**, `0x20`, and `dirty` = 2 at both transitions. The melody needs an index and a frame counter in `.data`; the table is read like `dx`.)

22.8 The functions that did not change

Eight parts of the file are those of Part 3 and are kept as they are. They are reprinted here so that this chapter contains the whole program.

```
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
```

Listing 22.23. Unchanged: `verm1_update`, buttons and edges.

```
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
```

```
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
```

Listing 22.24. Unchanged: `new_game`, the map and the worm.

```
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
```

Listing 22.25. Unchanged: `play_update`, the steering.

```
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
```

```
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
```

Listing 22.26. Unchanged: `step`, the table lookup.

```
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
1:  li   a0, 0
    li   a1, COLS-1
    call prg32_random_number
    mv   s0, a0                 # s0 = a column
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row
    li   t1, COLS
    mul  t2, a0, t1
    add  t2, t2, s0
    la   t0, grid
    add  t0, t0, t2
    lbu  t3, 0(t0)
    bnez t3, 1b                 # on the worm: try again
    la   t0, food_x
    sw   s0, 0(t0)
    la   t0, food_y
    sw   a0, 0(t0)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.27. Unchanged: `place_food`.

```
draw_worm:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: segment 0 is the head
1:  la   t0, length
    lw   t1, 0(t0)
    bge  s0, t1, 3f
    la   t0, head_i
    lw   t1, 0(t0)
```

```
    sub  t1, t1, s0
    andi t1, t1, 255            # ring index of segment i
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, BODY
    bnez s0, 2f
    li   a2, GREEN              # the head is brighter
2:  call draw_cell
    addi s0, s0, 1
    j    1b
3:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.28. Unchanged: `draw_worm`, used by `draw_full` only.

```
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listing 22.29. Unchanged: `draw_cell`.

```
.section .rodata                # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
```

Listing 22.30. Unchanged: `.rodata`.

22.9 Where we are, and what comes next

Verm1 is finished: about six hundred lines of RISC-V assembly, all written by the reader. It began as three functions that return, and grew one runnable step at a time. Every chapter of the book is in it: numbers, logic, state, memory, calls, pixels, devices.

A program that only its author can run is half finished. The next chapter gives it a name,

an icon, credits, a bundle for two architectures, and the Cartridge Store.

Part VI

Publication and Outlook

CHAPTER 23

Verm1 on the Cartridge Store

"Real artists ship."
— Steve Jobs, to the Macintosh team, 1983

In this chapter

By the end of this chapter you will have

- your Verm1 built for two architectures, the ESP32-C6 kit and QEMU, from the same source;
- a manifest that gives the game an identity: id, title, version, authors, licence;
- an icon, and a colophon with credits and controls;
- a bundle, one ZIP file, checked and ready for the Cartridge Store;
- a clear idea of what happens after you publish, and of what you are responsible for.

This is the last chapter of the project, and it is a tutorial again: at its end the reader has a bundle, `verm1-1.0.0.zip`, checked and ready for submission. Everything up to the upload was run on the reference game by the tests of the course material; the upload itself is a personal act made with a personal token, and was not run for this book.

23.1 Why publish?

A program that runs only on its author's computer is an exercise; one that others can install is a product. Publishing forces the questions that an exercise lets one skip: what is it called, who made it, on what does it run, what may others do with it. The Cartridge Store (Figure 23.1) is the shelf of PRG32: games and programs that install on the kit, on QEMU and on PRG32-QT. Every cartridge there was written by someone who started where the reader started, and the reader's can be next to them. The Store is at https://store.prg32.uniparthenope.it, and PRG32-QT has a Store browser of its own.

The path from the source to the shelf has six stations (Figure 23.2). The first four happen on the author's computer: nothing leaves it before the command that publishes.

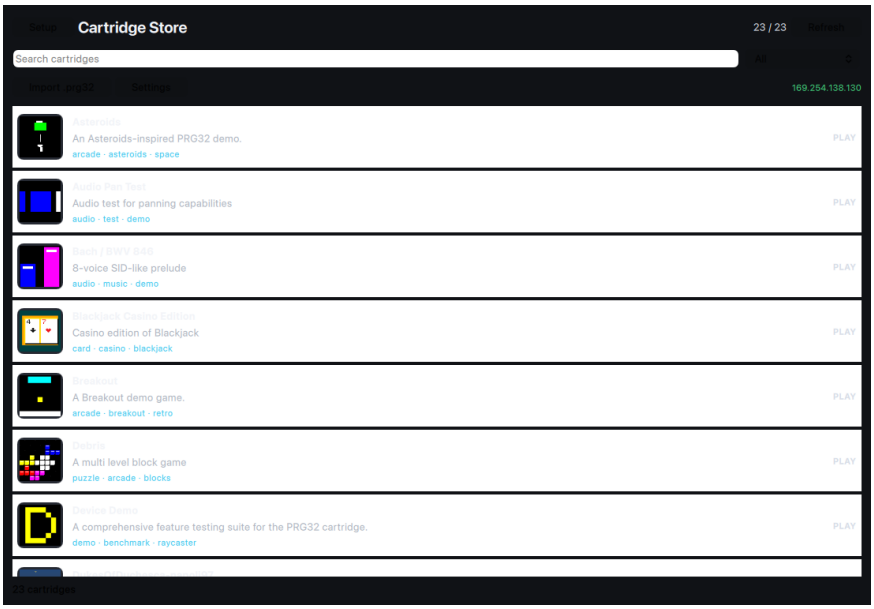


Figure 23.1. The Cartridge Store, seen from PRG32-QT.

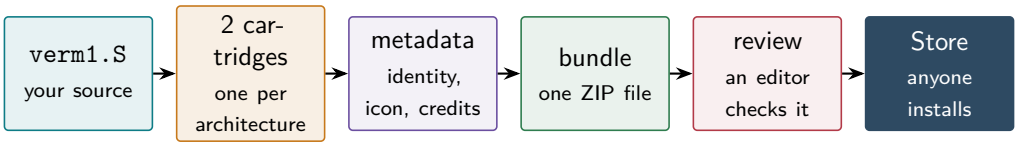


Figure 23.2. From source to shelf.

23.1.1 Before you package: is it finished?

The debugger is used one last time, as an instrument of quality (Table 23.1). A cartridge that was never run on a target should not be published for that target.

23.2 Step 1: one source, two cartridges

```
cd ~/PRG32          # your PRG32 folder, toolchain active

python3 -m prg32 cartridge build ~/verm1/verm1.S --portable \
    --entry-prefix verm1 --name verm1 \
    --architecture esp32c6 --out ~/verm1/verm1-esp32c6.prg32

python3 -m prg32 cartridge build ~/verm1/verm1.S --portable \
    --entry-prefix verm1 --name verm1 \
    --architecture qemu --out ~/verm1/verm1-qemu.prg32
```

Listing 23.1. Building for the kit and for QEMU.

Listing 23.1 is the command of every chapter with one more option: `--architecture` names the target of the file. The names follow a rule, `game-architecture.prg32`. The builder answers with the name, the load address `0x40800000` and the size of the code: the finished Verm1 is about 6 kilobytes (6 216 bytes for the reference game).

Table 23.1. Is the game finished?

Check	How	With
It starts, plays and ends	play three whole games, lose in three different ways	PRG32-QT
No frame is late	telemetry during a long game: late frames stay at zero	PRG32-QT, debug mode
Memory is consistent	pause at random and compare the ring, the map and the screen	PRG32-QT, memory view
It runs on the second architecture	build for QEMU, upload, play	QEMU
It runs on real hardware	upload to a kit, play with the real joystick and speaker	the kit, if you have one
It is yours	a title, a rule or a look that the reference does not have	you

```
python3 -m prg32 cartridge summary ~/verm1/verm1-esp32c6.prg32

{
  ...
  "import_model": "abi-table",
  ...
}
```

Listing 23.2. Is it portable?

The command `summary` prints what the file declares about itself (Listing 23.2). The value `abi-table` of `import_model` means that the cartridge reaches the runtime through a table of functions, the ABI of PRG32, of which `prg32_gfx_rect` and the others are entries. Such a cartridge does not depend on one build of the firmware: it is *portable*, and the Store accepts portable cartridges. A cartridge that is not portable is linked against the addresses of one firmware image, and breaks at the next update. Repeat the check on the file for QEMU.

23.3 Step 2: an identity

23.3.1 `manifest.json`: who is this game?

Create the file of Listing 23.3 in `~/verm1` with a text editor. (The description is cut in the listing; the complete file of the reference game is in Appendix D.)

```
{
  "abi": "prg32-metadata-1.0",
  "id": "it.uniparthenope.arc12.verm1",
  "title": "Verm1",
  "version": "1.0.0",
  "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
  "description": "Verm1 (say \"vermone\") is the course-long project of ...",
  "authors": [{"name": "Your Name"}],
  "license": "MIT",
  "tags": ["arcade", "snake", "assembly", "education"],
```

```
  "assets": {"icon": "icon.png"},
  "architectures": [
    {"id": "esp32c6", "file": "verm1-esp32c6.prg32"},
    {"id": "qemu", "file": "verm1-qemu.prg32"}
  ]
}
```

Listing 23.3. `manifest.json`.

The line `abi` says which version of the metadata format this is. The other fields are commitments (Table 23.2).

Table 23.2. The fields of the manifest, and what they commit their author to.

Field	Rule	For Verm1
`id`	by custom a reverse domain name; it never changes	one per author: put your own name in it
`version`	major.minor.patch; a published version is never replaced	1.0.0, then 1.0.1 for a fix
`title`	what the player reads	Verm1, or the name of your variant
`authors`	the people who wrote it	you, and whoever worked with you
`license`	what others may do with it	MIT, if you want others to learn from it
`tags`	how it will be found	arcade, snake, assembly, education

Watch Out

Treat an id and a version, once published, as permanent. A mistake is corrected by a new version, not by a replacement. And two authors cannot publish the same id: each uses an id of their own, for example `it.uniparthenope.arc12.`*surname*`.verm1`.

23.3.2 An icon

The icon is a small PNG that represents the game on the shelf (Figure 23.3). The reference icon is 64 by 64 pixels: a worm and a piece of food, in the colours of the game. Pixel art suits a machine of 320 by 200 pixels: draw it large, cell by cell, with any editor that saves PNG, or draw it with a program, as the reference icon was. Save it as `icon.png` next to the manifest, which names it under `assets`. An icon taken from the web is somebody else's work: draw your own.

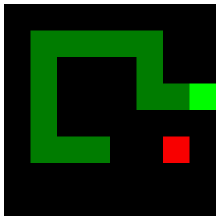


Figure 23.3. `icon.png`, enlarged.

23.3.3 `colophon.json`: credits and controls

```
{
  "abi": "prg32-colophon-1.0",
  "title": "Verm1",
  "subtitle": "A tribute to Snake, in RISC-V assembly",
  "version": "1.0.0",
  "release_date": "2026-12-18",
  "developer": {"name": "Your Name"},
  "authors": [
    {"role": "game design", "name": "Your Name"},
    {"role": "programming", "name": "Your Name"}
  ],
  "license": "MIT",
  "copyright": "Copyright 2026 Your Name",
  "acknowledgements": [
    "Giuseppe Coviello, reference implementation of Verm1",
    "ARC12 Computer Architecture, University of Naples Parthenope",
    "The PRG32 community"
  ],
  "controls": [
    {"input": "JOYSTICK", "action": "Steer the worm"},
    {"input": "A", "action": "Start, and return to the title after game over"}
  ],
  "start_prompt": "Press A to play"
}
```

Listing 23.4. `colophon.json`.

Colophon is the printers' word for the note at the end of a book that says who made it. Here it is the credits page of the cartridge (Listing 23.4): roles and names, giving credit to everyone; the controls, that is, what each input does, for the player; the acknowledgements. Replace `Your Name` everywhere, and set the release date. The first acknowledgement stays: a game that starts from the reference implementation of Verm1, contributed by Giuseppe Coviello, says so.

23.3.4 A tribute is not a copy

Verm1 is a tribute to Snake: the idea of a growing worm is as old as *Blockade* (1976) and is free for all. The name, the code, the icon and the sounds are original: nothing was taken from another game. The rules of a game are not protected; code, pictures, music and names are. Before publishing, ask of every piece: did I write or draw this myself? If not, does its licence allow me to publish it, and did I give credit? The names and the characters of commercial games are trademarks, and must not be used. The licence chosen tells others what they may do with the work: MIT lets them study, change and share it, keeping the author's name.

The rule

Your name goes on it. Publish only what you would sign.

23.4 Step 3: the cartridge carries its papers

```
# metadata.json = manifest.json WITHOUT the "assets" and "architectures"
    entries

python3 -m prg32 store attach-metadata ~/verm1/verm1-esp32c6.prg32 \
    --metadata ~/verm1/metadata.json --icon ~/verm1/icon.png \
    --colophon ~/verm1/colophon.json --architecture esp32c6 \
    --out ~/verm1/verm1-esp32c6.prg32

python3 -m prg32 store attach-metadata ~/verm1/verm1-qemu.prg32 \
    --metadata ~/verm1/metadata.json --icon ~/verm1/icon.png \
    --colophon ~/verm1/colophon.json --architecture qemu \
    --out ~/verm1/verm1-qemu.prg32
```

Listing 23.5. `metadata.json`, and attaching it.

The file `metadata.json` is a copy of the manifest without the two entries `assets` and `architectures`. The command `attach-metadata` writes title, icon and credits *inside* the cartridge, once per file and with its own architecture (Listing 23.5): a cartridge copied by hand still says what it is. The tool answers, for the reference game, `legacy=6316 blocks=3`: the three blocks are the metadata, the icon and the colophon, appended after the program, which is itself untouched.

Watch Out

Giving the full manifest to `--metadata` makes the tool fail, because of the entry `assets`.

```
python3 -m prg32 store inspect-metadata ~/verm1/verm1-esp32c6.prg32

{
  ...
    "id": "it.uniparthenope.arc12.verm1",
    "license": "MIT",
    "runtime": { "architecture": "esp32c6", ... },
    "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
    "title": "Verm1",
    "version": "1.0.0"
  },
  ...
  "trailer_present": true,
  "unknown_blocks": []
}
```

Listing 23.6. Reading the metadata back (output abridged).

Concept

Never trust a write that you have not read back. It is the habit of the whole book: the memory view of the debugger, and now this.

Check the id, the title, the version and the architecture (Listing 23.6); `trailer_present` says that the papers are in the file. Then run the cartridge once more: it must still work.

23.5 Step 4: the bundle

```
python3 -m prg32 store pack-bundle \
    --manifest ~/verm1/manifest.json --out ~/verm1/verm1-1.0.0.zip

Bundle: ~/verm1/verm1-1.0.0.zip
  manifest.json                  0.7 KB
  icon.png                       0.1 KB
  verm1-esp32c6.prg32            7.8 KB
  verm1-qemu.prg32               7.8 KB
Total: 16.4 KB
```

Listing 23.7. Packing the bundle (real output for the reference game).

The tool reads the manifest and collects the files that it names (Listing 23.7), which must be in the same folder as the manifest. A missing or misnamed file stops it here, and not at the Store. The name of the bundle carries the version. The whole game, for two machines, is 16 kilobytes.

Run it: test the bundle, not your folder

1. Unzip `verm1-1.0.0.zip` into an empty folder.
2. In PRG32-QT import `verm1-esp32c6.prg32` from that folder and run it: title, icon and credits must be yours.
3. Upload `verm1-qemu.prg32` to QEMU and play one game.
4. If you have a kit, upload the ESP32-C6 file and play with the real joystick.
5. Ask a colleague to do the same on another computer: the first user who is not you.

Expected result. The game runs from the files in the bundle, on every target declared, on a computer that is not yours. Testing the packaged files, and not those in the working folder, catches the classic mistake: a bundle built before the last fix.

23.6 Step 5: publication

```
# 1. sign in to the Store and create a token on the Tokens page
# 2. keep it in the environment of your shell: never in a file, never in Git
export PRG32_STORE_URL="https://store.prg32.uniparthenope.it"
export PRG32_STORE_TOKEN="paste-your-token-here"

# 3. submit
python3 -m prg32 store publish-bundle ~/verm1/verm1-1.0.0.zip \
    --store-url "$PRG32_STORE_URL" --token "$PRG32_STORE_TOKEN"
```

Listing 23.8. Publishing: a token and one command.

To publish, the author signs in to the Store, creates a *token* on the *Tokens* page, keeps it in the environment of the shell and submits the bundle (Listing 23.8).

The token is you

Whoever has the token publishes in your name. It is a secret: it does not go in the source, in a file, in a repository or in a screenshot. If it is lost or shown by mistake, revoke it and make a new one.

23.6.1 What happens next

Pending.
The bundle is stored as a submission: it is not yet on the shelf.

Review.
An editor checks that it installs, that it runs, that it is described honestly and that it contains nothing that is not the author's.

Published.
The game appears in the Store, for the architectures declared.

A bug?
Fix it, raise the version to 1.0.1, pack and submit again: 1.0.0 stays as it was.

Moderation protects the players and the authors. A rejection comes with a reason, and is answered with a new version.

23.6.2 The submission checklist

A checklist is not bureaucracy: it is how pilots and surgeons avoid the mistakes that experts make.

1. The game was played to the end on every architecture in the manifest.
2. `cartridge summary` says `import_model:  abi-table` for every file.
3. The id is yours and unique; the version is new; title, summary and description say what the game is.
4. Authors, licence and colophon are complete; every piece is yours or credited.
5. `inspect-metadata` shows the right id, version and architecture in each cartridge.
6. The bundle was unzipped and tested on another computer.
7. The token is in no file, no screenshot and no repository.

23.7 Check your understanding

Q23.1 You published version 1.0.0 and found a bug. What do you do?

A. upload 1.0.0 again
B. publish 1.0.1
C. change the id
D. ask for the old file to be edited

Q23.2 Why does `metadata.json` not contain the `architectures` entry?

A. it is optional
B. the metadata goes inside one cartridge, which has one architecture

C. QEMU does not support it
D. it would be too long

Q23.3 Where should the Store token be kept?

A. in `verm1.S`, as a comment
B. in the manifest
C. in an environment variable of your shell
D. in the `README` of the repository

23.8 Your release

Exercise 23.1 (the deliverable of the project)

1. Choose your id and your title, and write the manifest and the colophon for your variant of Verm1.
2. Draw your icon.
3. Build, attach, inspect, pack: produce your bundle, and pass the checklist with a colleague as reviewer.
4. If you want your game on the shelf, publish it.

Expected result. A bundle of your own game, reviewed by a colleague against the checklist. Publication is optional and personal; the bundle, with the source of your variant, is the deliverable of the project, and the oral examination can start from any line of it.

23.9 What you built

A game of six hundred lines of assembly, from three functions that only returned. With it, every layer of the machine: bits and numbers, gates and tables, state and memory, the processor, the screen, the devices. A method: one runnable step at a time, and a look inside whenever something is surprising. And the last step, which makes it engineering: a product with a name, a version and an author.

The next game is the reader's to choose. The next chapter shows where the same instructions run at scale.

CHAPTER 24

Beyond One Core: Superscalar RISC-V, HPC, GPUs and FPGAs

"If you were plowing a field, which would you rather use: two strong oxen or 1024 chickens?"
— attributed to Seymour Cray

In this chapter

By the end of this chapter you will be able to

- write the performance equation and say which factor each technique attacks;
- explain how a superscalar, out-of-order processor executes several instructions per cycle, and name RISC-V cores that do;
- read a loop written with the RISC-V vector extension;
- describe a supercomputer, compute a speed-up with Amdahl's law, and say why energy is the limit;
- say what a GPU and an FPGA are good at, and what each costs the programmer.

This closing chapter is an outlook. It starts from one loop of Verm1 and follows it onto four kinds of machine: the same instructions, on machines that execute many of them at once. It is not part of the midterm test. The RISC-V vector listings were tested on QEMU; the CUDA and Verilog fragments are standard textbook forms that were not compiled for this book. The figures for supercomputers are rounded and come from the TOP500 lists of 2022 to 2025.

24.1 Warm-up on the machine

```
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # one byte per turn of the loop
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
```

Listing 24.1. The loop that empties the map, from `new_game` of Verm1.

Listing 24.1 sets 960 bytes to zero: four instructions per byte, 3 840 instructions in all, one byte at a time and one instruction at a time. At best one instruction per clock cycle and 160 million cycles per second on the kit: about 24 microseconds. Nobody needs this loop to be faster; but it is small enough to be followed onto every machine of this chapter. How many ways are there of making it faster?

24.1.1 The performance equation

Theorem 24.1 (Performance equation)

The time that a program takes is the product of three numbers,

$$T = N \times CPI \times t,$$

where N is the number of instructions executed, CPI the average number of clock cycles per instruction and t the duration of a clock cycle.

For the loop above on the kit: $N = 3\,840$, about 1 cycle each, 6.25 nanoseconds per cycle. Each factor can be attacked: *fewer instructions*, with better algorithms, richer instructions, vectors; *fewer cycles per instruction*, with pipelines and superscalar execution; *a shorter cycle*, with faster transistors and deeper pipelines. And when one processor is not enough: more processors working together. Every machine of this chapter is a different answer to the same equation (Patterson and Hennessy, 2020).

24.1.2 Why not simply a faster clock?

For thirty years each new generation of transistors was smaller, faster, and no hotter per square millimetre. Round 2005 that stopped: smaller transistors still arrive, but a faster clock means more heat than a chip can shed (Chapter 10), and clock frequencies have stayed at a few gigahertz ever since. Moore's law, more transistors, went on; the rule that kept the density of power constant, known as Dennard scaling, ended. The extra transistors went elsewhere: wider processors, more cores, vector units, accelerators. Performance now comes from doing many things at once: from *parallelism*. The 160 MHz of the kit are slow by choice: they cost milliwatts.

24.2 Superscalar processors

24.2.1 From one at a time to many at once

Chapter 19 introduced the pipeline and named superscalar processors in one line; this section opens that line (Table 24.1). The *width* of a processor is the number of instructions that it can start in one cycle. With two pipelines side by side and the five classic stages, six instructions complete in seven cycles (Table 24.2); a single pipeline needs ten, and a processor without a pipeline thirty.

24.2.2 What gets in the way: dependences

```
    add  t0, t1, t2        # i1
    add  t3, t0, t4        # i2 needs the t0 of i1: it must wait
    lw   t5, 0(t6)         # i3 is independent: it could go now
```

Table 24.1. From one instruction at a time to many at once.

Processor	How it works	Instr. per cycle	Example
Multi-cycle	one instruction completes before the next begins	less than 1	the model of Chapter 18
Pipelined	several instructions in flight, each in a different stage	at most 1	the ESP32-C6 of the kit: four stages
Superscalar, in order	two or more pipelines side by side	up to 2 or more	small application cores
Superscalar, out of order	instructions run as soon as their operands are ready	up to 4, 6, 8, ...	laptops, telephones, servers

Table 24.2. Two pipelines side by side: six instructions in seven cycles, if nothing gets in the way.

cycle	1	2	3	4	5	6	7
i_1	IF	ID	EX	MEM	WB		
i_2	IF	ID	EX	MEM	WB		
i_3		IF	ID	EX	MEM	WB	
i_4		IF	ID	EX	MEM	WB	
i_5			IF	ID	EX	MEM	WB
i_6			IF	ID	EX	MEM	WB

```
    beqz t3, skip          # i4: which way? known only after i2
    addi t5, t5, 1         # i5: executed, or not?
skip:
```

Listing 24.2. Dependences between neighbouring instructions.

This is the ideal case, and it is rare. In Listing 24.2 the instruction i_2 cannot run beside i_1, because it uses its result: a *data dependence*. i_3 depends on nobody: why should it wait? And after the branch i_4 the processor does not know what comes next: a *control dependence*. Real code is full of such chains, and width alone is not enough. A superscalar processor that works in order stalls at i_2, and everything behind it waits. The remedy is to let i_3 overtake.

24.2.3 Out-of-order execution

1. **Fetch and decode, in order.** Several instructions per cycle enter a window of tens or hundreds of instructions.
2. **Rename.** Each result receives a fresh physical register, so that two uses of the name `t0` no longer collide.
3. **Execute when ready.** An instruction starts as soon as its operands exist, whatever its position in the program.
4. **Commit, in order.** Results become official in the order of the program: seen from outside, nothing ever overtook.

The algorithm is Tomasulo's, designed in 1967 for the floating-point unit of the IBM System/360 Model 91; the reorder buffer that commits in order came in the 1980s. The programmer's model, the one learnt in this book, is preserved exactly: this is the contract of

the instruction set.

24.2.4 Register renaming: thirty-two names, many registers

```
# as written                  # as executed, after renaming
    lw   t0, 0(a0)                lw   p41, 0(a0)
    addi t0, t0, 1                addi p42, p41, 1
    sw   t0, 0(a0)                sw   p42, 0(a0)
    lw   t0, 4(a0)                lw   p43, 4(a0)      # a new t0
    addi t0, t0, 1                addi p44, p43, 1
    sw   t0, 4(a0)                sw   p44, 4(a0)
```

Listing 24.3. The same code as written, and as executed after renaming.

In Listing 24.3 the second half reuses the *name* `t0`, not its value: a *false dependence*. Once renamed, the two halves share nothing, and can run at the same time on different registers. (The numbers `p41` to `p44` are invented.) The thirty-two registers known to the programmer are names; the chip has many more, a few hundred in a large core.

24.2.5 Guessing the future: branch prediction

A wide processor meets a branch every few instructions and cannot wait to know its outcome. It *predicts* the direction from the history of that branch, and goes on executing along the guess: a right guess costs nothing; a wrong one throws away the speculative work and starts again. The loop of the warm-up branches back 959 times and falls through once: a predictor learns it in the first turns. Modern predictors are right far more than nine times out of ten.

Speculation and security

Speculative execution has a dark side. In 2018 the attacks called Spectre and Meltdown showed that work which is thrown away can still leave traces in the cache that reveal secrets. Tricks of performance are also questions of security.

24.2.6 RISC-V goes superscalar

Table 24.3. Some RISC-V cores. The list ages quickly.

Core	From	Kind
HP core of the ESP32-C6	Espressif	in order, four stages, one instruction per cycle: the kit
BOOM	University of California, Berkeley	out of order, open source, for research and teaching
XiangShan	Chinese Academy of Sciences	out of order, open source, high performance
XuanTie C910	T-Head (Alibaba)	out of order, three-wide; released as open source
Performance P550	SiFive	out of order, three-wide, thirteen stages; sold as a design
Veyron	Ventana Micro Systems	out of order, for data centres

Because the instruction set is open, anyone may build such a core (Table 24.3), and several are published as source code that a student can read: this has no precedent among commercial architectures.

Definition 24.1 (Architecture and microarchitecture)

The *architecture* is the contract: the instructions, the registers, what each instruction does. The *microarchitecture* is one way of honouring it: stages, width, predictors, caches.

The `add` written for a microcontroller at 160 MHz is the `add` of a server core, and Verm1 would run unchanged on any of the cores of the table, given a screen. This is why an instruction set learnt once is knowledge that lasts. The distinction organises the whole book: Chapters 2 to 22 taught the architecture, and the kit is one small microarchitecture.

24.3 Vectors: one instruction, many data

24.3.1 Flynn's taxonomy

Table 24.4. The taxonomy of Flynn (1972).

Class	Instructions	Data	Example
SISD	one stream	one stream	the kit: one core, one datum at a time
SIMD	one stream	many at once	vector instructions; a GPU, to a first approximation
MISD	many	one stream	rare: redundant systems that vote
MIMD	many	many	multicore processors, clusters, supercomputers

The classification of Table 24.4 is crude, and is still the first vocabulary of parallel computing.

24.3.2 The same loop, with vectors

```
    la   t0, grid
    li   t1, COLS*ROWS
    vsetvli t2, t1, e8, m1, ta, ma  # elements are bytes
    vmv.v.i v0, 0                   # a register full of zeros
1:  vsetvli t2, t1, e8, m1, ta, ma  # t2 = bytes of this turn
    vse8.v  v0, (t0)                # store t2 bytes at once
    add  t0, t0, t2
    sub  t1, t1, t2
    bnez t1, 1b
```

Listing 24.4. Emptying the map with the vector extension.

The V extension of RISC-V, ratified in 2021, adds *vector registers*, many bytes wide (Listing 24.4). The instruction `vsetvli` asks: how many of the bytes that are left fit in a register? The answer comes back in `t2`. Then `vse8.v` stores `t2` bytes with one instruction. With registers of 128 bits the loop handles 16 bytes per turn: 60 turns instead of 960.

The code never says how wide the registers are: it is *vector-length agnostic*, and the same binary uses 128, 256 or 1 024 bits. The extension is not on the kit, since the ESP32-C6 is

RV32IMAC; the listing was tested on QEMU with registers of 128, 256, 512 and 1 024 bits.

24.3.3 A sum, lane by lane

```
    la   t0, numbers
    li   t1, 16                 # sixteen words to add
    vsetvli t2, zero, e32, m1, ta, ma
    vmv.v.i v1, 0               # v1 = partial sums, one per lane
1:  vsetvli t2, t1, e32, m1, tu, ma
    vle32.v v2, (t0)            # load t2 words
    vadd.vv v1, v1, v2          # add them, lane by lane
    slli t3, t2, 2
    add  t0, t0, t3
    sub  t1, t1, t2
    bnez t1, 1b
    vsetvli t2, zero, e32, m1, ta, ma
    vmv.s.x v3, zero
    vredsum.vs v3, v1, v3       # add the lanes together
    vmv.x.s a0, v3
```

Listing 24.5. The sum of sixteen words with vectors.

Listing 24.5 is the parallel sum of Chapters 4 and 20, in RISC-V. `vle32.v` loads several words at once; `vadd.vv` adds register to register, *lane* by lane; at the end `vredsum.vs` adds the lanes together. With the sixteen numbers from 1 to 16 the result is 136. (The setting `tu`, *tail undisturbed*, keeps the partial sums of the lanes that the last turn does not use.)

24.4 High-performance computing

24.4.1 What is a supercomputer?

A supercomputer is not one enormous processor. It is thousands of ordinary computers, the *nodes*, working on the same problem (Figure 24.1). Each node has one or two processors with tens of cores, often several GPUs, and a great deal of memory. A very fast network joins the nodes, and a parallel file system serves them all. Speed is measured in floating-point operations per second, FLOPS, on a standard problem: a huge system of linear equations (the benchmark called HPL, *High-Performance Linpack*). The TOP500 list has ranked the fastest machines twice a year since 1993.

Table 24.5. Orders of magnitude (rounded HPL figures from the TOP500 lists; the figure for the laptop is an order of magnitude).

Machine	Where	Floating-point operations per second
ESP32-C6	the kit	no floating-point unit: fixed point, as in Verm1
A laptop	your bag	about 10^{11}
Leonardo (2022)	CINECA, Bologna	about 2.4×10^{17}
Frontier (2022)	Oak Ridge, USA	over 10^{18}: the first exascale machine
El Capitan (2024)	Livermore, USA	about 1.7×10^{18}
JUPITER (2025)	Jülich, Germany	about 10^{18}: the first European exascale machine

Exa is 10^{18}. Seven orders of magnitude separate a laptop from an exascale machine (Ta-

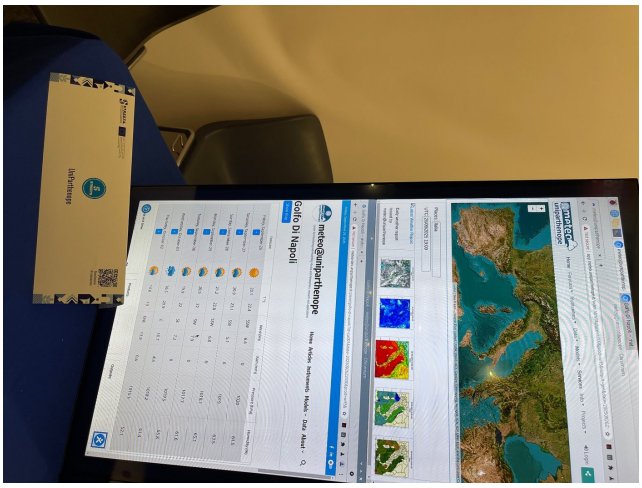


Figure 24.1. A small cluster: nodes, GPUs, a fast network. The structure of a supercomputer, in small.

ble 24.5): about fifty years of Moore's law, bought with parallelism instead.

24.4.2 Amdahl's law

Theorem 24.2 (Amdahl's law)

If a fraction p of the work of a program can be done in parallel, the speed-up obtained with n processors is

$$S(n) = \frac{1}{(1-p) + \dfrac{p}{n}}, \qquad \text{and} \qquad \lim_{n\to\infty} S(n) = \frac{1}{1-p}.$$

Proof. Take the time on one processor as 1. The serial part takes $1-p$ whatever the number of processors; the parallel part, shared among n, takes p/n. The speed-up is the ratio of the two times. □

The serial fraction does not shrink, however many processors are added (Amdahl, 1967). With $p = 0.9$ the speed-up can never exceed 10, even with a million processors (Table 24.6): a thousand processors on a program that is 90% parallel make it less than ten times faster. Before buying processors, measure p. The same law holds for any improvement that applies to only a part of the work: a faster GPU does not help the part that runs on the CPU.

Table 24.6. Amdahl's law in numbers: the speed-up.

Parallel fraction p	$n = 10$	$n = 100$	$n = 1000$	n unlimited
0.50	1.8	2.0	2.0	2
0.90	5.3	9.2	9.9	10
0.99	9.2	50	91	100
0.999	9.9	91	500	1000

As a check, $p = 0.9$ and $n = 100$ give $1/(0.1 + 0.009) = 9.17$. Gustafson's answer, in 1988, was that with more processors people solve bigger problems, and that in a bigger problem

the parallel part usually grows faster than the serial one.

24.4.3 Two ways to share the work

With *shared memory*, several cores of one node see the same memory, and the program runs as *threads* that read and write the same variables. It is easy to start: OpenMP adds one line above a loop. The danger is two threads that write the same variable. With *distributed memory* each node has its own memory and nothing is shared; the program runs as *processes* that send each other messages, and MPI, the *Message Passing Interface*, is the standard. It scales to a million cores, and the programmer pays for it. Real applications use both: MPI between the nodes, threads and GPUs inside each node. The weather and marine models of the Department work in this way.

24.4.4 The limit is energy

An exascale machine draws about twenty megawatts, the consumption of a small town (Frontier reached its exaflop at about 21 MW): some 50 thousand million operations per joule. The electricity of its lifetime costs as much as the machine. So the figure that matters is operations per watt, which has its own ranking, the Green500, led by GPUs and other accelerators: specialised hardware does more with each joule. The same reason puts a small RISC-V core in the kit: milliwatts. Energy efficiency is the thread that ties a microcontroller to a supercomputer.

24.4.5 RISC-V and European supercomputing

Today's largest machines use x86 or Arm processors with GPU accelerators. Europe designs its own processors and accelerators on RISC-V: the European Processor Initiative, and DARE, the project named in Chapter 0 (EuroHPC Joint Undertaking, 2025). Monte Cimone, built in 2022 at the University of Bologna with CINECA, is a complete HPC cluster on RISC-V processors; the Barcelona Supercomputing Center designs and fabricates RISC-V cores of its own. An open instruction set lets a continent build what it would otherwise have to buy.

24.5 GPU accelerators

24.5.1 Two ways to spend transistors

A CPU has a few large cores, tens of them, each out of order, with predictors and big caches: it is built to finish *one* task as soon as possible, and is at ease with branches and irregular data. A GPU has thousands of small, simple cores, all executing the same code on different data: it is built to finish a million tasks per second, and is at ease with regular arrays: pixels, matrices, grids. These are Cray's oxen and chickens, and the answer depends on the field. The GPU is not faster at any single thing; it does more things at once.

24.5.2 How a program uses a GPU

1. **Prepare.** The program on the CPU, the *host*, builds its arrays in main memory.
2. **Copy in.** The data travel to the memory of the GPU: a real cost, in time.
3. **Launch.** One small function, the *kernel*, runs in thousands of threads, one per element of the data.

4. **Copy out.** The results travel back, and the host goes on.

The GPU is a device, like those of Chapter 21: the CPU sends it data and commands through a driver. The programming models are CUDA, OpenCL and SYCL, and directives such as OpenACC and OpenMP offload.

```
// the kernel: this function runs once in every thread
__global__ void clear(unsigned char *grid, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // who am I?
    if (i < n)
        grid[i] = 0;                                 // my cell, and only mine
}

// on the host: 4 blocks of 256 threads cover the 960 cells
clear<<<(960 + 255) / 256, 256>>>(grid, 960);
```

Listing 24.6. The map of Verm1 with one thread per cell (CUDA C, in its usual textbook form; not compiled for this book).

In Listing 24.6 there is no loop: there are 960 threads. Each thread computes its own index and does one store. Threads come in blocks, and the last ones find nothing to do. For 960 bytes the two copies cost far more than the work, and the point is serious: a GPU pays off when the data are large and the work per element is real.

24.5.3 When does a GPU pay off?

It pays off when the same operation applies to very many data, independently: images, matrices, the grids of a simulation, neural networks; and when the data can stay on the GPU for many steps instead of travelling at every one. It does not when the work is a chain of decisions, each depending on the one before. And Amdahl's law applies again: what stays on the CPU sets the limit. A weather simulation updates every cell of a three-dimensional grid from its neighbours, millions of times: the same shape as the map of Verm1, a little larger. The training of today's neural networks is, almost entirely, matrix arithmetic on GPUs.

24.6 FPGA accelerators

24.6.1 A chip that becomes the circuit you describe

Definition 24.2 (FPGA)

A *field-programmable gate array* (FPGA) is a chip made of a sea of small programmable blocks: *look-up tables*, each able to be any function of a few inputs; flip-flops, one beside each table; and wires and switches that can join any block to any other. A configuration file, the *bitstream*, sets every table and every switch.

Load another bitstream, and the same chip is another circuit. The first FPGA, the Xilinx XC2064, was sold in 1985; today they go from a few euros to boards for data centres. An FPGA does not execute a program: it *becomes* a circuit. Nothing is fetched or decoded, and all the gates work at the same time.

24.6.2 A look-up table is a truth table in memory

a	b	stored for AND	stored for XOR	stored for NOT a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	0	0

The inputs are the address, and four stored bits choose the function: changing the bits changes the gate. Chapter 8 said that a truth table defines a function, and Chapter 16 that a ROM addressed by the inputs implements any function. A look-up table is that ROM, with four to six inputs and sixteen to sixty-four bits; the flip-flop next to it is that of Chapter 13. An FPGA is this book, repeated a hundred thousand times on one chip.

24.6.3 The U-turn rule of Verm1, as hardware

```
// Lecture 12: opposite directions differ in bit 1 only
module uturn (
    input  wire [1:0] dir,      // direction of the last step
    input  wire [1:0] want,     // direction asked by the player
    output wire       refuse    // 1 = this would be a U-turn
);
    assign refuse = (dir ^ want) == 2'b10;
endmodule
```

Listing 24.7. The U-turn rule in Verilog (not synthesised for this book).

Verilog, like VHDL, is a *hardware description language*: it describes circuits, not steps (Listing 24.7). The statement `assign` is a wire, always true: there is no before and no after. The circuit is two XOR gates and an AND with one inverted input,

$$refuse = (d_1 \oplus w_1) \cdot \overline{(d_0 \oplus w_0)}.$$

On the kit this rule took three instructions and three cycles (Chapter 12); here the gates answer in a fraction of a nanosecond, continuously.

24.6.4 From description to circuit

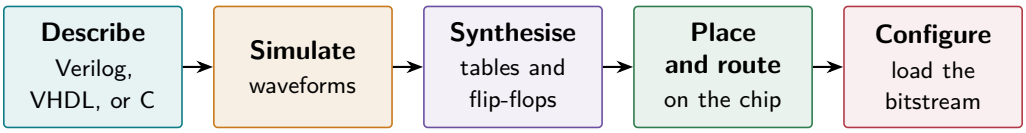


Figure 24.2. From a description to a circuit.

Each attempt takes minutes to hours (Figure 24.2): slower than building a cartridge, far faster than making a chip. The same flow, with a factory at the end instead of a bitstream, produces an *ASIC*, a chip made for one purpose. FPGAs are where chips are tried before they are made.

24.6.5 A processor is a circuit too

A RISC-V core is a description in a hardware language, and can be loaded on an FPGA. Such *soft cores* are free to read and to use: PicoRV32, VexRiscv, NEORV32, Rocket. A core

of the class of the kit's fits in a small, cheap FPGA, and beside it one can put a circuit of one's own for the one operation that a program repeats most: a custom accelerator. RISC-V reserves room in its encoding for custom instructions, to reach such circuits.

The circle closes

The gates of Chapter 8 build the adder of Chapter 9, the registers of Chapter 14 and the processor of Chapter 18; and that processor is a file that can be read. From gates to a processor, and back: the reader now knows every layer in between.

24.6.6 Four ways to compute

Table 24.7. Four ways to compute.

	CPU	**GPU**	**FPGA**	**ASIC**
What it is	a few complex cores	thousands of simple cores	a circuit you configure	a circuit made for one job
Programmed in	any language	kernels: CUDA, OpenCL, SYCL	Verilog, VHDL, HLS	the same, then a factory
Strong at	everything, moderately	regular data in bulk	streams, low latency	one thing, perfectly
Work per joule	low	high	high	highest
Effort to use	low	medium	high	very high
Change of mind	recompile	recompile	a new bitstream	a new chip

Generality is paid in energy; efficiency is paid in effort (Table 24.7). A modern system on a chip has all four: a telephone holds CPU cores, a GPU, and fixed circuits for video and radio.

24.7 Back to the machine

Table 24.8. Emptying the map of Verm1, 960 cells, on five machines.

Machine	**How**	**Steps in time**
The kit: ESP32-C6	one byte per turn, one instruction per cycle	3 840 cycles
Out-of-order RISC-V core	the same binary; several instructions per cycle, the branch predicted	a fraction of those cycles
RISC-V with vectors, 128 bits	16 bytes per turn	60 turns
GPU	960 threads, one per cell	one launch, plus two copies
FPGA	960 flip-flops with a common reset	one clock cycle

The first three rows of Table 24.8 are the same instruction set; the first and the third were run for this book. The work is the same everywhere; what changes is how much of it happens at the same time. And the costs change: the first row needs a chip of two euros and four lines of assembly; the last needs a hardware design.

Concept

The engineer's question is never which machine is fastest, but which is enough.

24.8 The program of this chapter

The listings of this chapter are parts of one program, `code/l24.S` in the repository of the course, printed in full in Listing 24.8. The automatic tests of the course material build it and run it on the headless runner of PRG32-QT, with scripted input, and check the frame; Figure 24.3 is the frame that the test wrote.

On the screen.
A green square: the 960 bytes of the map, filled with ones by init, were all set to zero by the four-instruction loop of the lecture. A red square would mean that some byte survived.

What to press.
Nothing: the program runs by itself.

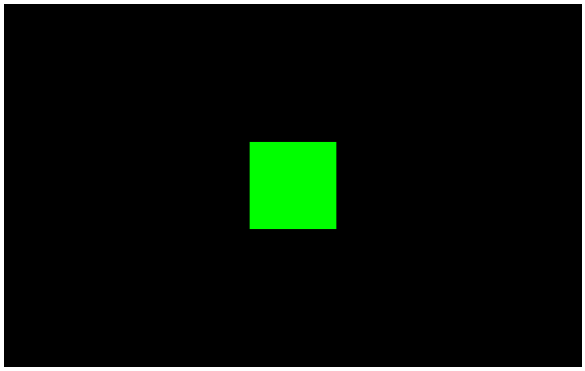


Figure 24.3. `l24.S` on PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 24: Beyond One Core
# l24.S: the loop that empties the map of Verm1, one byte at a time
#
# On the screen: a green square: the 960 bytes of the map, filled with ones by init,
# were all set to zero by the four-instruction loop of the lecture. A red square would
# mean that some byte survived.
# Build: python3 -m prg32 cartridge build l24.S --portable \
#            --entry-prefix l24 --name l24 --out l24.prg32
.option norelax

.equ BLACK, 0x0000
.equ GREEN, 0x07e0
.equ RED,   0xf800
.equ CELLS, 960              # 40 columns by 24 rows

.section .text
.global l24_init
.global l24_update
```

```
.global l24_draw

l24_init:                   # fill the map with ones, so that there is work to
    do
    la   t0, grid
    li   t1, CELLS
    li   t2, 1
1:  sb   t2, 0(t0)
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    ret

l24_update:
    la   t0, grid           # the loop of the lecture
    li   t1, CELLS
1:  sb   zero, 0(t0)        # one byte ...
    addi t0, t0, 1          # ... the next address ...
    addi t1, t1, -1         # ... one fewer to go ...
    bnez t1, 1b             # ... 960 times: 3840 instructions

    la   t0, grid           # count the bytes that are not zero
    li   t1, CELLS
    li   t3, 0
2:  lbu  t2, 0(t0)
    add  t3, t3, t2
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 2b
    la   t0, left
    sw   t3, 0(t0)
    ret

l24_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    la   t0, left
    lw   t1, 0(t0)
    li   a4, GREEN
    beqz t1, 1f
    li   a4, RED
1:  li   a0, 136
    li   a1, 76
    li   a2, 48
    li   a3, 48
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .data
left: .word 1
grid: .space CELLS
```

Listing 24.8. The complete program of the chapter, `code/l24.S`.

To run it on PRG32-QT the commands are those of Listing 24.9; Section B.8 gives the same steps for QEMU and for the board.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l24.S --portable \
    --entry-prefix l24 --name l24 --out l24.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l24.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listing 24.9. Building `l24.S` and running it on PRG32-QT.

24.9 Check your understanding

Q24.1 A program is 80% parallel. What is the largest speed-up that any number of processors can give?

A. 4
B. 5
C. 8
D. 80

Q24.2 What does register renaming remove?

A. branches
B. the need for a stack
C. false dependences between uses of the same register name
D. cache misses

Q24.3 With 256-bit vector registers, how many turns does the vector loop need for the 960 bytes?

A. 4
B. 30
C. 60
D. 960

Q24.4 A two-input look-up table stores, for the inputs 00, 01, 10, 11, the bits 0, 1, 1, 1. Which gate is it?

A. AND

B. OR
C. XOR
D. NAND

24.10 Exercises

Exercise 24.1 (paper first, then the machine)

1. Fill in the Amdahl table for $p = 0.95$ and $n = 10$, 100, 1000.
2. Draw the pipeline diagram of a three-wide processor for nine independent instructions with five stages.
3. Rename the registers of the code of `step` that moves and wraps the head in Verm1, Part 2: which instructions could run in the same cycle?
4. Write the truth table stored in a four-input look-up table for $refuse = (dir \oplus want) = 2$.
5. Build the test `vector_qemu.S` of the course material and run it on QEMU with vector registers of 128 and of 512 bits: why does the same binary work?

Expected result. Amdahl with $p = 0.95$: 6.9, 16.8, 19.6. Nine instructions on a three-wide processor: seven cycles. The look-up table has four ones out of sixteen entries, for $(dir, want) = (0, 2), (1, 3), (2, 0), (3, 1)$. QEMU is started as `qemu-system-riscv32 -machine virt -cpu rv32,v=true,vlen=128`.

24.11 Glossary

English	Italiano
superscalar	superscalare
register renaming	ridenominazione dei registri
out-of-order execution	esecuzione fuori ordine
branch prediction	predizione dei salti
speed-up	accelerazione (speed-up)
node	nodo
shared memory	memoria condivisa
message passing	scambio di messaggi
accelerator	acceleratore
kernel (GPU)	kernel
look-up table	tabella di look-up
bitstream	bitstream
hardware description language	linguaggio di descrizione dell'hardware
soft core	soft core

24.12 Conclusions and reading

One equation, three factors: every fast machine attacks one of them, or multiplies the processors. Superscalar, out-of-order cores keep the contract of the instruction set and execute many instructions at once. Vectors, GPUs and clusters exploit data that can be processed independently, and Amdahl's law says how far that goes. FPGAs turn a description into a circuit: the logic of this book, at scale.

The book ends here: from a bit to a supercomputer, with one instruction set.

Reading. Patterson and Hennessy (2020), chapters 4 and 6; Hennessy and Patterson (2019); https://top500.org; the RISC-V vector specification at https://riscv.org.

Part VII

Appendices

APPENDIX A

RISC-V Reference

This appendix is a complete reference of the RISC-V architecture as it is used in this book: the instruction set RV32IMAC of the ESP32-C6, the control and status registers of machine mode, the conventions of the assembler and the calling convention. It follows the ratified specifications (RISC-V International, 2024c,d,a,b), which prevail in case of doubt. Throughout, $x[r]$ denotes the content of register r, $M[a]$ the memory at address a, sext$(\cdot)$ sign extension to 32 bits and zext$(\cdot)$ zero extension; arithmetic is modulo 2^{32}.

A.1 The name of an instruction set

A RISC-V instruction set is named by a base and a list of extensions. The base gives the width of the registers; each letter adds a group of instructions (Table A.1). RV32IMAC is therefore the 32-bit base with multiplication and division, atomic instructions and compressed instructions. Since 2019 the instructions for the control and status registers and the instruction `fence.i` are formally separate extensions, Zicsr and Zifencei, which the ESP32-C6 also implements.

A.2 Registers

The hardware gives a special meaning only to `x0`, which always reads as zero and ignores writes. All the other roles of Table A.2 are conventions. The compressed instructions with a 3-bit register field can name only `x8`–`x15`, that is, `s0`, `s1` and `a0`–`a5`. The program counter is not a general register: it is read by `auipc`, `jal` and the branches, and written by jumps and branches.

A.3 Instruction formats

All the instructions of the base set are 32 bits long and are stored in little-endian order; the two least significant bits of a 32-bit instruction are `11`. The six formats are shown in Figure A.1.

Immediates. In every format bit 31 of the instruction is the sign bit of the immediate, which is always sign-extended to 32 bits.

- **I**: 12 bits, range $-2048 \ldots 2047$.

Table A.1. Bases and extensions of RISC-V.

Name	Meaning	What it adds	ESP32-C6
RV32I	base integer, 32 bits	32 registers of 32 bits; 40 instructions	yes
RV32E	embedded base	as RV32I with 16 registers	–
RV64I	base integer, 64 bits	registers and addresses of 64 bits	–
M	multiply, divide	`mul`, `mulh`, `mulhsu`, `mulhu`, `div`, `divu`, `rem`, `remu`	yes
A	atomic	`lr.w`, `sc.w` and the atomic memory operations	yes
F, D	floating point	registers `f0`–`f31`; IEEE 754 single and double precision	no
C	compressed	16-bit encodings of the most common instructions	yes
Zicsr	CSR access	`csrrw`, `csrrs`, `csrrc` and immediate forms	yes
Zifencei	instruction fence	`fence.i`	yes
B (Zba, Zbb, Zbs)	bit manipulation	shifts-and-adds, counts, rotations, single-bit operations	no
V	vectors	registers `v0`–`v31` of variable length (Chapter 24)	no

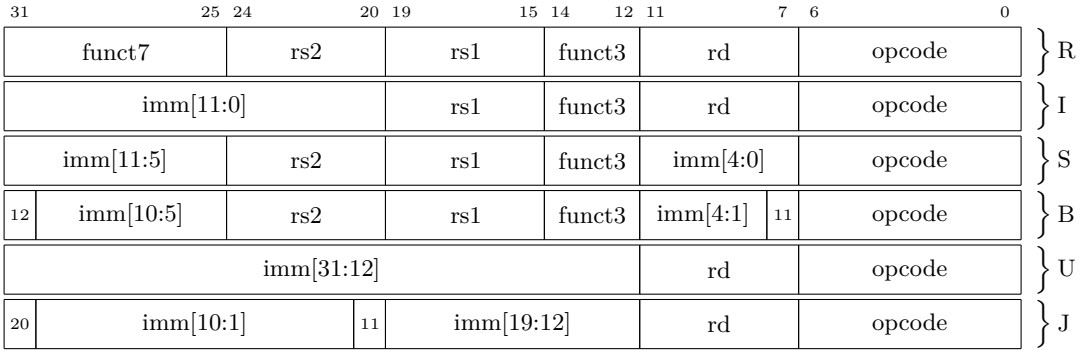


Figure A.1. The six formats of the 32-bit instructions.

- **S**: 12 bits, split between bits 31–25 and 11–7, so that `rs1` and `rs2` stay in place.
- **B**: 13 bits, of which bit 0 is always zero and is not stored; the offset is a multiple of 2 in the range $-4096 \ldots 4094$, relative to the address of the branch.
- **U**: the upper 20 bits; the lower 12 are zero.
- **J**: 21 bits, of which bit 0 is zero and not stored; the offset is a multiple of 2 in the range ± 1 MiB.

A.4 RV32I: the base integer instruction set

Table A.4 lists the forty instructions of RV32I. Signed comparisons read the registers in two's complement; "u" marks the unsigned forms.

Table A.2. The integer registers and their roles in the standard calling convention.

Register	ABI name	Role	Preserved across a call by
x0	zero	hard-wired zero	–
x1	ra	return address	caller
x2	sp	stack pointer	callee
x3	gp	global pointer	– (not to be changed)
x4	tp	thread pointer	– (not to be changed)
x5–x7	t0–t2	temporaries	caller
x8	s0 / fp	saved register / frame pointer	callee
x9	s1	saved register	callee
x10–x11	a0–a1	arguments / return values	caller
x12–x17	a2–a7	arguments	caller
x18–x27	s2–s11	saved registers	callee
x28–x31	t3–t6	temporaries	caller
pc	–	program counter	–

Table A.3. The major opcodes used by RV32IMAC (bits 6 to 0).

Opcode	Name	Format	Instructions
0110111	LUI	U	lui
0010111	AUIPC	U	auipc
1101111	JAL	J	jal
1100111	JALR	I	jalr
1100011	BRANCH	B	beq, bne, blt, bge, bltu, bgeu
0000011	LOAD	I	lb, lh, lw, lbu, lhu
0100011	STORE	S	sb, sh, sw
0010011	OP-IMM	I	addi, slti, sltiu, xori, ori, andi, shifts
0110011	OP	R	register-register operations; the M extension
0001111	MISC-MEM	I	fence, fence.i
1110011	SYSTEM	I	ecall, ebreak, mret, wfi, CSR instructions
0101111	AMO	R	the A extension

Table A.4. The instructions of RV32I.

Assembly	Fmt	opcode / f3 / f7	Effect
Constants and addresses			
lui rd, imm	U	0110111	$x[rd] = imm \ll 12$
auipc rd, imm	U	0010111	$x[rd] = pc + (imm \ll 12)$
Arithmetic and logic with an immediate			
addi rd, rs1, imm	I	0010011 / 000	$x[rd] = x[rs1] + \text{sext}(imm)$
slti rd, rs1, imm	I	0010011 / 010	$x[rd] = 1$ if $x[rs1] < \text{sext}(imm)$ (signed), else 0
sltiu rd, rs1, imm	I	0010011 / 011	the same, unsigned comparison
xori rd, rs1, imm	I	0010011 / 100	$x[rd] = x[rs1] \oplus \text{sext}(imm)$
ori rd, rs1, imm	I	0010011 / 110	$x[rd] = x[rs1] \mid \text{sext}(imm)$

Assembly	Fmt	opcode / f3 / f7	Effect
`andi rd, rs1, imm`	I	`0010011 / 111`	$x[rd] = x[rs1]$ & sext(imm)
`slli rd, rs1, sh`	I	`0010011 / 001 / 0000000`	$x[rd] = x[rs1] \ll sh$, $0 \leq sh \leq 31$
`srli rd, rs1, sh`	I	`0010011 / 101 / 0000000`	logical shift right: zeros enter
`srai rd, rs1, sh`	I	`0010011 / 101 / 0100000`	arithmetic shift right: the sign bit enters
Arithmetic and logic between registers			
`add rd, rs1, rs2`	R	`0110011 / 000 / 0000000`	$x[rd] = x[rs1] + x[rs2]$
`sub rd, rs1, rs2`	R	`0110011 / 000 / 0100000`	$x[rd] = x[rs1] - x[rs2]$
`sll rd, rs1, rs2`	R	`0110011 / 001 / 0000000`	shift left by $x[rs2]$ mod 32
`slt rd, rs1, rs2`	R	`0110011 / 010 / 0000000`	$x[rd] = 1$ if $x[rs1] < x[rs2]$ (signed), else 0
`sltu rd, rs1, rs2`	R	`0110011 / 011 / 0000000`	the same, unsigned
`xor rd, rs1, rs2`	R	`0110011 / 100 / 0000000`	bitwise exclusive or
`srl rd, rs1, rs2`	R	`0110011 / 101 / 0000000`	logical shift right by $x[rs2]$ mod 32
`sra rd, rs1, rs2`	R	`0110011 / 101 / 0100000`	arithmetic shift right by $x[rs2]$ mod 32
`or rd, rs1, rs2`	R	`0110011 / 110 / 0000000`	bitwise or
`and rd, rs1, rs2`	R	`0110011 / 111 / 0000000`	bitwise and
Loads and stores			
`lb rd, imm(rs1)`	I	`0000011 / 000`	$x[rd] = \text{sext}(M_8[x[rs1] + \text{sext}(imm)])$
`lh rd, imm(rs1)`	I	`0000011 / 001`	the same, 16 bits, sign-extended
`lw rd, imm(rs1)`	I	`0000011 / 010`	the same, 32 bits
`lbu rd, imm(rs1)`	I	`0000011 / 100`	8 bits, zero-extended
`lhu rd, imm(rs1)`	I	`0000011 / 101`	16 bits, zero-extended
`sb rs2, imm(rs1)`	S	`0100011 / 000`	$M_8[x[rs1] + \text{sext}(imm)] = x[rs2]_{7:0}$
`sh rs2, imm(rs1)`	S	`0100011 / 001`	the same, the low 16 bits
`sw rs2, imm(rs1)`	S	`0100011 / 010`	the same, 32 bits
Branches (the target is $pc + \text{sext}(imm)$)			
`beq rs1, rs2, label`	B	`1100011 / 000`	branch if $x[rs1] = x[rs2]$
`bne rs1, rs2, label`	B	`1100011 / 001`	branch if $x[rs1] \neq x[rs2]$
`blt rs1, rs2, label`	B	`1100011 / 100`	branch if $x[rs1] < x[rs2]$, signed
`bge rs1, rs2, label`	B	`1100011 / 101`	branch if $x[rs1] \geq x[rs2]$, signed
`bltu rs1, rs2, label`	B	`1100011 / 110`	branch if $x[rs1] < x[rs2]$, unsigned
`bgeu rs1, rs2, label`	B	`1100011 / 111`	branch if $x[rs1] \geq x[rs2]$, unsigned
Jumps			
`jal rd, label`	J	`1101111`	$x[rd] = pc + 4$; $pc = pc + \text{sext}(imm)$
`jalr rd, imm(rs1)`	I	`1100111 / 000`	$t = pc + 4$; $pc = (x[rs1] + \text{sext}(imm)) \;\&\; \sim 1$; $x[rd] = t$
Memory ordering and environment			
`fence`	I	`0001111 / 000`	orders memory accesses as seen by other harts and devices

Assembly	Fmt	opcode / f3 / f7	Effect
`ecall`	I	`1110011` / `000`, imm 0	environment call: raises an exception (cause 8, 9 or 11)
`ebreak`	I	`1110011` / `000`, imm 1	breakpoint: raises an exception (cause 3)

Remarks.

- There is no instruction to subtract an immediate: `addi` with a negative constant does it. There is no `nop` either: it is `addi x0, x0, 0`.
- There are no flags. Overflow and carry are computed with `slt` and `sltu` when they are needed (Chapter 5).
- A 32-bit constant is loaded with `lui` followed by `addi`. Because `addi` sign-extends its immediate, when bit 11 of the constant is 1 the upper part must be increased by one: the assembler does this when it expands `li`, and the relocation operators `%hi` and `%lo` do it for addresses.
- An instruction whose encoding is all zeros or all ones is illegal by definition: a jump into erased or empty memory is caught at once.
- With the C extension instructions are aligned to 2 bytes; without it, to 4, and a jump to a misaligned address raises an exception.

A.5 The M extension: multiplication and division

Table A.5. The M extension: format R, opcode `0110011`, funct7 `0000001`.

Assembly	funct3	Effect
`mul rd, rs1, rs2`	`000`	the low 32 bits of the product (the same for signed and unsigned operands)
`mulh rd, rs1, rs2`	`001`	the high 32 bits of the signed × signed product
`mulhsu rd, rs1, rs2`	`010`	the high 32 bits of the signed × unsigned product
`mulhu rd, rs1, rs2`	`011`	the high 32 bits of the unsigned × unsigned product
`div rd, rs1, rs2`	`100`	signed quotient, rounded towards zero
`divu rd, rs1, rs2`	`101`	unsigned quotient
`rem rd, rs1, rs2`	`110`	signed remainder; it has the sign of the dividend
`remu rd, rs1, rs2`	`111`	unsigned remainder

Division never raises an exception. A division by zero gives a quotient of all ones (-1, or $2^{32}-1$ unsigned) and a remainder equal to the dividend. The only signed overflow, $-2^{31}/-1$, gives the quotient -2^{31} and the remainder 0. A program that must detect a division by zero tests the divisor with a branch.

A.6 The A extension: atomic instructions

The A extension serves programs in which several threads of execution share memory. All its instructions have format R, opcode `0101111` and, for words, funct3 `010`; the five most significant bits select the operation (Table A.6).

Table A.6. The A extension (RV32).

Assembly	funct5	Effect
`lr.w rd, (rs1)`	00010	load reserved: $x[rd] = M[x[rs1]]$, and the address is reserved
`sc.w rd, rs2, (rs1)`	00011	store conditional: stores $x[rs2]$ only if the reservation still holds; $x[rd] = 0$ on success, non-zero on failure
`amoswap.w rd, rs2, (rs1)`	00001	atomically: $x[rd] = M[x[rs1]]$; $M[x[rs1]] = x[rs2]$
`amoadd.w rd, rs2, (rs1)`	00000	atomically: $x[rd] = M$; $M = M + x[rs2]$
`amoxor.w`, `amoand.w`, `amoor.w`	00100, 01100, 01000	the same with xor, and, or
`amomin.w`, `amomax.w`	10000, 10100	the same with signed minimum, maximum
`amominu.w`, `amomaxu.w`	11000, 11100	the same with unsigned minimum, maximum

A.7 The C extension: compressed instructions

The C extension gives a 16-bit encoding to the most frequent instructions. Every compressed instruction expands to exactly one 32-bit instruction, so that the extension adds nothing to what the processor can do: it makes programs about a quarter shorter. The assembler chooses the short form by itself whenever the operands allow it; a disassembler shows it with the prefix `c.` (Table A.7). A 16-bit instruction is recognised by its two least significant bits, which are different from `11`.

Table A.7. The compressed instructions of RV32C (without floating point).

Compressed	Expands to	Restrictions
`c.addi4spn rd', n`	`addi rd', sp, n`	n a non-zero multiple of 4, up to 1020
`c.lw rd', n(rs1')`	`lw rd', n(rs1')`	n a multiple of 4, up to 124
`c.sw rs2', n(rs1')`	`sw rs2', n(rs1')`	as above
`c.nop`	`addi x0, x0, 0`	
`c.addi rd, n`	`addi rd, rd, n`	$-32 \le n \le 31$, $n \neq 0$
`c.jal label`	`jal ra, label`	RV32 only; ±2 KiB
`c.li rd, n`	`addi rd, x0, n`	$-32 \le n \le 31$
`c.addi16sp n`	`addi sp, sp, n`	n a non-zero multiple of 16, $-512 \ldots 496$
`c.lui rd, n`	`lui rd, n`	$rd \neq$ `x0`, `x2`; $n \neq 0$
`c.srli rd', sh`	`srli rd', rd', sh`	
`c.srai rd', sh`	`srai rd', rd', sh`	
`c.andi rd', n`	`andi rd', rd', n`	$-32 \le n \le 31$
`c.sub rd', rs2'`	`sub rd', rd', rs2'`	
`c.xor`, `c.or`, `c.and`	`xor`, `or`, `and rd', rd', rs2'`	
`c.j label`	`jal x0, label`	±2 KiB
`c.beqz rs1', label`	`beq rs1', x0, label`	±256 bytes
`c.bnez rs1', label`	`bne rs1', x0, label`	±256 bytes
`c.slli rd, sh`	`slli rd, rd, sh`	
`c.lwsp rd, n(sp)`	`lw rd, n(sp)`	n a multiple of 4, up to 252

Compressed	Expands to	Restrictions
`c.swsp rs2, n(sp)`	`sw rs2, n(sp)`	as above
`c.jr rs1`	`jalr x0, 0(rs1)`	$rs1 \neq$ `x0`; `c.jr ra` is `ret`
`c.jalr rs1`	`jalr ra, 0(rs1)`	
`c.mv rd, rs2`	`add rd, x0, rs2`	
`c.add rd, rs2`	`add rd, rd, rs2`	
`c.ebreak`	`ebreak`	

The registers written `rd'`, `rs1'`, `rs2'` are named by 3 bits and can only be `x8`–`x15`. This explains what the debugger of PRG32-QT shows: an `addi t3, t3, 1` becomes `c.addi t3, 1`, a `ret` becomes `c.jr ra`, and the saving of `ra` at `12(sp)` becomes `c.swsp ra, 12(sp)`.

A.8 Pseudo-instructions

The assembler accepts a number of mnemonics that are not instructions and expands them (Table A.8).

Table A.8. The pseudo-instructions of the RISC-V assembler.

Pseudo-instruction	Expansion	Meaning
`nop`	`addi x0, x0, 0`	no operation
`li rd, imm`	`addi rd, x0, imm`, or `lui` + `addi`	load a constant
`la rd, symbol`	`auipc rd, hi` + `addi rd, rd, lo`	load an address, relative to the PC
`lw rd, symbol`	`auipc rd, hi` + `lw rd, lo(rd)`	load a global variable
`sw rs, symbol, rt`	`auipc rt, hi` + `sw rs, lo(rt)`	store a global variable, through `rt`
`mv rd, rs`	`addi rd, rs, 0`	copy a register
`not rd, rs`	`xori rd, rs, -1`	bitwise complement
`neg rd, rs`	`sub rd, x0, rs`	two's complement
`seqz rd, rs`	`sltiu rd, rs, 1`	set if zero
`snez rd, rs`	`sltu rd, x0, rs`	set if not zero
`sltz rd, rs`	`slt rd, rs, x0`	set if negative
`sgtz rd, rs`	`slt rd, x0, rs`	set if positive
`beqz rs, label`	`beq rs, x0, label`	branch if zero
`bnez rs, label`	`bne rs, x0, label`	branch if not zero
`blez rs, label`	`bge x0, rs, label`	branch if ≤ 0
`bgez rs, label`	`bge rs, x0, label`	branch if ≥ 0
`bltz rs, label`	`blt rs, x0, label`	branch if < 0
`bgtz rs, label`	`blt x0, rs, label`	branch if > 0
`bgt rs, rt, label`	`blt rt, rs, label`	branch if greater, signed
`ble rs, rt, label`	`bge rt, rs, label`	branch if less or equal, signed
`bgtu rs, rt, label`	`bltu rt, rs, label`	branch if greater, unsigned
`bleu rs, rt, label`	`bgeu rt, rs, label`	branch if less or equal, unsigned
`j label`	`jal x0, label`	unconditional jump
`jal label`	`jal ra, label`	jump and link

Pseudo-instruction	Expansion	Meaning
`jr rs`	`jalr x0, 0(rs)`	jump to a register
`jalr rs`	`jalr ra, 0(rs)`	call through a register
`ret`	`jalr x0, 0(ra)`	return from a function
`call symbol`	`auipc ra, hi` + `jalr ra, lo(ra)`	call a function (the linker may shorten it to `jal`)
`tail symbol`	`auipc t1, hi` + `jalr x0, lo(t1)`	jump to a function without linking
`csrr rd, csr`	`csrrs rd, csr, x0`	read a CSR
`csrw csr, rs`	`csrrw x0, csr, rs`	write a CSR
`csrs csr, rs`	`csrrs x0, csr, rs`	set bits in a CSR
`csrc csr, rs`	`csrrc x0, csr, rs`	clear bits in a CSR
`csrwi`, `csrsi`, `csrci`	`csrrwi`, `csrrsi`, `csrrci` with `rd` = `x0`	the same with a 5-bit immediate

A.9 The assembler: directives, labels and expressions

The assembler of the course is the GNU assembler of the RISC-V toolchain of ESP-IDF. A source file has the extension `.S`, which means that it passes through the C preprocessor first: `#include` and `#define` are available, and a `#` begins a comment only when it cannot be a preprocessor directive. Comments can also be written as in C, between `/*` and `*/`.

Table A.9. The directives used in this book.

Directive	Effect
`.section .text` (or `.text`)	what follows is executable code
`.section .rodata`	read-only data: strings, tables
`.section .data` (or `.data`)	initialised variables
`.section .bss` (or `.bss`)	variables initialised to zero, which occupy no space in the file
`.global name` (or `.globl`)	makes the label visible outside the file
`.equ NAME, value` (or `.set`)	gives a name to a constant; no memory is used
`.byte`, `.half`, `.word`	store values of 8, 16, 32 bits, separated by commas
`.ascii "text"`	stores the characters of a string
`.asciz "text"` (or `.string`)	the same, followed by a zero byte
`.space n` (or `.zero n`)	reserves n bytes set to zero
`.balign n`	advances to the next multiple of n bytes
`.align n`, `.p2align n`	advances to the next multiple of 2^n bytes
`.option norelax`	forbids the linker to shorten sequences (used in every PRG32 cartridge)
`.option push`, `.option pop`	save and restore the options
`.option rvc`, `.option norvc`	allow or forbid compressed instructions
`.macro name args ... .endm`	define a macro
`.include "file"`	insert another source file

Labels. A name followed by a colon is a label and stands for the address of what follows. A label made of a single number is *local* and can be defined many times: `1b` refers to the nearest label `1:` *backwards*, and `1f` to the nearest *forwards*.

Expressions. Wherever a constant is expected an expression can be written, with the operators of C, and is computed by the assembler: `COLS*ROWS`, `grid+12*COLS`, `~0x10`, `1 << 4`. Numbers are decimal, hexadecimal (`0x`), binary (`0b`) or characters (`'A'`).

Relocation operators. `%hi(sym)` and `%lo(sym)` give the upper 20 and the lower 12 bits of an absolute address, already corrected for the sign extension of the lower part; `%pcrel_hi(sym)` and `%pcrel_lo(label)` do the same for an address relative to the PC, and are what `la` and `call` use.

A.10 The calling convention

The standard calling convention for RV32 without floating point (the ABI called ILP32) is the following (RISC-V International, 2024a).

1. **Arguments.** The first eight arguments of a function travel in `a0`–`a7`, in order; further arguments go on the stack. A value of 64 bits takes two registers, the low word first.
2. **Results.** The result comes back in `a0` (and `a1` for 64 bits).
3. **Caller-saved registers.** `ra`, `t0`–`t6` and `a0`–`a7` may be destroyed by any call. A caller that needs their values afterwards saves them itself.
4. **Callee-saved registers.** `sp` and `s0`–`s11` must have, at the return, the value that they had at the call. A function that uses them saves them on entry and restores them on exit.
5. **The stack.** It grows towards the low addresses; `sp` points to the last occupied word and is a multiple of 16 at every call. A function may use the memory below `sp` only after having lowered `sp`.
6. **`gp` and `tp`** are not to be modified by ordinary code.

Listing A.1 is the template of a function that calls other functions and uses two saved registers; a *leaf* function that uses only `t` and `a` registers needs none of it.

```
my_function:
    addi sp, sp, -16        # prologue: a frame of 16 bytes
    sw   ra, 12(sp)         # the return address
    sw   s0, 8(sp)          # the saved registers that will be used
    sw   s1, 4(sp)

    mv   s0, a0             # values that must survive the calls
    call other_function     # may destroy ra, t0-t6, a0-a7
    add  a0, a0, s0         # the result goes in a0

    lw   ra, 12(sp)         # epilogue, in reverse order
    lw   s0, 8(sp)
    lw   s1, 4(sp)
    addi sp, sp, 16
    ret
```

Listing A.1. The frame of a non-leaf function.

On PRG32 the three entry points of a cartridge are ordinary functions that follow this convention, and so are all the functions of the runtime (Appendix B).

A.11 Privilege, control and status registers, traps

A.11.1 Privilege modes

Table A.10. The privilege modes.

Level	Name	Runs	ESP32-C6
0	user (U)	applications	yes
1	supervisor (S)	the kernel of an operating system	no
3	machine (M)	firmware; the only mandatory mode	yes

A.11.2 The CSR instructions (Zicsr)

The control and status registers form a separate address space of 12 bits, 4 096 registers. They are read and written atomically by six instructions of format I, opcode `1110011`, whose immediate field is the address of the CSR (Table A.11).

Table A.11. The CSR instructions.

Assembly	funct3	Effect
`csrrw rd, csr, rs1`	`001`	$t = CSR$; $CSR = x[rs1]$; $x[rd] = t$
`csrrs rd, csr, rs1`	`010`	$t = CSR$; $CSR = t \mid x[rs1]$; $x[rd] = t$ (sets bits)
`csrrc rd, csr, rs1`	`011`	$t = CSR$; $CSR = t \,\&\, \sim x[rs1]$; $x[rd] = t$ (clears bits)
`csrrwi rd, csr, uimm`	`101`	as `csrrw`, with a 5-bit unsigned immediate
`csrrsi rd, csr, uimm`	`110`	as `csrrs`, with the immediate
`csrrci rd, csr, uimm`	`111`	as `csrrc`, with the immediate

A.11.3 Machine-mode registers

A.11.4 Trap entry and return

When a trap is taken in machine mode the hardware performs, in one step, the following actions.

1. `mepc` = the address of the instruction interrupted (for an interrupt) or of the instruction that caused the exception.
2. `mcause` = the cause; `mtval` = the additional information.
3. In `mstatus`: MPIE = MIE; MIE = 0; MPP = the current mode.
4. The mode becomes M.
5. pc = *base* of `mtvec`; or, in vectored mode and for an interrupt, $pc = base + 4 \times cause$.

The instruction `mret` (encoding `0x30200073`) returns: the mode becomes that in MPP, MIE = MPIE, MPIE = 1, and pc = `mepc`. The instruction `wfi` (`0x10500073`), *wait for interrupt*, lets the processor stop until an interrupt is pending.

An interrupt of machine mode is taken when its bit is set in both `mip` and `mie` and, if the processor is in machine mode, MIE is 1. After an exception, a handler that wants to resume *after* the offending instruction (for example after an `ecall`) must add 4, or 2 for a compressed instruction, to `mepc`.

Table A.12. The principal CSRs of machine mode.

Address	Name	Content
0x300	mstatus	status: MIE (bit 3), global interrupt enable; MPIE (bit 7), its value before the trap; MPP (bits 12–11), the mode before the trap
0x301	misa	the base and the extensions implemented
0x304	mie	interrupt enables: MSIE (bit 3), MTIE (bit 7), MEIE (bit 11)
0x305	mtvec	trap vector: base address (bits 31–2) and mode (bits 1–0): 0 direct, 1 vectored
0x340	mscratch	a word for the use of the trap handler
0x341	mepc	the address of the instruction that was interrupted, or that caused the exception
0x342	mcause	bit 31: 1 for an interrupt, 0 for an exception; the other bits: the code of the cause
0x343	mtval	additional information: the faulting address, or the illegal instruction
0x344	mip	interrupts pending: MSIP (bit 3), MTIP (bit 7), MEIP (bit 11)
0xB00, 0xB02	mcycle, minstret	counters of clock cycles and of instructions retired (low 32 bits)
0xF11–0xF14	mvendorid, marchid, mimpid, mhartid	identification of the manufacturer, of the architecture, of the implementation and of the hardware thread

A.12 Encoding and decoding by hand

To decode a 32-bit word: write it in binary; read bits 6–0 and find the format in Table A.3; cut the fields as in Figure A.1; find the instruction with `funct3` (and `funct7`) in Table A.4 or A.5; rebuild the immediate and sign-extend it; translate the register numbers with Table A.2.

To encode, proceed in the opposite direction. For instance `sw a0, 8(sp)` is a store of format S with `rs2` = `a0` = 10, `rs1` = `sp` = 2, `funct3` = 010 and the immediate $8 = 0000000\,01000_2$:

$$\underbrace{0000000}_{imm[11:5]}\ \underbrace{01010}_{rs2}\ \underbrace{00010}_{rs1}\ \underbrace{010}_{funct3}\ \underbrace{01000}_{imm[4:0]}\ \underbrace{0100011}_{opcode} = \texttt{0x00A12423}.$$

Table A.14 collects the encodings met in the book.

A.13 Idioms

Table A.15 gathers the short sequences that recur in the programs of this book.

Table A.13. The standard values of `mcause`.

Interrupts (bit 31 = 1)		**Exceptions** (bit 31 = 0)	
Code	**Cause**	**Code**	**Cause**
1	supervisor software interrupt	0	instruction address misaligned
3	machine software interrupt	1	instruction access fault
5	supervisor timer interrupt	2	illegal instruction
7	machine timer interrupt	3	breakpoint
9	supervisor external interrupt	4	load address misaligned
11	machine external interrupt	5	load access fault
≥ 16	defined by the platform	6	store address misaligned
		7	store access fault
		8	environment call from U mode
		9	environment call from S mode
		11	environment call from M mode
		12	instruction page fault
		13	load page fault
		15	store page fault

Table A.14. Encodings met in the book (uncompressed forms).

Instruction	**Encoding**	**Instruction**	**Encoding**
`addi t1, t1, 1`	`0x00130313`	`sub t0, t1, t2`	`0x407302B3`
`lw t1, 0(t0)`	`0x0002A303`	`sub t1, t0, t1`	`0x40628333`
`sw t1, 0(t0)`	`0x0062A023`	`addi t0, t0, 10`	`0x00A28293`
`add t2, t0, t1`	`0x006283B3`	`sw a0, 8(sp)`	`0x00A12423`
`ret`	`0x00008067`	`nop`	`0x00000013`
`ecall`	`0x00000073`	`ebreak`	`0x00100073`
`mret`	`0x30200073`	`wfi`	`0x10500073`

Table A.15. Common idioms.

Purpose	**Code**
test bit n of `t0`	`srli t1, t0, n` ; `andi t1, t1, 1`
set, clear, toggle bits with a mask m	`ori t0, t0, m` ; `andi t0, t0, ~m` ; `xori t0, t0, m`
multiply by 2^k; unsigned and signed division by 2^k	`slli` ; `srli` ; `srai` (which rounds towards $-\infty$)
element i of an array of words at `t0`	`slli t1, t1, 2` ; `add t0, t0, t1` ; `lw a0, 0(t0)`
are `a0` and `a1` equal? (1 or 0)	`xor a0, a0, a1` ; `seqz a0, a0`
clamp `t2` to $[0, max]$	`bgez t2, 1f` ; `li t2, 0` ; `1: ble t2, t3, 2f` ; `mv t2, t3` ; `2:`
index modulo a power of two n	`andi t0, t0, n-1`
unsigned overflow of `add t2, t0, t1`	`sltu t3, t2, t0`
edge of a signal: now and not before	`not t1, t1` ; `and s1, s0, t1`
fixed point Q24.8: to integer; product	`srai a0, t1, 8` ; `mul` then `srai ..., 8`
decimal digit to ASCII character	`addi t0, t0, 48`
a counted loop	`li t1, n` ; `1: ...` ; `addi t1, t1, -1` ; `bnez t1, 1b`

APPENDIX B

PRG32 Reference

This appendix is a reference of the PRG32 platform as it is used in the book: the contract between a cartridge and the runtime, the programming interface, the cartridge file, the command-line tools, the three hosts and the Cartridge Store. It was compiled from the sources and the documentation of the repositories in October 2026 (Montella et al., 2026a; Montella, 2026; PRG32 Project, 2026); the platform evolves, and the repositories prevail over this text. The laboratory manual of the platform is the companion volume (Montella et al., 2026b).

B.1 The platform at a glance

Table B.1. The PRG32 platform in numbers.

Processor	Espressif ESP32-C6: one RISC-V core, RV32IMAC, up to 160 MHz, machine and user mode, no floating point
Memory	512 KB of SRAM, 320 KB of ROM, external flash
Display	ILI9341 TFT, 320 × 240 pixels, on SPI
Game viewport	320 × 200 pixels; two status bands of 20 pixels above and below
Frame buffer	8 bits per pixel (palette indices), with a palette of 256 RGB565 colours
Text grid	40 × 25 characters of 8 × 8 pixels
Tiles	8 × 8 pixels; a playfield of 64 × 32 tiles on 2 layers
Input	joystick (4 directions) and buttons A, B, START/SELECT, for one or two players
Sound	I2S amplifier (MAX98357A); notes, samples, tracks
Network	Wi-Fi: access point `PRG32` or a station on an existing network
Cartridge	a `.prg32` file, loaded at `0x40800000`; 4 slots of 131 072 bytes in flash
Cartridge RAM	64 KiB by default (profiles of 32 and of 128 KiB exist)
Hosts	the ESP32-C6 board; Espressif QEMU (emulated ESP32-C3); PRG32-QT
Licence	MIT

Table B.2. The repositories of the organisation `riscv-prg32` on GitHub.

Repository	Content
`PRG32`	the resident firmware, the cartridge tools (`python3 -m prg32`), the examples, the documentation
`PRG32-QT`	the desktop and mobile host, with the RV32IMAC interpreter and the debugger
`PRG32-Construction-Kit`	a studio for building games with blocks and sprites
`Playing-with-RISC-V_PRG32_textbook`	the laboratory manual, with its verified laboratories

B.2 The contract

A cartridge supplies three functions, whose names are a prefix chosen by the programmer followed by `_init`, `_update` and `_draw`; the prefix is passed to the builder with `--entry-prefix`.

`prefix_init`
is called once, when the cartridge starts. It sets up the state.

`prefix_update`
is called once per frame. It reads the input and changes the state.

`prefix_draw`
is called once per frame, after `update`. It turns the state into a picture.

The three functions follow the standard RISC-V calling convention (Appendix A), must return, and must not disable interrupts or touch the control and status registers. Registers do not keep their values between one call and the next: whatever must last lives in the sections `.data` or `.bss` of the cartridge. A function that never returns blocks the frame: PRG32-QT reports "instruction budget exhausted", and on the board a watchdog reports the stall.

The ABI table. A *portable* cartridge does not contain the addresses of the functions of the runtime. When the runtime enters `init`, `update` or `draw` it passes in `a0` a pointer to a table of function pointers, the ABI table; small stubs added by the builder store that pointer and let the source keep the familiar form `call prg32_gfx_clear`. The table has a magic number (`PABI`), a version, a hash and a set of feature bits (audio, Wi-Fi, multiplayer, metrics, keyboard, tile map, platformer, sprites); a cartridge declares in its header the hash and the features that it requires, and a host accepts it if it can honour them. New functions are only ever appended to the table, so that an old cartridge keeps working on a new host.

```
# mygame.S
.option norelax

.equ BLACK, 0x0000

.section .text
.global mygame_init
.global mygame_update
.global mygame_draw

mygame_init:                # once: set up the state
```

```
    ret

mygame_update:              # every frame: input, state
    ret

mygame_draw:                # every frame: state to picture
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata            # constants: strings, tables
.section .data              # variables
```

Listing B.1. The skeleton of a PRG32 cartridge in assembly.

B.3 Screen, colours and input

Coordinates. The origin is the top-left corner of the game viewport; x grows to the right from 0 to 319 and y grows downwards from 0 to 199. Drawing outside the viewport is clipped.

Colours. A colour is a 16-bit value in RGB565: 5 bits of red (bits 15–11), 6 of green (bits 10–5), 5 of blue (bits 4–0). The predefined constants are in Table B.3.

Table B.3. The colour constants (`PRG32_COLOR_*`).

BLACK	WHITE	RED	GREEN	BLUE	YELLOW	CYAN	MAGENTA
`0x0000`	`0xffff`	`0xf800`	`0x07e0`	`0x001f`	`0xffe0`	`0x07ff`	`0xf81f`

Buttons. `prg32_input_read` returns a word with one bit per button, 1 when pressed (Table B.4). START and SELECT are two names of the same input.

Table B.4. The button masks (`PRG32_BTN_*` and `PRG32_P2_BTN_*`).

	LEFT	RIGHT	UP	DOWN	A	B	START
Player 1: bit	0	1	2	3	4	5	6
Player 1: mask	`0x0001`	`0x0002`	`0x0004`	`0x0008`	`0x0010`	`0x0020`	`0x0040`
Player 2: bit	8	9	10	11	12	13	14
Player 2: mask	`0x0100`	`0x0200`	`0x0400`	`0x0800`	`0x1000`	`0x2000`	`0x4000`

B.4 The programming interface

The functions that a portable cartridge can call are the entries of the ABI table: 139 of them in version 1.6. The tables that follow give their C prototypes, as declared in

`components/prg32/include/prg32.h`. In assembly the arguments go, in order, in `a0`, `a1`, ..., and the result comes back in `a0`; a call may destroy every `t` and `a` register and `ra`.

B.4.1 Time, input, random numbers

Prototype	Effect
`uint32_t prg32_ticks_ms(void)`	milliseconds since the start
`uint32_t prg32_input_read(void)`	the buttons of both players, one bit each
`uint32_t prg32_input_read_player(uint8_t player)`	the buttons of one player
`uint32_t prg32_input_read_menu(void)`	the buttons as the menus read them
`void prg32_input_wait_released(uint32_t mask)`	waits until the buttons of the mask are released
`uint32_t prg32_controller_read(void)`	the raw state of the controllers
`uint32_t prg32_random_number(uint32_t min, uint32_t max)`	a uniform random number in $[min, max]$, both included; returns min if $max \leq min$

B.4.2 Graphics

Prototype	Effect
`void prg32_gfx_clear(uint16_t color)`	fills the viewport with a colour
`void prg32_gfx_pixel(int x, int y, uint16_t color)`	draws one pixel
`void prg32_gfx_rect(int x, int y, int w, int h, uint16_t color)`	draws a filled rectangle
`void prg32_gfx_text8(int x, int y, const char *s, uint16_t fg, uint16_t bg)`	writes a zero-terminated string with the 8×8 font, on its background colour
`void prg32_gfx_present(void)`	sends the frame to the display (the runtime does it after `draw`)
`void prg32_gfx_set_fullscreen(int enabled)`	uses the whole 320×240 display
`int prg32_gfx_fullscreen_enabled(void)`	whether full screen is active
`int prg32_gfx_snapshot_row_rgb565(int y, uint16_t *out, size_t pixels)`	copies a row of the frame as RGB565
`void prg32_palette_set(uint8_t index, uint16_t rgb565)`	changes an entry of the palette: every pixel with that index changes at the next frame
`uint16_t prg32_palette_get(uint8_t index)`	reads an entry of the palette
`void prg32_gfx_clear_indexed(uint8_t index)`	as `clear`, with a palette index
`void prg32_gfx_pixel_indexed(int x, int y, uint8_t index)`	as `pixel`, with a palette index
`void prg32_gfx_rect_indexed(int x, int y, int w, int h, uint8_t index)`	as `rect`, with a palette index

B.4.3 Console, status bands, splash screens

Prototype	Effect
`void prg32_console_clear(void)`	clears the text console
`void prg32_console_putc(int ch)`	writes a character
`void prg32_console_write(const char *s)`	writes a string
`void prg32_console_hex32(uint32_t value)`	writes a word as eight hexadecimal digits

Prototype	Effect
`void prg32_band_set_mode(uint8_t band, prg32_band_mode_t mode)`	chooses what a status band shows (band 0 top, 1 bottom)
`void prg32_band_set_text(uint8_t band, const char *text)`	writes a text in a band
`void prg32_band_set_game_info(const char *text)`	writes the information line of the game
`void prg32_band_log(const char *message)`	appends a message to the band log
`void prg32_band_set_colors(uint8_t band, uint16_t fg, uint16_t bg)`	sets the colours of a band
`void prg32_splash_show_game(const char *title, const char *subtitle, uint32_t duration_ms, uint16_t bg, uint16_t fg, uint16_t accent)`	shows a title screen for a time
`void prg32_debug_overlay_draw(int enabled, int x, int y, uint32_t input_mask, uint32_t frame)`	draws the input mask and the frame number

On the board and in QEMU the console is drawn on the screen; PRG32-QT collects it in its log. The other functions of the groups `prg32_band_*` and `prg32_splash_*` of the table (`band_mode`, `band_mode_name`, `band_use_default_colors`, `band_load_config`, `band_save_config`, `splash_draw`, `splash_show`, `splash_draw_game`, `splash_show_default`) are variants of these.

B.4.4 Sound

Prototype	Effect
`void prg32_audio_note(uint8_t channel, uint8_t instrument, uint8_t note, uint8_t volume, uint32_t duration_ms)`	plays a MIDI note (0–127; 69 is A4, 440 Hz; 72 is C5) for a duration
`void prg32_audio_note_off(uint8_t channel)`	silences a channel
`void prg32_audio_notes(uint8_t channel, uint8_t instrument, uint8_t volume, const prg32_midi_note_t *notes, size_t count)`	plays a sequence of notes
`void prg32_buzzer_tone(uint32_t hz, uint32_t ms, uint16_t duty)`	a tone of a given frequency on the buzzer
`void prg32_buzzer_play_notes(const prg32_note_t *notes, size_t count)`	a sequence of tones
`void prg32_buzzer_sample_u8(const uint8_t *samples, size_t count, uint32_t sample_rate)`	plays unsigned 8-bit PCM samples centred on 128

Audio calls are safe when no output is available: they do nothing. The runtime owns the initialisation of the audio and the master volume. The table also contains the polyphonic interface (`prg32_audio_note_on`, `note_on_pan`, `play_sample`, `play_sample_pan`, `play_track`, `stop_track`, `stop_channel`, `stop_all`, `set_tempo`, `set_master_volume`, `set_channel_volume`, `set_channel_pan`), described in `docs/software/audio_polyphony.md`.

B.4.5 Tiles, playfields and platforms

Prototype	Effect
`void prg32_tile_clear(uint16_t color)`	clears the tile screen
`void prg32_tile_define(uint8_t id, const uint8_t *bitmap8x8, uint16_t fg, uint16_t bg)`	defines a tile from 8 bytes, one per row
`void prg32_tile_put(uint8_t tx, uint8_t ty, uint8_t id)`	puts a tile in a cell of the 40×25 grid
`void prg32_tile_present(void)`	draws the tile screen
`void prg32_playfield_clear(uint8_t layer, uint8_t tile_id)`	fills a layer of the 64×32 playfield
`void prg32_playfield_put(uint8_t layer, uint8_t tx, uint8_t ty, uint8_t id)`	puts a tile in the playfield
`uint8_t prg32_playfield_get(uint8_t layer, uint8_t tx, uint8_t ty)`	reads a tile of the playfield
`void prg32_playfield_scroll(uint8_t layer, int x, int y)`	sets the scroll position of a layer
`void prg32_playfield_scroll_by(uint8_t layer, int dx, int dy)`	moves the scroll position
`void prg32_playfield_parallax(uint8_t layer, int x_q8, int y_q8)`	sets the speed of a layer relative to the camera, in Q8 fixed point ($256 = 1$)
`void prg32_playfield_camera(int x, int y)`	moves the camera
`void prg32_playfield_draw(uint8_t layer, int transparent_zero)`	draws a layer
`void prg32_playfield_draw_dual(void)`	draws both layers
`void prg32_platform_tile_flags(uint8_t tile_id, uint8_t flags)`	marks a tile as solid, platform, hazard or collectable
`int prg32_platform_solid_at(uint8_t layer, int pixel_x, int pixel_y)`	whether a pixel is inside a solid tile
`void prg32_platform_actor_init(prg32_platform_actor_t *actor, uint8_t layer, int x, int y, int w, int h)`	prepares an actor of a platform game
`uint16_t prg32_platform_actor_step(prg32_platform_actor_t *actor, uint32_t input_mask, int move_speed, int jump_speed, int gravity, int max_fall)`	one step of movement with gravity and collisions; returns contact flags
`void prg32_platform_camera_follow(const prg32_platform_actor_t *actor, int deadzone_x, int deadzone_y)`	makes the camera follow an actor

B.4.6 Sprites

Prototype	Effect
`void prg32_sprite_draw_8x8(int x, int y, const uint8_t *bits, uint16_t fg, uint16_t bg)`	draws a one-bit sprite of 8×8
`void prg32_sprite_draw_16x16(int x, int y, const uint16_t *rgb565)`	draws an RGB565 sprite of 16×16
`void prg32_sprite_draw_24x24(int x, int y, const uint16_t *rgb565)`	draws an RGB565 sprite of 24×24
`void prg32_sprite_draw_frame(int x, int y, int w, int h, const uint16_t *frames, uint32_t frame, uint16_t transparent)`	draws one frame of an animation, with a transparent colour
`uint32_t prg32_sprite_anim_frame(uint32_t now_ms, uint32_t frame_count, uint32_t frame_ms)`	which frame of an animation is due

Prototype	Effect
`void prg32_sprite_draw_indexed(int x, int y, const prg32_indexed_sprite_t *sprite, uint32_t frame)`	draws a sprite with 1, 2, 4 or 8 bits per pixel and its own palette
`void prg32_sprite_draw_bitplanes(int x, int y, const prg32_indexed_sprite_t *sprite, uint32_t frame)`	the same, from bit planes
`int prg32_sprite_hitbox(int ax, int ay, int aw, int ah, int bx, int by, int bw, int bh)`	whether two rectangles overlap

B.4.7 Scores, text input and other services

Prototype	Effect
`int prg32_score_submit(const char *game, const char *player, uint32_t score)`	records a score in the local table
`int prg32_score_submit_current_player(const char *game, uint32_t score)`	the same, for the current player
`int prg32_score_count(const char *game)`	how many scores are recorded
`int prg32_score_get(const char *game, int index, prg32_score_t *out_score)`	reads a recorded score
`int prg32_scoreboard_show(const char *game, const char *title)`	shows the table of the best scores
`int prg32_score_sync_remote(void)`	synchronises the scores with the Store
`int prg32_text_input(char *buffer, size_t capacity, const char *title)`	asks the player for a text with an on-screen keyboard

The remaining entries of the table serve the on-screen keyboard (`prg32_keyboard_*`), the name of the player (`prg32_score_player_*`), the cartridge slots (`prg32_cart_*`), the Wi-Fi state (`prg32_wifi_*`), multiplayer games (`prg32_multiplayer_*`) and performance measurement (`prg32_perf_*`); they are documented in `docs/software` and `docs/measurement` of the repository.

B.5 The cartridge file

A `.prg32` file is a header followed by the code and the data of the program, optionally followed by a trailer of metadata. Multi-byte fields are little-endian.

The first group of fields of Table B.12 is the original header, which old tools can still read; the second group extends it for portable cartridges. The command `cartridge summary` prints all of them.

Limits. The memory needed (`mem_size`) must fit in the cartridge RAM of the host: 64 KiB in the default profile, 32 KiB in the "classroom" profile, 128 KiB in an optional profile of the board. The option `--cart-ram-kib` of the builder sets the limit to check against.

Writing portable cartridges. A portable cartridge may be loaded at different addresses on different hosts. Code must therefore be position-relative, which the assembler guarantees for `la`, `call` and the branches; a cartridge should not store addresses of code or of strings in initialised tables of pointers in `.data`.

Table B.12. The header of a cartridge (`prg32_cart_header_v2_t`).

Field	Size	Content
`magic`	4 bytes	the characters `PRG2`
`abi_major`, `abi_minor`	2 + 2	the version of the cartridge format
`header_size`	2	the length of the header: it tells the two versions apart
`flags`	2	audio block (bit 0), multiplayer (1), ABI table (2), relocatable (3)
`load_addr`	4	`0x40800000`
`code_size`	4	the bytes of code and initialised data in the file
`mem_size`	4	the memory needed when running: code, data and `.bss`
`init_offset`, `update_offset`, `draw_offset`	4 + 4 + 4	the positions of the three entry points
`payload_crc32`	4	a checksum of the payload
`name`	32	the name of the cartridge
`abi_hash`	4	the hash of the ABI table for which it was built
`required_features`, `optional_features`	4 + 4	the feature bits needed, and those used if present
`isa_flags`	4	the instruction-set extensions used
`relocation_offset`, `relocation_count`	4 + 4	reserved for relocation
`import_model`	4	1 = `abi-table` (portable); 0 = legacy absolute imports

The metadata trailer. The command `store attach-metadata` appends after the program a trailer, marked `PRG32META`, made of blocks: `META` (the metadata, format `prg32-metadata-1.0`), `ICON` (the icon), `COLO` (the colophon, format `prg32-colophon-1.0`) and, optionally, `SCRN` (a screenshot) and `SIGN` (a signature). The program is not modified.

B.6 The command-line tools

All the tools are subcommands of one Python module, run from the folder of the PRG32 repository with ESP-IDF active (Chapter 1):

```
python3 -m prg32 <command> [subcommand] [options]
```

Every command accepts `--help`. On Windows the interpreter is called `python`. Commands and subcommands may be abbreviated to any prefix that is not ambiguous: `python3 -m prg32 c b` is `cartridge build`. The Store commands read `--store-url` and `--token` from the command line or, when they are missing, from the file `~/.prg32/config.json`; an address without a scheme is taken as `https://`.

Table B.13. The commands of `python3 -m prg32`.

Command	Effect
`doctor`	checks the toolchain: `idf.py`, the compiler, the partitions

Command	Effect
`runtime --url URL`	prints the state of a running host
`cartridge build SRC`	builds a portable cartridge from a `.S` or `.c` file. Options: `--out`, `--name`, `--entry-prefix`, `--portable`, `--architecture {esp32c6,qemu}`, `--cart-ram-kib`, `--multiplayer`, `--audio-block`, `--required-feature`, `--optional-feature`, `--build-dir`
`cartridge summary FILE`	prints the header of a cartridge
`esp32c6 upload FILE`	uploads a cartridge to a host over HTTP (`--url`, `--slot cart0…cart3`)
`esp32c6 run`	starts a stored cartridge
`esp32c6 upload-and-run FILE`	both
`esp32c6 build`, `flash`, `build-and-flash`	build the firmware of the board and write it through USB
`esp32c6 erase-flash`, `reset`	erase the flash; erase and write again
`esp32c6 screenshot`, `memory`, `performance`	a screenshot, a memory analysis, the results of the performance test
`qemu build`, `run`, `build-and-run`	build the firmware for QEMU and start the emulator
`qemu upload FILE`	stage a cartridge into the flash image of QEMU (with QEMU closed)
`store attach-metadata FILE`	append or replace the metadata trailer (`--metadata`, `--icon`, `--colophon`, `--architecture`, `--out`)
`store inspect-metadata FILE`	print the trailer of a cartridge
`store pack-bundle`	pack the ZIP bundle named by a manifest (`--manifest`, `--out`)
`store publish-bundle ZIP`	submit a bundle (`--store-url`, `--token`)
`store list`, `download`, `discover`	list and download cartridges; find Stores on the local network
`store set-url`, `clear-url`	set the Store used by local builds
`wifi set`, `wifi clear`	the Wi-Fi credentials and mode of the next firmware build (`--ssid`, `--password`, `--mode`)
`abi gen`, `abi check`	generate and verify the headers of the ABI table
`debug --url URL ACTION`	control the debugger of PRG32-QT: `state`, `enable`, `disable`, `pause`, `step`, `resume`, `speed`
`test`	run the tests of the tools

```
# build
python3 -m prg32 cartridge build mygame.S --portable \
    --entry-prefix mygame --name mygame --out mygame.prg32

# run on PRG32-QT, on this computer
python3 -m prg32 esp32c6 upload-and-run mygame.prg32 \
    --url http://localhost:8080 --slot cart0

# run on the board, joined to its Wi-Fi network
python3 -m prg32 esp32c6 upload-and-run mygame.prg32 \
    --url http://192.168.4.1 --slot cart0

# run on QEMU
```

```
python3 -m prg32 qemu upload mygame.prg32
python3 -m prg32 qemu run
```

Listing B.2. The loop of every chapter.

B.7 The hosts

B.7.1 The board

Table B.14. The reference wiring of the kit: the default pins of the firmware (`components/prg32/Kconfig`, October 2026).

Display (SPI)		**Player 1 (switches to ground)**	
GPIO7	MOSI	GPIO18	LEFT
GPIO2	MISO	GPIO19	RIGHT
GPIO6	SCLK	GPIO3	UP
GPIO10	CS (chip select)	GPIO13	DOWN
GPIO8	DC (data/command)	GPIO21	A
GPIO9	RST (reset)	GPIO22	B
GPIO5	BL (backlight)	GPIO20	START / SELECT
Audio (I2S)		**Other**	
GPIO4	BCLK (bit clock)	—	on-board RGB LED (disabled)
GPIO11	LRC / WS (word select)	USB	firmware, console
GPIO23	DIN (data)		

The pins are options of the firmware (`idf.py menuconfig`), and they changed in October 2026 with the new prototype of the printed circuit board: DC and RST of the display moved from GPIO1 and GPIO0 to GPIO8 and GPIO9, and the on-board RGB LED, which is wired to GPIO8, was disabled. At the time of writing the tables of `docs/hardware/hardware.md` still show the earlier wiring: when a display stays black, compare the wires with the options of the firmware that was flashed.

Each button joins its pin to ground, and the firmware enables the internal pull-up of the pin and inverts the level (Chapter 10). When it starts, the board creates the Wi-Fi network `PRG32` (password `prg32game`) and answers at `http://192.168.4.1`; with `wifi set --mode sta` it joins an existing network instead. By default the firmware starts on the *Setup* menu, from which a stored cartridge is run (*Run Cartridge*, then the slot); a firmware built with `PRG32_BOOT_SETUP_MODE=0` starts the stored cartridge directly.

B.7.2 QEMU

The emulator of Espressif runs the real firmware on an emulated ESP32-C3, with a window for the display. The keyboard is read from the terminal that launched it. A cartridge is staged into the flash image while the emulator is closed; like the board, the emulator starts on the *Setup* menu.

B.7.3 PRG32-QT

PRG32-QT is an application written with Qt 6 that interprets RV32IMAC and offers a cartridge the same ABI table as the firmware. It listens on port 8080 like a board, imports `.prg32` files, and browses the Store.

Table B.15. PRG32-QT: keys, profiles and debugger.

Keys	arrows or **W A S D**: joystick; **Z** or **J**: A; **X** or **K**: B; **Enter** or **Space**: Start/Select; **Escape**: back to Setup. Game controllers are recognised
Performance profiles	*Accurate* (timed like the ESP32-C6, the default), *Optimal* (30 frames per second), *Unlimited*
Debug mode	*Settings*, *Run cartridges in RISC-V debug mode*
Execution	**Pause**, **Resume**, **Step** (one instruction): three transport buttons drawn as on a video player; speed from 0.01x to 4x. The bug button switches instruction-level debugging on and off
Views	**Init**, **Update**, **Draw**, **PC**: the listing of each function, or at the program counter
Breakpoints	a click in the gutter of the listing; **Resume** ignores the current breakpoint once
State	the 32 registers; 128 bytes of memory at an address typed in the *Memory* field
Telemetry	frames, retired instructions, frames per second, late frames, the input mask
Errors	"instruction budget exhausted" (a frame that does not end); "unsupported CSR" (an access to a control register)

B.8 Testing a program on the three hosts

The same source file gives a cartridge for each host. This section is the step-by-step guide that the code deck of every lecture repeats with the name of its own program; here the program is `code/l05.S` of Chapter 5, whose entry prefix is `l05`. The repository of the course is assumed to be in `~/arc12` (`git clone https://github.com/riscv-prg32/computer-architecture-with-prg32 ~/arc12`), and every command is given from the folder of the PRG32 repository, in a terminal in which ESP-IDF is active (Chapter 1).

Table B.16. How each host is covered by the automatic tests of the course material.

Host	What the tests do
PRG32-QT	every program is built, run on the headless runner with scripted input, and the pixels of its frame are checked
QEMU	every program is built for the architecture `qemu` and staged into the flash image of the emulator; the picture is not checked, because the emulator needs its window
the board	every program is built for the architecture `esp32c6`; it is not run: there is no board on the test machine

B.8.1 On PRG32-QT

1. Start PRG32-QT on the computer. It listens on port 8080, like a board; its *Setup* screen shows the address to use if `localhost` does not answer.

2. Build the cartridge.
3. Send it to PRG32-QT and start it.
4. Look at the screen and press the keys: arrows or **W A S D** for the joystick, **Z** or **J** for A, **X** or **K** for B, **Enter** for Start, **Escape** to go back to Setup.
5. To look inside, enable *Run cartridges in RISC-V debug mode* in *Settings* and run the cartridge again.

```
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --out l05.prg32
python3 -m prg32 esp32c6 upload-and-run l05.prg32 \
    --url http://localhost:8080 --slot cart0
```

Listing B.3. Steps 2 and 3 on PRG32-QT.

Without the toolchain, a cartridge that somebody else has built is imported in PRG32-QT as a file and started with *Run Cartridge*.

B.8.2 On QEMU

1. Once: build the firmware for the emulator. It takes several minutes.
2. Build the cartridge for the architecture `qemu`.
3. With QEMU closed, stage the cartridge into the flash image.
4. Start the emulator. It opens on the *Setup* menu: choose *Run Cartridge*, then `cart0`.
5. Type in the *terminal* that launched QEMU, not in the window with the picture: arrows or **W A S D**, **Z** or **J** for A, **X** or **K** for B, **Enter** or **Space** for Select.

```
python3 -m prg32 qemu build
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --architecture qemu --out l05-qemu.prg32
python3 -m prg32 qemu upload l05-qemu.prg32
python3 -m prg32 qemu run
```

Listing B.4. Steps 1 to 4 on QEMU.

B.8.3 On the board

1. Once per board: connect it with a USB cable and write the firmware.
2. The board creates the Wi-Fi network `PRG32` (password `prg32game`) and answers at `192.168.4.1`: join the network with the computer.
3. Build the cartridge for the architecture `esp32c6`.
4. Send it over Wi-Fi and start it.
5. Use the joystick and the buttons of the kit.

```
python3 -m prg32 esp32c6 build-and-flash
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --architecture esp32c6 \
    --out l05-esp32c6.prg32
python3 -m prg32 esp32c6 upload-and-run l05-esp32c6.prg32 \
    --url http://192.168.4.1 --slot cart0
```

Listing B.5. Steps 1, 3 and 4 on the board.

B.8.4 The programs of the lectures

Table B.17 lists the programs in the folder `code`. The tutorial chapters have no program of their own there: their program is Verm1, in the folder `verm1` (Appendix D).

Table B.17. The program of each lecture, and its entry prefix.

Ch.	File	It shows	Ch.	File	It shows
0	`100.S`	a first program	11	`111.S`	a multiplexer in software
1	`101.S`	a test pattern	13	`113.S`	a state machine
2	`102.S` (`first`)	load, modify, store	14	`114.S`	serial and parallel
3	`103.S`	the stored program	15	`115.S`	bytes and words
4	`104.S`	an array	16	`116.S`	what survives a restart
5	`105.S`	two's complement	18	`118.S`	load, compute, store
6	`106.S`	fixed point	19	`119.S`	instructions as numbers
8	`108.S`	masks and De Morgan	20	`120.S`	one pixel
9	`109.S`	the full adder	21	`121.S`	polling the joystick
10	`110.S`	one button, one bit	24	`124.S`	the loop on the map

B.9 The Cartridge Store

The Store is a service that catalogues, reviews and distributes cartridges; the public instance is at https://store.prg32.uniparthenope.it. A submission is a *bundle*: one ZIP file that contains a manifest, an icon and one cartridge per architecture (Chapter 23).

Table B.18. The fields of `manifest.json` (format `prg32-metadata-1.0`).

Field	Content
`abi`	the format of the metadata: `prg32-metadata-1.0`
`id`	a unique and permanent identifier, by custom a reverse domain name
`title`	the name that the player reads
`version`	*major.minor.patch*; a published version is never replaced
`summary`, `description`	one line for the shelf; a paragraph
`authors`	a list of objects with a `name`
`license`	the licence, for example `MIT`
`tags`	words by which the cartridge is found
`assets`	the files of the bundle that are not cartridges: `icon`
`architectures`	a list of pairs `id` (`esp32c6`, `qemu`) and `file`

The file `metadata.json` attached to each cartridge is the manifest without `assets` and `architectures`. The *colophon* (`colophon.json`, format `prg32-colophon-1.0`) holds the credits: `title`, `subtitle`, `version`, `release_date`, `developer`, `authors` with their `role`, `license`, `copyright`, `acknowledgements`, `controls` (pairs `input` and `action`) and `start_prompt`.

The path of a submission. A bundle submitted with a personal token is *pending* until an editor has reviewed it; when it is approved it is *published* for the architectures declared. The Store also keeps score tables, which a cartridge reaches through the functions `prg32_score_*`.

Watch Out

The token of the Store is a personal credential. It is kept in an environment variable of the shell (`PRG32_STORE_TOKEN`), never in a file, in a repository or in a screenshot.

B.10 When something does not work

The symptoms met most often during installation are in Table 1.5. Three more concern the programs.

The builder says that an entry point is missing.
The prefix passed to `--entry-prefix` is not the one in the source, or a `.global` line is missing.

The game freezes, or returns to the menu.
A function does not return: a loop that never ends, or a `ra` that was not saved before a call and restored after it.

A value changes "by itself".
It was kept in a `t` or `a` register across a call, or `sp` was not restored: check the prologue and the epilogue, instruction by instruction, in the debugger.

APPENDIX C

Git and GitHub Reference

The platform of this book, the tools, the companion manual and the book itself live on GitHub, and so should the reader's project. This appendix is a reference of Git, the program that records the history of a project, and of GitHub, the service that hosts it: the concepts, the commands of every day, the workflow with which one contributes to PRG32, and the way in which a project such as Verm1 is kept and released. It assumes no previous experience. The authoritative sources are Chacon and Straub (2014) and the documentation of GitHub (GitHub, 2026).

C.1 Why version control

Chapter 7 gave an instruction that every tutorial repeated: *keep a copy of each finished part.* A folder full of files called `verm1_part2_ok.S` and `verm1_final_really.S` is the handmade form of version control. Git does the same thing properly: it remembers every recorded state of the files, says exactly what changed between two states, and returns to any of them.

Concept

Git records snapshots of a project, called *commits*; each one is a labelled state to which one can return. **GitHub** hosts a Git repository on line, so that the work is backed up, can be shared and can be reviewed. Together they turn "I think I broke something" into "let me see exactly what I changed".

C.2 Concepts

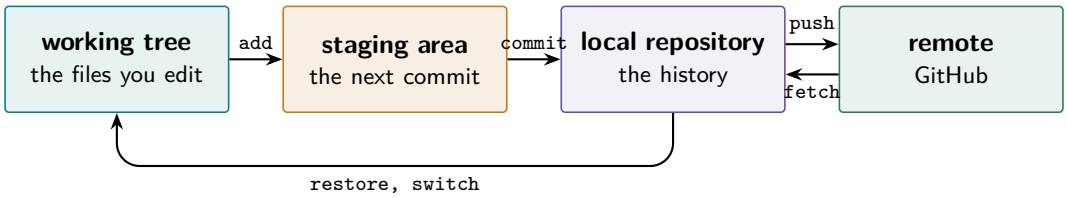


Figure C.1. The four places in which a change can be, and the commands that move it.

A change travels through four places (Figure C.1). It is made in the working tree; `git add` chooses it for the next commit; `git commit` records it in the local repository; `git push`

Table C.1. The vocabulary of Git and GitHub.

Term	Meaning
repository	a project with its whole history: the folder `.git` inside the project holds it
working tree	the files as they are now in the folder
staging area (index)	the changes chosen for the next commit
commit	a recorded snapshot, with an author, a date, a message and a unique identifier (a hash of 40 hexadecimal digits, usually abbreviated to 7)
branch	a movable name for a line of commits; the default one is `main`
`HEAD`	the commit on which the working tree is based
tag	a fixed name for one commit, typically a version: `v1.0.0`
remote	a copy of the repository elsewhere, typically on GitHub; the default name is `origin`
clone	a complete local copy of a remote repository
fork	a copy of somebody else's repository under one's own GitHub account
push / fetch / pull	send commits to a remote / receive them / receive and merge them
merge	to join the histories of two branches
pull request (PR)	a proposal, on GitHub, to merge a branch into another, with a discussion and a review
issue	a report or a task attached to a repository on GitHub
release	a tag published on GitHub with notes and, optionally, files to download

sends it to the remote. Nothing leaves the computer before the push.

C.3 Installing and configuring

Git is installed with the system's package manager (`brew install git` on macOS, `sudo apt install git` on Debian and Ubuntu) or, on Windows, with *Git for Windows* from https://git-scm.com; it was already installed in Chapter 1. It is configured once per computer (Listing C.1).

```
git --version
git config --global user.name  "Your Name"
git config --global user.email "your.address@studenti.uniparthenope.it"
git config --global init.defaultBranch main
git config --list
```

Listing C.1. Configuring Git, once.

A GitHub account is created at https://github.com. To let Git talk to GitHub without typing a password at every push, the simplest way is the official command-line client, `gh`, whose command `gh auth login` opens a browser and stores the credential in the keychain of the system; the alternative is an SSH key (`ssh-keygen -t ed25519`), whose *public* half is pasted in the settings of the account.

Watch Out

A private key, a password and a token are never committed, pasted in a file of the project or shown in a screenshot. This applies to the token of the Cartridge Store exactly as to the credentials of GitHub.

C.4 Getting a repository

There are two ways of obtaining a repository: copying an existing one, or making a folder into one (Listing C.2).

```
# 1. clone an existing repository: the platform, the debugger, the manual
git clone https://github.com/riscv-prg32/PRG32
git clone https://github.com/riscv-prg32/PRG32-QT
git clone https://github.com/riscv-prg32/Playing-with-RISC-V_PRG32_textbook

# 2. start a new repository in the folder of your game
cd ~/verm1
git init
git add verm1.S
git commit -m "Verm1 part 1: a head that crosses the world"
```

Listing C.2. Cloning the platform; starting a project of one's own.

To bring a clone up to date with the remote: `git pull`.

C.5 The everyday cycle

Three commands cover most of the daily use: `status`, to see what has changed; `add`, to choose what to keep; `commit`, to record it with a message (Listing C.3).

```
git status                      # what has changed?
git diff                        # exactly which lines?
git add verm1.S                 # choose the changes to record
git diff --staged               # what is about to be recorded?
git commit -m "Part 2: steering, with the U-turn rule as one XOR"
git log --oneline               # the history, one line per commit
```

Listing C.3. Inspect, stage, commit.

Commit after every *Run it*

The method of this book is one runnable step at a time. Commit at each checkpoint at which the program runs and behaves as stated: then any break is, at worst, one step's worth of work to inspect or to undo.

A commit message is a note to one's future self. Its first line says in the imperative what the commit does, in fifty characters or so: "Add the edge detector for button A", not "stuff" or "fix". If more is needed, a blank line follows and then an explanation of *why*.

C.5.1 What not to commit: `.gitignore`

Generated files do not belong in a repository: they can be rebuilt from the sources, and they change at every build. A file named `.gitignore` lists the patterns that Git must ignore (Listing C.4).

```
# build products
build/
*.o
*.elf
*.prg32
*.zip

# editors and operating systems
.vscode/
.DS_Store
Thumbs.db

# secrets: never
.env
*.token
```

Listing C.4. A `.gitignore` for a cartridge project.

Whether the built cartridge is kept under version control is a matter of choice; the usual answer is no, because a release (Section C.10) is the right place for it.

C.6 Looking at the history

```
git log --oneline --graph --all     # all the branches, as a graph
git log -p verm1.S                  # the changes made to one file
git show 3f2a9c1                    # one commit: message and changes
git diff v1.0.0 HEAD                # from a version to now
git blame verm1.S                   # who wrote each line, and when
git bisect start                    # find the commit that broke something
```

Listing C.5. Reading the history.

`git bisect` deserves a word. Given a commit in which the program worked and one in which it does not, it proposes the commit half-way, asks whether that one is good or bad, and repeats: a binary search in the history, which finds the guilty commit among a thousand in ten steps.

C.7 Undoing

Watch Out

The commands that rewrite history, such as `git reset --hard` and `git push --force`, destroy work, one's own and other people's. History that has been pushed is corrected with `git revert`, which adds a commit, and never by rewriting it.

Table C.2. How to undo, from the least to the most drastic.

Situation	Command
discard the changes of a file since the last commit	`git restore verm1.S`
take a file out of the staging area, keeping its changes	`git restore --staged verm1.S`
correct the message, or add a forgotten file, of the last commit not yet pushed	`git commit --amend`
bring back one file as it was in an older commit	`git restore --source 3f2a9c1 verm1.S`
cancel the effect of a commit already published, with a new commit	`git revert 3f2a9c1`
put work aside for a moment, and take it back	`git stash` ; `git stash pop`
look at an old state without changing anything	`git switch --detach 3f2a9c1`

C.8 Branches

A *branch* is an independent line of work. Before a risky change, for example the incremental drawing of Chapter 22, one makes a branch: if the experiment fails the branch is discarded and the main line is untouched; if it succeeds it is merged (Listing C.6).

```
git switch -c feature/pause-state   # create a branch and move to it
# ... edit, build, run, commit, as many times as needed ...
git switch main                     # back to the main line
git merge feature/pause-state       # bring the work in
git branch -d feature/pause-state   # the branch is no longer needed

git branch                          # list the branches
git switch -                        # go back to the previous branch
```

Listing C.6. Working on a branch.

Conflicts. If two branches have changed the same lines, the merge stops and Git marks the place in the file:

```
<<<<<<< HEAD
.equ SPEED_MIN, 32
=======
.equ SPEED_MIN, 40
>>>>>>> feature/levels
```

The conflict is resolved by editing the file until it says what is wanted, removing the three marker lines, and then running `git add` and `git commit`. `git merge --abort` gives up and returns to the state before the merge.

C.9 Working with GitHub

C.9.1 Publishing a repository

```
# with the GitHub client: create the repository and push, in one step
gh repo create verm1-yourname --public --source . --push
```

```
# or: create an empty repository on github.com, then
git remote add origin https://github.com/yourname/verm1-yourname.git
git push -u origin main

# afterwards
git push            # send new commits
git pull            # receive the commits made elsewhere
git remote -v       # where does this repository point?
```

Listing C.7. Publishing a local repository on GitHub.

C.9.2 The files of a good repository

The PRG32 repository is a model of the conventions (Table C.3): the files that orient a newcomer and make a project easy to evaluate.

Table C.3. The files that a repository should contain.

File	Content
`README.md`	what the project is, a screenshot, how to build it and run it, the controls
`LICENSE`	what others may do with it: MIT for PRG32 and for the programs of this book
`.gitignore`	what is not to be recorded
`CITATION.cff`	how to cite the project in a report or in a thesis
`CONTRIBUTING.md`	how to propose a change
`CHANGELOG.md`	what changed from one version to the next
`.github/workflows/`	the automatic checks run by GitHub Actions

```
# Verm1 Deluxe

A hungry worm on a 40 by 24 grid: a tribute to Snake, written in RISC-V
assembly for the PRG32 platform. My variant adds walls and a pause.

![The game](screenshot.png)

## Build and run

    python3 -m prg32 cartridge build verm1.S --portable \
        --entry-prefix verm1 --name verm1 --out verm1.prg32
    python3 -m prg32 esp32c6 upload-and-run verm1.prg32 \
        --url http://localhost:8080 --slot cart0

## Controls

Joystick: steer. A: start. B: pause.

## Credits and licence

Written by Your Name for ARC12, University of Naples "Parthenope".
MIT License: see LICENSE.
```

Listing C.8. A `README.md` for a variant of Verm1.

Files with the extension `.md` are written in Markdown: a line that begins with `#` is a title, `**bold**` is bold, a line indented by four spaces is code, and `[text](address)` is a link.

C.9.3 Forks and pull requests

One cannot push to a repository of which one is not a maintainer. The way of proposing a change to somebody else's project is the *pull request* (Figure C.2, Listing C.9).



Figure C.2. The path of a contribution.

```
gh repo fork riscv-prg32/PRG32 --clone       # fork, and clone the fork
cd PRG32
git switch -c docs/cartridge-workflow-notes  # a branch from main

# ... make the smallest change that proves the idea; run the checks ...

git add docs/
git commit -m "Clarify the cartridge workflow for first-time users"
git push -u origin docs/cartridge-workflow-notes
gh pr create --fill                          # open the pull request

# keep the fork up to date with the original
git remote add upstream https://github.com/riscv-prg32/PRG32.git
git fetch upstream
git switch main
git merge upstream/main
```

Listing C.9. A contribution to PRG32, from the fork to the pull request.

The rules of the PRG32 repository, in its `CONTRIBUTING.md`, are written for students first: pick one clear goal; create a branch from `main`; make the smallest change that proves the idea; update the documentation when behaviour, commands, paths or interfaces change; run the checks; and open a pull request that says what the *context* was, what *method* was followed and how the change was *validated*, on QEMU or on hardware. Branch names say the kind of work: `lab/...`, `game/...`, `docs/...`

Reviewing. A pull request shows the changes line by line, and anyone may comment on a line. A reviewer approves, or asks for changes; the author answers by pushing further commits to the same branch, which update the pull request. The checklist of Chapter 23, used by a colleague on one's bundle, is a review.

C.9.4 Issues

An *issue* reports a defect or proposes a task. A good report of a defect contains what was done (the exact command), what was expected, what happened (the *whole* message, as text), and the environment: the operating system, the version of the tools, the host. A commit or a pull request whose message contains `Fixes #12` closes issue 12 when it is merged.

C.9.5 GitHub Actions: checks that run by themselves

A file in `.github/workflows` describes a job that GitHub runs at every push or pull request, on a clean machine: typically, building the project and running its tests. The workflow of the PRG32 repository builds the in-tree cartridges with the same command-line tool that is used locally, for the ESP32-C6 and for QEMU, and exposes them as files to download. A green mark next to a commit means that the checks passed; a red cross means that something which worked on the author's computer does not work on a clean one, which is exactly what one wants to know before a merge.

```
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and test
        run: ./scripts/build-and-test.sh
```

Listing C.10. The outline of a workflow file (YAML). The build step is project-specific: for a cartridge it requires the ESP-IDF toolchain.

C.10 Versions, tags and releases

Chapter 23 gave Verm1 a version, 1.0.0, with the rule that a published version is never replaced. The numbering is called *semantic versioning*: *major.minor.patch*, where the patch grows for a fix, the minor for a new feature that breaks nothing, and the major for a change that is not compatible. In Git a version is a *tag*, and on GitHub a tag becomes a *release*, to which files can be attached (Listing C.11).

```
git tag -a v1.0.0 -m "Verm1 1.0.0: first release"
git push origin v1.0.0

# publish a release on GitHub and attach the bundle of the Store
gh release create v1.0.0 verm1-1.0.0.zip \
    --title "Verm1 1.0.0" --notes "First release: the finished game."

git tag                 # list the versions
git switch --detach v1.0.0   # look at the sources of a version
```

Listing C.11. Tagging version 1.0.0 and releasing the bundle.

The tag guarantees that the sources from which a published bundle was built can always be found again: it ties the file that players install to the lines that the author wrote.

C.11 This book and Overleaf

The sources of this book are a folder of LaTeX files that compiles on Overleaf (upload the folder as a ZIP file, or import the repository; the main document is `main.tex` and the compiler is pdfLaTeX). Overleaf can be linked to a GitHub repository, with which it

exchanges commits; and every Overleaf project is itself a Git repository that can be cloned, so that the text can be edited in a browser and versioned in the same way as a program. The slide decks of the course are generated from sources kept in the same manner, and corrections are welcome as pull requests.

C.12 Citing software

Software that is used in a report, in a thesis or in a paper is cited, like any other source. A repository that contains a file `CITATION.cff` shows on GitHub a button, *Cite this repository*, which produces the reference in the usual formats. The PRG32 repository has one, and its entry in the bibliography of this book (Montella et al., 2026a) is the model to follow.

C.13 Command summary

Table C.4. The commands of Git and of the GitHub client used in this appendix.

Command	Effect
`git config --global user.name "..."`	set the name of the author
`git init`	make the current folder a repository
`git clone URL`	copy a remote repository
`git status`	what has changed
`git diff` ; `git diff --staged`	the changes not yet staged; those staged
`git add FILE` ; `git add -p`	stage a file; stage a part of the changes
`git commit -m "message"`	record a snapshot
`git log --oneline --graph`	the history
`git show COMMIT`	one commit
`git restore FILE`	discard the changes of a file
`git restore --staged FILE`	unstage a file
`git revert COMMIT`	undo a commit with a new commit
`git stash` ; `git stash pop`	put work aside; take it back
`git switch -c NAME`	create a branch and move to it
`git switch NAME`	move to a branch
`git merge NAME`	merge a branch into the current one
`git branch` ; `git branch -d NAME`	list the branches; delete one
`git remote -v` ; `git remote add NAME URL`	list the remotes; add one
`git fetch` ; `git pull`	receive commits; receive and merge
`git push` ; `git push -u origin NAME`	send commits; the first push of a branch
`git tag -a v1.0.0 -m "..."`	name a version
`git bisect start` / `good` / `bad`	find the commit that introduced a defect
`gh auth login`	sign in to GitHub from the terminal
`gh repo create NAME --source . --push`	create a repository on GitHub from the current folder
`gh repo fork OWNER/REPO --clone`	fork a repository and clone the fork
`gh pr create` ; `gh pr status` ; `gh pr checkout N`	open a pull request; see its state; try one locally
`gh issue create` ; `gh issue list`	open an issue; list them

Command	Effect
`gh release create TAG FILES`	publish a release

Checkpoint

You should now be able to clone the platform, record your work step by step with meaningful messages, experiment on a branch, undo an unwanted change, publish your repository, propose a change with a pull request and release a version of your game. These operations are enough for a first year of coursework, and for a first contribution to an open project.

APPENDIX D

Verm1: The Complete Sources

This appendix prints in full the four reference programs of the project and the files of the Store bundle. The reference implementation of Verm1 was contributed by Giuseppe Coviello; the tutorial chapters are built on it. They are the files that the automatic tests of the course material build, run on the headless runner of PRG32-QT and play with a scripted player; the listings of the tutorial chapters are regions of these same files. A reader whose program refuses to work at the beginning of a tutorial starts from the file of the previous part (Table D.1).

Table D.1. The reference programs.

File	Chapter	Lines	The game at this point
`verm1_part1.S`	7	78	a grid, a head that moves by itself, a world without edges
`verm1_part2.S`	12	231	steering, the U-turn rule, food, a score in marks, a sound
`verm1_part3.S`	17	435	a body that grows, collisions, three states, a best score
`verm1_part4.S`	22	565	digits, levels, two sounds, incremental drawing: the finished game

Every part is built and run with the same two commands:

```
python3 -m prg32 cartridge build verm1_part4.S --portable \
    --entry-prefix verm1 --name verm1 --out verm1.prg32
python3 -m prg32 esp32c6 upload-and-run verm1.prg32 \
    --url http://localhost:8080 --slot cart0
```

D.1 Part 1: a head that crosses the world

```
# Verm1 (say "vermone"), a tribute to Snake - part 1: a head that crosses the
    world
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ CELL,  8               # a cell is 8 x 8 pixels
```

```
.equ COLS,  40              # 320 / 8 cells across
.equ ROWS,  24              # 192 / 8 cells down
.equ TOP,   8               # the top 8 pixels are the title line
.equ SPEED_Q8, 40           # 40/256 of a cell per frame (Q24.8)

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
verm1_init:
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)              # the head starts in column 5 ...
    la   t0, head_y
    li   t1, 12
    sw   t1, 0(t0)              # ... of row 12
    la   t0, step_q8
    sw   zero, 0(t0)            # no fraction of a step yet
    ret
verm1_update:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 2f             # less than one whole cell: wait
    sub  t1, t1, t2             # one cell done: keep the remainder
    sw   t1, 0(t0)

    la   t0, head_x
    lw   t3, 0(t0)
    addi t3, t3, 1              # one cell to the right
    li   t4, COLS
    blt  t3, t4, 1f
    li   t3, 0                  # past the right edge: back to column 0
1:  sw   t3, 0(t0)
    ret
2:  sw   t1, 0(t0)
    ret
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0                  # x
    li   a1, 0                  # y
    la   a2, title              # address of the string
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    la   t0, head_x
    lw   a0, 0(t0)
    slli a0, a0, 3              # x = column * 8
    la   t0, head_y
    lw   a1, 0(t0)
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
```

```
    li   a2, CELL               # width
    li   a3, CELL               # height
    li   a4, GREEN
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
.section .rodata
title:   .asciz "VERM1"

.section .data
head_x:  .word 5                # column of the head, 0 .. COLS-1
head_y:  .word 12               # row of the head, 0 .. ROWS-1
step_q8: .word 0                # fraction of a step, in 1/256
```

Listing D.1. `verm1_part1.S`.

D.2 Part 2: steering, food and a score

```
# Verm1 (say "vermone"), a tribute to Snake - part 2: steering and food
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ RED,   0xf800
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01        # one bit per button
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ DIR_RIGHT, 0           # a direction is a 2-bit code:
.equ DIR_DOWN,  1           #   bit 0 = vertical
.equ DIR_LEFT,  2           #   bit 1 = backwards
.equ DIR_UP,    3

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
verm1_init:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)
    la   t0, head_y
    li   t1, 12
    sw   t1, 0(t0)
```

```
    la   t0, dir
    sw   zero, 0(t0)            # moving right
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read       # a0 = one bit per button
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)              # remember it for the next step
5:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
```

```
92      sw   t1, 0(t0)              # the wanted direction is now the direction
93      slli t1, t1, 2              # table offset = direction * 4 bytes
94      la   t0, dx
95      add  t0, t0, t1
96      lw   t2, 0(t0)              # t2 = dx[direction]: -1, 0 or +1
97      la   t0, dy
98      add  t0, t0, t1
99      lw   t3, 0(t0)              # t3 = dy[direction]
100     la   t0, head_x
101     lw   t4, 0(t0)
102     add  t4, t4, t2
103     bgez t4, 1f
104     li   t4, COLS-1             # off the left edge: wrap
105 1:  li   t5, COLS
106     blt  t4, t5, 2f
107     li   t4, 0                  # off the right edge: wrap
108 2:  sw   t4, 0(t0)              # t4 = new column
109     la   t0, head_y
110     lw   t6, 0(t0)
111     add  t6, t6, t3
112     bgez t6, 3f
113     li   t6, ROWS-1
114 3:  li   t5, ROWS
115     blt  t6, t5, 4f
116     li   t6, 0
117 4:  sw   t6, 0(t0)              # t6 = new row
118     la   t0, food_x
119     lw   t1, 0(t0)
120     xor  t1, t1, t4             # 0 if same column
121     la   t0, food_y
122     lw   t2, 0(t0)
123     xor  t2, t2, t6             # 0 if same row
124     or   t1, t1, t2             # 0 only if both are equal
125     bnez t1, 5f                 # not on the food
126     la   t0, score
127     lw   t1, 0(t0)
128     addi t1, t1, 1
129     sw   t1, 0(t0)
130     call place_food
131     call eat_sound
132 5:  lw   ra, 12(sp)
133     addi sp, sp, 16
134     ret
135 place_food:
136     addi sp, sp, -16
137     sw   ra, 12(sp)
138     li   a0, 0
139     li   a1, COLS-1
140     call prg32_random_number    # a0 = a column, 0 .. 39
141     la   t0, food_x
142     sw   a0, 0(t0)
143     li   a0, 0
144     li   a1, ROWS-1
145     call prg32_random_number    # a0 = a row, 0 .. 23
146     la   t0, food_y
147     sw   a0, 0(t0)
```

```
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a2, 72                 # note: C5
    li   a3, 255                # volume
    li   a4, 60                 # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    la   t0, head_x
    lw   a0, 0(t0)
    la   t0, head_y
    lw   a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
```

```
    ret
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
.section .rodata
title:   .asciz "VERM1"
dx:      .word  1, 0, -1,  0    # right, down, left, up
dy:      .word  0, 1,  0, -1

.section .data
head_x:  .word 5
head_y:  .word 12
dir:     .word 0                # direction of the last step
want:    .word 0                # direction asked by the player
step_q8: .word 0
score:   .word 0
food_x:  .word 20
food_y:  .word 12
```

Listing D.2. `verm1_part2.S`.

D.3 Part 3: a body, rules and states

```
# Verm1 (say "vermone"), a tribute to Snake - part 3: a body, rules and states
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
.equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
.equ DIR_DOWN,  1
```

```
22 .equ DIR_LEFT,  2
23 .equ DIR_UP,    3
24 .equ TITLE, 0               # the three states of the game
25 .equ PLAY,  1
26 .equ OVER,  2
27 .section .text
28 .global verm1_init
29 .global verm1_update
30 .global verm1_draw
31
32 verm1_init:
33     la   t0, state
34     sw   zero, 0(t0)            # start on the title screen
35     la   t0, prev_btn
36     sw   zero, 0(t0)
37     ret
38 verm1_update:
39     addi sp, sp, -16
40     sw   ra, 12(sp)
41     sw   s0, 8(sp)
42     sw   s1, 4(sp)
43     call prg32_input_read
44     mv   s0, a0                 # s0 = buttons held in this frame
45     la   t0, prev_btn
46     lw   t1, 0(t0)              # t1 = buttons held one frame ago
47     sw   s0, 0(t0)
48     not  t1, t1
49     and  s1, s0, t1             # s1 = buttons pressed just now
50     la   t0, state
51     lw   t1, 0(t0)              # t1 = present state
52     li   t2, PLAY
53     beq  t1, t2, u_play
54     andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
55     beqz t3, u_done
56     li   t2, TITLE
57     beq  t1, t2, u_start
58     sw   t2, 0(t0)              # OVER --A--> TITLE
59     j    u_done
60 u_start:
61     call new_game               # TITLE --A--> PLAY
62     j    u_done
63 u_play:
64     mv   a0, s0
65     call play_update
66 u_done:
67     lw   s1, 4(sp)
68     lw   s0, 8(sp)
69     lw   ra, 12(sp)
70     addi sp, sp, 16
71     ret
72 new_game:
73     addi sp, sp, -16
74     sw   ra, 12(sp)
75     la   t0, grid
76     li   t1, COLS*ROWS
77 1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
```

```
78      addi t0, t0, 1
79      addi t1, t1, -1
80      bnez t1, 1b
81      li   t1, 0                  # i = 0, 1, 2: three cells in row 12
82  2:  la   t0, body_x
83      add  t0, t0, t1
84      addi t2, t1, 3
85      sb   t2, 0(t0)              # columns 3, 4, 5
86      la   t0, body_y
87      add  t0, t0, t1
88      li   t3, 12
89      sb   t3, 0(t0)
90      la   t0, grid+12*COLS
91      add  t0, t0, t2
92      li   t3, 1
93      sb   t3, 0(t0)              # mark the cell as occupied
94      addi t1, t1, 1
95      li   t3, 3
96      blt  t1, t3, 2b
97      la   t0, head_i
98      li   t1, 2
99      sw   t1, 0(t0)              # the head is entry 2 of the ring
100     la   t0, length
101     li   t1, 3
102     sw   t1, 0(t0)
103     la   t0, dir
104     sw   zero, 0(t0)
105     la   t0, want
106     sw   zero, 0(t0)
107     la   t0, step_q8
108     sw   zero, 0(t0)
109     la   t0, score
110     sw   zero, 0(t0)
111     call place_food
112     la   t0, state
113     li   t1, PLAY
114     sw   t1, 0(t0)
115     lw   ra, 12(sp)
116     addi sp, sp, 16
117     ret
118 play_update:                   # a0 = buttons held
119     addi sp, sp, -16
120     sw   ra, 12(sp)
121     la   t0, dir
122     lw   t1, 0(t0)              # t1 = direction of the last step
123     la   t0, want
124     lw   t2, 0(t0)              # t2 = wanted direction, so far
125     andi t3, a0, BTN_RIGHT
126     beqz t3, 1f
127     li   t2, DIR_RIGHT
128 1:  andi t3, a0, BTN_DOWN
129     beqz t3, 2f
130     li   t2, DIR_DOWN
131 2:  andi t3, a0, BTN_LEFT
132     beqz t3, 3f
133     li   t2, DIR_LEFT
```

```
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
    add  s0, s0, t2
    bgez s0, 1f
    li   s0, COLS-1             # off the left edge: wrap
1:  li   t4, COLS
    blt  s0, t4, 2f
    li   s0, 0
2:  la   t0, body_y
    add  t0, t0, t5
    lbu  s1, 0(t0)
    add  s1, s1, t3
    bgez s1, 3f
    li   s1, ROWS-1
3:  li   t4, ROWS
```

```
    blt  s1, t4, 4f
    li   s1, 0
4:
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, s0
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, s1
    or   s2, t1, t2            # s2 = 0 if the new head is on the food
    la   t0, length
    lw   t1, 0(t0)
    bnez s2, 5f                # not eating: the tail moves on
    li   t2, 255
    bge  t1, t2, 5f            # the ring is full: no more growth
    addi t1, t1, 1
    sw   t1, 0(t0)             # eating: one cell longer, the tail stays
    j    6f
5:  la   t0, head_i
    lw   t2, 0(t0)
    sub  t2, t2, t1
    addi t2, t2, 1
    andi t2, t2, 255           # ring index of the tail
    la   t0, body_x
    add  t0, t0, t2
    lbu  t3, 0(t0)
    la   t0, body_y
    add  t0, t0, t2
    lbu  t4, 0(t0)
    li   t5, COLS
    mul  t4, t4, t5
    add  t4, t4, t3            # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)           # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0            # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                # a free cell: go on
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)             # the worm bit itself: PLAY --> OVER
    la   t0, score
    lw   t1, 0(t0)
    la   t0, best
    lw   t2, 0(t0)
    ble  t1, t2, 9f
    sw   t1, 0(t0)             # a new best score
    j    9f
7:  li   t1, 1
    sb   t1, 0(t0)             # occupy the new cell
    la   t0, head_i
    lw   t1, 0(t0)
```

```
246     addi t1, t1, 1
247     andi t1, t1, 255            # the ring wraps after entry 255
248     sw   t1, 0(t0)
249     la   t0, body_x
250     add  t0, t0, t1
251     sb   s0, 0(t0)              # write the new head into the ring
252     la   t0, body_y
253     add  t0, t0, t1
254     sb   s1, 0(t0)
255     bnez s2, 9f
256     la   t0, score
257     lw   t1, 0(t0)
258     addi t1, t1, 1
259     sw   t1, 0(t0)
260     call place_food
261     call eat_sound
262 9:  lw   s2, 0(sp)
263     lw   s1, 4(sp)
264     lw   s0, 8(sp)
265     lw   ra, 12(sp)
266     addi sp, sp, 16
267     ret
268 place_food:
269     addi sp, sp, -16
270     sw   ra, 12(sp)
271     sw   s0, 8(sp)
272 1:  li   a0, 0
273     li   a1, COLS-1
274     call prg32_random_number
275     mv   s0, a0                 # s0 = a column
276     li   a0, 0
277     li   a1, ROWS-1
278     call prg32_random_number    # a0 = a row
279     li   t1, COLS
280     mul  t2, a0, t1
281     add  t2, t2, s0
282     la   t0, grid
283     add  t0, t0, t2
284     lbu  t3, 0(t0)
285     bnez t3, 1b                 # on the worm: try again
286     la   t0, food_x
287     sw   s0, 0(t0)
288     la   t0, food_y
289     sw   a0, 0(t0)
290     lw   s0, 8(sp)
291     lw   ra, 12(sp)
292     addi sp, sp, 16
293     ret
294 eat_sound:
295     addi sp, sp, -16
296     sw   ra, 12(sp)
297     li   a0, 0                  # channel
298     li   a1, 0                  # instrument
299     li   a2, 72                 # note: C5
300     li   a3, 255                # volume
301     li   a4, 60                 # milliseconds
```

```
302     call prg32_audio_note
303     lw   ra, 12(sp)
304     addi sp, sp, 16
305     ret
306 verm1_draw:
307     addi sp, sp, -16
308     sw   ra, 12(sp)
309     li   a0, BLACK
310     call prg32_gfx_clear
311     li   a0, 0
312     li   a1, 0
313     la   a2, title
314     call draw_text
315     la   t0, state
316     lw   t1, 0(t0)
317     li   t2, TITLE
318     bne  t1, t2, 1f
319     li   a0, 132                # TITLE: only an invitation
320     li   a1, 96
321     la   a2, press_a
322     call draw_text
323     j    2f
324 1:  call draw_score             # PLAY and OVER: the whole scene
325     call draw_worm
326     la   t0, food_x
327     lw   a0, 0(t0)
328     la   t0, food_y
329     lw   a1, 0(t0)
330     li   a2, RED
331     call draw_cell
332     la   t0, state
333     lw   t1, 0(t0)
334     li   t2, OVER
335     bne  t1, t2, 2f
336     li   a0, 124
337     li   a1, 96
338     la   a2, game_over
339     call draw_text
340 2:  lw   ra, 12(sp)
341     addi sp, sp, 16
342     ret
343
344 draw_text:                      # a0 = x, a1 = y, a2 = address of the string
345     addi sp, sp, -16
346     sw   ra, 12(sp)
347     li   a3, WHITE
348     li   a4, BLACK
349     call prg32_gfx_text8
350     lw   ra, 12(sp)
351     addi sp, sp, 16
352     ret
353 draw_worm:
354     addi sp, sp, -16
355     sw   ra, 12(sp)
356     sw   s0, 8(sp)
357     li   s0, 0                  # s0 = i: segment 0 is the head
```

```
1:  la   t0, length
    lw   t1, 0(t0)
    bge  s0, t1, 3f
    la   t0, head_i
    lw   t1, 0(t0)
    sub  t1, t1, s0
    andi t1, t1, 255            # ring index of segment i
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, BODY
    bnez s0, 2f
    li   a2, GREEN              # the head is brighter
2:  call draw_cell
    addi s0, s0, 1
    j    1b
3:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_score:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
```

```
    ret
.section .rodata                # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
.section .data                  # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0              # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0              # lost when the power goes
food_x:    .word 0
food_y:    .word 0
head_i:    .word 0              # ring index of the head
length:    .word 0              # number of cells of the worm
body_x:    .space 256           # the ring: one byte per segment
body_y:    .space 256
grid:      .space COLS*ROWS     # one byte per cell: 1 = worm
```

Listing D.3. `verm1_part3.S`.

D.4 Part 4: the finished game

```
# Verm1 (say "vermone"), a tribute to Snake - part 4: the finished game
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_MIN, 32          # 32/256 of a cell per frame at the start
.equ SPEED_MAX, 96          # the worm never goes faster than this
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
.equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
.equ DIR_DOWN,  1
.equ DIR_LEFT,  2
.equ DIR_UP,    3
.equ TITLE, 0               # the three states of the game
.equ PLAY,  1
.equ OVER,  2
```

```
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # draw the whole title screen, once
    ret
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    j    u_done
u_start:
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, grid
    li   t1, COLS*ROWS
```

```
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
    la   t0, head_i
    li   t1, 2
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
```

```
140     li   t2, DIR_DOWN
141 2:  andi t3, a0, BTN_LEFT
142     beqz t3, 3f
143     li   t2, DIR_LEFT
144 3:  andi t3, a0, BTN_UP
145     beqz t3, 4f
146     li   t2, DIR_UP
147 4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
148     li   t4, 2
149     beq  t3, t4, 5f             # a U-turn: ignore the request
150     sw   t2, 0(t0)
151 5:
152     la   t0, score
153     lw   t3, 0(t0)
154     slli t3, t3, 2
155     addi t3, t3, SPEED_MIN      # speed = 32 + 4 * score, in 1/256 of a cell
156     li   t4, SPEED_MAX
157     ble  t3, t4, 8f
158     mv   t3, t4                 # ... but never more than 96
159 8:  la   t0, step_q8
160     lw   t1, 0(t0)
161     add  t1, t1, t3             # add a fraction of a cell
162     li   t2, 256
163     blt  t1, t2, 6f             # less than one whole cell: wait
164     sub  t1, t1, t2
165     sw   t1, 0(t0)
166     call step                   # one whole cell: move
167     j    7f
168 6:  sw   t1, 0(t0)
169 7:  lw   ra, 12(sp)
170     addi sp, sp, 16
171     ret
172 step:
173     addi sp, sp, -16
174     sw   ra, 12(sp)
175     sw   s0, 8(sp)              # s0 = column of the new head
176     sw   s1, 4(sp)              # s1 = row of the new head
177     sw   s2, 0(sp)              # s2 = 0 when the worm eats
178     la   t0, want
179     lw   t1, 0(t0)
180     la   t0, dir
181     sw   t1, 0(t0)              # the wanted direction is now the direction
182     slli t1, t1, 2              # table offset = direction * 4 bytes
183     la   t0, dx
184     add  t0, t0, t1
185     lw   t2, 0(t0)              # t2 = dx[direction]
186     la   t0, dy
187     add  t0, t0, t1
188     lw   t3, 0(t0)              # t3 = dy[direction]
189     la   t0, head_i
190     lw   t5, 0(t0)              # t5 = ring index of the head
191     la   t0, body_x
192     add  t0, t0, t5
193     lbu  s0, 0(t0)              # one byte: the column of the head
194     la   t0, old_x
195     sw   s0, 0(t0)              # remember where the head was
```

```
196     add  s0, s0, t2
197     bgez s0, 1f
198     li   s0, COLS-1             # off the left edge: wrap
199 1:  li   t4, COLS
200     blt  s0, t4, 2f
201     li   s0, 0
202 2:  la   t0, body_y
203     add  t0, t0, t5
204     lbu  s1, 0(t0)
205     la   t0, old_y
206     sw   s1, 0(t0)
207     add  s1, s1, t3
208     bgez s1, 3f
209     li   s1, ROWS-1
210 3:  li   t4, ROWS
211     blt  s1, t4, 4f
212     li   s1, 0
213 4:
214     la   t0, food_x
215     lw   t1, 0(t0)
216     xor  t1, t1, s0
217     la   t0, food_y
218     lw   t2, 0(t0)
219     xor  t2, t2, s1
220     or   s2, t1, t2             # s2 = 0 if the new head is on the food
221     la   t0, length
222     lw   t1, 0(t0)
223     bnez s2, 5f                 # not eating: the tail moves on
224     li   t2, 255
225     bge  t1, t2, 5f             # the ring is full: no more growth
226     addi t1, t1, 1
227     sw   t1, 0(t0)              # eating: one cell longer, the tail stays
228     la   t0, erase_on
229     sw   zero, 0(t0)            # ... so there is no cell to erase
230     j    6f
231 5:  la   t0, head_i
232     lw   t2, 0(t0)
233     sub  t2, t2, t1
234     addi t2, t2, 1
235     andi t2, t2, 255            # ring index of the tail
236     la   t0, body_x
237     add  t0, t0, t2
238     lbu  t3, 0(t0)
239     la   t0, body_y
240     add  t0, t0, t2
241     lbu  t4, 0(t0)
242     la   t0, erase_x
243     sw   t3, 0(t0)              # remember the cell the tail leaves
244     la   t0, erase_y
245     sw   t4, 0(t0)
246     la   t0, erase_on
247     li   t5, 1
248     sw   t5, 0(t0)
249     li   t5, COLS
250     mul  t4, t4, t5
251     add  t4, t4, t3             # cell number of the tail = row * 40 + column
```

```
252     la   t0, grid
253     add  t0, t0, t4
254     sb   zero, 0(t0)            # the tail cell is free again
255 6:  li   t5, COLS
256     mul  t4, s1, t5
257     add  t4, t4, s0             # cell number of the new head
258     la   t0, grid
259     add  t0, t0, t4
260     lbu  t1, 0(t0)
261     beqz t1, 7f                 # a free cell: go on
262     call game_is_over           # the worm bit itself
263     j    9f
264 7:  li   t1, 1
265     sb   t1, 0(t0)              # occupy the new cell
266     la   t0, head_i
267     lw   t1, 0(t0)
268     addi t1, t1, 1
269     andi t1, t1, 255            # the ring wraps after entry 255
270     sw   t1, 0(t0)
271     la   t0, body_x
272     add  t0, t0, t1
273     sb   s0, 0(t0)              # write the new head into the ring
274     la   t0, body_y
275     add  t0, t0, t1
276     sb   s1, 0(t0)
277     la   t0, dirty
278     li   t1, 1
279     sw   t1, 0(t0)              # the screen must follow: a few cells
280     bnez s2, 9f
281     la   t0, score
282     lw   t1, 0(t0)
283     addi t1, t1, 1
284     sw   t1, 0(t0)
285     call place_food
286     li   a2, 72                 # a short high note
287     li   a4, 60
288     call play_note
289 9:  lw   s2, 0(sp)
290     lw   s1, 4(sp)
291     lw   s0, 8(sp)
292     lw   ra, 12(sp)
293     addi sp, sp, 16
294     ret
295 game_is_over:
296     addi sp, sp, -16
297     sw   ra, 12(sp)
298     la   t0, state
299     li   t1, OVER
300     sw   t1, 0(t0)              # PLAY --> OVER
301     la   t0, dirty
302     li   t1, 2
303     sw   t1, 0(t0)              # a new screen: draw everything
304     la   t0, score
305     lw   t1, 0(t0)
306     la   t0, best
307     lw   t2, 0(t0)
```

```
308     ble  t1, t2, 1f
309     sw   t1, 0(t0)              # a new best score
310 1:  li   a2, 36                 # a long low note
311     li   a4, 400
312     call play_note
313     lw   ra, 12(sp)
314     addi sp, sp, 16
315     ret
316 place_food:
317     addi sp, sp, -16
318     sw   ra, 12(sp)
319     sw   s0, 8(sp)
320 1:  li   a0, 0
321     li   a1, COLS-1
322     call prg32_random_number
323     mv   s0, a0                 # s0 = a column
324     li   a0, 0
325     li   a1, ROWS-1
326     call prg32_random_number    # a0 = a row
327     li   t1, COLS
328     mul  t2, a0, t1
329     add  t2, t2, s0
330     la   t0, grid
331     add  t0, t0, t2
332     lbu  t3, 0(t0)
333     bnez t3, 1b                 # on the worm: try again
334     la   t0, food_x
335     sw   s0, 0(t0)
336     la   t0, food_y
337     sw   a0, 0(t0)
338     lw   s0, 8(sp)
339     lw   ra, 12(sp)
340     addi sp, sp, 16
341     ret
342 play_note:                      # a2 = MIDI note, a4 = milliseconds
343     addi sp, sp, -16
344     sw   ra, 12(sp)
345     li   a0, 0                  # channel
346     li   a1, 0                  # instrument
347     li   a3, 255                # volume
348     call prg32_audio_note
349     lw   ra, 12(sp)
350     addi sp, sp, 16
351     ret
352 verm1_draw:
353     la   t0, dirty
354     lw   t1, 0(t0)
355     beqz t1, 3f                 # nothing changed: nothing to draw
356     sw   zero, 0(t0)
357     addi sp, sp, -16
358     sw   ra, 12(sp)
359     li   t2, 2
360     beq  t1, t2, 1f
361     call draw_changes           # one step: a few cells
362     j    2f
363 1:  call draw_full              # a new screen: everything
```

```
2:  lw   ra, 12(sp)
    addi sp, sp, 16
3:  ret
draw_changes:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, erase_on
    lw   t1, 0(t0)
    beqz t1, 1f
    la   t0, erase_x
    lw   a0, 0(t0)
    la   t0, erase_y
    lw   a1, 0(t0)
    li   a2, BLACK
    call draw_cell              # the cell the tail has left
1:  la   t0, old_x
    lw   a0, 0(t0)
    la   t0, old_y
    lw   a1, 0(t0)
    li   a2, BODY
    call draw_cell              # the old head is now body
    call draw_head
    call draw_food
    call draw_score
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_full:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    call draw_score
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
    call draw_text
    j    2f
1:  call draw_worm              # PLAY and OVER: the whole scene
    call draw_food
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
    la   a2, game_over
    call draw_text
```

```
2:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_head:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_i
    lw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_text:                      # a0 = x, a1 = y, a2 = address of the string
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_worm:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: segment 0 is the head
1:  la   t0, length
    lw   t1, 0(t0)
    bge  s0, t1, 3f
    la   t0, head_i
    lw   t1, 0(t0)
    sub  t1, t1, s0
    andi t1, t1, 255            # ring index of segment i
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
```

```
476     add  t0, t0, t1
477     lbu  a1, 0(t0)
478     li   a2, BODY
479     bnez s0, 2f
480     li   a2, GREEN              # the head is brighter
481 2:  call draw_cell
482     addi s0, s0, 1
483     j    1b
484 3:  lw   s0, 8(sp)
485     lw   ra, 12(sp)
486     addi sp, sp, 16
487     ret
488 draw_score:
489     addi sp, sp, -16
490     sw   ra, 12(sp)
491     la   t0, score
492     lw   a0, 0(t0)
493     la   a1, score_txt
494     call to_decimal             # the score, as three characters
495     li   a0, 56
496     li   a1, 0
497     la   a2, score_txt
498     call draw_text
499     la   t0, best
500     lw   a0, 0(t0)
501     la   a1, best_txt+5         # after the letters "BEST "
502     call to_decimal
503     li   a0, 256
504     li   a1, 0
505     la   a2, best_txt
506     call draw_text
507     lw   ra, 12(sp)
508     addi sp, sp, 16
509     ret
510 to_decimal:                     # a0 = value 0..999, a1 = address of 3 chars
511     li   t0, 10
512     remu t1, a0, t0             # the units: the remainder of value / 10
513     addi t1, t1, 48             # 48 is the ASCII code of '0'
514     sb   t1, 2(a1)
515     divu a0, a0, t0
516     remu t1, a0, t0             # the tens
517     addi t1, t1, 48
518     sb   t1, 1(a1)
519     divu a0, a0, t0
520     remu t1, a0, t0             # the hundreds
521     addi t1, t1, 48
522     sb   t1, 0(a1)
523     ret                         # a leaf function: no stack frame needed
524 draw_cell:                      # a0 = column, a1 = row, a2 = colour
525     addi sp, sp, -16
526     sw   ra, 12(sp)
527     slli a0, a0, 3              # x = column * 8
528     slli a1, a1, 3
529     addi a1, a1, TOP            # y = row * 8 + 8
530     mv   a4, a2                 # the colour is the fifth argument
531     li   a2, CELL
```

```
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
.section .rodata                # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
.section .data                  # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0              # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0              # lost when the power goes
food_x:    .word 0
food_y:    .word 0
head_i:    .word 0              # ring index of the head
length:    .word 0              # number of cells of the worm
body_x:    .space 256           # the ring: one byte per segment
body_y:    .space 256
grid:      .space COLS*ROWS     # one byte per cell: 1 = worm
dirty:     .word 2              # 0 nothing to draw, 1 a step, 2 everything
old_x:     .word 0              # where the head was before the step
old_y:     .word 0
erase_on:  .word 0              # 1 if the tail left a cell
erase_x:   .word 0
erase_y:   .word 0
score_txt: .asciz "000"         # filled in by to_decimal
best_txt:  .asciz "BEST 000"
```

Listing D.4. `verm1_part4.S`.

D.5 The files of the Store bundle

The three files that follow are those of Chapter 23, for the reference game. In a bundle of the reader's own, the identifier, the title and every occurrence of `Your Name` are replaced; the acknowledgement of the reference implementation stays, as for any work that starts from somebody else's.

```
{
  "abi": "prg32-metadata-1.0",
  "id": "it.uniparthenope.arc12.verm1",
  "title": "Verm1",
  "version": "1.0.0",
  "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
  "description": "Verm1 (say \"vermone\") is the course-long project of ARC12
    Computer Architecture at the University of Naples Parthenope. Steer the
    worm with the joystick, eat the red food, and do not bite yourself. Written
     from scratch in RISC-V assembly.",
```

```
  "authors": [{"name": "Your Name"}],
  "license": "MIT",
  "tags": ["arcade", "snake", "assembly", "education"],
  "assets": {"icon": "icon.png"},
  "architectures": [
    {"id": "esp32c6", "file": "verm1-esp32c6.prg32"},
    {"id": "qemu", "file": "verm1-qemu.prg32"}
  ]
}
```

Listing D.5. `manifest.json`.

```
{
  "abi": "prg32-metadata-1.0",
  "id": "it.uniparthenope.arc12.verm1",
  "title": "Verm1",
  "version": "1.0.0",
  "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
  "description": "Verm1 (say \"vermone\") is the course-long project of ARC12
    Computer Architecture at the University of Naples Parthenope. Steer the
    worm with the joystick, eat the red food, and do not bite yourself. Written
     from scratch in RISC-V assembly.",
  "authors": [
    {
      "name": "Your Name"
    }
  ],
  "license": "MIT",
  "tags": [
    "arcade",
    "snake",
    "assembly",
    "education"
  ]
}
```

Listing D.6. `metadata.json`: the manifest without `assets` and `architectures`.

```
{
  "abi": "prg32-colophon-1.0",
  "title": "Verm1",
  "subtitle": "A tribute to Snake, in RISC-V assembly",
  "version": "1.0.0",
  "release_date": "2026-12-18",
  "developer": {"name": "Your Name"},
  "authors": [
    {"role": "game design", "name": "Your Name"},
    {"role": "programming", "name": "Your Name"}
  ],
  "license": "MIT",
  "copyright": "Copyright 2026 Your Name",
  "acknowledgements": [
    "Giuseppe Coviello, reference implementation of Verm1",
    "ARC12 Computer Architecture, University of Naples Parthenope",
```

```
    "The PRG32 community"
  ],
  "controls": [
    {"input": "JOYSTICK", "action": "Steer the worm"},
    {"input": "A", "action": "Start, and return to the title after game over"}
  ],
  "start_prompt": "Press A to play"
}
```

Listing D.7. `colophon.json`.

APPENDIX E

Answers to the Questions

Each topical and tutorial chapter ends with multiple-choice questions in the style of the midterm test (section *Check your understanding*). The answers follow: the letter, the text of the right option, and a line of explanation. Answer first, and read afterwards: a question whose answer has been read teaches nothing.

Chapter 1: Setting Up

Q1.1 **B.** *ESP-IDF is not active in this terminal.* Activation lasts as long as the terminal: run export.sh, or open the ESP-IDF PowerShell.

Q1.2 **C.** *PRG32-QT.* PRG32-QT interprets the cartridge and has a built-in debugger; a compiler is needed only to build cartridges.

Q1.3 **C.** *it calls the runtime through the same table of functions on every host.* A portable cartridge depends only on the ABI, the table of functions of the runtime, which every host provides.

Chapter 2: First Contact

Q2.1 **B.** *The address of the counter.* la loads the address of the label; lw then reads the value stored at that address into t1.

Q2.2 **C.** *ra, because call overwrites it.* call writes the return address in ra. The callee's own calls would overwrite it, so a non-leaf function saves ra on the stack and restores it before ret.

Q2.3 **D.** *44.* 300 = 256 + 44 = 1 0010 1100 in binary. AND with 1111 1111 keeps the low eight bits: 0010 1100 = 44.

Chapter 3: History of Computing

Q3.1 **B.** *Program and data are stored in the same memory.* The stored-program concept: instructions are data in memory, fetched and executed one after another.

Q3.2 **C.** *Its instruction set is an open standard, free to implement.* RISC-V is an open standard instruction set architecture; anyone may design and sell a RISC-V processor without an architecture licence. It was not the first RISC (Berkeley RISC, MIPS, ARM came earlier).

Q3.3 **C.** *Ada Lovelace.* Ada Lovelace, for her 1843 notes on the Analytical Engine.

Chapter 4: Digital Information

Q4.1 **D.** *256.* A byte is 8 bits: $2^8 = 256$ patterns, for example the naturals from 0 to 255.

Q4.2 **B.** *magenta.* 0xF81F = 11111 000000 11111: red and blue at maximum, green at zero: magenta.

Q4.3 **C.** *5.* Four characters plus the terminating zero byte.

Q4.4 **C.** *0xCD.* RISC-V is little-endian: the least significant byte, 0xCD, goes at the lowest address.

Chapter 5: Integers

Q5.1 **B.** *-4.* Complement and add one: 0000 0011 + 1 = 0000 0100 = 4, so the pattern is -4. (252 is its unsigned reading.)

Q5.2 **B.** *100 + 28.* The range is [-128, 127]. 100 + 27 = 127 fits; 100 + 28 = 128 does not: two positives give a negative pattern (1000 0000 = -128).

Q5.3 **C.** *t2 = 1, t3 = 0.* Signed: -1 < 1 is true, so t2 = 1. Unsigned: 4,294,967,295 < 1 is false, so t3 = 0.

Q5.4 **A.** *0.101.* 0.625 × 2 = 1.25 (1); 0.25 × 2 = 0.5 (0); 0.5 × 2 = 1.0 (1): 0.101 = 1/2 + 1/8.

Chapter 6: Real Numbers

Q6.1 **D.** *Excess-k.* In excess-k, X = x + k: a larger number always has a larger pattern. This is why it encodes floating-point exponents, which must be compared quickly.

Q6.2 **B.** *1.5.* 0x180 = 384; 384 / 256 = 1.5. Integer part 0x01, fraction 0x80 = 128/256 = 0.5.

Q6.3 **B.** *3.0.* 0100 0000 0100 0000 ...: S = 0, E = 1000 0000 = 128, F = 100...: $1.1_2 \times 2^{128-127} = 1.5 \times 2 = 3.0$.

Q6.4 **C.** *infinity.* All ones in the exponent: infinity if the fraction is zero, NaN otherwise.

Chapter 7: Verm1, Part 1

Q7.1 **B.** *4.* 64/256 = 1/4 of a cell per frame: a whole cell every 4 frames.

Q7.2 **C.** *(80, 32).* x = 10 × 8 = 80; y = 3 × 8 + 8 = 32.

Q7.3 **B.** *draw calls other functions, which overwrite ra.* A call writes the return address in ra. A function that makes calls must keep its own ra safe on the stack; a leaf function need not.

Chapter 8: Boolean Algebra: Operators and Properties

Q8.1 **C.** *x = 1, y = 1.* NAND is the complement of AND: it is 0 only when both inputs are 1.

Q8.2 **B.** $\overline{a} + b$. $\overline{a \cdot \overline{b}} = \overline{a} + \overline{\overline{b}} = \overline{a} + b$, using involution on the second term.

Q8.3 **C.** *0x0000000A.* 0x5A = 0101 1010; AND with 0000 1111 keeps the low nibble: 1010 = 0xA.

Q8.4 **B.** *ori t0, t0, 0x20.* OR with a mask: x + 1 = 1 on bit 5, x + 0 = x elsewhere. xori would toggle it; addi would carry into bit 6 if the bit was already 1.

Chapter 9: Boolean Algebra: Functions and Minimisation

Q9.1 **D.** *256.* $2^{2^3} = 2^8 = 256$.

Q9.2 **B.** $A\,\overline{B}\,C$. Row 5 is 101: $A = 1$, $B = 0$, $C = 1$, so the minterm is $A\,\overline{B}\,C$.

Q9.3 **B.** *two variables.* A group of 2^k cells eliminates k variables: four cells, two variables.

Q9.4 **C.** *at least two inputs are 1.* C_{out} is the majority function: $AB + C_{in}(A \oplus B) = AB + AC_{in} + BC_{in}$. An odd number of 1s describes the sum bit S instead.

Chapter 10: Electronics

Q10.1 **B.** *260* Ω. R = (3.3 - 2) / 0.005 = 260 Ω.

Q10.2 **B.** *the pMOS conducts and the output is 1.* A low gate closes the pMOS, which joins the output to the supply; the nMOS is open.

Q10.3 **C.** *6.* A NAND (4) followed by an inverter (2).

Q10.4 **B.** *To make the pin read a defined 1 when the button is released.* Without it the released pin would float; the pull-up ties it to 3.3 V, and the pressed switch overrides it to ground.

Chapter 11: Combinational Machines

Q11.1 **B.** *5.* $\lceil \log_2 26 \rceil = 5$, since $2^4 = 16 < 26 \leq 32 = 2^5$. Six words remain unassigned: the code is incomplete.

Q11.2 **C.** *8 outputs, one at 1.* $2^3 = 8$ outputs, and exactly one is asserted: the decoded (one-hot) representation of the input word.

Q11.3 **B.** *3.* n selection lines choose among 2^n inputs: 3 lines for 8 inputs.

Q11.4 **C.** *does not detect the error.* Two alterations leave the parity unchanged. A single parity bit detects only an odd number of errors.

Chapter 12: Verm1, Part 2

Q12.1 **C.** *0x0a.* 0x08 OR 0x02 = 0x0a: one bit per button.

Q12.2 **B.** *DOWN (1).* 3 XOR 1 = 2: DOWN is the opposite of UP.

Q12.3 **B.** *each entry of the table is 4 bytes.* The entries are words: entry k is at offset 4k.

Q12.4 **C.** *t1 and t4 are equal.* XOR is 1 where bits differ: no difference, all zeros.

Chapter 13: Sequential Machines

Q13.1 **B.** *the present state only.* Moore: w maps S to U. In a Mealy machine the output depends on the state and on the input.

Q13.2 **C.** *Q stays at 1.* S = R = 0 is the hold condition: the feedback keeps the stored value.

Q13.3 **C.** *The flip-flop changes only on a clock edge.* The latch is transparent for the whole active level of the clock; the flip-flop samples D only on the edge.

Q13.4 **B.** *4.* $\lceil \log_2 10 \rceil = 4$ state variables, with the minimum-length binary code of Chapter 11. Ten would be needed with a one-hot code.

Chapter 14: Registers and Buses

Q14.1 **B.** *So that only the enabled source drives the lines.* Several sources share the same lines; all but one must be electrically disconnected (high impedance), otherwise two outputs would fight over the wire.

Q14.2 **C.** *32.* One bit per pulse: n pulses for n bits. A parallel transfer needs one pulse and n wires.

Q14.3 **C.** *8,192.* 2^{10} = 1,024 words of 8 bits: 8,192 bits, that is 1 KiB.

Chapter 15: Memory

Q15.1 **C.** *64 KiB.* 2^{16} = 65,536 locations of one byte: 64 KiB, the address space of a Commodore 64.

Q15.2 **D.** *0x01.* Little-endian: the least significant byte, 0x01, is at the lowest address.

Q15.3 **A.** *Because its capacitors lose their charge.* Each bit is a charge in a capacitor that leaks away; a circuit inside the memory rewrites every cell periodically.

Q15.4 **B.** *With sw to the address of a device register.* RISC-V has no I/O instructions: devices are memory-mapped and reached with ordinary loads and stores.

Chapter 16: Permanent Memory

Q16.1 **D.** *Flash.* Flash is the block-erasable evolution of EEPROM. EPROM needs ultraviolet light; ROM and PROM cannot be erased.

Q16.2 **B.** *The datum is fetched from main memory and copied into the cache, with its neighbours.* The access is simply slower: the block containing the address is loaded into the cache, betting that nearby addresses will be needed soon.

Q16.3 **B.** *Latency.* Latency is the wait for the sector to arrive under the head; seek time depends on the movement of the arm.

Q16.4 **C.** *a transition between pit and land.* A change of reflectivity, in either direction, is a 1; no change is a 0.

Chapter 17: Verm1, Part 3

Q17.1 **C.** *254, 255, 0, 1.* From the tail, head_i - length + 1 = -2, that is 254 after AND 255, up to the head: 254, 255, 0, 1.

Q17.2 **B.** *byte 127.* $3 \times 40 + 7 = 127$.

Q17.3 **B.** *the game starts once.* Only the frame in which A goes down gives s1 = 0x10: one edge, one transition.

Q17.4 B. *so that the head can enter the cell the tail leaves.* In the same step the tail moves away: that cell is free for the head.

Chapter 18: The Processor

Q18.1 B. *the status register with the flags.* RISC-V has no condition codes: branches compare registers directly.

Q18.2 C. *only loads and stores.* It is a load-store architecture: lw, lh, lb, sw, sh, sb and their unsigned forms.

Q18.3 B. *t0 was pushed on the stack.* The pointer moves down by one word and the value is stored on the new top: a push.

Q18.4 B. *accumulator := accumulator + [y].* The accumulator is the implicit operand and the destination.

Chapter 19: Instructions and Interrupts

Q19.1 B. *the seven least significant.* Bits 6 to 0, in every format: the processor always knows where to look first.

Q19.2 C. *4.* Addresses count bytes and an instruction is four bytes long.

Q19.3 C. *mepc.* mepc, written by the hardware at the trap and used by mret. mtvec holds the address of the handler.

Q19.4 C. *an illegal instruction is fetched.* It is caused by the instruction being executed, synchronously; the others come from outside, at any moment.

Chapter 20: Buses and the Video Card

Q20.1 C. *921,600 bytes.* 307,200 pixels × 3 bytes = 921,600 bytes.

Q20.2 B. *100 MB/s.* 2 bytes × 50 million transfers per second = 100 MB/s.

Q20.3 C. *the position of its colour in a palette.* The byte is an index; the actual colour is looked up in a table of 256 entries, the palette.

Q20.4 B. *Using graphics processors for non-graphical computation.* General-purpose computing on graphics processing units.

Chapter 21: Input and Output

Q21.1 B. *It turns a row and column combination into a key code.* From a decoded position in the key matrix to a compact n-bit code, as the encoder of Chapter 11.

Q21.2 B. *three cells with red, green and blue filters.* Three sub-pixels, each a liquid crystal cell behind a colour filter; a TFT in each stores its state.

Q21.3 B. *Its pixels are driven one at a time and must hold their state in between.* Persistence: without a transistor per pixel, the cells respond slowly and keep their state badly while the others are scanned.

Q21.4 C. *polling.* The program asks the device periodically. With interrupts the device would call the program.

Chapter 22: Verm1, Part 4

Q22.1 **B.** *32 30 35.* The characters '2', '0', '5': 0x32, 0x30, 0x35. The units are stored at offset 2.

Q22.2 **B.** *8.* One step every 4 frames is 64/256: 32 + 4 × score = 64, so score = 8.

Q22.3 **B.** *5.* la is two instructions, auipc and addi; then lw, beqz and ret.

Q22.4 **B.** *it calls no function and uses only a and t registers.* A leaf function that touches only caller-saved registers has nothing to save.

Chapter 23: The Cartridge Store

Q23.1 **B.** *publish 1.0.1.* A published version is treated as permanent: a correction is a new version with the same id.

Q23.2 **B.** *the metadata goes inside one cartridge, which has one architecture.* The list of files belongs to the bundle; a single cartridge states its own architecture, given on the command line.

Q23.3 **C.** *in an environment variable of your shell.* A token is a credential: it never goes into a file that is shared or versioned.

Chapter 24: Beyond One Core

Q24.1 **B.** *5.* With n unlimited, S = 1 / (1 - p) = 1 / 0.2 = 5.

Q24.2 **C.** *false dependences between uses of the same register name.* Two unrelated values that share a name get different physical registers and can be computed at the same time.

Q24.3 **B.** *30.* 256 bits are 32 bytes per turn: 960 / 32 = 30. The code does not change.

Q24.4 **B.** *OR.* The output is 1 whenever at least one input is 1: OR.

APPENDIX F

English–Italian Glossary

The course is taught in a bilingual setting: the lectures are in Italian, and the slides, this book and the technical documentation are in English. This glossary gathers, in alphabetical order, the terms of the glossaries that close the chapters, with the chapter in which each is introduced.

English	Italiano	Ch.
accelerator	acceleratore	24
access time	tempo di accesso	15
accumulator	accumulatore	18
active matrix	matrice attiva	21
address	indirizzo	2
address / data bus	bus indirizzi / bus dati	15
addressing mode	modo di indirizzamento	19
aligned word	parola allineata	15
analysis / synthesis	analisi / sintesi	8
arithmetic logic unit	unità aritmetico-logica	3
array	vettore (array)	4
backlight	retroilluminazione	21
base (radix)	base	5
bit / byte / word	bit / byte / parola	4
bitstream	bitstream	24
branch	salto condizionato	2
branch prediction	predizione dei salti	24
breadboard	basetta sperimentale (breadboard)	10
breakpoint	punto di interruzione	1
bus	bus	14
bus width	larghezza del bus	20
byte order	ordinamento dei byte	15
byte order (endianness)	ordinamento dei byte	4
cache hit / miss	successo / fallimento di cache	16
calling convention	convenzione di chiamata	2
canonical form	forma canonica	9
capacitor	condensatore	10
cardinality	cardinalità	11

English	Italiano	Ch.
carry / borrow	riporto / prestito	5
carry in / carry out	riporto entrante / uscente	9
cartridge	cartuccia	1, 2
cathode-ray tube	tubo a raggi catodici	21
chip select	selezione del chip	14
clause, product term	clausola, termine prodotto	9
clock period	periodo di clock	13
code word	parola codice	11
colour depth	profondità di colore	20
combinational network	rete combinatoria	11
comparator	comparatore	11
complement	complemento (negazione)	8
computer	calcolatore, elaboratore	3
computer science / informatics	informatica	3
context	contesto	19
control unit	unità di controllo	3, 18
controller	controller (unità di controllo della periferica)	20
current	corrente	10
decoder / encoder	decodificatore / codificatore	11
dedicated / common bus	bus dedicato / comune	14
denormalised	denormalizzato	6
device driver	driver di periferica	20, 21
don't care	condizione di indifferenza	9
dot-matrix / ink-jet printer	stampante ad aghi / a getto d'inchiostro	21
drum / toner	tamburo / toner	21
duality	dualità	8
emulator	emulatore	1
enable	abilitazione	11
exception / trap	eccezione / trap	19
excess-k (biased)	eccesso-k	6
exponent	esponente	6
factor, sum term	fattore, termine somma	9
feedback	retroazione	13
fetch / decode / execute	prelievo / decodifica / esecuzione	19
finite-state machine	automa a stati finiti	13
firmware	firmware	1, 16
fixed / variable length	lunghezza fissa / variabile	11
fixed point	virgola fissa	6
floating input	ingresso flottante	10
floating point	virgola mobile	6
gate, source, drain	gate, source, drain	10
general / special register	registro generico / speciale	18
ground	massa	10
half adder / full adder	semisommatore / sommatore completo	9
hardware description language	linguaggio di descrizione dell'hardware	24
hexadecimal	esadecimale	4
hidden bit	bit nascosto	6

English	Italiano	Ch.
high impedance	alta impedenza	14
high-performance computing	calcolo ad alte prestazioni	3
host	macchina ospite	1
immediate	immediato (costante)	2
instruction set	insieme (set) di istruzioni	3, 18
integrated circuit	circuito integrato	10
interpreter	interprete	1
interrupt	interruzione	19
interrupt latency	latenza di interruzione	19
interrupt service routine	routine di servizio dell'interruzione	19
involution	involuzione	8
Karnaugh map	mappa di Karnaugh	9
kernel (GPU)	kernel	24
keyboard / key	tastiera / tasto	21
label	etichetta	2
latch / flip-flop	latch / flip-flop (bistabile)	13
latency	tempo di latenza	16
level- / edge-triggered	sensibile al livello / al fronte	13
linked list	lista concatenata	4
liquid crystals	cristalli liquidi	21
literal	letterale	9
load / store	caricare / memorizzare	2
load, strobe	caricamento, strobe	14
logic circuit	circuito logico	8
logic gate	porta logica	8
logic level	livello logico	10
logical product (AND)	prodotto logico (congiunzione)	8
logical sum (OR)	somma logica (disgiunzione)	8
look-up table	tabella di look-up	24
loom	telaio	3
machine code	codice macchina	2
machine learning	apprendimento automatico	4
machine register	registro di macchina	18
magnetic core	nucleo magnetico (di ferrite)	15
main memory	memoria centrale	15
mainframe	mainframe, grande elaboratore	3
mantissa / fraction	mantissa	6
mask	maschera	8
mass storage	memoria di massa	16
memory-mapped I/O	I/O mappato in memoria	15
message passing	scambio di messaggi	24
minterm / maxterm	mintermine / maxtermine	9
most significant bit	bit più significativo	5
motherboard	scheda madre	16
multiplexer / demultiplexer	multiplexer / demultiplexer	11
natural number	numero naturale	5
nesting	annidamento	19

English	Italiano	Ch.
next-state function	funzione stato prossimo	13
node	nodo	24
noise margin	margine di rumore	10
non-volatile	non volatile	16
normalisation	normalizzazione	6
Not a Number (NaN)	non un numero (NaN)	6
one's complement	complemento a uno (alla base diminuita)	6
operand	operando	19
operation code (opcode)	codice operativo	19
out-of-order execution	esecuzione fuori ordine	24
overflow	overflow (traboccamento)	5
palette	tavolozza (palette)	20
parity bit	bit di parità	11
peripheral	periferica	20
pipeline stage	stadio della pipeline	19
pointing device	periferica di puntamento	21
polling	interrogazione ciclica	21
positional notation	notazione posizionale	4
present / next state	stato presente / prossimo	13
privileged instruction	istruzione privilegiata	18
product of sums (POS)	prodotto di somme (tipo S)	9
program counter	contatore di programma	13, 18
propagation delay	ritardo di propagazione	10, 11
pull-up resistor	resistenza di pull-up	10
punched card	scheda perforata	3
quantisation	quantizzazione	4
read / write enable	abilitazione lettura / scrittura	15
redundant code	codice ridondante	11
refresh	rinfresco (refresh)	15
register	registro	2, 13, 14
register renaming	ridenominazione dei registri	24
remainder	resto	5
resistor / resistance	resistore / resistenza	10
resolution	risoluzione	20
return address	indirizzo di ritorno	2
ripple-carry adder	sommatore a propagazione del riporto	9
rising / falling edge	fronte di salita / di discesa	13
rounding	arrotondamento	6
runtime	ambiente di esecuzione	1
sampling	campionamento	4
seek time	tempo di posizionamento	16
semiconductor, doping	semiconduttore, drogaggio	10
sequential network	rete sequenziale	13
serial / parallel transfer	trasferimento seriale / parallelo	14
shared memory	memoria condivisa	24
shift	scorrimento (shift)	5
shift register	registro a scorrimento	14

English	Italiano	Ch.
sign and magnitude	segno e modulo	5
sign extension	estensione del segno	5
signed integer	numero relativo (intero con segno)	5
single / double precision	singola / doppia precisione	6
soft core	soft core	24
solid-state disk	disco a stato solido	16
speed-up	accelerazione (speed-up)	24
stack	pila (stack)	2
stack	pila	4
stack pointer	puntatore alla pila	2
stack pointer	puntatore allo stack	18
state table / diagram	tabella / diagramma degli stati	13
status register, flags	registro di stato, indicatori	18
stored program	programma memorizzato	3
string	stringa	4
sum of products (SOP)	somma di prodotti (tipo P)	9
superscalar	superscalare	24
supervisor / user state	stato supervisore / utente	18
switching network	rete di commutazione	8
tape library	libreria a nastri	16
three-state (tri-state)	a tre stati (tristate)	14
to build	compilare (costruire)	1
to clamp	limitare a un intervallo	5
to flash	scrivere nella memoria flash	1
to mask an interrupt	mascherare un'interruzione	19
to set / clear / toggle	porre a 1 / azzerare / invertire	8
toolchain	catena di strumenti di sviluppo	1
track / sector / block	traccia / settore / blocco	16
transcoder	transcodificatore	11
transfer rate	velocità di trasferimento	20
tree	albero	4
truth table	tabella di verità	8
two's complement	complemento a due (alla base)	5
video card	scheda video	20
video memory	memoria video	20
volatile / persistent	volatile / persistente	15
voltage	tensione	10
word gate	porta di parola	14

Bibliography

Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

George Boole. *An Investigation of the Laws of Thought*. Walton and Maberly, London, 1854.

Giacomo Bucci. *Calcolatori elettronici: architettura e organizzazione*. McGraw-Hill Education, 4 edition, 2017.

Paul E. Ceruzzi. *A History of Modern Computing*. MIT Press, 2 edition, 2003.

Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. https://git-scm.com/book.

Espressif Systems. ESP32-C6 technical reference manual, 2025. https://www.espressif.com/en/support/documents/technical-documents.

Espressif Systems. ESP-IDF programming guide, 2026. https://docs.espressif.com/projects/esp-idf/.

EuroHPC Joint Undertaking. Advancing European Sovereignty in HPC with RISC-V: the DARE Project. Press release, European High-Performance Computing Joint Undertaking, 2025. https://eurohpc-ju.europa.eu/.

Michael J. Flynn. Some computer organizations and their effectiveness. *IEEE Transactions on Computers*, C-21(9):948–960, 1972.

GitHub. GitHub docs, 2026. https://docs.github.com.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

Richard W. Hamming. Error detecting and error correcting codes. *Bell System Technical Journal*, 29(2):147–160, 1950.

John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019, Institute of Electrical and Electronics Engineers, 2019.

Maurice Karnaugh. The map method for synthesis of combinational logic circuits. *Transactions of the American Institute of Electrical Engineers, Part I*, 72(5):593–599, 1953.

George H. Mealy. A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34(5):1045–1079, 1955.

Raffaele Montella. PRG32-QT: the portable C++20/Qt 6 host and debugger for PRG32 cartridges. https://github.com/riscv-prg32/PRG32-QT, 2026. Version 0.4.1, MIT License.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32: Platform for RISC-V Gaming Development (32-bit). Software repository, University of Naples "Parthenope", 2026a. https://github.com/riscv-prg32/PRG32.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. *Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform*. University of Naples "Parthenope", 2026b. https://github.com/riscv-prg32/Playing-with-RISC-V_PRG32_textbook.

Edward F. Moore. Gedanken-experiments on sequential machines. In *Automata Studies*, pages 129–153. Princeton University Press, 1956.

Gordon E. Moore. Cramming more components onto integrated circuits. *Electronics*, 38(8): 114–117, 1965.

David A. Patterson and John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2 edition, 2020.

PRG32 Project. PRG32 Cartridge Store: catalogue, review, scores, metrics and multiplayer service. https://github.com/riscv-prg32/CartridgeStore, 2026. MIT License.

RISC-V International. RISC-V ABIs specification (ELF psabi). Ratified specification, 2024a. https://github.com/riscv-non-isa/riscv-elf-psabi-doc.

RISC-V International. RISC-V assembly programmer's manual, 2024b. https://github.com/riscv-non-isa/riscv-asm-manual.

RISC-V International. The RISC-V Instruction Set Manual. Technical specification, RISC-V International, 2024c. https://riscv.org/technical/specifications/.

RISC-V International. The RISC-V instruction set manual, volume II: Privileged architecture. Ratified specification, 2024d. https://riscv.org/specifications/ratified/.

Claude E. Shannon. A symbolic analysis of relay and switching circuits. *Transactions of the American Institute of Electrical Engineers*, 57(12):713–723, 1938.

William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 11 edition, 2021.

Andrew S. Tanenbaum and Todd Austin. *Structured Computer Organization*. Pearson, 6 edition, 2013.

Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936.

John von Neumann. First draft of a report on the EDVAC. Technical report, Moore School of Electrical Engineering, University of Pennsylvania, 1945.